

Quantum Computing: Lecture Notes

Ronald de Wolf

QuSoft, CWI and University of Amsterdam

*Dedicated to the memory of my father
Abraham de Wolf (1942–2019)*

Preface from 2011

These lecture notes were formed in small chunks during my “Quantum computing” course at the University of Amsterdam, Feb-May 2011, and compiled into one text thereafter. Each chapter was covered in a lecture of 2×45 minutes, with an additional 45-minute lecture for exercises and homework. The first half of the course (Chapters 1–7) covers quantum algorithms, the second half covers quantum complexity (Chapters 8–9), stuff involving Alice and Bob (Chapters 10–13), and error-correction (Chapter 14). A 15th lecture about physical implementations and general outlook was more sketchy, and I didn’t write lecture notes for it.

These chapters may also be read as a general introduction to the area of quantum computation and information from the perspective of a theoretical computer scientist. While I made an effort to make the text self-contained and consistent, it may still be somewhat rough around the edges; I hope to continue polishing and adding to it. Comments & constructive criticism are very welcome, and can be sent to rdewolf@cwi.nl

Those who want to read more (much more...): see the book by Nielsen and Chuang [196] for the general area, the book of John Watrous [247] for quantum information theory, and the lecture notes of John Preskill [200] for the theoretical physics perspective.

Attribution, acknowledgments, subsequent updates

Most of the material in Chapters 1–6 [chapter numbers in this paragraph are for the 2011 version] comes from the first chapter of my PhD thesis [248], with a number of additions: the lower bound for Simon, the Fourier transform, the geometric explanation of Grover. Chapter 7 is newly written for these notes, inspired by Santha’s survey [216]. Chapters 8 and 9 are largely new as well. Section 3 of Chapter 8, and most of Chapter 10 are taken (with many changes) from my “quantum proofs” survey paper with Andy Drucker [100]. Chapters 11 and 12 are partly taken from my non-locality survey with Harry Buhrman, Richard Cleve, and Serge Massar [72]. Chapters 13 and 14 are new. Thanks to Giannicola Scarpa (the teaching assistant for the first two editions of this course) for useful comments on some of the chapters.

January’13: Updated and corrected a few things for the Feb-Mar 2013 version of this course, and included exercises for each chapter. Thanks to Harry Buhrman, Florian Speelman, and Jeroen Zuiddam for spotting some typos in the earlier version.

April’13: More updates, clarifications and corrections; moved some material from Chapter 2 to 1; changed and added some exercises. Thanks to Jouke Witteveen for useful comments.

April’14: Fixed and clarified a few more things. Thanks to Maarten Wegewijs for spotting a typo in Chapter 4.

March’15: Updated a few small things.

July’15: Updated and corrected a few small things, added more exercises. Thanks to Srinivasan Arunachalam, Carla Groenland, and Koen Groenland for useful comments.

May’16: A few more corrections, thanks to Ralph Bottesch for useful comments.

January’18: Many more corrections, more exercises, a new chapter about the Hidden Subgroup

Problem (Chapter 6; the above-mentioned chapter numbers are for the earlier version of the notes), and moved the hints about exercises to an Appendix for students who want to try the exercises first without hints. Thanks to Joran van Apeldoorn, Srinivasan Arunachalam, Rens Baardman, Alexander Belov, Koen de Boer, Daniel Chernowitz, András Gilyén, Ronald de Haan, Leon Ingelse, Stacey Jeffery, Rafael Kiesel, Jens Klooster, Sam Kuypers, Christian Nesenberend, and Christian Schaffner for useful comments.

January'19: More corrections, clarifications and exercises, and new chapters about Hamiltonian simulation (Chapter 9) and the HHL algorithm (Chapter 10). These two chapters can be taught together in two lectures, with the longer Chapter 9 spilling over into the second lecture if necessary. I marked by '(H)' the exercises having a hint in Appendix C, and removed citations from exercises to prevent students looking up the original papers when doing the exercises (which is neither necessary nor helpful). Those references are [49, 96, 103, 67, 102, 115, 75, 37, 50, 35, 118, 86, 19]. Thanks to Arjan Cornelissen, Sven Cornets de Groot, Gerrit Vos, and Harm de Vries for useful comments, and to András Gilyén for much help with Chapters 9 and 10. Thanks to my father and Mieke Beer for hosting me for two months while I was recovering from an ankle fracture in a wheelchair, from which much of these two chapters was written.

July'19: More corrections, clarifications and exercises. Thanks to Joran van Apeldoorn, András Gilyén, Stephanie Gonzalez, Sander Gribling, Jaco ter Hoeve, Arnold Kole, Lotte Mertens, Stefano Pironio, Merel Schalkers, Jim Skulte, Iris Smit, Manuel Van, and Sebastian Zur for useful comments. Thanks to Barbara Terhal for suggesting the possibility of dedicating these notes.

January'21: More corrections, clarifications and exercises, and a new chapter about QMA and the local Hamiltonian problem (Chapter 14). Thanks to Dorit Aharonov, Tomas Ehrencron, Alex Grilo, Joris Kattemölle, Stan de Lange, Noah Linden, Tessel Majtli, Nikhil Mande, Andrea Mazzocco, Robert Modderman, Thomas Preu, Philip Verduyn Lunel, Baer Ververgaert, and Carel Wagenaar for useful comments.

January'22: More corrections, clarifications and exercises. Thanks to Simon Apers, Christiaan van Asperen, Yanlin Chen, Lynn Engelberts, Sevag Gharibian, András Gilyén, Diego González-Sánchez, Bruno Jedynak, Joris Kattemölle, Julius Krebbekx, Zeph Landau, Noah Linden, Frédéric Magniez, Ryan Mann, and Yanelle Stolwijk for useful comments.

August'22: More corrections, clarifications and exercises, and a new chapter about some aspects of quantum machine learning (Chapter 19). Thanks to Danish Alvi, Srinivasan Arunachalam, Nikolaos Chatzis, Li Chen, Yanlin Chen, Iordanis Kerenidis, Mikhail Kudinov, Casper Loman, Frédéric Magniez, Galina Pass, Benjamin Shaffrey, Mario Szegedy, and Michael Walter for useful comments.

January'23: More corrections and clarifications, and a new chapter about the generalized adversary bound (Chapter 12). Thanks to Srinivasan Arunachalam, Martijn Brehm, Yanlin Chen, Lynn Engelberts, Noah Linden, Frédéric Magniez, Mehrdad Tahmasbi, Quinten Tupker, and Jordi Weggemans for useful comments.

Contents

1	Quantum Computing	1
1.1	Introduction	1
1.2	Quantum mechanics	2
1.2.1	Superposition	3
1.2.2	Measurement	3
1.2.3	Unitary evolution	6
1.3	Qubits and quantum memory	7
1.4	Elementary gates	8
1.5	Example: quantum teleportation	10
2	The Circuit Model and the Deutsch-Jozsa Algorithm	13
2.1	Quantum computation	13
2.1.1	Classical circuits	13
2.1.2	Quantum circuits	14
2.2	Universality of various sets of elementary gates	16
2.3	Quantum parallelism	16
2.4	The early algorithms	17
2.4.1	Deutsch-Jozsa	18
2.4.2	Bernstein-Vazirani	19
3	Simon's Algorithm	23
3.1	The problem	23
3.2	The quantum algorithm	23
3.3	Classical algorithms for Simon's problem	24
3.3.1	Upper bound	24
3.3.2	Lower bound	25
4	The Fourier Transform	29
4.1	The classical discrete Fourier transform	29
4.2	The Fast Fourier Transform	30
4.3	Application: multiplying two polynomials	31
4.4	The quantum Fourier transform	31
4.5	An efficient quantum circuit	32
4.6	Application: phase estimation	33

5	Shor's Factoring Algorithm	37
5.1	Factoring	37
5.2	Reduction from factoring to period-finding	37
5.3	Shor's period-finding algorithm	39
5.4	Continued fractions	41
6	Hidden Subgroup Problem	45
6.1	Hidden Subgroup Problem	45
6.1.1	Group theory reminder	45
6.1.2	Definition and some instances of the HSP	46
6.2	An efficient quantum algorithm if G is Abelian	47
6.2.1	Representation theory and the quantum Fourier transform	47
6.2.2	A general algorithm for Abelian HSP	48
6.3	General non-Abelian HSP	50
6.3.1	The symmetric group and the graph isomorphism problem	50
6.3.2	Non-Abelian QFT on coset states	50
6.3.3	Query-efficient algorithm	51
7	Grover's Search Algorithm	53
7.1	The problem	53
7.2	Grover's algorithm	53
7.3	Amplitude amplification	56
7.4	Application: satisfiability	58
8	Quantum Walk Algorithms	63
8.1	Classical random walks	63
8.2	Quantum walks	64
8.3	Applications	67
8.3.1	Grover search	67
8.3.2	Collision problem	67
8.3.3	Finding a triangle in a graph	68
9	Hamiltonian Simulation	71
9.1	Hamiltonians	71
9.2	Method 1: Lie-Suzuki-Trotter methods	72
9.3	Method 2: Linear combination of unitaries (LCU)	74
9.3.1	Hamiltonian simulation via LCU	75
9.4	Method 3: Transforming block-encoded matrices	76
9.4.1	Hamiltonian simulation via transforming block-encoded matrices	78
10	The HHL Algorithm	83
10.1	The linear-system problem	83
10.2	The basic HHL algorithm for linear systems	84
10.3	Improving the efficiency of the HHL algorithm	85

11 Quantum Query Lower Bounds	89
11.1 Introduction	89
11.2 The polynomial method	90
11.3 The quantum adversary method	92
12 Quantum Algorithms from the Generalized Adversary Bound	99
12.1 The generalized adversary bound	99
12.2 The dual of the generalized adversary bound	101
12.3 $\text{ADV}^\pm$ is an upper bound on quantum query complexity	102
12.4 Applications	104
12.5 Perfect composition and AND-OR trees	105
13 Quantum Complexity Theory	107
13.1 Most functions need exponentially many gates	107
13.2 Classical and quantum complexity classes	108
13.3 Classically simulating quantum computers in polynomial space	110
14 QMA and the Local Hamiltonian Problem	113
14.1 Quantum Merlin-Arthur (QMA)	113
14.2 The local Hamiltonian problem	114
14.3 Local Hamiltonian is QMA-complete	116
14.3.1 Completeness and soundness	117
14.3.2 Reducing the locality	119
14.4 Other interesting problems in QMA	119
14.5 Quantum interactive proofs	120
15 Quantum Encodings, with a Non-Quantum Application	125
15.1 Mixed states and general measurements	125
15.2 Quantum encodings and their limits	126
15.3 Lower bounds on locally decodable codes	128
16 Quantum Communication Complexity	133
16.1 Classical communication complexity	133
16.2 The quantum question	134
16.3 Example 1: Distributed Deutsch-Jozsa	135
16.4 Example 2: The Intersection problem	136
16.5 Example 3: The vector-in-subspace problem	137
16.6 Example 4: Quantum fingerprinting	137
17 Entanglement and Non-Locality	143
17.1 Quantum non-locality	143
17.2 CHSH: Clauser-Horne-Shimony-Holt	145
17.3 Magic square game	146
17.4 A non-local version of distributed Deutsch-Jozsa	148

18 Quantum Cryptography	151
18.1 Saving cryptography from Shor	151
18.2 Quantum key distribution	151
18.3 Reduced density matrices and the Schmidt decomposition	154
18.4 The impossibility of perfect bit commitment	155
18.5 More quantum cryptography	156
19 Quantum Machine Learning	159
19.1 Introduction	159
19.2 Supervised learning from quantum data	160
19.2.1 The PAC model of learning	160
19.2.2 Learning from quantum examples under the uniform distribution	161
19.2.3 Learning from quantum examples under all distributions	162
19.2.4 Learning quantum states from classical data	163
19.3 Unsupervised learning from quantum data	163
19.4 Optimization	164
19.4.1 Variational quantum algorithms	165
19.4.2 Some provable quantum speed-ups for optimization	165
20 Error-Correction and Fault-Tolerance	169
20.1 Introduction	169
20.2 Classical error-correction	169
20.3 Quantum errors	170
20.4 Quantum error-correcting codes	171
20.5 Fault-tolerant quantum computation	174
20.6 Concatenated codes and the threshold theorem	174
A Some Useful Linear Algebra	179
A.1 Vector spaces	179
A.2 Matrices	179
A.3 Inner product	180
A.4 Unitary matrices	180
A.5 Diagonalization and singular values	181
A.6 Tensor products	182
A.7 Trace	183
A.8 Rank	183
A.9 The Pauli matrices	184
A.10 Dirac notation	184
B Some other Useful Math and CS	185
B.1 Some notation, equalities and inequalities	185
B.2 Algorithms and probabilities	186
C Hints for Exercises	189

Chapter 1

Quantum Computing

1.1 Introduction

Today’s computers—both in theory (Turing machines) and practice (PCs, HPCs, laptops, tablets, smartphones, ...)—are based on classical physics. They are limited by locality (operations have only local effects) and by the classical fact that systems can be in only one state at the time. However, modern quantum physics tells us that the world behaves quite differently. A quantum system can be in a *superposition* of many different states at the same time, and can exhibit *interference* effects during the course of its evolution. Moreover, spatially separated quantum systems may be *entangled* with each other and operations may have “non-local” effects because of this.

Quantum computation is the field that investigates the computational power and other properties of computers based on quantum-mechanical principles. It combines two of the most important strands of 20th-century science: quantum mechanics (developed by Planck, Einstein, Bohr, Heisenberg, Schrödinger and others in the period 1900–1925) and computer science (whose birth may be dated to Turing’s 1936 paper [237]). An important objective is to find quantum algorithms that are significantly faster than any classical algorithm solving the same problem.

Quantum computation started in the early 1980s with suggestions for analog quantum computers by Yuri Manin [185] (and appendix of [186]), Richard Feynman [111, 112], and Paul Benioff [47], and reached more digital ground when in 1985 David Deutsch defined the universal quantum Turing machine [97]. See Preskill [201] for more on this early history. The following years saw only sparse activity, notably the development of the first algorithms by Deutsch and Jozsa [99] and by Simon [230], and the development of quantum complexity theory by Bernstein and Vazirani [53]. However, interest in the field increased tremendously after Peter Shor’s very surprising discovery of efficient quantum algorithms for the problems of integer factorization and discrete logarithms in 1994 [228], which was inspired by Simon’s work. Since most of current classical cryptography is based on the assumption that these two problems are computationally hard, the ability to actually build and use a quantum computer would allow us to break most current cryptographic systems, notably the RSA system [210, 211]. In contrast, a *quantum* form of cryptography due to Bennett and Brassard [51] is unbreakable even for quantum computers.

Here are three reasons to study quantum computers, from practical to more philosophical:

1. The process of miniaturization that has made current classical computers so powerful and cheap, has already reached micro-levels where quantum effects occur. Chipmakers tend to go to great lengths to suppress those quantum effects, forcing their bits and logical operations

to behave classically, but instead one might also try to work with them, enabling further miniaturization.

2. Making use of quantum effects allows one to speed up certain computations enormously (sometimes exponentially), and even enables some things that are impossible for classical computers. The main purpose of these lecture notes is to explain these advantages of quantum computing (algorithms, crypto, etc.) in detail.
3. Finally, one might say that the main goal of theoretical computer science is to “study the power and limitations of the strongest-possible computational devices that Nature allows us.” Since our current understanding of Nature is quantum mechanical, theoretical computer science should arguably be studying the power of quantum computers, not classical ones.

Before limiting ourselves to theory, let us say a few words about practice: *to what extent will quantum computers ever be built?* At this point in time, it is just too early to tell. The first small 2-qubit quantum computer was built in 1997 and in 2001 a 5-qubit quantum computer was used to successfully factor the number 15 [240]. Since then, experimental progress on a number of different technologies has been steady but slow. The most advanced implementations currently use superconducting qubits and ion-trap qubits. The largest quantum computation done at the time of writing is Google’s “quantum supremacy” experiment on 53 qubits [30], which performs a complicated (but rather useless) sampling task that appears to be no longer simulatable in a reasonable amount of the time on even the largest existing classical supercomputer.

The practical problems facing physical realizations of quantum computers seem formidable. The problems of noise and decoherence have to some extent been solved in theory by the discovery of quantum error-correcting codes and fault-tolerant computing (see, e.g., Chapter 20 in these notes), but these problems are by no means solved in practice. On the other hand, we should realize that the field of physical realization of quantum computing is still in its infancy and that classical computing had to face and solve many formidable technical problems as well—interestingly, often these problems were even of the same nature as those now faced by quantum computing (e.g., noise-reduction and error-correction). Moreover, while the difficulties facing the implementation of a full quantum computer may seem daunting, more limited applications involving quantum communication have already been implemented with some success, for example teleportation (which is the process of sending qubits using entanglement and *classical* communication), and versions of BB84 quantum key distribution are nowadays even commercially available.

Even if the theory of quantum computing never materializes to a real large-scale physical computer, quantum-mechanical computers are still an extremely interesting idea which will bear fruit in other areas than practical fast computing. On the physics side, it may improve our understanding of quantum mechanics. The emerging theories of entanglement and of Hamiltonian complexity have already done this to some extent. On the computer science side, the theory of quantum computation generalizes and enriches classical complexity theory and may help resolve some of its problems (see Section 15.3 for an example).

1.2 Quantum mechanics

Here we give a brief and abstract introduction to quantum mechanics. In short: a quantum state is a *superposition* of classical states, written as a vector of amplitudes, to which we can apply either a *measurement* or a *unitary operation*. For the required linear algebra we refer to Appendix A.

1.2.1 Superposition

Consider some physical system that can be in N different, mutually exclusive classical states. Because we will typically start counting from 0 in these notes, we call these states $|0\rangle, |1\rangle, \dots, |N-1\rangle$. Roughly, by a “classical” state we mean a state in which the system can be found if we observe it. A *pure quantum state* (usually just called *state*) $|\phi\rangle$ is a *superposition* of classical states, written

$$|\phi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle + \dots + \alpha_{N-1}|N-1\rangle.$$

Here α_i is a complex number that is called the *amplitude* of $|i\rangle$ in $|\phi\rangle$. Intuitively, a system in quantum state $|\phi\rangle$ is “in *all* classical states *at the same time*,” each state having a certain amplitude. It is in state $|0\rangle$ with amplitude α_0 , in state $|1\rangle$ with amplitude α_1 , and so on. Mathematically, the states $|0\rangle, \dots, |N-1\rangle$ form an orthonormal basis of an N -dimensional *Hilbert space* (i.e., an N -dimensional vector space equipped with an inner product). A quantum state $|\phi\rangle$ is a vector in this space, usually written as an N -dimensional column vector of its amplitudes:

$$|\phi\rangle = \begin{pmatrix} \alpha_0 \\ \vdots \\ \alpha_{N-1} \end{pmatrix}.$$

Such a vector is sometimes called a “ket.” Its conjugate transpose is the following row vector, sometimes called a “bra”:

$$\langle\phi| = (\alpha_0^*, \dots, \alpha_{N-1}^*).$$

The reason for this terminology (often called “Dirac notation” after Paul Dirac) is that an inner product $\langle\phi|\psi\rangle$ between two states corresponds to the dot product between a bra and a ket vector (“bracket”): $\langle\phi|\psi\rangle = \langle\phi| \cdot |\psi\rangle$.

We can combine different Hilbert spaces using tensor product: if $|0\rangle, \dots, |N-1\rangle$ are an orthonormal basis of space $\mathcal{H}_A$ and $|0\rangle, \dots, |M-1\rangle$ are an orthonormal basis of space $\mathcal{H}_B$, then the tensor product space $\mathcal{H} = \mathcal{H}_A \otimes \mathcal{H}_B$ is an NM -dimensional space spanned by the set of states $\{|i\rangle \otimes |j\rangle \mid i \in \{0, \dots, N-1\}, j \in \{0, \dots, M-1\}\}$. An arbitrary state in $\mathcal{H}$ is of the form $\sum_{i=0}^{N-1} \sum_{j=0}^{M-1} \alpha_{ij} |i\rangle \otimes |j\rangle$. Such a state is called *bipartite*. Similarly we can have *tripartite* states that “live” in a Hilbert space that is the tensor product of three smaller Hilbert spaces, etc.

There are two things we can do with a quantum state: measure it or let it evolve unitarily without measuring it. We will deal with measurement first.

1.2.2 Measurement

Measurement in the computational basis

Suppose we measure state $|\phi\rangle$. We cannot “see” a superposition itself, but only classical states. Accordingly, if we measure state $|\phi\rangle$ we will see one and only one classical state $|j\rangle$. Which specific $|j\rangle$ will we see? This is not determined in advance; the only thing we can say is that we will see state $|j\rangle$ with probability $|\alpha_j|^2$, which is the squared norm of the corresponding amplitude α_j . This is known as “Born’s rule.” Accordingly, observing a quantum state induces a probability distribution on the classical states, given by the squared norms of the amplitudes. This implies $\sum_{j=0}^{N-1} |\alpha_j|^2 = 1$, so the vector of amplitudes has (Euclidean) norm 1. If we measure $|\phi\rangle$ and get outcome j as a result¹, then $|\phi\rangle$ itself has “disappeared,” and all that is left is $|j\rangle$. In other words,

¹Don’t use the ambiguous phrase “we measure j ” in this case, since it’s not clear in that phrasing whether $|j\rangle$ is the state you’re applying the measurement to, or the outcome of the measurement.

observing $|\phi\rangle$ “collapses” the quantum superposition $|\phi\rangle$ to the classical state $|j\rangle$ that we saw, and all “information” that might have been contained in the amplitudes α_i is gone. Note that the probabilities of the various measurement outcomes are exactly the same when we measure $|\phi\rangle$ or when we measure state $e^{i\theta}|\phi\rangle$; because of this we sometimes say that the “global phase” $e^{i\theta}$ has no physical significance.

Projective measurement

For most of the topics in these notes, the above “measurement in the computational (or standard) basis” suffices. However, somewhat more general kinds of measurement than the above are possible and sometimes useful. The remainder of this subsection may be skipped on a first reading, but will become more relevant in the later parts of these notes, starting from Chapter 15.

A *projective* measurement on some space, with m possible outcomes, is a collection of projectors $P_1, \dots, P_m$ that all act on that same space and that sum to identity, $\sum_{j=1}^m P_j = I$.² These projectors are then pairwise orthogonal, meaning that $P_i P_j = 0$ if $i \neq j$. The projector P_j projects on some subspace V_j of the total Hilbert space V , and every state $|\phi\rangle \in V$ can be decomposed in a unique way as $|\phi\rangle = \sum_{j=1}^m |\phi_j\rangle$, with $|\phi_j\rangle = P_j |\phi\rangle \in V_j$. Because the projectors are orthogonal, the subspaces V_j are orthogonal as well, as are the states $|\phi_j\rangle$. When we apply this measurement to the pure state $|\phi\rangle$, then we will get outcome j with probability $\| |\phi_j\rangle \|^2 = \text{Tr}(P_j |\phi\rangle \langle \phi|) = \langle \phi | P_j | \phi \rangle$ and the measured state will then “collapse” to the new state $|\phi_j\rangle / \| |\phi_j\rangle \| = P_j |\phi\rangle / \| P_j |\phi\rangle \|$.³ The probabilities sum to 1 thanks to our assumption that $\sum_{j=1}^m P_j = I$ and the fact that trace is a linear function:

$$\sum_{j=1}^m \text{Tr}(P_j |\phi\rangle \langle \phi|) = \text{Tr}\left(\left(\sum_{j=1}^m P_j\right) |\phi\rangle \langle \phi|\right) = \text{Tr}(|\phi\rangle \langle \phi|) = \langle \phi | \phi \rangle = 1.$$

Note carefully that we cannot *choose* which P_j will be applied to the state but can only give a probability distribution. However, if the state $|\phi\rangle$ that we measure lies fully within one of the subspaces V_j , then the measurement outcome will be that j with certainty.

For example, a measurement in the computational basis on an N -dimensional state is the specific projective measurement where $m = N$ and $P_j = |j\rangle \langle j|$. That is, P_j projects onto the computational basis state $|j\rangle$ and the corresponding subspace $V_j \subseteq V$ is the 1-dimensional subspace spanned by $|j\rangle$. Consider the state $|\phi\rangle = \sum_{j=0}^{N-1} \alpha_j |j\rangle$. Note that $P_j |\phi\rangle = \alpha_j |j\rangle$, so applying our measurement to $|\phi\rangle$ will give outcome j with probability $\| \alpha_j |j\rangle \|^2 = |\alpha_j|^2$, and in that case the state collapses to $\alpha_j |j\rangle / \| \alpha_j |j\rangle \| = \frac{\alpha_j}{|\alpha_j|} |j\rangle$. The norm-1 factor $\frac{\alpha_j}{|\alpha_j|}$ may be disregarded because it has no physical significance, so we end up with the state $|j\rangle$ as we saw before.

Instead of the standard orthonormal basis, with basis states $|0\rangle, \dots, |N-1\rangle$, we may consider any other orthonormal basis B of states $|\psi_0\rangle, \dots, |\psi_{N-1}\rangle$, and consider the projective measurement defined by the projectors $P_j = |\psi_j\rangle \langle \psi_j|$. This is called “measuring in basis B .” Applying this measurement to state $|\phi\rangle$ gives outcome j with probability $\langle \phi | P_j | \phi \rangle = |\langle \phi | \psi_j \rangle|^2$. Note that if $|\phi\rangle$ equals one of the basis vectors $|\psi_j\rangle$, then the measurement gives that outcome j with probability 1.

In the previous two examples the projectors had rank 1 (i.e., project on 1-dimensional subspaces), but this is not necessary. For example, a measurement that distinguishes between $|j\rangle$

²The m projectors together form *one* measurement; don’t use the word “measurement” for individual P_j s.

³Don’t confuse the *outcome* of the measurement, which is the label j of the projector P_j that was applied, and the *post-measurement state*, which is $P_j |\phi\rangle / \| P_j |\phi\rangle \|$.

with $j < N/2$ and $|j\rangle$ with $j \geq N/2$ corresponds to the two projectors $P_1 = \sum_{j < N/2} |j\rangle\langle j|$ and $P_2 = \sum_{j \geq N/2} |j\rangle\langle j|$, each of rank $N/2$ (assume N is even). Applying this measurement to the state $|\phi\rangle = \frac{1}{\sqrt{3}}|1\rangle + \sqrt{\frac{2}{3}}|N\rangle$ gives outcome 1 with probability $\|P_1|\phi\rangle\|^2 = 1/3$, in which case the state collapses to $|1\rangle$. It gives outcome 2 with probability $\|P_2|\phi\rangle\|^2 = 2/3$, the state then collapses to $|N\rangle$.

Observables

A projective measurement with projectors $P_1, \dots, P_m$ and associated distinct outcomes $\lambda_1, \dots, \lambda_m \in \mathbb{R}$, can be written as one matrix $M = \sum_{i=1}^m \lambda_i P_i$, which is called an *observable*. This is a succinct way of writing down the projective measurement in one matrix, and has the added advantage that the *expected value* of the outcome can be easily calculated: if we are measuring a state $|\phi\rangle$, then the probability of outcome λ_i is $\|P_i|\phi\rangle\|^2 = \text{Tr}(P_i|\phi\rangle\langle\phi|)$, so the expected value of the outcome is $\sum_{i=1}^m \lambda_i \text{Tr}(P_i|\phi\rangle\langle\phi|) = \text{Tr}(\sum_{i=1}^m \lambda_i P_i |\phi\rangle\langle\phi|) = \text{Tr}(M|\phi\rangle\langle\phi|)$. Note that M is Hermitian: $M = M^*$. Conversely, since every Hermitian M has a spectral decomposition $M = \sum_{i=1}^m \lambda_i P_i$, there is a direct correspondence between observables and Hermitian matrices.

The Pauli matrices I, X, Y, Z (see Appendix A.9) are examples of 2-dimensional observables, with eigenvalues ± 1 . For example, $Z = |0\rangle\langle 0| - |1\rangle\langle 1|$ corresponds to measurement in the computational basis (with measurement outcomes $+1$ and -1 for $|0\rangle$ and $|1\rangle$, respectively).

Suppose we have a bipartite state. An observable A on the first part of the state corresponds to an observable $A \otimes I$ on the bipartite state. Similarly, an observable B on the second part of the state corresponds to an observable $I \otimes B$ on the bipartite state. Separately measuring observables A and B on the two parts of a bipartite state is different from measuring the joint observable $A \otimes B$: the separate measurements give one outcome each, while the joint measurement gives only one outcome, and the distribution on the post-measurement state may be different. What is true, however, is that the measurement statistics of the product of outcomes is the same as the measurement statistics of the outcome of the joint measurement. For example consider the case when $A = B = Z$ (these correspond to measurement in the computational basis), and the bipartite state is $|\phi\rangle = \frac{1}{\sqrt{2}}(|0\rangle \otimes |0\rangle + |1\rangle \otimes |1\rangle)$. With the separate measurements, the outcomes will be $++$ or $--$ (note that in both cases the product of the two outcomes is $+1$) and the state $|\phi\rangle$ will collapse to either $|0\rangle \otimes |0\rangle$ or $|1\rangle \otimes |1\rangle$. Yet $|\phi\rangle$ remains undisturbed by a joint measurement with ± 1 -valued observable $Z \otimes Z$, because $|\phi\rangle$ is a $+1$ -eigenstate of $Z \otimes Z$.

POVM measurement

If we only care about the final probability distribution on the m outcomes, not about the resulting post-measurement state, then the most general type of measurement we can do is a so-called *positive-operator-valued measure* (POVM). This is specified by m positive semidefinite (psd) matrices $E_1, \dots, E_m$ that sum to identity. When measuring a state $|\phi\rangle$, the probability of outcome i is given by $\text{Tr}(E_i|\phi\rangle\langle\phi|)$. A projective measurement is the special case of a POVM where the measurement elements E_i are projectors.⁴

There are situations where a POVM can do things a projective measurement cannot do.⁵ For

⁴Note that if E_i is a projector, then $\text{Tr}(E_i|\phi\rangle\langle\phi|) = \text{Tr}(E_i^2|\phi\rangle\langle\phi|) = \text{Tr}(E_i|\phi\rangle\langle\phi|E_i) = \|E_i|\phi\rangle\|^2$, using the fact that $E_i = E_i^2$ and the cyclic property of the trace. These inequalities can fail if E_i is psd but not a projector.

⁵Even though POVMs strictly generalize projective measurements, one can show that every POVM can be “simulated” by a projective measurement on a slightly larger space that yields the exact same probability distribution over measurement outcomes (this follows from Neumark’s theorem).

example, suppose you have a state in a 2-dimensional space, and you know it is either in state $|0\rangle$ or in state $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. These two states are not orthogonal, so there is no measurement that distinguishes them perfectly. However, there is a POVM measurement that never makes a mistake, but sometimes gives another outcome 2, meaning “I don’t know.” That is, you would like to do a measurement with three possible outcome: 0, 1, and 2, such that:

- If the state is $|0\rangle$, then you get correct outcome 0 with probability $1/4$, and outcome 2 with probability $3/4$, but never get incorrect outcome 1.
- If the state is $|+\rangle$, then you get correct outcome 1 with probability $1/4$, and outcome 2 with probability $3/4$, but never get incorrect outcome 0.

You cannot achieve this with a projective measurement on the qubit, but the following 3-outcome POVM does the job:

$$\begin{aligned} E_0 &= \frac{1}{2}|- \rangle \langle -| \text{ (where } |- \rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle), \text{ which is orthogonal to the } |+\rangle \text{ state);} \\ E_1 &= \frac{1}{2}|1\rangle \langle 1| \text{ (note that this is orthogonal to the } |0\rangle \text{ state);} \\ E_2 &= I - E_0 - E_1. \end{aligned}$$

You can check that E_0, E_1, E_2 are psd and add up to identity, so they form a valid POVM. None of the 3 matrices is a projector. The success probability $1/4$ can be improved further, see Exercise 9.

1.2.3 Unitary evolution

Instead of measuring $|\phi\rangle$, we can also apply some operation to it, i.e., change the state to some

$$|\psi\rangle = \beta_0|0\rangle + \beta_1|1\rangle + \cdots + \beta_{N-1}|N-1\rangle.$$

Quantum mechanics only allows *linear* operations to be applied to quantum states. What this means is: if we view a state like $|\phi\rangle$ as an N -dimensional vector $(\alpha_0, \dots, \alpha_{N-1})^T$, then applying an operation that changes $|\phi\rangle$ to $|\psi\rangle$ corresponds to multiplying $|\phi\rangle$ with an $N \times N$ complex-valued matrix U :

$$U \begin{pmatrix} \alpha_0 \\ \vdots \\ \alpha_{N-1} \end{pmatrix} = \begin{pmatrix} \beta_0 \\ \vdots \\ \beta_{N-1} \end{pmatrix}.$$

Note that by linearity we have $|\psi\rangle = U|\phi\rangle = U(\sum_i \alpha_i |i\rangle) = \sum_i \alpha_i U|i\rangle$.

Because measuring $|\psi\rangle$ should also give a probability distribution, we have the constraint $\sum_{j=0}^{N-1} |\beta_j|^2 = 1$ on the new state. This implies that the operation U must preserve the norm of vectors, and hence must be a *unitary* transformation (often just called “a unitary”). A matrix U is *unitary* if its inverse U^{-1} equals its conjugate transpose U^* . This is equivalent to saying that U always maps a vector of norm 1 to a vector of norm 1. Because a unitary transformation always has an inverse, it follows that any (non-measuring) operation on quantum states must be reversible: by applying U^{-1} we can always “undo” the action of U , and nothing is lost in the process. On the other hand, a measurement is clearly non-reversible, because we cannot reconstruct $|\phi\rangle$ from the observed classical state $|j\rangle$.

1.3 Qubits and quantum memory

In classical computation, the unit of information is a *bit*, which can be 0 or 1. In *quantum* computation, this unit is a *quantum* bit (*qubit*), which is a superposition of 0 and 1. Consider a system with 2 basis states, call them $|0\rangle$ and $|1\rangle$. We identify these basis states with the two orthogonal vectors $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $\begin{pmatrix} 0 \\ 1 \end{pmatrix}$, respectively. A single qubit can be in any superposition

$$\alpha_0|0\rangle + \alpha_1|1\rangle, \quad |\alpha_0|^2 + |\alpha_1|^2 = 1.$$

Accordingly, a single qubit “lives” in the vector space $\mathbb{C}^2$.

Similarly we can think of systems of more than 1 qubit, which “live” in the tensor product space of several qubit systems. For instance, a 2-qubit system has 4 basis states: $|0\rangle \otimes |0\rangle$, $|0\rangle \otimes |1\rangle$, $|1\rangle \otimes |0\rangle$, $|1\rangle \otimes |1\rangle$. Here for instance $|1\rangle \otimes |0\rangle$ means that the first qubit is in its basis state $|1\rangle$ and the second qubit is in its basis state $|0\rangle$. We will often abbreviate this to $|1\rangle|0\rangle$, $|1, 0\rangle$, or even $|10\rangle$.

More generally, a register of n qubits has 2^n basis states, each of the form $|b_1\rangle \otimes |b_2\rangle \otimes \dots \otimes |b_n\rangle$, with $b_i \in \{0, 1\}$. We can abbreviate this to $|b_1 b_2 \dots b_n\rangle$. We will often abbreviate $0 \dots 0$ to 0^n . Since bitstrings of length n can be viewed as integers between 0 and $2^n - 1$ (see Appendix B.2), we can also write the basis states as numbers $|0\rangle, |1\rangle, |2\rangle, \dots, |2^n - 1\rangle$. Note that the vector corresponding to n -qubit basis state $|x\rangle$ is the 2^n -dimensional vector that has a 1 at the x -th position and 0s elsewhere (here we view x as an integer in $\{0, \dots, 2^n - 1\}$ and we count the positions in the vector starting from position 0). This implies that two n -qubit basis states $|x\rangle$ and $|y\rangle$ are orthogonal iff $x \neq y$. A different way to see this orthogonality is to use the rules of tensor product (Appendix A.6):

$$\langle x|y\rangle = \langle x_1|y_1\rangle \otimes \dots \otimes \langle x_n|y_n\rangle = \langle x_1|y_1\rangle \dots \langle x_n|y_n\rangle.$$

Since $\langle x_k|y_k\rangle = \delta_{x_k, y_k}$, we see that basis states $|x\rangle$ and $|y\rangle$ will be orthogonal as soon as there is at least one position k at which the bits of x and y differ.

A quantum register of n qubits can be in any superposition⁶

$$\alpha_0|0\rangle + \alpha_1|1\rangle + \dots + \alpha_{2^n-1}|2^n - 1\rangle, \quad \sum_{j=0}^{2^n-1} |\alpha_j|^2 = 1.$$

Measuring this in the computational basis, we obtain the n -bit state $|j\rangle$ with probability $|\alpha_j|^2$.

Measuring just the first qubit of a state would correspond to the projective measurement that has the two projectors $P_0 = |0\rangle\langle 0| \otimes I_{2^{n-1}}$ and $P_1 = |1\rangle\langle 1| \otimes I_{2^{n-1}}$. For example, applying this measurement to the state $\frac{1}{\sqrt{3}}|0\rangle|\phi\rangle + \sqrt{\frac{2}{3}}|1\rangle|\psi\rangle$ gives outcome 0 with probability $1/3$; the state then becomes $|0\rangle|\phi\rangle$. We get outcome 1 with probability $2/3$; the state then becomes $|1\rangle|\psi\rangle$. Similarly, measuring the first n qubits of an $(n + m)$ -qubit state in the computational basis corresponds to the projective measurement that has 2^n projectors $P_j = |j\rangle\langle j| \otimes I_{2^m}$ for $j \in \{0, 1\}^n$.

An important property that deserves to be mentioned is *entanglement*, which refers to quantum correlations between different qubits. For instance, consider a 2-qubit register that is in the state

$$\frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle.$$

⁶Don’t call such a multi-qubit state or register a “qubit” or an “ n -qubit”—the term “qubit” only refers to the state of a 2-dimensional system. You can use “ n -qubit” as an adjective but not as a noun.

Such 2-qubit states are sometimes called *EPR-pairs* in honor of Einstein, Podolsky, and Rosen [106], who examined such states and their seemingly paradoxical properties. Initially neither of the two qubits has a classical value $|0\rangle$ or $|1\rangle$. However, if we measure the first qubit and observe, say, a $|0\rangle$, then the whole state collapses to $|00\rangle$. Thus observing the first qubit immediately fixes also the second, unobserved qubit to a classical value. Since the two qubits that make up the register may be far apart, this example illustrates some of the non-local effects that quantum systems can exhibit. In general, a bipartite state $|\phi\rangle$ is called *entangled* if it cannot be written as a tensor product $|\phi_A\rangle \otimes |\phi_B\rangle$ where $|\phi_A\rangle$ lives in the first space and $|\phi_B\rangle$ lives in the second.⁷

At this point, a comparison with classical probability distributions may be helpful. Suppose we have two probability spaces, A and B , the first with 2^n possible outcomes, the second with 2^m possible outcomes. A probability distribution on the first space can be described by 2^n numbers (nonnegative reals summing to 1; actually there are only $2^n - 1$ degrees of freedom here) and a distribution on the second by 2^m numbers. Accordingly, a *product* distribution on the joint space can be described by $2^n + 2^m$ numbers. However, an arbitrary (non-product) distribution on the joint space takes 2^{n+m} real numbers, since there are 2^{n+m} possible outcomes in total. Analogously, an n -qubit state $|\phi_A\rangle$ can be described by 2^n numbers (complex numbers whose squared moduli sum to 1), an m -qubit state $|\phi_B\rangle$ by 2^m numbers, and their tensor product $|\phi_A\rangle \otimes |\phi_B\rangle$ by $2^n + 2^m$ numbers. However, an arbitrary (possibly entangled) state in the joint space takes 2^{n+m} numbers, since it lives in a 2^{n+m} -dimensional space. We see that the number of parameters required to describe quantum states is the same as the number of parameters needed to describe probability distributions. Also note the analogy between statistical independence⁸ of two random variables A and B and non-entanglement of the product state $|\phi_A\rangle \otimes |\phi_B\rangle$. However, despite the similarities between probabilities and amplitudes, quantum states are much more powerful than distributions, because amplitudes may have negative (or even complex) parts which can lead to *interference* effects. Amplitudes only become probabilities when we square them. The art of quantum computing is to use these special properties for interesting computational purposes.

1.4 Elementary gates

A unitary that acts on a small number of qubits (say, at most 3) is often called a *gate*, in analogy to classical logic gates like AND, OR, and NOT; more about that in the next chapter. The Pauli matrices I, X, Y, Z (Appendix A.9) are examples of 1-qubit gates. For example, the bitflip gate X (a.k.a. NOT-gate) negates the bit in the computational basis, i.e., it swaps $|0\rangle$ and $|1\rangle$. The phaseflip gate Z puts a $-$ in front of $|1\rangle$. Represented as 2×2 unitary matrices, these are

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

⁷We often omit the tensor product symbol for such unentangled states, abbreviating $|\phi_A\rangle \otimes |\phi_B\rangle$ to $|\phi_A\rangle|\phi_B\rangle$ (you shouldn't abbreviate this further to $|\phi_A\phi_B\rangle$ though, unless both $|\phi_A\rangle$ and $|\phi_B\rangle$ are computational basis states). Note that there cannot be ambiguity between tensor product and the usual matrix product in this abbreviation, because both $|\phi_A\rangle$ and $|\phi_B\rangle$ are column vectors and hence their matrix product wouldn't even be well-defined (the dimensions "don't fit").

⁸Two random variables A and B are *independent* if their joint probability distribution can be written as a product of individual distributions for A and for B : $\Pr[A = a \wedge B = b] = \Pr[A = a] \cdot \Pr[B = b]$ for all possible values a, b .

Another important 1-qubit gate is the *phase gate* R_ϕ , which merely rotates the phase of the $|1\rangle$ -state by an angle ϕ :

$$\begin{aligned} R_\phi|0\rangle &= |0\rangle \\ R_\phi|1\rangle &= e^{i\phi}|1\rangle \end{aligned}$$

This corresponds to the unitary matrix

$$R_\phi = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix}.$$

Note that Z is a special case of this: $Z = R_\pi$, because $e^{i\pi} = -1$. The $R_{\pi/4}$ -gate is often just called the T -gate.

Possibly the most important 1-qubit gate is the *Hadamard* transform, specified by:

$$\begin{aligned} H|0\rangle &= \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \\ H|1\rangle &= \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle \end{aligned}$$

As a unitary matrix, this is represented as

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

If we apply H to initial state $|0\rangle$ and then measure, we have equal probability of observing $|0\rangle$ or $|1\rangle$. Similarly, applying H to $|1\rangle$ and observing gives equal probability of $|0\rangle$ or $|1\rangle$. However, if we apply H to the superposition $\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$ then we obtain

$$H\left(\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle\right) = \frac{1}{\sqrt{2}}H|0\rangle + \frac{1}{\sqrt{2}}H|1\rangle = \frac{1}{2}(|0\rangle + |1\rangle) + \frac{1}{2}(|0\rangle - |1\rangle) = |0\rangle.$$

The positive and negative amplitudes for $|1\rangle$ have canceled each other out! This effect is called *interference*, and is analogous to interference patterns between light or sound waves.

An example of a 2-qubit gate is the *controlled-not* gate CNOT. It negates the second bit of its input if the first bit is 1, and does nothing if the first bit is 0:

$$\begin{aligned} \text{CNOT}|0\rangle|b\rangle &= |0\rangle|b\rangle \\ \text{CNOT}|1\rangle|b\rangle &= |1\rangle|1-b\rangle \end{aligned}$$

The first qubit is called the *control* qubit, the second the *target* qubit. In matrix form, this is

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

More generally, if U is some n -qubit unitary matrix, then the controlled- U operation corresponds to the following $2^{n+1} \times 2^{n+1}$ unitary matrix:

$$\begin{pmatrix} I & 0 \\ 0 & U \end{pmatrix},$$

where I is the 2^n -dimensional identity matrix and the two 0s denote $2^n \times 2^n$ all-0 matrices.

1.5 Example: quantum teleportation

In the next chapter we will look in more detail at how we can use and combine such elementary gates, but as an example we will here already explain *teleportation* [48]. Suppose there are two parties, Alice and Bob. Alice has a qubit $\alpha_0|0\rangle + \alpha_1|1\rangle$ that she wants to send to Bob via a *classical* channel. Without further resources this would be impossible, because the amplitudes α_0, α_1 may require an infinite number of bits of precision to write them down exactly. However, suppose Alice also shares an EPR-pair

$$\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

with Bob (say Alice holds the first qubit and Bob the second). Initially, their joint state is

$$(\alpha_0|0\rangle + \alpha_1|1\rangle) \otimes \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle).$$

The first two qubits belong to Alice, the third to Bob. Alice performs a CNOT on her two qubits and then a Hadamard transform on her first qubit. Their joint 3-qubit state can now be written as

$$\begin{aligned} & \frac{1}{2} |00\rangle(\alpha_0|0\rangle + \alpha_1|1\rangle) + \\ & \frac{1}{2} |01\rangle(\alpha_0|1\rangle + \alpha_1|0\rangle) + \\ & \frac{1}{2} |10\rangle(\alpha_0|0\rangle - \alpha_1|1\rangle) + \\ & \frac{1}{2} |11\rangle(\alpha_0|1\rangle - \alpha_1|0\rangle). \end{aligned}$$

$\underbrace{\hspace{10em}}_{\text{Alice}}$
 $\underbrace{\hspace{10em}}_{\text{Bob}}$

Alice then measures her two qubits in the computational basis and sends the result (2 random classical bits ab) to Bob over a classical channel. Bob now knows which transformation he must do on his qubit in order to regain the qubit $\alpha_0|0\rangle + \alpha_1|1\rangle$. First, if $b = 1$ then he applies a bitflip (X -gate) on his qubit; second if $a = 1$ then he applies a phaseflip (Z -gate). For instance, if Alice sent $ab = 11$, then Bob knows that his qubit is $\alpha_0|1\rangle - \alpha_1|0\rangle$. A bitflip followed by a phaseflip will give him Alice's original qubit $\alpha_0|0\rangle + \alpha_1|1\rangle$. In fact, if Alice's qubit had been *entangled* with some other qubits, then teleportation preserves this entanglement: Bob then receives a qubit that is entangled in the same way as Alice's original qubit was.

Note that the qubit on Alice's side has been destroyed: teleporting moves a qubit from Alice to Bob, rather than copying it. In fact, *copying an unknown qubit is impossible* [249], see Exercise 10.

Exercises

1. (a) What is the inner product between the real vectors $(0, 1, 0, 1)$ and $(0, 1, 1, 1)$?
 (b) What is the inner product between the states $|0101\rangle$ and $|0111\rangle$?
2. Compute the result of applying a Hadamard transform to both qubits of $|0\rangle \otimes |1\rangle$ in two ways (the first way using tensor product of vectors, the second using tensor product of matrices), and show that the two results are equal:

$$H|0\rangle \otimes H|1\rangle = (H \otimes H)(|0\rangle \otimes |1\rangle).$$

3. Show that a bitflip operation, preceded and followed by Hadamard transforms, equals a phaseflip operation: $HXH = Z$.

4. Show that surrounding a CNOT gate with Hadamard gates switches the role of the control-bit and target-bit of the CNOT: $(H \otimes H)\text{CNOT}(H \otimes H)$ is the 2-qubit gate where the second bit controls whether the first bit is negated (i.e., flipped).
5. Simplify the following: $(\langle 0| \otimes I)(\alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle)$.
6. Prove that an EPR-pair $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ is an *entangled* state, i.e., that it cannot be written as the tensor product of two separate qubits.
7. Suppose we have the state $\frac{1}{\sqrt{2}}(|0\rangle|\phi\rangle + |1\rangle|\psi\rangle)$, where $|\phi\rangle$ and $|\psi\rangle$ are unknown normalized quantum states with the same number of qubits. Suppose we apply a Hadamard gate to the first qubit and then measure that first qubit in the computational basis. Give the probability of measurement outcome 1, as a function of the states $|\phi\rangle$ and $|\psi\rangle$.
8. Give the 2-outcome projective measurement on a 2-qubit space that measures the parity (i.e., sum modulo 2) of 2-bit basis states. Also give the corresponding observable.
9. (H) Show that the success probability of the POVM at the end of Section 1.2.2 can be increased from $1/4$ to $1/(2 + \sqrt{2})$.
10. (H) Prove the *quantum no-cloning theorem*: there does not exist a 2-qubit unitary U that maps

$$|\phi\rangle|0\rangle \mapsto |\phi\rangle|\phi\rangle$$

for every qubit $|\phi\rangle$.

11. Show that unitaries cannot “delete” information: there is no 1-qubit unitary U that maps $|\phi\rangle \mapsto |0\rangle$ for every 1-qubit state $|\phi\rangle$.
12. Suppose Alice and Bob are not entangled. If Alice sends a qubit to Bob, then this can give Bob at most one bit of information about Alice.⁹ However, if they share an EPR-pair, $|\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, then they can transmit *two* classical bits by sending one qubit over the channel; this is called *superdense coding*. This exercise will show how this works.
 - (a) They start with a shared EPR-pair, $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Alice has classical bits a and b . Suppose she does an X -gate on her half of the EPR-pair if $a = 1$, followed by a Z -gate if $b = 1$ (she does both if $ab = 11$, and neither if $ab = 00$). Write the resulting 2-qubit state for the four different cases that ab could take.
 - (b) Suppose Alice sends her half of the state to Bob, who now has two qubits. Show that Bob can determine both a and b from his state, using Hadamard and CNOT gates, followed by a measurement in the computational basis.
13. Alice and Bob share an EPR-pair, $|\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.
 - (a) Let C be a 2×2 matrix. Show that $\text{Tr}((C \otimes I)|\psi\rangle\langle\psi|) = \frac{1}{2}\text{Tr}(C)$.
 - (b) (H) Alice could apply one of the 4 Pauli matrices (I, X, Y, Z) to her qubit. Use part (a) to show that the 4 resulting 2-qubit states form an orthonormal set.

⁹This is actually a deep statement, a special case of *Holevo's theorem*. More about this may be found in Chapter 15.

- (c) Suppose Alice applies one of the 4 Pauli matrices to her qubit and then sends that qubit to Bob. Give the 4 projectors of a 4-outcome projective measurement that Bob could do on his 2 qubits to find out which Pauli matrix Alice actually applied.
14. Let $\theta \in [0, 2\pi)$, $U_\theta = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$, $|\phi\rangle = U_\theta|0\rangle$ and $|\phi^\perp\rangle = U_\theta|1\rangle$.
- (a) Show that $ZX|\phi^\perp\rangle = |\phi\rangle$.
 - (b) Show that an EPR-pair, $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, can also be written as $\frac{1}{\sqrt{2}}(|\phi\rangle|\phi\rangle + |\phi^\perp\rangle|\phi^\perp\rangle)$.
 - (c) Suppose Alice and Bob start with an EPR-pair. Alice applies U_θ^{-1} to her qubit and then measures it in the computational basis. What pure state does Bob have if her outcome was 0, and what pure state does he have if her outcome was 1?
 - (d) Suppose Alice knows the number θ but Bob does not. Give a protocol that uses one EPR-pair and 1 classical bit of communication where Bob ends up with the qubit $|\phi\rangle$ (in contrast to general teleportation of an unknown qubit, which uses 1 EPR-pair and 2 bits of communication).

Chapter 2

The Circuit Model and the Deutsch-Jozsa Algorithm

2.1 Quantum computation

Below we explain how a quantum computer can apply computational steps to its register of qubits. Two models exist for this: the quantum Turing machine [97, 53] and the quantum circuit model [98, 251]. These models are equivalent, in the sense that they can simulate each other in polynomial time, assuming the circuits are appropriately “uniform.” We only explain the circuit model here, which is more popular among researchers.

2.1.1 Classical circuits

In classical complexity theory, a *Boolean circuit* is a finite directed acyclic graph with AND, OR, and NOT gates. It has n input nodes, which contain the n input bits ($n \geq 0$). The internal nodes are AND, OR, and NOT gates, and there are one or more designated output nodes. The initial input bits are fed into AND, OR, and NOT gates according to the circuit, and eventually the output nodes assume some value. We say that a circuit *computes* some Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ if the output nodes get the right value $f(x)$ for every input $x \in \{0, 1\}^n$.

A *circuit family* is a set $\mathcal{C} = \{C_n\}$ of circuits, one for each input size n . Each circuit has one output bit. Such a family *recognizes* or *decides* a language $L \subseteq \{0, 1\}^* = \cup_{n \geq 0} \{0, 1\}^n$ if, for every n and every input $x \in \{0, 1\}^n$, the circuit C_n outputs 1 if $x \in L$ and outputs 0 otherwise.¹ Such a circuit family is *uniformly polynomial* if there is a deterministic Turing machine that outputs C_n given n as input, using space logarithmic in n .² Note that the size (number of gates) of the circuits C_n can then grow at most polynomially with n . It is known that uniformly polynomial circuit families are equal in power to polynomial-time deterministic Turing machines: a language L can be decided by a uniformly polynomial circuit family iff $L \in \mathbf{P}$ [198, Theorem 11.5], where $\mathbf{P}$ is the class of languages decidable by polynomial-time Turing machines.

Similarly we can consider *randomized* circuits. These receive, in addition to the n input bits, also some random bits (“coin flips”) as input. A randomized circuit computes a function f if it

¹We can think of a language L as a sequence of Boolean functions $f_n : \{0, 1\}^n \rightarrow \{0, 1\}$, where f_n takes value 1 exactly on the n -bit strings that are in L . The circuit C_n then computes the function f_n .

²Logarithmic space implies time that’s at most polynomial in n , because such a machine will have only $\text{poly}(n)$ different internal states, so it either halts after $\text{poly}(n)$ steps or cycles forever.

successfully outputs the right answer $f(x)$ with probability at least $2/3$ for every x (probability taken over the values of the random bits). Randomized circuits are equal in power to randomized Turing machines: a language L can be decided by a uniformly polynomial randomized circuit family iff $L \in \mathbf{BPP}$, where $\mathbf{BPP}$ (“Bounded-error Probabilistic Polynomial time”) is the class of languages that can efficiently be recognized by randomized Turing machines with success probability at least $2/3$. Because we can efficiently reduce the error probability of randomized algorithms (see Appendix B.2), the particular value $2/3$ doesn’t really matter here and may be replaced by any fixed constant in $(1/2, 1)$.

2.1.2 Quantum circuits

A *quantum circuit* (also called quantum network or quantum gate array) generalizes the idea of classical circuit families, replacing the AND, OR, and NOT gates by elementary *quantum gates*. A quantum gate is a unitary transformation on a small (usually 1, 2, or 3) number of qubits. We saw a number of examples already in the previous chapter: the bitflip gate X , the phaseflip gate Z , the Hadamard gate H . The main 2-qubit gate we have seen is the controlled-NOT (CNOT) gate. Adding another control qubit, we get the 3-qubit *Toffoli* gate, also called controlled-controlled-not (CCNOT) gate. This negates the third bit of its input if *both* of the first two bits are 1. The Toffoli gate is important because it is complete for classical reversible computation: any classical computation can be implemented by a circuit of Toffoli gates. This is easy to see: using auxiliary wires with fixed values, Toffoli can implement AND (fix the 3rd ingoing wire to 0) and NOT (fix the 1st and 2nd ingoing wire to 1). It is known that AND and NOT-gates together suffice to implement any classical Boolean circuit, so if we can apply (or simulate) Toffoli gates, we can implement any classical computation in a reversible manner.

Mathematically, such elementary quantum gates can be composed into bigger unitary operations by taking tensor products (if gates are applied *in parallel* to different parts of the register), and ordinary matrix products (if gates are applied *sequentially*). We have already seen a simple example of such a circuit of elementary gates in the previous chapter, namely to implement teleportation.

For example, if we apply the Hadamard gate H to each bit in a register of n zeroes, we obtain

$$\frac{1}{\sqrt{2^n}} \sum_{j \in \{0,1\}^n} |j\rangle,$$

which is a superposition of all n -bit strings. More generally, if we apply $H^{\otimes n}$ to an initial state $|i\rangle$, with $i \in \{0,1\}^n$, we obtain

$$H^{\otimes n}|i\rangle = \frac{1}{\sqrt{2^n}} \sum_{j \in \{0,1\}^n} (-1)^{i \cdot j} |j\rangle, \quad (2.1)$$

where $i \cdot j = \sum_{k=1}^n i_k j_k$ denotes the inner product of the n -bit strings $i, j \in \{0,1\}^n$. For example:

$$H^{\otimes 2}|01\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = \frac{1}{2} \sum_{j \in \{0,1\}^2} (-1)^{01 \cdot j} |j\rangle.$$

Note that Hadamard happens to be its own inverse (it’s unitary and Hermitian, hence $H = H^* = H^{-1}$), so applying it once more on the right-hand side of the above equation would give us back $|01\rangle$. The n -fold Hadamard transform will be very useful for quantum algorithms.

As in the classical case, a quantum circuit is a finite directed acyclic graph of input nodes, gates, and output nodes. There are n nodes that contain the input (as classical bits); in addition we may have some more input nodes that are initially $|0\rangle$ (“workspace”). The internal nodes of the quantum circuit are quantum gates that each operate on at most two or three qubits of the state. The gates in the circuit transform the initial state vector into a final state, which will generally be a superposition. We measure some or all qubits of this final state in the computational basis in order to (probabilistically) obtain a classical output to the algorithm. We can think of the measurement of one qubit in the computational basis as one special type of gate. We may assume without much loss of generality that such measurements only happen at the very end of the circuit (see Exercise 7).

What about the more general kinds of measurements discussed in Section 1.2.2? If we want to apply such a measurement in the circuit model, we will have to implement it using a circuit of elementary gates followed by a measurement in the computational basis. For example, suppose projectors P_0 and P_1 form a 2-outcome projective measurement on an n -qubit space ($P_0 + P_1 = I_{2^n}$). Assume for simplicity that P_0 and P_1 both have rank 2^{n-1} . Then there exists a unitary U that maps an n -qubit state $|\phi\rangle$ to a state whose first qubit is $|0\rangle$ whenever $P_0|\phi\rangle = |\phi\rangle$, and that maps n -qubit $|\psi\rangle$ to a state whose first qubit is $|1\rangle$ whenever $P_1|\psi\rangle = |\psi\rangle$. We can now implement the projective measurement by first applying a circuit that implements U , and then measuring (in the computational basis) the first qubit of the resulting state. The minimal-size circuit to implement U could be very large (i.e., expensive) if the projective measurement is complicated, but that is how it should be.

To draw quantum circuits, the convention is to let time progress from left to right: we start with the initial state on the left. Each qubit is pictured as a horizontal wire, and the circuit prescribes which gates are to be applied to which wires. Single-qubit gates like X and H just act on one wire, while multi-qubit gates such as the CNOT act on multiple wires simultaneously.³ When one qubit “controls” the application of a gate to another qubit, then the controlling wire is drawn with a dot linked vertically to the gate that is applied to the target qubit. This happens for instance with the CNOT, where the applied single-qubit gate is X , usually drawn as ‘ $\oplus$ ’ in a circuit picture (similarly, the Toffoli gate is drawn in a circuit with a dot on the two control wires and an ‘ $\oplus$ ’ on the target wire). Figure 2.1 gives a simple example on two qubits, initially in basis state $|00\rangle$: first apply H to the 1st qubit, then CNOT to both qubits (with the first qubit acting as the control), and then Z to the last qubit. The resulting state is $\frac{1}{\sqrt{2}}(|00\rangle - |11\rangle)$.

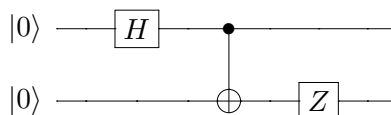


Figure 2.1: Simple circuit for turning $|00\rangle$ into an entangled state

Note that if we have a circuit for unitary U , it is very easy to find a circuit for the inverse U^{-1} with the same complexity: just reverse the order of the gates, and take the inverse of each gate. For example, if $U = U_1U_2U_3$, then $U^{-1} = U_3^{-1}U_2^{-1}U_1^{-1}$.

³Note that the number of wires (qubits) going into a unitary must equal the number of wires going out because a unitary is always invertible (reversible). This differs from the case of classical circuits, where non-reversible gates like AND have more wires going in than out.

In analogy to the classical class **BPP**, we will define **BQP** (“Bounded-error Quantum Polynomial time”) as the class of languages that can efficiently be computed with success probability at least $2/3$ by (a family of) quantum circuits whose size grows at most polynomially with the input length. We will study this quantum complexity class and its relation with various classical complexity classes in more detail in Chapter 13.

2.2 Universality of various sets of elementary gates

Which set of elementary gates should we allow? There are several reasonable choices.

- (1) The set of all 1-qubit operations together with the 2-qubit CNOT gate is universal, meaning that any other unitary transformation can be built from these gates.

Allowing all 1-qubit gates is not very realistic from an implementational point of view, as there are continuously many of them, and we cannot expect experimentalists to implement gates to infinite precision. However, the model is usually restricted, only allowing a small finite set of 1-qubit gates from which all other 1-qubit gates can be efficiently approximated.

- (2) The set consisting of CNOT, Hadamard, and the phase-gate $T = R_{\pi/4}$ is universal in the sense of approximation, meaning that any other unitary can be arbitrarily well approximated using circuits of only these gates. The *Solovay-Kitaev theorem* [196, Appendix 3] says that this approximation is quite efficient: we can approximate any gate on 1 or 2 qubits up to error ε using a number of gates (from our small set) that is only $\text{polylog}(1/\varepsilon)$, i.e., polynomial in the logarithm of $1/\varepsilon$; in particular, simulating arbitrary gates up to exponentially small error costs only a polynomial overhead.

It is often convenient to restrict to real numbers and use an even smaller set of gates:

- (3) The set of Hadamard and Toffoli (CCNOT) is universal for all unitaries with real entries in the sense of approximation, meaning that any unitary with only real entries can be arbitrarily well approximated using circuits of only these gates.

2.3 Quantum parallelism

One uniquely quantum-mechanical effect that we can use for building quantum algorithms is *quantum parallelism*. Suppose we have a classical algorithm that computes some function $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$. Then we can build a quantum circuit U (consisting only of Toffoli gates) that maps $|z\rangle|0\rangle \rightarrow |z\rangle|f(z)\rangle$ for every $z \in \{0, 1\}^n$. Now suppose we apply U to a superposition of *all* inputs z (which is easy to build using n Hadamard transforms):

$$U \left(\frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} |z\rangle|0\rangle \right) = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} |z\rangle|f(z)\rangle.$$

We applied U just once, but the final superposition contains $f(z)$ for *all* 2^n input values z ! However, by itself this is not very useful and does not give more than classical randomization, since observing the final superposition will give just one uniformly random $|z\rangle|f(z)\rangle$ and all other information will be lost. As we will see below, quantum parallelism needs to be combined with the effects of interference and entanglement in order to get something that is better than classical.

2.4 The early algorithms

The two best-known successes of quantum algorithms so far are Shor’s factoring algorithm from 1994 [228] and Grover’s search algorithm from 1996 [125], which will be explained in later chapters. Here we describe some of the earlier quantum algorithms that preceded Shor’s and Grover’s.

Virtually all quantum algorithms work with *queries* in some form or other. We will explain this model here. It may look contrived at first, but eventually will lead smoothly to Shor’s and Grover’s algorithm. We should, however, emphasize that the query complexity model differs from the standard model described above, because the input is now given as a “black-box” (also sometimes called an “oracle”). This means that the exponential quantum-classical separations that we describe below do not by themselves give exponential quantum-classical separations in the standard circuit model (the same applies to Simon’s algorithm in the next chapter).

To explain the query setting, consider an N -bit input $x = (x_0, \dots, x_{N-1}) \in \{0, 1\}^N$. Usually we will have $N = 2^n$, so that we can address bit x_i using an n -bit index i . One can think of the input as an N -bit memory which we can access at any point of our choice (a “Random Access Memory” or RAM). For example, a memory of $N = 1024$ bits can be indexed by addresses $i \in \{0, 1\}^{10}$ of $n = 10$ bits each. A memory access is via a so-called “black-box,” which is equipped to output the bit x_i on input i . As a quantum operation, this is the following unitary mapping on $n + 1$ qubits:

$$O_x : |i, 0\rangle \rightarrow |i, x_i\rangle.$$

The first n qubits of the state are called the *address bits* (or address register), while the $(n + 1)$ st qubit is called the *target bit*.⁴ Since this mapping must be unitary, we also have to specify what happens if the initial value of the target bit is 1. Therefore we actually let O_x be the following unitary transformation:

$$O_x : |i, b\rangle \rightarrow |i, b \oplus x_i\rangle,$$

here $i \in \{0, 1\}^n$, $b \in \{0, 1\}$, and $\oplus$ denotes exclusive-or (addition modulo 2). In matrix representation, this O_x is now a permutation matrix and hence unitary. Note that a quantum computer can apply O_x on a superposition of various i , something a classical computer cannot do. One application of this black-box is called a *query*, and counting the required number of queries to compute this or that function of x is something we will do a lot in the first half of these notes.

Given the ability to make a query of the above type, we can also make a query of the form $|i\rangle \mapsto (-1)^{x_i} |i\rangle$ by setting the target bit to the state $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = H|1\rangle$:

$$O_x(|i\rangle|-\rangle) = |i\rangle \frac{1}{\sqrt{2}}(|x_i\rangle - |1 - x_i\rangle) = (-1)^{x_i} |i\rangle|-\rangle.$$

This $\pm$ -kind of query puts the output variable in the phase of the state: if x_i is 1 then we get a -1 in the phase of basis state $|i\rangle$; if $x_i = 0$ then nothing happens to $|i\rangle$.⁵ This “phase-query” or “phase-oracle” is sometimes more convenient than the standard type of query. We denote the corresponding n -qubit unitary transformation by $O_{x,\pm}$.

⁴It is a common rookie mistake to confuse the N bits of x with the n address bits; don’t fall for this!

⁵This is sometimes called the “phase kick-back trick.” Note that for $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, we have $O_x(|i\rangle|+\rangle) = |i\rangle|+\rangle$ irrespective of what x is. This allows us to control on which part of the state a phase-query is applied: we put the control qubit in state $|-\rangle$ for indices i where we want to apply the phase-query, and in state $|+\rangle$ for the indices where we do not want to apply a phase-query.

2.4.1 Deutsch-Jozsa

Deutsch-Jozsa problem [99]:

For $N = 2^n$, we are given $x \in \{0, 1\}^N$ such that either

- (1) all x_i have the same value (“constant”), or
- (2) $N/2$ of the x_i are 0 and $N/2$ are 1 (“balanced”).

The goal is to find out whether x is constant or balanced.

The algorithm of Deutsch and Jozsa is as follows. We start in the n -qubit zero state $|0^n\rangle$, apply a Hadamard transform to each qubit, apply a query (in its $\pm$ -form), apply another Hadamard to each qubit, and then measure the final state. As a unitary transformation, the algorithm would be $H^{\otimes n} O_{x,\pm} H^{\otimes n}$. We have drawn the corresponding quantum circuit in Figure 2.2 (where time again progresses from left to right). Note that the number of wires going into the query is n , not N ; the basis states on this sequence of wires specify an n -bit address.

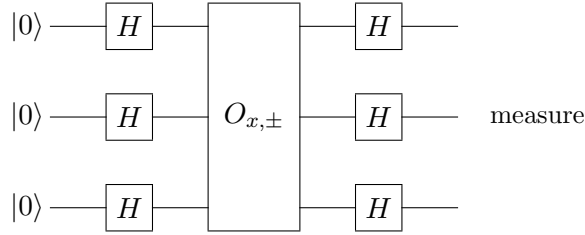


Figure 2.2: The Deutsch-Jozsa algorithm for $n = 3$

Let us follow the state through these operations. Initially we have the state $|0^n\rangle$. By Equation (2.1) on page 14, after the first Hadamard transforms we have obtained the uniform superposition of all i :

$$\frac{1}{\sqrt{2^n}} \sum_{i \in \{0,1\}^n} |i\rangle.$$

The $O_{x,\pm}$ -query turns this into

$$\frac{1}{\sqrt{2^n}} \sum_{i \in \{0,1\}^n} (-1)^{x_i} |i\rangle.$$

Applying the second batch of Hadamards gives (again by Equation (2.1)) the final superposition

$$\frac{1}{2^n} \sum_{i \in \{0,1\}^n} (-1)^{x_i} \sum_{j \in \{0,1\}^n} (-1)^{i \cdot j} |j\rangle,$$

where $i \cdot j = \sum_{k=1}^n i_k j_k$ as before. Since $i \cdot 0^n = 0$ for all $i \in \{0, 1\}^n$, we see that the amplitude of the $|0^n\rangle$ -state in the final superposition is

$$\frac{1}{2^n} \sum_{i \in \{0,1\}^n} (-1)^{x_i} = \begin{cases} 1 & \text{if } x_i = 0 \text{ for all } i, \\ -1 & \text{if } x_i = 1 \text{ for all } i, \\ 0 & \text{if } x \text{ is balanced.} \end{cases}$$

Hence the final observation will yield $|0^n\rangle$ if x is constant and will yield some other state if x is balanced. Accordingly, the Deutsch-Jozsa problem can be solved with certainty using only 1

quantum query and $O(n)$ other operations (the original solution of Deutsch and Jozsa used 2 queries, the 1-query solution is from [91]).

In contrast, it is easy to see that any *classical* deterministic algorithm needs at least $N/2 + 1$ queries: if it has made only $N/2$ queries and seen only 0s, the correct output is still undetermined. However, a classical algorithm can solve this problem efficiently if we allow a small error probability: just query x at two random positions, output “constant” if those bits are the same and “balanced” if they are different. This algorithm outputs the correct answer with probability 1 if x is constant and outputs the correct answer with probability $1/2$ if x is balanced. Thus the quantum-classical separation of this problem only holds if we consider algorithms without error probability.

2.4.2 Bernstein-Vazirani

Bernstein-Vazirani problem [53]:

For $N = 2^n$, we are given $x \in \{0, 1\}^N$ with the property that there is some unknown $a \in \{0, 1\}^n$ such that $x_i = (i \cdot a) \bmod 2$. The goal is to find a .

The Bernstein-Vazirani algorithm is *exactly* the same as the Deutsch-Jozsa algorithm, but now the final observation miraculously yields a . Since $(-1)^{x_i} = (-1)^{(i \cdot a) \bmod 2} = (-1)^{i \cdot a}$, we can write the state obtained after the query as:

$$\frac{1}{\sqrt{2^n}} \sum_{i \in \{0,1\}^n} (-1)^{x_i} |i\rangle = \frac{1}{\sqrt{2^n}} \sum_{i \in \{0,1\}^n} (-1)^{i \cdot a} |i\rangle.$$

Since Hadamard is its own inverse, from Equation (2.1) we can see that applying a Hadamard to each qubit of the above state will turn it into the classical state $|a\rangle$. This solves the Bernstein-Vazirani problem with 1 query and $O(n)$ other operations. In contrast, any classical algorithm (even a randomized one with small error probability) needs to ask n queries for information-theoretic reasons: the final answer consists of n bits and one classical query gives at most 1 bit of information.

Bernstein and Vazirani also defined a recursive version of this problem, which can be solved exactly by a quantum algorithm in $\text{poly}(n)$ steps, but for which every classical randomized algorithm needs $n^{\Omega(\log n)}$ steps.

Exercises

1. Is the controlled-NOT operation C Hermitian? Determine C^{-1} .
2. Construct a CNOT from two Hadamard gates and one controlled- Z (the controlled- Z gate maps $|11\rangle \mapsto -|11\rangle$ and acts like the identity on the other basis states).
3. A SWAP-gate interchanges two qubits: it maps basis state $|a, b\rangle$ to $|b, a\rangle$. Implement a SWAP-gate using a few CNOTs (when using a CNOT, you’re allowed to use either of the 2 bits as the control, but be explicit about this).
4. Show that every 1-qubit unitary with real entries can be written as a rotation matrix, possibly preceded and followed by Z -gates. In other words, show that for every 2×2 real unitary U , there exist signs $s_1, s_2, s_3 \in \{1, -1\}$ and angle $\theta \in [0, 2\pi)$ such that

$$U = s_1 \begin{pmatrix} 1 & 0 \\ 0 & s_2 \end{pmatrix} \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & s_3 \end{pmatrix}.$$

5. Let U be a 1-qubit unitary that we would like to implement in a controlled way, i.e., we want to implement a map $|c\rangle|b\rangle \mapsto |c\rangle U^c|b\rangle$ for all $c, b \in \{0, 1\}$ (here $U^0 = I$ and $U^1 = U$). One can show there exist 1-qubit unitaries A , B , and C , such that $ABC = I$ and $AXBXC = U$ (X is the NOT-gate); you may assume this without proof. Give a circuit that acts on two qubits and implements a controlled- U gate, using CNOTs and (uncontrolled) A , B , and C gates.
6. (H) Let C be a given quantum circuit consisting of T many gates, which may be CNOTs and single-qubit gates. Show that we can implement C in a controlled way using $O(T)$ Toffoli gates, CNOTs and single-qubit gates, and no auxiliary qubits other than the controlling qubit.
7. (H) It is possible to avoid doing any intermediate measurements in a quantum circuit, using one auxiliary qubit for each 1-qubit measurement that needs to be delayed until the end of the computation. Show how.
8. (a) Give a circuit that maps $|0^n, b\rangle \mapsto |0^n, 1 - b\rangle$ for $b \in \{0, 1\}$, and that maps $|i, b\rangle \mapsto |i, b\rangle$ whenever $i \in \{0, 1\}^n \setminus \{0^n\}$. You are allowed to use every type of elementary gate mentioned in the lecture notes (incl. Toffoli gates), as well as auxiliary qubits that are initially $|0\rangle$ and that should be put back to $|0\rangle$ at the end of the computation.
 (b) Suppose we can make queries of the type $|i, b\rangle \mapsto |i, b \oplus x_i\rangle$ to input $x \in \{0, 1\}^N$, with $N = 2^n$. Let x' be the input x with its first bit flipped (e.g., if $x = 0110$ then $x' = 1110$). Give a circuit that implements a query to x' . Your circuit may use one query to x .
 (c) Give a circuit that implements a query to an input x'' that is obtained from x (analogously to (b)) by setting its first bit to 0. Your circuit may use one query to x .
9. In Section 2.4 we showed that a standard query, which maps $|i, b\rangle \mapsto |i, b \oplus x_i\rangle$ (where $i \in \{0, \dots, N - 1\}$ and $b \in \{0, 1\}$), can be used to implement a phase-query to x , i.e., one of the type $|i\rangle \mapsto (-1)^{x_i}|i\rangle$ (this is an *uncontrolled* phase-query).
 (a) Show that a standard query can be implemented using one *controlled* phase-query to x (which maps $|c, i\rangle \mapsto (-1)^{cx_i}|c, i\rangle$, so the phase is added only if the control bit is $c = 1$), and possibly some auxiliary qubits and other gates.
 (b) Can you also implement a standard query using one or more *uncontrolled* phase-queries to x , and possibly some auxiliary qubits and other gates? If yes, show how. If no, prove why not.
10. Suppose we have a 2-bit input $x = x_0x_1$ and a phase-query that maps

$$O_{x,\pm} : |b\rangle \mapsto (-1)^{x_b}|b\rangle \text{ for } b \in \{0, 1\}.$$

- (a) Suppose we run the 1-qubit circuit $HO_{x,\pm}H$ on initial state $|0\rangle$ and then measure (in the computational basis). What is the probability distribution on the output bit, as a function of x ?
- (b) Now suppose the query leaves some workspace in a second qubit, which is initially $|0\rangle$:

$$O'_{x,\pm} : |b, 0\rangle \mapsto (-1)^{x_b}|b, b\rangle \text{ for } b \in \{0, 1\}.$$

Suppose we just ignore the workspace and run the algorithm of (a) on the first qubit with $O'_{x,\pm}$ instead of $O_{x,\pm}$ (and $H \otimes I$ instead of H , and initial state $|00\rangle$). What is now

the probability distribution on the output bit (i.e., if we measure the first of the two bits)?

Comment: This exercise illustrates why it's important to “clean up” (i.e., set back to $|0\rangle$) workspace qubits of some subroutine before running it on a superposition of inputs: the unintended entanglement between the address and workspace registers can thwart the intended interference effects.

11. Give a randomized classical algorithm (i.e., one that can flip coins during its operation) that makes only two queries to x , and decides the Deutsch-Jozsa problem with success probability at least $2/3$ on every possible input. A high-level description is enough, no need to write out the classical circuit.
12. Suppose our N -bit input x satisfies the following promise:
either (1) the first $N/2$ bits of x are all 0 and the second $N/2$ bits are all 1; or (2) the number of 1s in the first half of x plus the number of 0s in the second half, equals $N/2$. Modify the Deutsch-Jozsa algorithm to efficiently distinguish these two cases (1) and (2).
13. (H) Let $N = 2^n$. A *parity query* to input $x \in \{0, 1\}^N$ corresponds to the $(N + 1)$ -qubit unitary map $Q_x : |y, b\rangle \mapsto |y, b \oplus (x \cdot y)\rangle$, where $x \cdot y = \sum_{i=0}^{N-1} x_i y_i \bmod 2$. For a fixed function $f : \{0, 1\}^N \rightarrow \{0, 1\}$, give a quantum algorithm that computes $f(x)$ using only one such query (i.e., one application of Q_x), and as many elementary gates as you want. You do not need to give the circuit in full detail, an informal description of the algorithm is good enough.

Chapter 3

Simon's Algorithm

The Deutsch-Jozsa problem showed an exponential quantum improvement over the best *deterministic* classical algorithms; the Bernstein-Vazirani problem showed a polynomial improvement over the best *randomized* classical algorithms that have error probability $\leq 1/3$. In this chapter we will combine these two features: we will see a computational problem due to Simon [230] where quantum computers are provably exponentially more efficient (in terms of number of queries) than bounded-error randomized algorithms. Simon's problem may look rather contrived at first sight, but his quantum algorithm to solve it was the main inspiration for Shor's important quantum algorithm for the very natural problem of integer factorization, which we will see in Chapter 5. More recently, Simon's algorithm itself was also used to break some classical cryptographic systems [147, 217].

3.1 The problem

Let $N = 2^n$, and identify the set $\{0, \dots, N-1\}$ with $\{0, 1\}^n$. Let $j \oplus s$ be the n -bit string obtained by bitwise adding the n -bit strings j and s mod 2, so for example $00110 \oplus 10101 = 10011$.

Simon's problem [230]:

For $N = 2^n$, we are given $x = (x_0, \dots, x_{N-1})$, with $x_i \in \{0, 1\}^n$, with the property that there is some unknown nonzero $s \in \{0, 1\}^n$ such that $x_i = x_j$ iff $(i = j \text{ or } i = j \oplus s)$. The goal is to find s .

Note that x , viewed as a function from $\{0, \dots, N-1\}$ to $\{0, \dots, N-1\}$, is a 2-to-1 function, where the 2-to-1-ness is determined by the unknown *mask* s . The queries to the input here are slightly different from before: the input $x = (x_0, \dots, x_{N-1})$ now has variables x_i that themselves are n -bit strings, and one query gives such a string completely ($|i, 0^n\rangle \mapsto |i, x_i\rangle$). However, we can also view this problem as having $n2^n$ binary variables that we can query individually. Since we can simulate one x_i -query using only n binary queries (just query all n bits of x_i), this alternative view will not affect the number of queries very much.

3.2 The quantum algorithm

Simon's algorithm starts out very similar to Deutsch-Jozsa: start in a state of $2n$ zero qubits $|0^n\rangle|0^n\rangle$ and apply Hadamard transforms to the first n qubits to put them in a uniform superposition, giving

$$\frac{1}{\sqrt{2^n}} \sum_{i \in \{0,1\}^n} |i\rangle|0^n\rangle.$$

At this point, the second n -qubit register still holds only zeroes. A query turns this into

$$\frac{1}{\sqrt{2^n}} \sum_{i \in \{0,1\}^n} |i\rangle |x_i\rangle.$$

Now the algorithm measures the second n -qubit register in the computational basis (see Exercise 1); this measurement is actually not necessary, but it facilitates analysis. The measurement outcome will be some value x_i and the first register will collapse to the superposition of the two indices having that x_i -value:

$$\frac{1}{\sqrt{2}}(|i\rangle + |i \oplus s\rangle)|x_i\rangle.$$

We will now ignore the second register and apply Hadamard transforms to the first n qubits. Using Equation (2.1) and the fact that $(i \oplus s) \cdot j = (i \cdot j) \oplus (s \cdot j)$, we can write the resulting state as

$$\begin{aligned} \frac{1}{\sqrt{2^{n+1}}} \left(\sum_{j \in \{0,1\}^n} (-1)^{i \cdot j} |j\rangle + \sum_{j \in \{0,1\}^n} (-1)^{(i \oplus s) \cdot j} |j\rangle \right) = \\ \frac{1}{\sqrt{2^{n+1}}} \left(\sum_{j \in \{0,1\}^n} (-1)^{i \cdot j} (1 + (-1)^{s \cdot j}) |j\rangle \right). \end{aligned}$$

Note that $|j\rangle$ has nonzero amplitude iff $s \cdot j = 0 \pmod{2}$. Measuring the state gives a uniformly random element from the set $\{j \mid s \cdot j = 0 \pmod{2}\}$. Accordingly, we get a linear equation that gives information about s . We repeat this algorithm until we have obtained $n - 1$ independent linear equations involving s . The solutions to these equations will be 0^n and the correct s , which we can compute efficiently by a classical algorithm (Gaussian elimination modulo 2). This can be done by means of a classical circuit of size roughly $O(n^3)$.

Note that if the j 's you have generated at some point span a space of size 2^k , for some $k < n - 1$, then the probability that your next run of the algorithm produces a j that is linearly independent of the earlier ones, is $(2^{n-1} - 2^k)/2^{n-1} \geq 1/2$. Hence an expected number of $O(n)$ runs of the algorithm suffices to find $n - 1$ linearly independent j 's. Simon's algorithm thus finds s using an expected number of $O(n)$ x_i -queries and polynomially many other operations.

3.3 Classical algorithms for Simon's problem

3.3.1 Upper bound

Let us first sketch a classical randomized algorithm that solves Simon's problem using $O(\sqrt{2^n})$ queries. The algorithm is based on the so-called "birthday paradox," which is the phenomenon that in a group of only 23 people, there is already a large probability that two people share the same birthday, despite the fact that the number of possible birthdays (365) is much larger than the number of people (23). The intuitive explanation is that the number of *pairs* of people is actually quadratic in the number of people, and each pair has a $1/365$ probability to have the same birthday (assuming birthdays are distributed uniformly random among people). Of course, different pairs may overlap and hence are not independent, but the idea still works.

Our algorithm will make T randomly chosen distinct queries $i_1, \dots, i_T$, for some T to be determined later. If there is a *collision* among those queries (i.e., $x_{i_k} = x_{i_\ell}$ for some $k \neq \ell$, so i_k

and i_ℓ happen to have the same “birthday”), then we are done, because then we know $i_k = i_\ell \oplus s$, equivalently $s = i_k \oplus i_\ell$. There won’t be any collisions if $s = 0^n$, but how large should T be such that we are likely to see a collision in case $s \neq 0^n$? There are $\binom{T}{2} = \frac{1}{2}T(T-1) \approx T^2/2$ pairs in our sequence that could be a collision, and since the indices are chosen randomly, the probability for a fixed pair to form a collision is $1/(2^n - 1)$. Hence by linearity of expectation, the *expected* number of collisions in our sequence will be roughly $T^2/2^{n+1}$. If we choose $T = \sqrt{2^{n+1}}$, we expect to have roughly 1 collision in our sequence, which is good enough to find s . Of course, an *expected value* of 1 collision does not mean that we will have at least one collision *with high probability*, but a slightly more involved calculation shows the latter statement as well.

3.3.2 Lower bound

Simon [230] proved that any classical randomized algorithm that finds s with high probability needs to make $\Omega(\sqrt{2^n})$ queries, so the above classical algorithm is essentially optimal. This was the first proven exponential separation between quantum algorithms and classical bounded-error algorithms (let us stress again that this does not prove an exponential separation in the usual circuit model, because we are counting queries rather than ordinary operations here). Simon’s algorithm inspired Shor to his factoring algorithm, which we describe in Chapter 5.

We will prove the classical lower bound for a decision version of Simon’s problem:

Given: input $x = (x_0, \dots, x_{N-1})$, where $N = 2^n$ and $x_i \in \{0, 1\}^n$
Promise: $\exists s \in \{0, 1\}^n$ such that: $x_i = x_j$ iff $(i = j \text{ or } i = j \oplus s)$
Task: decide whether $s = 0^n$

Consider the input distribution μ that is defined as follows. With probability $1/2$, x is a uniformly random permutation of $\{0, 1\}^n$; this corresponds to the case $s = 0^n$. With probability $1/2$, we pick a nonzero string s at random, and for each pair $(i, i \oplus s)$, we pick a unique value for $x_i = x_{i \oplus s}$ at random. If there exists a *randomized* T -query algorithm that achieves success probability $\geq 2/3$ under this input distribution μ , then there also is *deterministic* T -query algorithm that achieves success probability $\geq 2/3$ under μ (because the behavior of the randomized algorithm is an average over a number of deterministic algorithms). Now consider a deterministic algorithm with error $\leq 1/3$ under μ , that makes T queries to x . We want to show that $T = \Omega(\sqrt{2^n})$.

First consider the case $s = 0^n$. We can assume the algorithm never queries the same point twice. Then the T outcomes of the queries are T distinct n -bit strings, and each sequence of T strings is equally likely.

Now consider the case $s \neq 0^n$. Suppose the algorithm queries the indices $i_1, \dots, i_T$ (this sequence depends on x) and gets outputs $x_{i_1}, \dots, x_{i_T}$. Call a sequence of queries $i_1, \dots, i_T$ *good* if it shows a collision (i.e., $x_{i_k} = x_{i_\ell}$ for some $k \neq \ell$), and *bad* otherwise. If the sequence of queries of the algorithm is good, then we can find s , since $i_k \oplus i_\ell = s$. On the other hand, if the sequence is bad, then each sequence of T distinct outcomes is equally likely—just as in the $s = 0^n$ case! We will now show that the probability of the bad case is very close to 1 for small T .

If $i_1, \dots, i_{k-1}$ is bad, then we have excluded at most $\binom{k-1}{2}$ possible values of s (namely all values $i_j \oplus i_{j'}$ for all distinct $j, j' \in [k-1]$), and all other values of s are equally likely. The probability that the next query i_k makes the sequence good, is the probability that $x_{i_k} = x_{i_j}$ for some $j < k$, equivalently, that the set $S_k = \{i_k \oplus i_j \mid j < k\}$ happens to contain the string s . But S_k has only $k-1$ members, while there are at least $2^n - 1 - \binom{k-1}{2}$ equally likely remaining possibilities for s .

This means that the probability that the sequence is still bad after query i_k is made, is very close to 1. In formulas:

$$\begin{aligned}
\Pr[i_1, \dots, i_T \text{ is bad}] &= \prod_{k=2}^T \Pr[i_1, \dots, i_k \text{ is bad} \mid i_1, \dots, i_{k-1} \text{ is bad}] \\
&\geq \prod_{k=2}^T \left(1 - \frac{k-1}{2^n - 1 - \binom{k-1}{2}} \right) \\
&\geq 1 - \sum_{k=2}^T \frac{k-1}{2^n - 1 - \binom{k-1}{2}}.
\end{aligned}$$

Here we used the fact that $(1-a)(1-b) \geq 1 - (a+b)$ if $a, b \geq 0$. Note that $\sum_{k=2}^T (k-1) = T(T-1)/2 \approx T^2/2$, and $2^n - 1 - \binom{k-1}{2} \approx 2^n$ as long as $k \ll \sqrt{2^n}$. Hence we can approximate the last formula by $1 - T^2/2^{n+1}$. Accordingly, if $T \ll \sqrt{2^n}$ then with probability nearly 1 (probability taken over the input distribution μ) the algorithm's sequence of queries is bad. If it gets a bad sequence, it cannot "see" the difference between the $s = 0^n$ case and the $s \neq 0^n$ case, since both cases result in a uniformly random sequence of T distinct n -bit strings as answers to the T queries. This shows that T has to be $\Omega(\sqrt{2^n})$ in order to enable the algorithm to get a good sequence of queries with high probability.

Exercises

1. Give the projectors of the 2^n -outcome projective measurement that is applied to the whole $2n$ -qubit state in Simon's algorithm right after the query.
2. Analyze the different steps of Simon's algorithm if $s = 0^n$ (so all x_i -values are distinct), and show that the final output j is uniformly distributed over $\{0, 1\}^n$.
3. Suppose we run Simon's algorithm on the following input x (with $N = 8$ and hence $n = 3$):

$$\begin{aligned}
x_{000} &= x_{111} = 000 \\
x_{001} &= x_{110} = 001 \\
x_{010} &= x_{101} = 010 \\
x_{011} &= x_{100} = 011
\end{aligned}$$

Note that x is 2-to-1 and $x_i = x_{i \oplus 111}$ for all $i \in \{0, 1\}^3$, so $s = 111$.

- (a) Give the starting state of Simon's algorithm.
- (b) Give the state after the first Hadamard transforms on the first 3 qubits.
- (c) Give the state after applying the oracle query.
- (d) Give the state after measuring the second register (suppose the measurement gave $|001\rangle$).
- (e) Using $H^{\otimes n}|i\rangle = \frac{1}{\sqrt{2^n}} \sum_{j \in \{0,1\}^n} (-1)^{i \cdot j} |j\rangle$, give the state after the final Hadamards.
- (f) Why does a measurement of the first 3 qubits of the final state give information about s ?
- (g) Suppose the first run of the algorithm gives $j = 011$ and a second run gives $j = 101$. Show that, assuming $s \neq 000$, those two runs of the algorithm already determine s .

4. Consider the following generalization of Simon's problem: the input is $x = (x_0, \dots, x_{N-1})$, with $N = 2^n$ and $x_i \in \{0, 1\}^n$, with the property that there is some unknown *subspace* $V \subseteq \{0, 1\}^n$ (where $\{0, 1\}^n$ is the vector space of n -bit strings with entrywise addition modulo 2) such that $x_i = x_j$ iff there exists a $v \in V$ such that $i = j \oplus v$. The usual definition of Simon's problem corresponds to the case of 1-dimensional subspace $V = \{0, s\}$.

Show that one run of Simon's algorithm now produces a $j \in \{0, 1\}^n$ that is orthogonal to the whole subspace (i.e., $j \cdot v = 0 \pmod 2$ for every $v \in V$).

5. Let $f : \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}$ be a 2-to-1 function, meaning that every $y \in \{0, 1\}^{n-1}$ has exactly two distinct pre-images $x, x' \in \{0, 1\}^n$. Suppose there is an efficient quantum circuit (i.e., with number of elementary gates that's polynomial in n) to compute f , but no efficient circuit that can produce from given $x \in \{0, 1\}^n$ an $x' \neq x$ such that $f(x) = f(x')$.

Show how a quantum computer can efficiently generate a uniformly random $y \in \{0, 1\}^{n-1}$ and an associated n -qubit state $|\phi_y\rangle$ such that:

(1) when asked, from $|\phi_y\rangle$ you can efficiently generate an x such that $f(x) = y$;
and

(2) when asked, you can efficiently sample uniformly from the set $\{a \in \{0, 1\}^n : a \cdot (x \oplus x') = 0 \pmod 2\}$, where x and x' are the two pre-images of y .

Comment: You're not supposed to do *both* tasks (1) and (2) one after another, only either one of the two (whichever you're asked to do). This problem may look arbitrary but was recently used to design an efficient protocol through which a *classical* computer can efficiently verify that a quantum computer works as intended [184].

6. (a) Suppose x is an N -bit string. What happens if we apply a Hadamard transform to each qubit of the N -qubit state $\frac{1}{\sqrt{2^N}} \sum_{y \in \{0, 1\}^N} (-1)^{x \cdot y} |y\rangle$?
- (b) Give a quantum algorithm that uses T queries to N -bit string x , and that maps $|y\rangle \mapsto (-1)^{x \cdot y} |y\rangle$ for every $y \in \{0, 1\}^N$ that contains at most T 1s (i.e., for every y of Hamming weight $\leq T$). You can argue on a high level, no need to write out circuits in detail.
- (c) (H) Give a quantum algorithm that with high probability outputs x , using at most $N/2 + 2\sqrt{N}$ queries to x .
- (d) Argue that a classical algorithm needs at least N queries in order to have success probability $> 1/2$ of outputting the correct x .

Chapter 4

The Fourier Transform

4.1 The classical discrete Fourier transform

The Fourier transform occurs in many different versions throughout classical computing, in areas ranging from signal-processing to data compression to complexity theory.

For our purposes, the Fourier transform is going to be an $N \times N$ unitary matrix, all of whose entries have the same magnitude. For $N = 2$, it's just our familiar Hadamard transform:

$$F_2 = H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

Doing something similar in 3 dimensions is impossible with real numbers: we can't give three orthogonal vectors in $\{+1, -1\}^3$. However, using *complex* numbers allows us to define the Fourier transform for any N . Let $\omega_N = e^{2\pi i/N}$ be an N -th root of unity ("root of unity" means that $\omega_N^k = 1$ for some integer k , in this case $k = N$). The rows of the matrix will be indexed by $j \in \{0, \dots, N-1\}$ and the columns by $k \in \{0, \dots, N-1\}$. Define the (j, k) -entry of the matrix F_N by $\frac{1}{\sqrt{N}}\omega_N^{jk}$, where the exponent jk is the usual product of two integers:

$$F_N = \frac{1}{\sqrt{N}} \begin{pmatrix} & \vdots & \\ \cdots & \omega_N^{jk} & \cdots \\ & \vdots & \end{pmatrix}$$

This F_N is a unitary matrix, because each column has norm 1 and any two distinct columns (say those indexed by k and k') are orthogonal:

$$\sum_{j=0}^{N-1} \frac{1}{\sqrt{N}} (\omega_N^{jk})^* \frac{1}{\sqrt{N}} \omega_N^{jk'} = \frac{1}{N} \sum_{j=0}^{N-1} \omega_N^{j(k'-k)} = \begin{cases} 1 & \text{if } k = k' \\ 0 & \text{otherwise} \end{cases}$$

using the formula for geometric sums from Appendix B.1.

Since F_N is unitary and symmetric, the inverse $F_N^{-1} = F_N^*$ only differs from F_N by having minus signs in the exponent of the entries. For a vector $v \in \mathbb{R}^N$, the vector $\hat{v} = F_N v$ is called the Fourier transform of v .¹ Its entries are given by $\hat{v}_j = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega_N^{jk} v_k$.

¹The literature on Fourier analysis usually talks about the Fourier transform of a *function* rather than of a vector,

4.2 The Fast Fourier Transform

The naive way of computing the Fourier transform $\widehat{v} = F_N v$ of $v \in \mathbb{R}^N$ just does the matrix-vector multiplication to compute all the entries of $\widehat{v}$. This would take $O(N)$ steps (additions and multiplications) per entry, and $O(N^2)$ steps to compute the whole vector $\widehat{v}$. However, there is a more efficient way of computing $\widehat{v}$. This algorithm is called the *Fast Fourier Transform* (FFT, due to Cooley and Tukey in 1965 [93]), and takes only $O(N \log N)$ steps. This difference between the quadratic N^2 steps and the near-linear $N \log N$ is tremendously important in practice when N is large, and is the main reason that Fourier transforms are so widely used.

We will assume $N = 2^n$, which is usually fine because we can add zeroes to our vector to make its dimension a power of 2 (but similar FFTs can be given also directly for most N that aren't a power of 2). The key to the FFT is to rewrite the entries of $\widehat{v}$ as follows:

$$\begin{aligned}\widehat{v}_j &= \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega_N^{jk} v_k \\ &= \frac{1}{\sqrt{N}} \left(\sum_{\text{even } k} \omega_N^{jk} v_k + \omega_N^j \sum_{\text{odd } k} \omega_N^{j(k-1)} v_k \right) \\ &= \frac{1}{\sqrt{2}} \left(\frac{1}{\sqrt{N/2}} \sum_{\text{even } k} \omega_{N/2}^{jk/2} v_k + \omega_N^j \frac{1}{\sqrt{N/2}} \sum_{\text{odd } k} \omega_{N/2}^{j(k-1)/2} v_k \right)\end{aligned}$$

Note that we've rewritten the entries of the N -dimensional Fourier transform $\widehat{v}$ in terms of two $N/2$ -dimensional Fourier transforms, one of the even-numbered entries of v , and one of the odd-numbered entries of v .

This suggests a recursive procedure for computing $\widehat{v}$: first separately compute the Fourier transform $\widehat{v_{\text{even}}}$ of the $N/2$ -dimensional vector of even-numbered entries of v and the Fourier transform $\widehat{v_{\text{odd}}}$ of the $N/2$ -dimensional vector of odd-numbered entries of v , and then compute the N entries

$$\widehat{v}_j = \frac{1}{\sqrt{2}} (\widehat{v_{\text{even}}}_j + \omega_N^j \widehat{v_{\text{odd}}}_j).$$

Strictly speaking this is not well-defined, because $\widehat{v_{\text{even}}}$ and $\widehat{v_{\text{odd}}}$ are just $N/2$ -dimensional vectors. However, if we take two copies of these $N/2$ -dimensional vectors to get an N -dimensional vector, defining $\widehat{v_{\text{even}}}_{j+N/2} = \widehat{v_{\text{even}}}_j$ (and similarly for $\widehat{v_{\text{odd}}}$), then it all works out.

The time $T(N)$ it takes to implement F_N this way can be written recursively as $T(N) = 2T(N/2) + O(N)$, because we need to compute two $N/2$ -dimensional Fourier transforms and do $O(N)$ additional operations to compute $\widehat{v}$. This recursion works out to time $T(N) = O(N \log N)$, as promised. Similarly, we have an equally efficient algorithm for the *inverse* Fourier transform $F_N^{-1} = F_N^*$, whose entries are $\frac{1}{\sqrt{N}} \omega_N^{-jk}$.

but on finite domains that's just a notational variant of what we do here: a vector $v \in \mathbb{R}^N$ can also be viewed as a function $v : \{0, \dots, N-1\} \rightarrow \mathbb{R}$ defined by $v(i) = v_i$. Also, in the classical literature people sometimes use the term "Fourier transform" for what we call the inverse Fourier transform.

4.3 Application: multiplying two polynomials

Suppose we are given two real-valued polynomials p and q , each of degree at most d :

$$p(x) = \sum_{j=0}^d a_j x^j \text{ and } q(x) = \sum_{k=0}^d b_k x^k$$

We would like to compute the product of these two polynomials, which is

$$(p \cdot q)(x) = \left(\sum_{j=0}^d a_j x^j \right) \left(\sum_{k=0}^d b_k x^k \right) = \sum_{\ell=0}^{2d} \underbrace{\left(\sum_{j=0}^{2d} a_j b_{\ell-j} \right)}_{c_\ell} x^\ell,$$

where implicitly we set $a_j = b_j = 0$ for $j > d$ and $b_{\ell-j} = 0$ if $j > \ell$. Clearly, each coefficient c_ℓ by itself takes $O(d)$ steps (additions and multiplications) to compute, which suggests an algorithm for computing the coefficients of $p \cdot q$ that takes $O(d^2)$ steps. However, using the fast Fourier transform we can do this in $O(d \log d)$ steps, as follows.

The *convolution* of two vectors $a, b \in \mathbb{R}^N$ is a vector $a * b \in \mathbb{R}^N$ whose ℓ -th entry is defined by $(a * b)_\ell = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} a_j b_{\ell-j \bmod N}$. Let us set $N = 2d + 1$ (the number of nonzero coefficients of $p \cdot q$) and make the above $(d + 1)$ -dimensional vectors of coefficients a and b N -dimensional by adding d zeroes. Then the coefficients of the polynomial $p \cdot q$ are proportional to the entries of the convolution: $c_\ell = \sqrt{N}(a * b)_\ell$. It is easy to show that the Fourier coefficients of the convolution of a and b are the products of the Fourier coefficients of a and b : for every $\ell \in \{0, \dots, N - 1\}$ we have $(\widehat{a * b})_\ell = \widehat{a}_\ell \widehat{b}_\ell$. This immediately suggests an algorithm for computing the vector of coefficients c_ℓ : apply the FFT to a and b to get $\widehat{a}$ and $\widehat{b}$, multiply those two vectors entrywise to get $\widehat{a * b}$, apply the inverse FFT to get $a * b$, and finally multiply $a * b$ with $\sqrt{N}$ to get the vector c of the coefficients of $p \cdot q$. Since the FFTs and their inverse take $O(N \log N)$ steps, and pointwise multiplication of two N -dimensional vectors takes $O(N)$ steps, this algorithm takes $O(N \log N) = O(d \log d)$ steps.

Note that if two numbers $a_d \cdots a_1 a_0$ and $b_d \cdots b_1 b_0$ are given in decimal notation, then we can interpret their digits as coefficients of single-variate degree- d polynomials p and q , respectively: $p(x) = \sum_{j=0}^d a_j x^j$ and $q(x) = \sum_{k=0}^d b_k x^k$. The two numbers will now be $p(10)$ and $q(10)$. Their product is the evaluation of the product-polynomial $p \cdot q$ at the point $x = 10$. This suggests that we can use the above procedure (for fast multiplication of polynomials) to multiply two numbers in $O(d \log d)$ steps, which would be a lot faster than the standard $O(d^2)$ algorithm for multiplication that one learns in primary school. However, in this case we have to be careful since the steps of the above algorithm are themselves multiplications between numbers, which we cannot count at unit cost anymore if our goal is to implement a multiplication between numbers! Still, it turns out that implementing this idea carefully allows one to multiply two d -digit numbers in $O(d \log d \log \log d)$ elementary operations. This is known as the Schönhage-Strassen algorithm [218] (slightly improved further by Fürer [116] and Harvey and van der Hoeven [135]), and is one of the ingredients of Shor's algorithm in the next chapter. We'll skip the details.

4.4 The quantum Fourier transform

Since F_N is an $N \times N$ unitary matrix, we can interpret it as a quantum operation, mapping an N -dimensional vector of amplitudes to another N -dimensional vector of amplitudes. This is called the

quantum Fourier transform (QFT). In case $N = 2^n$ (which is the only case we will care about), this will be an n -qubit unitary. Notice carefully that this quantum operation does something different from the classical Fourier transform: in the classical case we are given a vector v , written on a piece of paper so to say, and we compute the vector $\hat{v} = F_N v$, and also write the result on a piece of paper. In the quantum case, we are working on *quantum states*; these are vectors of amplitudes, but we don't have those written down anywhere—they only exist as the amplitudes in a superposition. We will see below that the QFT can be implemented by a quantum circuit using $O(n^2)$ elementary gates. This is exponentially faster than even the FFT (which takes $O(N \log N) = O(2^n n)$ steps), but it achieves something different: computing the QFT won't give us the entries of the Fourier transform written down on a piece of paper, but only as the amplitudes of the resulting state.

4.5 An efficient quantum circuit

Here we will describe the efficient circuit for the n -qubit QFT. The elementary gates we will allow ourselves are Hadamards and controlled- R_s gates, where

$$R_s = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^s} \end{pmatrix}.$$

Note that $R_1 = Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$, $R_2 = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$. For large s , $e^{2\pi i/2^s}$ is close to 1 and hence the R_s -gate is close to the identity-gate I . We could implement R_s -gates using Hadamards and controlled- $R_{1/2/3}$ gates, but for simplicity we will just treat each R_s as an elementary gate.

Since the QFT is linear, it suffices if our circuit implements it correctly on all n -qubit basis states $|k\rangle$, i.e., it should map

$$|k\rangle \mapsto F_N |k\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \omega_N^{jk} |j\rangle.$$

The key to doing this efficiently is to rewrite $F_N |k\rangle$, which turns out to be a *product state* (so F_N does not introduce entanglement when applied to a basis state $|k\rangle$), as follows. Let $|k\rangle = |k_1 \dots k_n\rangle$, k_1 being the most significant bit. Note that for integer $j = j_1 \dots j_n$, we can write $j/2^n = \sum_{\ell=1}^n j_\ell 2^{-\ell}$. For example, binary 0.101 is $1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 5/8$. We have the following sequence of equalities (which is probably most easily verified by working backwards from the last formula):

$$\begin{aligned} F_N |k\rangle &= \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} e^{2\pi i j k / 2^n} |j\rangle \\ &= \frac{1}{\sqrt{2^n}} \sum_{j \in \{0,1\}^n} e^{2\pi i (\sum_{\ell=1}^n j_\ell 2^{-\ell}) k} |j_1 \dots j_n\rangle \\ &= \frac{1}{\sqrt{2^n}} \sum_{j \in \{0,1\}^n} \prod_{\ell=1}^n e^{2\pi i j_\ell k / 2^\ell} |j_1 \dots j_n\rangle \\ &= \bigotimes_{\ell=1}^n \frac{1}{\sqrt{2}} \left(|0\rangle + e^{2\pi i k / 2^\ell} |1\rangle \right). \end{aligned}$$

Note that $e^{2\pi i k / 2^\ell} = e^{2\pi i k_1 \dots k_{n-\ell} \cdot k_{n-\ell+1} \dots k_n} = e^{2\pi i 0 \cdot k_{n-\ell+1} \dots k_n}$: the $n - \ell$ most significant bits of k don't matter for this value, because $e^{2\pi i m} = 1$ if m is an integer.

As an example, for $n = 3$ we have the 3-qubit product state

$$F_8|k_1k_2k_3\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i 0.k_3}|1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i 0.k_2k_3}|1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i 0.k_1k_2k_3}|1\rangle).$$

This example suggests what the circuit should be. To prepare the first qubit of the desired state $F_8|k_1k_2k_3\rangle$, we can just apply a Hadamard to $|k_3\rangle$, giving state $\frac{1}{\sqrt{2}}(|0\rangle + (-1)^{k_3}|1\rangle)$ and observe that $(-1)^{k_3} = e^{2\pi i 0.k_3}$. To prepare the second qubit of the desired state, apply a Hadamard to $|k_2\rangle$, giving $\frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i 0.k_2}|1\rangle)$, and then conditioned on k_3 (*before* we apply the Hadamard to $|k_3\rangle$) apply R_2 . This multiplies $|1\rangle$ with a phase $e^{2\pi i 0.k_3}$, producing the correct qubit $\frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i 0.k_2k_3}|1\rangle)$. Finally, to prepare the third qubit of the desired state, we apply a Hadamard to $|k_1\rangle$, apply R_2 conditioned on k_2 , and R_3 conditioned on k_3 . This produces the correct qubit $\frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i 0.k_1k_2k_3}|1\rangle)$. We have now produced all three qubits of the desired state $F_8|k_1k_2k_3\rangle$, *but in the wrong order*: the first qubit should be the third and vice versa. So the final step is just to swap qubits 1 and 3. Figure 4.1 illustrates the circuit in the case $n = 3$. Here the black circles indicate the control-qubits for each of the controlled- R_s operations, and the operation at the end of the circuit swaps qubits 1 and 3. The general case works analogously: starting with $\ell = 1$, we apply a Hadamard to $|k_\ell\rangle$ and then “rotate in” the additional phases required, conditioned on the values of the later bits $k_{\ell+1} \dots k_n$. Some swap gates at the end then put the qubits in the right order.²

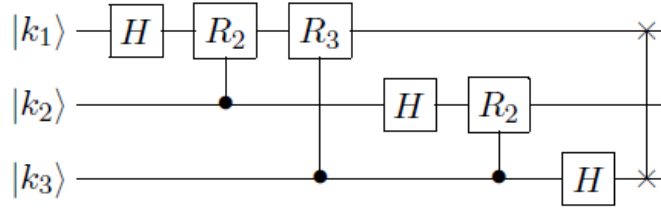


Figure 4.1: The circuit for the 3-qubit QFT

Since the circuit involves n qubits, and at most n gates are applied to each qubit, the overall circuit uses at most n^2 gates. In fact, many of those gates are phase gates R_s with $s \gg \log n$, which are very close to the identity and hence don't do much anyway. As observed by Coppersmith [94], we can actually omit those from the circuit, keeping only $O(\log n)$ gates per qubit and $O(n \log n)$ gates overall. Intuitively, the overall error caused by these omissions will be small (Exercise 4 asks you to make this precise). Finally, note that by inverting the circuit (i.e., reversing the order of the gates and taking the adjoint U^* of each gate U) we obtain an equally efficient circuit for the inverse Fourier transform $F_N^{-1} = F_N^*$.

4.6 Application: phase estimation

An important applications of the QFT is in *phase estimation*. This was originally due to Kitaev [155], it was put in a broader context by Cleve et al. [91], and is now a very common subroutine

²We can implement a SWAP-gate using CNOTs (Exercise 2.3); CNOTs and controlled- R_s gates can be constructed from Hadamard and controlled- R_1 (= controlled- Z) gates, which are in the allowed set of elementary gates here.

in many quantum algorithms.

Suppose we can apply a unitary U and we are given an eigenvector $|\psi\rangle$ of U with corresponding unknown eigenvalue λ (i.e., $U|\psi\rangle = \lambda|\psi\rangle$), and we would like to compute or at least approximate the λ . Since U is unitary, λ must have magnitude 1, so we can write it as $\lambda = e^{2\pi i\phi}$ for some real number $\phi \in [0, 1)$; the only thing that matters is this phase ϕ . Suppose for simplicity that we know that $\phi = 0.\phi_1 \dots \phi_n$ can be written exactly with n bits of precision. Then here's the algorithm for phase estimation:

1. Start with $|0^n\rangle|\psi\rangle$.
2. For $N = 2^n$, apply F_N to the first n qubits to get $\frac{1}{\sqrt{2^n}} \sum_{j=0}^{N-1} |j\rangle|\psi\rangle$ (in fact, $H^{\otimes n} \otimes I$ would have the same effect).
3. Apply the map $|j\rangle|\psi\rangle \mapsto |j\rangle U^j |\psi\rangle = e^{2\pi i\phi j} |j\rangle|\psi\rangle$. In other words, apply U to the second register for a number of times given by the first register.
4. Apply the inverse Fourier transform F_N^{-1} to the first n qubits and measure the result.

Note that after step 3, the first n qubits are in state $\frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} e^{2\pi i\phi j} |j\rangle = F_N |2^n \phi\rangle$, hence (under the assumption that ϕ can be written exactly with n bits) the inverse Fourier transform is going to give us $|2^n \phi\rangle = |\phi_1 \dots \phi_n\rangle$ with probability 1.

In case ϕ cannot be written exactly with n bits of precision, then one can show that this procedure still (with high probability) spits out a good n -bit approximation to ϕ . We'll omit the calculation.

Exercises

1. For $\omega = e^{2\pi i/3}$ and $F_3 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 & 1 & 1 \\ 1 & \omega & \omega^2 \\ 1 & \omega^2 & \omega \end{pmatrix}$, calculate $F_3 \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$ and $F_3 \begin{pmatrix} 1 \\ \omega^2 \\ \omega \end{pmatrix}$.
2. Prove that the Fourier coefficients of the convolution of vectors a and b are the product of the Fourier coefficients of a and b . In other words, prove that for every $a, b \in \mathbb{R}^N$ and every $\ell \in \{0, \dots, N-1\}$ we have $\widehat{(a * b)}_\ell = \widehat{a}_\ell \cdot \widehat{b}_\ell$. Here the Fourier transform $\widehat{a}$ is defined as the vector $F_N a$, and the ℓ -entry of the convolution-vector $a * b$ is $(a * b)_\ell = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} a_j b_{(\ell-j) \bmod N}$.
3. (H) The *total variation distance* between two probability distributions P and Q on the same set, is defined as $d_{TV D}(P, Q) = \frac{1}{2} \sum_i |P(i) - Q(i)|$. An equivalent alternative way to define this: $d_{TV D}(P, Q)$ is the maximum, over all events E , of $|P(E) - Q(E)|$. Hence $d_{TV D}(P, Q)$ is small iff all events have roughly the same probability under P and under Q .

The *Euclidean distance* between two states $|\phi\rangle = \sum_i \alpha_i |i\rangle$ and $|\psi\rangle = \sum_i \beta_i |i\rangle$ is defined as $\| |\phi\rangle - |\psi\rangle \| = \sqrt{\sum_i |\alpha_i - \beta_i|^2}$. Assume the two states are unit vectors with (for simplicity) real amplitudes. Suppose the Euclidean distance is small: $\| |\phi\rangle - |\psi\rangle \| = \epsilon$. If we measure $|\phi\rangle$ in the computational basis then the probability distribution over the outcomes is given by the $|\alpha_i|^2$, and if we measure $|\psi\rangle$ then the probabilities are $|\beta_i|^2$. Show that these distributions are close: the total variation distance $\frac{1}{2} \sum_i |\alpha_i^2 - \beta_i^2|$ is $\leq \epsilon$.

4. (H) The *operator norm* of a matrix A is defined as $\|A\| = \max_{v: \|v\|=1} \|Av\|$.

An equivalent definition is that $\|A\|$ is the largest singular value of A (see Appendix A.5).

The *distance* between two matrices A and B is defined as $\|A - B\|$.

- (a) What is the distance between the 2×2 identity matrix and the phase-gate $\begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix}$?
- (b) What is the distance between the 4×4 identity matrix and the *controlled* version of the phase gate of (a)?
- (c) What is the distance between the $2^n \times 2^n$ identity matrix I_{2^n} and the controlled phase gate of (b) tensored with $I_{2^{n-2}}$?
- (d) Suppose we have a product of n -qubit unitaries $U = U_T U_{T-1} \cdots U_1$ (for instance, each U_i could be an elementary gate on a few qubits, tensored with identity on the other qubits). Suppose we drop the j -th gate from this sequence: $U' = U_T U_{T-1} \cdots U_{j+1} U_{j-1} \cdots U_1$. Show that $\|U' - U\| = \|I - U_j\|$.
- (e) Now we also drop the k -th unitary: $U'' = U_T U_{T-1} \cdots U_{j+1} U_{j-1} \cdots U_{k+1} U_{k-1} \cdots U_1$. Show that $\|U'' - U\| \leq \|I - U_j\| + \|I - U_k\|$.
- (f) Give a quantum circuit with $O(n \log n)$ elementary gates that has distance less than $1/n$ from the Fourier transform F_{2^n} .

Comment: The above exercise shows the important fact that if we have a quantum circuit C that has various subparts (“subroutines”), then a circuit $\tilde{C}$ where those subroutines are implemented with small operator-norm error, rather than perfectly, still works well: if $\|C - \tilde{C}\|$ is small then (by definition of operator norm) for all initial states $|\phi\rangle$ the states $C|\phi\rangle$ and $\tilde{C}|\phi\rangle$ are close in Euclidean distance. By Exercise 3 then also the final output distributions are close (in total variation distance).

5. Suppose $a \in \mathbb{R}^N$ is a vector (indexed by $\ell = 0, \dots, N-1$) which is r -periodic in the following sense: there exists an integer r such that $a_\ell = 1$ whenever ℓ is an integer multiple of r , and $a_\ell = 0$ otherwise. Compute the Fourier transform $F_N a$ of this vector, i.e., write down a formula for the entries of the vector $F_N a$. Assuming r divides N , write down a simple closed form for the formula for the entries. Which are the nonzero entries in the vector $F_N a$, and what is their magnitude?
6. (a) The squared Fourier transform, F_N^2 , turns out to map computational basis states to computational basis states. Describe this map, i.e., determine to which basis state a basis state $|k\rangle$ gets mapped for each $k \in \{0, 1\}^n$.
- (b) Show that $F_N^4 = I$. What can you conclude about the eigenvalues of F_N ?

Chapter 5

Shor's Factoring Algorithm

5.1 Factoring

Probably the most important quantum algorithm so far is Shor's factoring algorithm [228]. It can find a factor of a composite number N in roughly $(\log N)^2$ steps, which is polynomial in the length $\log N$ of the input. On the other hand, there is no known classical (deterministic or randomized) algorithm that can factor N in polynomial time. The best known classical randomized algorithms run in time roughly

$$2^{(\log N)^\alpha},$$

where $\alpha = 1/3$ for a heuristic upper bound [165] and $\alpha = 1/2$ for a less-heuristic but still not fully proven upper bound [166]. In fact, much of modern cryptography is based on the conjecture that no fast classical factoring algorithm exists [211]. All this cryptography (for example RSA) would be broken if Shor's algorithm could be physically realized. In terms of complexity classes: factoring (rather, the decision problem equivalent to it) is provably in **BQP** but is not known to be in **BPP**. If indeed factoring is not in **BPP**, then the quantum computer would be the first counterexample to the “strong” Church-Turing thesis, which states that all “reasonable” models of computation are polynomially equivalent (see [107] and [198, p.31,36]).

5.2 Reduction from factoring to period-finding

The crucial observation of Shor was that there is an efficient quantum algorithm for the problem of *period-finding* and that factoring can be reduced to this, in the sense that an efficient algorithm for period-finding implies an efficient algorithm for factoring.

We first explain the reduction. Suppose we want to find factors of the composite number $N > 1$. We may assume N is odd and not a prime power, since those cases can easily be filtered out by a classical algorithm. Now randomly choose some integer $x \in \{2, \dots, N-1\}$ which is coprime¹ to N . If x is not coprime to N , then the greatest common divisor of x and N is a nontrivial factor of N , so then we are already done. From now on consider x and N are coprime, so x is an element

¹The *greatest common divisor* of two integers a and b is the largest positive integer c that divides both a and b . If $\gcd(a, b) = 1$, then a and b are called *coprime*. The gcd can be computed efficiently (in time roughly quadratic in the number of bits of a and b) on a classical computer by *Euclid's algorithm*.

of the multiplicative group $\mathbb{Z}_N^*$. Consider the sequence

$$1 = x^0 \pmod{N}, \quad x^1 \pmod{N}, \quad x^2 \pmod{N}, \dots$$

This sequence will cycle after a while: there is a least $0 < r \leq N$ such that $x^r = 1 \pmod{N}$. This r is called the *period* of the sequence (a.k.a. the *order of the element x in the group $\mathbb{Z}_N^*$*). Assuming N is odd and not a prime power (those cases are easy to factor anyway), it can be shown that with probability $\geq 1/2$, the period r is even and $x^{r/2} + 1$ and $x^{r/2} - 1$ are not multiples of N [196, Theorem A4.13]. In that case we have:

$$\begin{aligned} x^r &\equiv 1 \pmod{N} && \iff \\ (x^{r/2})^2 &\equiv 1 \pmod{N} && \iff \\ (x^{r/2} + 1)(x^{r/2} - 1) &\equiv 0 \pmod{N} && \iff \\ (x^{r/2} + 1)(x^{r/2} - 1) &= kN \text{ for some } k. \end{aligned}$$

Note that $k > 0$ because both $x^{r/2} + 1 > 0$ and $x^{r/2} - 1 > 0$ ($x > 1$). Hence $x^{r/2} + 1$ or $x^{r/2} - 1$ will share a factor with N . Because $x^{r/2} + 1$ and $x^{r/2} - 1$ are not multiples of N this factor will be $< N$, and in fact *both* these numbers will share a non-trivial factor with N . Accordingly, if we have r then we can compute the *greatest common divisors* $\gcd(x^{r/2} + 1, N)$ and $\gcd(x^{r/2} - 1, N)$, and both of these two numbers will be non-trivial factors of N . If we are unlucky we might have chosen an x that does not give a factor (which we can detect efficiently), but trying a few different random x gives a high probability of finding a factor.

Thus the problem of factoring reduces to finding the period r of the function given by modular exponentiation $f(a) = x^a \pmod{N}$. In general, the period-finding problem can be stated as follows:

The period-finding problem:

We are given some function $f : \mathbb{N} \rightarrow \{0, \dots, N-1\}$ with the property that there is some unknown $r \in \{0, \dots, N-1\}$ such that $f(a) = f(b)$ iff $a = b \pmod{r}$. The goal is to find r .

One might think that if f itself is efficiently computable, then period-finding is an easy problem to solve even on a classical computer: just compute $f(0), f(1), f(2), \dots$ until we encounter the value $f(0)$ for the second time. The input at which this happens is the period r that we're trying to find. The problem with this approach is that r could be huge, for instance $N^{1/2}$ or $N^{1/100}$, which is exponentially large in the number of inputs bits. To be efficient, we would like a runtime that is polynomial in $\log N$, since that is the bitsize of the inputs to f . It is generally believed that classical computers cannot solve period-finding efficiently.

We will show below how we *can* solve this problem efficiently on a quantum computer, using only $O(\log \log N)$ evaluations of f and $O(\log \log N)$ quantum Fourier transforms. An evaluation of f can be viewed as analogous to the application of a query in the algorithms of the previous chapters. Even a somewhat more general kind of period-finding can be solved by Shor's algorithm with very few f -evaluations, whereas any classical bounded-error algorithm would need to evaluate the function $\Omega(N^{1/3}/\sqrt{\log N})$ times in order to find the period [88].

How many steps (elementary gates) does Shor's algorithm take? For $a = N^{O(1)}$, we can compute $f(a) = x^a \pmod{N}$ in $O((\log N)^2 \log \log N \log \log \log N)$ steps by the "square-and-multiply" method, using known algorithms for fast integer multiplication mod N , see Exercise 1.

Moreover, as explained in the previous chapter, the quantum Fourier transform can be implemented using $O((\log N)^2)$ steps. Accordingly, Shor's algorithm finds a factor of N using an

expected number of $O((\log N)^2(\log \log N)^2 \log \log \log N)$ gates, which is only slightly worse than quadratic in the input length.

5.3 Shor's period-finding algorithm

Now we will show how Shor's algorithm finds the period r of the function f , given a “black-box” that maps $|a\rangle|0^n\rangle \mapsto |a\rangle|f(a)\rangle$. We can always efficiently pick some $q = 2^\ell$ such that $N^2 < q \leq 2N^2$. Then we can implement the Fourier transform F_q using $O((\log N)^2)$ gates. Let O_f denote the unitary that maps $|a\rangle|0^n\rangle \mapsto |a\rangle|f(a)\rangle$, where the first register consists of ℓ qubits, and the second of $n = \lceil \log N \rceil$ qubits.

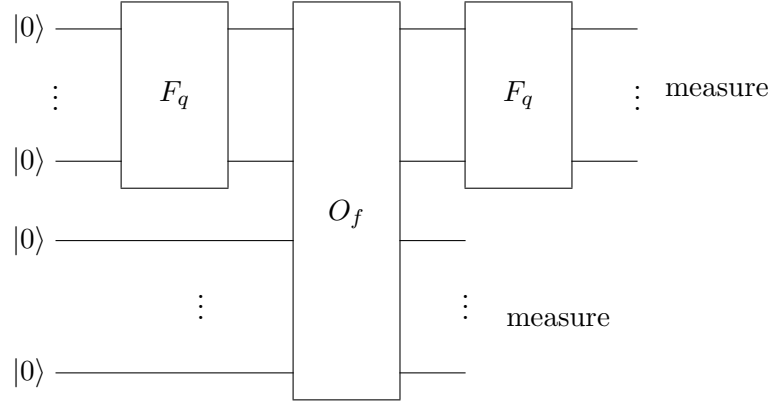


Figure 5.1: Shor's period-finding algorithm

Shor's period-finding algorithm is illustrated in Figure 5.1.² Start with $|0^\ell\rangle|0^n\rangle$. Apply the QFT (or just ℓ Hadamard gates) to the first register to build the uniform superposition

$$\frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle|0^n\rangle.$$

The second register still consists of zeroes. Now use the “black-box” to compute $f(a)$ in quantum parallel:

$$\frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle|f(a)\rangle.$$

Observing the second register gives some value $f(s)$, with $s < r$. Let m be the number of elements of $\{0, \dots, q-1\}$ that map to the observed value $f(s)$. Because $f(a) = f(s)$ iff $a = s \pmod r$, the a of the form $a = jr + s$ ($0 \leq j < m$) are exactly the a for which $f(a) = f(s)$. Thus the first register collapses to a superposition of $|s\rangle, |r+s\rangle, |2r+s\rangle, |3r+s\rangle, \dots$; this superposition runs until the last number of the form $jr + s$ that is $< q$, let's define m to be the number of elements in this superposition, i.e., the number of integers j such that $jr + s \in \{0, \dots, q-1\}$ (depending on s ,

²Notice the resemblance of the basic structure (Fourier, f -evaluation, Fourier) with the basic structure of Simon's algorithm (Hadamard, query, Hadamard). This is not a coincidence, because Shor was inspired by reading Simon's paper. The number of qubits used is roughly $3 \log N$: $\log q \approx 2 \log N$ qubits for the first register, and $\lceil \log N \rceil$ for the second register. This number can be reduced to slightly more than $2 \log N$ qubits [38, 131]. Accordingly, to factor for instance a 2048-bit integer N , slightly more than 4096 (perfect) qubits suffice.

this m will be $\lceil q/r \rceil$ or $\lfloor q/r \rfloor$). The second register collapses to the classical state $|f(s)\rangle$. We can now ignore the second register, and have in the first:

$$\frac{1}{\sqrt{m}} \sum_{j=0}^{m-1} |jr + s\rangle.$$

Applying the QFT again gives

$$\frac{1}{\sqrt{m}} \sum_{j=0}^{m-1} \frac{1}{\sqrt{q}} \sum_{b=0}^{q-1} e^{2\pi i \frac{(jr+s)b}{q}} |b\rangle = \frac{1}{\sqrt{mq}} \sum_{b=0}^{q-1} e^{2\pi i \frac{sb}{q}} \left(\sum_{j=0}^{m-1} e^{2\pi i \frac{jr b}{q}} \right) |b\rangle.$$

We want to see which $|b\rangle$ have amplitudes with large squared absolute value—those are the b we are likely to see if we now measure. Using that $\sum_{j=0}^{m-1} z^j = (1 - z^m)/(1 - z)$ for $z \neq 1$ (see Appendix B), we compute:

$$\sum_{j=0}^{m-1} e^{2\pi i \frac{jr b}{q}} = \sum_{j=0}^{m-1} \left(e^{2\pi i \frac{r b}{q}} \right)^j = \begin{cases} m & \text{if } e^{2\pi i \frac{r b}{q}} = 1 \\ \frac{1 - e^{2\pi i \frac{m r b}{q}}}{1 - e^{2\pi i \frac{r b}{q}}} & \text{if } e^{2\pi i \frac{r b}{q}} \neq 1 \end{cases} \quad (5.1)$$

Easy case: r divides q . Let us do an easy case first. Suppose r divides q , so the whole period “fits” an integer number of times in the domain $\{0, \dots, q-1\}$ of f , and $m = q/r$. For the first case of Eq. (5.1), note that $e^{2\pi i r b/q} = 1$ iff $r b/q$ is an integer iff b is a multiple of q/r . Such b will have squared amplitude equal to $(m/\sqrt{mq})^2 = m/q = 1/r$. Since there are exactly r such basis states b , together they have all the amplitude: the sum of squares of those amplitudes is 1, so the amplitudes of b that are *not* integer multiples of q/r must all be 0. Thus we are left with a superposition where only the b that are integer multiples of q/r have nonzero amplitude. Observing this final superposition gives some random multiple $b = cq/r$, with c a uniformly random number in $\{0, \dots, r-1\}$. Thus we get a b such that

$$\frac{b}{q} = \frac{c}{r},$$

where b and q are known to the algorithm, and c and r are not. There are $\phi(r) \in \Omega(r/\log \log r)$ numbers smaller than r that are coprime to r [133, Theorem 328], so c will be coprime to r with probability $\Omega(1/\log \log r) \geq \Omega(1/\log \log N)$. Accordingly, an expected number of $O(\log \log N)$ repetitions of the procedure of this section suffices to obtain a $b = cq/r$ with c coprime to r .³ Once we have such a b , we can obtain r as the denominator by writing b/q in lowest terms. Of course, our algorithm doesn’t actually know whether c and r are coprime in some particular run of the algorithm, but it can efficiently check if the purported factors $\gcd(x^{r/2} \pm 1)$ are actual factors of N by division (which, like multiplication, can be done classically with a near-linear number of gates).

Hard case: r does not divide q . Because our q is a power of 2, it is actually quite likely that r does not divide q . However, the same algorithm will still yield with high probability a b which is *close* to a multiple of q/r . Note that q/r is no longer an integer, and $m = \lfloor q/r \rfloor$, possibly $+1$.

³The number of required f -evaluations for period-finding can actually be reduced from $O(\log \log N)$ to $O(1)$.

All calculations up to and including Eq. (5.1) are still valid. Using $|1 - e^{i\theta}| = 2|\sin(\theta/2)|$, we can rewrite the absolute value of the second case of Eq. (5.1) to

$$\frac{|1 - e^{2\pi i \frac{mrb}{q}}|}{|1 - e^{2\pi i \frac{rb}{q}}|} = \frac{|\sin(\pi mrb/q)|}{|\sin(\pi rb/q)|}.$$

The right-hand side is the ratio of two sine-functions of b , where the numerator oscillates much faster than the denominator because of the additional factor of m . Note that the denominator is close to 0 (making the ratio large) iff b is close to an integer multiple of q/r . For most of those b , the numerator won't be close to 0. Hence, roughly speaking, the ratio will be small if b is far from an integer multiple of q/r , and large for most b that are close to a multiple of q/r . Doing the calculation precisely, one can show that with high probability (see [228, 196] for details) the measurement yields a b such that

$$\left| \frac{b}{q} - \frac{c}{r} \right| \leq \frac{1}{2q},$$

for a random $c \in \{0, \dots, r-1\}$. Equivalently, $|b - cq/r| \leq 1/2$, so the measurement outcome b will be an integer multiple of q/r rounded up or down to an integer. As in the easy case, b and q are known to us while c and r are unknown.

Because the known ratio b/q is now not exactly equal to the unknown ratio c/r , we cannot just try to find r by writing b/q in lowest terms like we did in the easy case. However, two distinct fractions, each with denominator $\leq N$, must be at least $1/N^2 > 1/q$ apart.⁴ Therefore c/r is the *only* fraction with denominator $\leq N$ at distance $\leq 1/2q$ from the known ratio b/q . Applying a classical method called “continued-fraction expansion” to b/q efficiently gives us the fraction with denominator $\leq N$ that is closest to b/q (see next section). This fraction must be c/r . Again, c and r will be coprime with probability $\Omega(1/\log \log r)$, in which case writing c/r in lowest terms gives r .

5.4 Continued fractions

Let $[a_0, a_1, a_2, \dots]$ (finite or infinite) denote the real number

$$a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{\dots}}}$$

This is called a *continued fraction* (CF). The a_i are the *partial quotients*. We assume these to be positive natural numbers ([133, p.131] calls such CF “simple”). $[a_0, \dots, a_n]$ is the n -th *convergent* of the fraction. [133, Theorem 149 & 157] gives a simple way to compute numerator and denominator of the n -th convergent from the partial quotients:

If

$$\begin{aligned} p_0 &= a_0, & p_1 &= a_1 a_0 + 1, & p_n &= a_n p_{n-1} + p_{n-2} \\ q_0 &= 1, & q_1 &= a_1, & q_n &= a_n q_{n-1} + q_{n-2} \end{aligned}$$

then $[a_0, \dots, a_n] = \frac{p_n}{q_n}$. Moreover, this fraction is in lowest terms.

⁴Consider two fractions c/r and c'/r' with integer c, c', r, r' , and $r, r' \leq N$. If $c/r \neq c'/r'$ then $cr' - c'r$ is a nonzero integer, and hence $|c/r - c'/r'| = |(cr' - c'r)/rr'| \geq 1/|rr'| \geq 1/N^2$.

Note that q_n increases at least exponentially with n ($q_n \geq 2q_{n-2}$). Given a real number x , the following “algorithm” gives a continued fraction expansion of x [133, p.135]:

$$\begin{aligned} a_0 &:= \lfloor x \rfloor, & x_1 &:= 1/(x - a_0) \\ a_1 &:= \lfloor x_1 \rfloor, & x_2 &:= 1/(x_1 - a_1) \\ a_2 &:= \lfloor x_2 \rfloor, & x_3 &:= 1/(x_2 - a_2) \\ &\dots \end{aligned}$$

Informally, we just take the integer part of the number as the partial quotient and continue with the inverse of the decimal part of the number. The convergents of the CF approximate x as follows [133, Theorem 164 & 171]:

$$\text{If } x = [a_0, a_1, \dots] \text{ then } \left| x - \frac{p_n}{q_n} \right| < \frac{1}{q_n^2}.$$

Recall that q_n increases exponentially with n , so this convergence is quite fast. Moreover, p_n/q_n provides the *best* approximation of x among all fractions with denominator $\leq q_n$ [133, Theorem 181]:

$$\text{If } n > 1, q \leq q_n, p/q \neq p_n/q_n, \text{ then } \left| x - \frac{p_n}{q_n} \right| < \left| x - \frac{p}{q} \right|.$$

Exercises

1. This exercise is about efficient classical implementation of modular exponentiation.
 - (a) (H) Given n -bit numbers x and N , compute the whole sequence $x^0 \bmod N, x^1 \bmod N, x^2 \bmod N, x^4 \bmod N, x^8 \bmod N, x^{16} \bmod N, \dots, x^{2^{n-1}} \bmod N$, using $O(n^2 \log(n) \log \log(n))$ steps.
 - (b) Suppose n -bit number a can be written as $a = a_{n-1} \dots a_1 a_0$ in binary. Express $x^a \bmod N$ as a product of the numbers computed in part (a).
 - (c) Show that you can compute $f(a) = x^a \bmod N$ in $O(n^2 \log(n) \log \log(n))$ steps.
2. Consider the function $f(a) = 7^a \bmod 10$.
 - (a) What is the period r of f ?
 - (b) Show how Shor’s algorithm finds the period of f , using a Fourier transform over $q = 128$ elements. Write down all intermediate superpositions of the algorithm for this case (don’t just copy the general expressions from the notes, but instantiate them with actual numbers as much as possible, incl. with the value of the period found in (a)). You may assume you’re lucky, meaning the first run of the algorithm already gives a measurement outcome $b = cq/r$ with c coprime to r .
3. (H) This exercise explains basic RSA encryption. Suppose Alice wants to allow other people to send encrypted messages to her, such that she is the only one who can decrypt them. She believes that factoring an n -bit number can’t be done efficiently (efficient = in time polynomial in n). So in particular, she doesn’t believe in quantum computing.

Alice chooses two large random prime numbers, p and q , and computes their product $N = p \cdot q$ (a typical size is to have N a number of $n = 1024$ bits, which corresponds to both

p and q being numbers of roughly 512 bits). She computes the so-called Euler ϕ -function: $\phi(N) = (p-1)(q-1)$; she also chooses an *encryption exponent* e , which doesn't share any nontrivial factor with $\phi(N)$ (i.e., e and $\phi(N)$ are coprime). Group theory guarantees there is an efficiently computable *decryption exponent* d such that $de = 1 \bmod \phi(N)$. The *public key* consists of e and N (Alice puts this on her homepage), while the *secret key* consists of d and N . Any number $m \in \{1, \dots, N-1\}$ that is coprime to N , can be used as a message. There are $\phi(N)$ such m , and these numbers form a group under the operation of multiplication mod N . The number of bits $n = \lceil \log N \rceil$ of N is the maximal length (in bits) of a message m and also the length (in bits) of the encryption. The encryption function is defined as $C(m) = m^e \bmod N$, and the decryption function is $D(c) = c^d \bmod N$.

- (a) Give a randomized algorithm by which Alice can efficiently generate the secret and public key.
- (b) Show that Bob can efficiently compute the encoding $C(m)$ of the message m that he wants to send to Alice, knowing the public key but not the private key.
- (c) Show that $D(C(m)) = m$ for all possible messages.
- (d) Show that Alice can efficiently decrypt the encryption $C(m)$ she receives from Bob.
- (e) Show that if Charlie could factor N , then he could efficiently decrypt Bob's message.

Chapter 6

Hidden Subgroup Problem

6.1 Hidden Subgroup Problem

6.1.1 Group theory reminder

A group G consists of a set of elements (which is usually denoted by G as well) and an operation $\circ : G \times G \rightarrow G$ (often written as addition or multiplication), such that

1. the operation is associative: $g \circ (h \circ k) = (g \circ h) \circ k$ for all $g, h, k \in G$;
2. there is an *identity element* $e \in G$ satisfying $e \circ g = g \circ e = g$ for every $g \in G$;
3. and every $g \in G$ has an *inverse* $g^{-1} \in G$, such that $g \circ g^{-1} = g^{-1} \circ g = e$ (if the group operation is written as addition, then g^{-1} is written as $-g$).

We often abbreviate $g \circ h$ to gh . The group is *Abelian* (or *commutative*) if $gh = hg$ for all $g, h \in G$. Simple examples of finite additive Abelian groups are $G = \{0, 1\}^n$ with bitwise addition mod 2 as the group operation, and $G = \mathbb{Z}_N$, the “cyclic group” of integers mod N . The set $G = \mathbb{Z}_N^*$ is the multiplicative group consisting of all integers in $\{1, \dots, N-1\}$ that are coprime to N , with multiplication mod N as the group operation.¹ An important example of a non-Abelian group is the “symmetric group” S_n , which is the group of $n!$ permutations of n elements, using composition as the group operation.

A *subgroup* H of G , denoted $H \leq G$, is a subset of G that is itself a group, i.e., it contains e and is closed under taking products and inverses. A (left) *coset* of H is a set $gH = \{gh \mid h \in H\}$, i.e., a translation of H by the element g . All cosets of H have size $|H|$, and it is easy to show that two cosets gH and $g'H$ are either equal or disjoint, so the set of cosets partitions G into equal-sized parts.² Note that g and g' are in the same coset of H iff $g^{-1}g' \in H$.

If $T \subseteq G$, then we use $\langle T \rangle$ to denote the set of elements of G that we can write as products of elements from T and their inverses. This $H = \langle T \rangle$ is a subgroup of G , and T is called a *generating set* of H . Note that adding one more element $t \notin \langle T \rangle$ to T at least doubles the size of the generated subgroup, because H and tH are disjoint and $H \cup tH \subseteq \langle T \cup \{t\} \rangle$. This implies that every $H \leq G$

¹Euler’s ϕ -function counts the number of elements of $\{1, \dots, N-1\}$ that are coprime to N , so $|\mathbb{Z}_N^*| = \phi(N)$. Note that for prime p , the multiplicative Abelian group $\mathbb{Z}_p^*$ is isomorphic to the additive group $\mathbb{Z}_{p-1}$. However, for general N , $\mathbb{Z}_N^*$ need not be isomorphic to $\mathbb{Z}_{\phi(N)}$.

²This also proves Lagrange’s theorem for finite groups: if $H \leq G$ then $|H|$ divides $|G|$.

has a generating set of size $\leq \log |H| \leq \log |G|$. We abbreviate $\langle \{\gamma\} \rangle$ to $\langle \gamma \rangle$, which is the cyclic group generated by γ ; every cyclic group of size N is isomorphic to $\mathbb{Z}_N$.

6.1.2 Definition and some instances of the HSP

The *Hidden Subgroup Problem* is the following:

Given a known group G and a function $f : G \rightarrow S$ where S is some finite set.
 Suppose f has the property that there exists a subgroup $H \leq G$ such that f is constant within each coset, and distinct on different cosets: $f(g) = f(g')$ iff $gH = g'H$.
 Goal: find H .

We assume f can be computed efficiently, meaning in time polynomial in $\log |G|$ (the latter is the number of bits needed to describe an input $g \in G$ for f). Since H may be large, “finding H ” typically means finding a generating set for H .

This looks like a rather abstract algebraic problem, but many important problems can be written as an instance of the HSP. We will start with some examples where G is Abelian.

Simon’s problem. This is a very natural instance of HSP. Here G is the additive group $\mathbb{Z}_2^n = \{0, 1\}^n$ of size 2^n , $H = \{0, s\}$ for a “hidden” $s \in \{0, 1\}^n$, and f satisfies $f(x) = f(y)$ iff $x - y \in H$. Clearly, finding the generator of H (i.e., finding s) solves Simon’s problem.

Period-finding. As we saw in Chapter 5, we can factor a large number N if we can solve the following: given an x that is coprime to N and associated function $f : \mathbb{Z} \rightarrow \mathbb{Z}_N^*$ by $f(a) = x^a \bmod N$, find the period r of f .³ Since $\langle x \rangle$ is a size- r subgroup of the group $\mathbb{Z}_N^*$, the period r divides $|\mathbb{Z}_N^*| = \phi(N)$. Hence we can restrict the domain of f to $\mathbb{Z}_{\phi(N)}$.

Period-finding is an instance of the HSP as follows. Let $G = \mathbb{Z}_{\phi(N)}$ and consider its subgroup $H = \langle r \rangle$ of all multiples of r up to $\phi(N)$ (i.e., $H = r\mathbb{Z}_{\phi(N)} = \{0, r, 2r, \dots, \phi(N) - r\}$). Note that because of its periodicity, f is constant on each coset $s + H$ of H , and distinct on different cosets. Also, f is efficiently computable by repeated squaring. Since the hidden subgroup H is generated by r , finding the generator of H solves the period-finding problem.

Discrete logarithm. Another problem often used in classical public-key cryptography is the discrete logarithm problem: given a generator γ of a cyclic multiplicative group C of size N (so $C = \{\gamma^a \mid a \in \{0, \dots, N-1\}\}$), and $A \in C$, can we find the unique $a \in \{0, 1, \dots, N-1\}$ such that $\gamma^a = A$? This a is called the discrete logarithm of A (w.r.t. generator γ). It is generally believed that classical computers need time roughly exponential in $\log N$ to compute a from A (and one can actually *prove* this in a model where we can only implement group operations via some “black-box” [229]). This assumption underlies for instance the security of Diffie-Hellman key exchange (where $C = \mathbb{Z}_p^*$ for some large prime p , see Exercise 3), as well as elliptic-curve cryptography.

Discrete log is an instance of the HSP as follows. We take $G = \mathbb{Z}_N \times \mathbb{Z}_N$ and define function $f : G \rightarrow C$ by $f(x, y) = \gamma^x A^{-y}$, which is efficiently computable by repeated squaring. For group elements $g_1 = (x_1, y_1), g_2 = (x_2, y_2) \in G$ we have

$$f(g_1) = f(g_2) \iff \gamma^{x_1 - ay_1} = \gamma^{x_2 - ay_2} \iff (x_1 - x_2) = a(y_1 - y_2) \bmod N \iff g_1 - g_2 \in \langle (a, 1) \rangle.$$

³This r is also known as the *order* of the element x in the group $\mathbb{Z}_N^*$, so this problem is also known as *order-finding*.

Let H be the subgroup of G generated by the element $(a, 1)$, then we have an instance of the HSP. Finding the generator of the hidden subgroup H gives us a , solving the discrete log problem.

6.2 An efficient quantum algorithm if G is Abelian

In this section we show that HSPs where G (and hence H) is Abelian, and where f is efficiently computable, can be solved efficiently by a quantum algorithm. This generalizes Shor's factoring algorithm, and will also give an efficient quantum algorithm for computing discrete logarithms.

6.2.1 Representation theory and the quantum Fourier transform

We start by quickly explaining the basics of representation theory. The idea here is to replace group elements by matrices, so that linear algebra can be used as a tool in group theory. A d -dimensional *representation* of a multiplicative group G is a map $\rho : g \mapsto \rho(g)$ from G to the set of $d \times d$ invertible complex matrices, satisfying $\rho(gh) = \rho(g)\rho(h)$ for all $g, h \in G$. The latter property makes the map ρ a *homomorphism*. It need not be an *isomorphism* (i.e., bijective), for example the constant-1 function is a trivial representation of any group. A representation of G is *irreducible* if it cannot be decomposed further into the direct sum of lower-dimensional representations of G . A 1-dimensional representation of G is called a *character* of G (sometimes *linear character*). Note that a character χ is irreducible, and the complex values $\chi(g)$ must have modulus 1 because $|\chi(g^k)| = |\chi(g)|^k$ for all integers k . For example, the group $\mathbb{Z}_2 = \{0, 1\}$ has two characters: the χ that maps both elements to 1, and the χ that maps 0 to 1 and 1 to -1 .

In the remainder of this section we will restrict attention to the case where G is Abelian (and usually finite). In the Abelian case the characters are exactly the irreducible representations (irreps): there are no irreps of dimension > 1 . The “Basis Theorem” of group theory says that every finite Abelian group G is isomorphic to a direct product $\mathbb{Z}_{N_1} \times \cdots \times \mathbb{Z}_{N_\ell}$ of cyclic groups. First consider just one cyclic group $\mathbb{Z}_N$, written additively. Consider the discrete Fourier transform (Chapter 4), which is an $N \times N$ matrix. Ignoring the normalizing factor of $1/\sqrt{N}$, its k -th column may be viewed as a map $\chi_k : \mathbb{Z}_N \rightarrow \mathbb{C}$ defined by $\chi_k(j) = \omega_N^{jk}$, where $\omega_N = e^{2\pi i/N}$. Note that $\chi_k(j+j') = \chi_k(j)\chi_k(j')$, so χ_k is actually a 1-dimensional representation (i.e., a character) of $\mathbb{Z}_N$. In fact, the N characters corresponding to the N columns of the Fourier matrix are *all* the characters of $\mathbb{Z}_N$. For Abelian groups G that are (isomorphic to) a product $\mathbb{Z}_{N_1} \times \cdots \times \mathbb{Z}_{N_\ell}$ of cyclic groups, the $|G| = N_1 \cdots N_\ell$ characters are just the products of the characters of the individual cyclic groups $\mathbb{Z}_{N_j}$. Note that the characters are pairwise orthogonal.

The set of all characters of G forms a group $\widehat{G}$ with the operation of pointwise multiplication. This is called the *dual group* of G . If $H \leq G$, then the following is a subgroup of $\widehat{G}$ of size $|G|/|H|$:

$$H^\perp = \{\chi_k \mid \chi_k(h) = 1 \text{ for all } h \in H\}.$$

Let us interpret the quantum Fourier transform in terms of the characters. For $k \in \mathbb{Z}_N$, define the state whose amplitudes are the (normalized) values of χ_k :

$$|\chi_k\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \chi_k(j) |j\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \omega_N^{jk} |j\rangle.$$

With this notation, the QFT just maps the standard (computational) basis of $\mathbb{C}^N$ to the orthonormal basis corresponding to the characters:

$$F_N : |k\rangle \mapsto |\chi_k\rangle.$$

As we saw in Chapter 4, this map can be implemented by an efficient quantum circuit if N is a power of 2. The QFT corresponding to a group G that is isomorphic to $\mathbb{Z}_{N_1} \times \cdots \times \mathbb{Z}_{N_\ell}$ is just the tensor product of the QFTs for the individual cyclic groups. For example, the QFT corresponding to $\mathbb{Z}_2$ is the Hadamard gate H , so the QFT corresponding to $\mathbb{Z}_2^n$ is $H^{\otimes n}$ (which is of course very different from the QFT corresponding to $\mathbb{Z}_{2^n}$).

6.2.2 A general algorithm for Abelian HSP

The following is an efficient quantum algorithm for solving the HSP for some Abelian group G (written additively) and function $f : G \rightarrow S$. This algorithm, sometimes called the “standard algorithm” for HSP, was first observed by Kitaev [155] (inspired by Shor’s algorithm) and worked out further by many, for instance Mosca and Ekert [192].

1. Start with $|0\rangle|0\rangle$, where the two registers have dimension $|G|$ and $|S|$, respectively.
2. Create a uniform superposition over G in the first register: $\frac{1}{\sqrt{|G|}} \sum_{g \in G} |g\rangle|0\rangle$.
3. Compute f in superposition: $\frac{1}{\sqrt{|G|}} \sum_{g \in G} |g\rangle|f(g)\rangle$.
4. Measure the second register. This yields some value $f(s)$ for unknown $s \in G$. The first register collapses to a superposition over the g with the same f -value as s (i.e., the coset $s + H$): $\frac{1}{\sqrt{|H|}} \sum_{h \in H} |s + h\rangle$.
5. Apply the QFT corresponding to G to this state, giving $\frac{1}{\sqrt{|H|}} \sum_{h \in H} |\chi_{s+h}\rangle$.
6. Measure and output the resulting g .

The key to understanding this algorithm is to observe that step 5 maps the uniform superposition over the coset $s + H$ to a uniform superposition over the labels of $H^\perp$:

$$\begin{aligned} \frac{1}{\sqrt{|H|}} \sum_{h \in H} |\chi_{s+h}\rangle &= \frac{1}{\sqrt{|H||G|}} \sum_{h \in H} \sum_{g \in G} \chi_{s+h}(g) |g\rangle \\ &= \frac{1}{\sqrt{|H||G|}} \sum_{g \in G} \chi_s(g) \sum_{h \in H} \chi_h(g) |g\rangle = \sqrt{\frac{|H|}{|G|}} \sum_{g: \chi_g \in H^\perp} \chi_s(g) |g\rangle, \end{aligned}$$

where the last equality follows from the orthogonality of characters of the group H (note that χ_g restricted to H is a character of H , and it’s the constant-1 character iff $\chi_g \in H^\perp$):

$$\sum_{h \in H} \chi_h(g) = \sum_{h \in H} \chi_g(h) = \begin{cases} |H| & \text{if } \chi_g \in H^\perp \\ 0 & \text{if } \chi_g \notin H^\perp \end{cases}$$

The phases $\chi_s(g)$ do not affect the probabilities of the final measurement, since $|\chi_s(g)|^2 = 1$. The above algorithm thus samples uniformly from the (labels of) elements of $H^\perp$. Each such element $\chi_g \in H^\perp$ gives us a constraint on H because $\chi_g(h) = 1$ for all $h \in H$.⁴ Generating a small number of such elements will give sufficient information to find the generators of H itself. Consider our earlier examples of Abelian HSP:

Simon's problem. Recall that $G = \mathbb{Z}_2^n = \{0, 1\}^n$ and $H = \{0, s\}$ for the HSP corresponding to Simon's problem. Setting up the uniform superposition over G can be done by applying $H^{\otimes n}$ to the initial state $|0^n\rangle$ of the first register. The QFT corresponding to G is just $H^{\otimes n}$. The 2^n characters are $\chi_g(x) = (-1)^{x \cdot g}$. The algorithm will uniformly sample from labels of elements of

$$H^\perp = \{\chi_g \mid \chi_g(h) = 1 \text{ for all } h \in H\} = \{\chi_g \mid g \cdot s = 0\}.$$

Accordingly, the algorithm samples uniformly from the $g \in \{0, 1\}^n$ such that $g \cdot s = 0 \pmod{2}$. Doing this an expected $O(n)$ times gives $n - 1$ linearly independent equations about s , from which we can find s using Gaussian elimination.

Period-finding. For the HSP corresponding to period-finding, $G = \mathbb{Z}_{\phi(N)}$ and $H = \langle r \rangle$, and

$$H^\perp = \{\chi_b \mid e^{2\pi i b h / \phi(N)} = 1 \text{ for all } h \in H\} = \{\chi_b \mid b r / \phi(N) \in \{0, \dots, r - 1\}\}.$$

Accordingly, the output of the algorithm is an integer multiple $b = c\phi(N)/r$ of $\phi(N)/r$, for uniformly random $c \in \{0, \dots, r - 1\}$.

Notice that the algorithm doesn't actually know $\phi(N)$, which creates two problems. First, of the 4 numbers $b, c, \phi(N), r$ involved in the equation $b = c\phi(N)/r$ we only know the measurement outcome b , which is not enough to compute r . Second, step 5 of the algorithm wants to do a QFT corresponding to the group $\mathbb{Z}_{\phi(N)}$ but it doesn't know $\phi(N)$; and even if we knew $\phi(N)$, we've only seen how to efficiently implement a QFT over $\mathbb{Z}_q$ when q is a power of 2. Fortunately, if we actually use the QFT over $\mathbb{Z}_q$ for q a power of 2 that is roughly N^2 (and in step 1 set up a uniform superposition over $\mathbb{Z}_q$ instead of over G), then one can show that the above algorithm still works: the measurement yields an integer b that (with high probability) is close to an integer multiple of q/r .⁵ This is basically just Shor's algorithm as described in Chapter 5.

Discrete logarithm. For the HSP corresponding to the discrete log problem, where $G = \mathbb{Z}_N \times \mathbb{Z}_N$ and $H = \langle (a, 1) \rangle$, a small calculation shows that $H^\perp = \{\chi_{(c, -ac)} \mid c \in \mathbb{Z}_N\}$ (see Exercise 2). Hence sampling from $H^\perp$ yields some label $(c, -ac) \in G$ of an element of $H^\perp$, from which we can compute the discrete logarithm a . The QFT corresponding to G is $F_N \otimes F_N$, which we don't know how to implement efficiently for arbitrary N , but which we can replace by $F_q \otimes F_q$ for some power-of-2 q chosen to be somewhat larger than N .

In the above algorithm we assumed G is a *finite* Abelian group. These techniques have been much extended to the case of infinite groups such as $G = \mathbb{Z}$ and even $\mathbb{R}^d$, to obtain efficient quantum algorithms for problems like Pell's equation [128], and computing properties in number fields [59].

⁴This is a *linear* constraint mod N . For example, say $G = \mathbb{Z}_{N_1} \times \mathbb{Z}_{N_2}$, and $g = (g_1, g_2)$ is the label of an element of $H^\perp$. Then $1 = \chi_g(h) = \omega_{N_1}^{g_1 h_1} \omega_{N_2}^{g_2 h_2}$ for all $h = (h_1, h_2) \in H$, equivalently $g_1 h_1 N_2 + g_2 h_2 N_1 = 0 \pmod{N}$.

⁵There is something to be proved here, but we will skip the details. In fact one can even use a Fourier transform for $q = O(N)$ instead of $O(N^2)$ [127]. Note that this also reduces the number of qubits used by Shor's algorithm from roughly $3 \log N$ to roughly $2 \log N$.

6.3 General non-Abelian HSP

6.3.1 The symmetric group and the graph isomorphism problem

The Abelian HSP covers a number of interesting computational problems, including period-finding and discrete log. However, there are also some interesting computational problems that can be cast as an instance of HSP with a *non-Abelian* G . Unfortunately we do not have an efficient algorithm for most non-Abelian HSPs.

A good example is the *graph isomorphism* (GI) problem: given two undirected n -vertex graphs $\mathcal{G}_1$ and $\mathcal{G}_2$, decide whether there exists a bijection taking the vertices of $\mathcal{G}_1$ to those of $\mathcal{G}_2$ that makes the two graphs equal. No efficient classical algorithm is known for GI, so it would be great if we could solve this efficiently on a quantum computer.⁶

How can we try to solve this via the HSP? Let $\mathcal{G}$ be the $2n$ -vertex graph that is the disjoint union of the two graphs $\mathcal{G}_1$ and $\mathcal{G}_2$. Let $G = S_{2n}$. Let f map $\pi \in S_{2n}$ to $\pi(\mathcal{G})$, which means that edge (i, j) becomes edge $(\pi(i), \pi(j))$. Let H be the automorphism group $\text{Aut}(\mathcal{G})$ of $\mathcal{G}$, which is the set of all $\pi \in S_{2n}$ that map $\mathcal{G}$ to itself. This gives an instance of the HSP, and solving it would give us a generating set of $H = \text{Aut}(\mathcal{G})$.

Assume for simplicity that each of $\mathcal{G}_1$ and $\mathcal{G}_2$ is connected. If $\mathcal{G}_1$ and $\mathcal{G}_2$ are not isomorphic, then the only automorphisms of $\mathcal{G}$ are the ones that permute vertices inside $\mathcal{G}_1$ and inside $\mathcal{G}_2$: $\text{Aut}(\mathcal{G}) = \text{Aut}(\mathcal{G}_1) \times \text{Aut}(\mathcal{G}_2)$. However, if the two graphs are isomorphic, then $\text{Aut}(\mathcal{G})$ will also contain a permutation that swaps the first n with the second n vertices. Accordingly, if we were able to find a generating set of the hidden subgroup $H = \text{Aut}(\mathcal{G})$, then we can just check whether all generators are in $\text{Aut}(\mathcal{G}_1) \times \text{Aut}(\mathcal{G}_2)$ and decide graph isomorphism.

6.3.2 Non-Abelian QFT on coset states

One can try to design a quantum algorithm for general, non-Abelian instances of the HSP along the lines of the earlier standard algorithm: set up a uniform superposition over a random coset of H , apply the QFT corresponding to G , measure the final state, and hope that the result gives useful information about H . QFTs corresponding to non-Abelian G are much more complicated than in the Abelian case, because the irreducible representations ρ can now have dimension $d > 1$. For completeness, let's write down the QFT anyway. Let $\hat{G}$ denote the set of irreducible representations of G , and $\dim(\rho)$ be the dimension of a particular $\rho \in \hat{G}$. We can assume without loss of generality that the $\dim(\rho) \times \dim(\rho)$ matrices $\rho(g)$ are unitary. The QFT corresponding to G is defined as follows:

$$|g\rangle \mapsto \sum_{\rho \in \hat{G}} \sqrt{\frac{\dim(\rho)}{|G|}} |\rho\rangle \sum_{i,j=1}^{\dim(\rho)} \rho(g)_{ij} |i, j\rangle,$$

where $|\rho\rangle$ denotes a name or label of ρ . It can be shown that this map is unitary. In particular, $|G| = \sum_{\rho \in \hat{G}} \dim(\rho)^2$, which implies that the dimensions on the left and the right are the same, and that the right-hand state has norm 1. In many cases this QFT can still be implemented with an efficient quantum circuit, including for the symmetric group $G = S_{2n}$ that is relevant for graph isomorphism [36, 190]. However, that is not enough for an efficient algorithm: the standard algorithm does not always yield much information about the hidden $H \leq S_{2n}$ [124, 191, 129].

⁶For a long time, the best algorithm for GI took time roughly $2^{\sqrt{n}}$ [33], but in a recent breakthrough Babai gave a “quasi-polynomial” algorithm, which is $2^{(\log n)^{O(1)}}$ time [32]. That's not yet polynomial, but a lot faster than before.

There are some special cases of non-Abelian HSP that can be computed efficiently, for instance for normal subgroups [130], solvable groups [244, 141], and nil-2 groups [142].

6.3.3 Query-efficient algorithm

While we do not have a general efficient quantum algorithm for the non-Abelian HSP, there does exist an algorithm that needs to compute f only a few times, i.e., a *query-efficient* algorithm, due to Ettinger et al. [108]. We will sketch this now.

Consider steps 1–3 of the standard algorithm for the Abelian case. Even in the general non-Abelian case, this produces a coset state, i.e., a two-register superposition where the second register ranges over the values of f , and the first register will be a uniform superposition over the coset of H that corresponds to that value of f . Suppose we do this m times, producing a state $|\psi_H\rangle$ which is the tensor product of m coset states for the same unknown H (for simplicity, below we'll ignore the fact that this state also depends on the particular values f takes on the cosets of H). One can show that the coset states corresponding to different possible H are pairwise almost orthogonal: $|\langle\psi_H|\psi_{H'}\rangle|$ is exponentially small in m . How large should we take m to ensure that these states are “sufficiently orthogonal” to enable us to learn H from $|\psi_H\rangle$? The hidden subgroup H is generated by a set of $\leq \log |G|$ elements. Hence the total number of possible H that we want to distinguish is at most $\binom{|G|}{\log |G|} \leq 2^{(\log |G|)^2}$. This upper bound on the number of possible H allows us to define a POVM measurement $\{E_H\}$ (see Section 1.2.2 for the definition of POVM), with one element for each possible hidden subgroup H , such that if we measure $|\psi_H\rangle$ with this POVM, then we are likely to get the correct outcome H . Choosing $m = O((\log |G|)^2)$ make the states “sufficiently orthogonal” for this idea to work (see Exercise 4). This POVM need not be efficiently implementable: circuits to implement it (using only a computational-basis measurement at the end) may require a number of elementary gates that's polynomial in $|G|$. But at least the number of times we need to *query* the function f is only polylogarithmic in $|G|$.

For those interested in more HSP results, a good source is Childs's lecture notes [82, Chapter 4–14].

Exercises

1. Show that the Deutsch-Jozsa problem for $n = 1$ (i.e., where $f : \{0, 1\} \rightarrow \{0, 1\}$) is an instance of the HSP. Explicitly say what G , f , H , and $H^\perp$ are, and how sampling from $H^\perp$ allows you to solve the problem.
2. Show that for the HSP corresponding to discrete log, we indeed have $H^\perp = \{\chi_{(c, -ac)} \mid c \in \mathbb{Z}_N\}$ as claimed near the end of Section 6.2.2.
3. This exercise explains Diffie-Hellman key exchange, which is secure under the assumption that the adversary cannot efficiently compute discrete logarithms. Alice and Bob choose a public key consisting of a large prime p (say, of 1000 or 2000 bits) and generator γ of the group $\mathbb{Z}_p^*$, which has size $\phi(p) = p - 1$. To agree on a shared secret key K , Alice chooses a uniformly random $a \in \{0, \dots, p - 2\}$ and sends Bob the group element $A = \gamma^a$; Bob chooses a uniformly random $b \in \{0, \dots, p - 2\}$ and sends Alice $B = \gamma^b$. Alice and Bob use $K = \gamma^{ab}$ as their secret key, which they can use for instance to encrypt messages using a one-time pad.
 - (a) Show that both Alice and Bob can efficiently compute K given the communication.

- (b) Show that an adversary who can efficiently compute discrete logarithms, can compute K from the public key and the communication tapped from the channel (i.e., A , B , p and γ , but not a and b).
4. Suppose we are given an unknown state $|\psi_i\rangle$ from a known set of K states $\{|\psi_j\rangle \mid j \in [K]\}$.
- (a) Suppose the states are pairwise orthogonal: $\langle\psi_j|\psi_k\rangle = \delta_{jk}$. Give a projective measurement that determines i with probability 1.
- (b) (H) Suppose the states are pairwise *almost* orthogonal: $|\langle\psi_j|\psi_k\rangle| \ll 1/K^2$ for all distinct $j, k \in [K]$. Define $E_i = \frac{2}{3}|\psi_i\rangle\langle\psi_i|$. Show that $I - \sum_{i=1}^K E_i$ is positive semidefinite.
- (c) Under the same assumption as (b), give a POVM that determines i with success probability at least $2/3$.
5. (H) Suppose we have an efficient algorithm to produce, from a given undirected n -vertex graph $\mathcal{G}$, the following n^2 -qubit state:

$$a_{\mathcal{G}} \sum_{\pi \in S_n} |\pi(\mathcal{G})\rangle,$$

where the basis states correspond to $n \times n$ adjacency matrices. Here $a_{\mathcal{G}}$ is a scalar that makes the norm equal to 1. Use this procedure to efficiently decide (with high success probability) whether two given graphs $\mathcal{G}_1$ and $\mathcal{G}_2$ are isomorphic or not.

Chapter 7

Grover's Search Algorithm

The second-most important quantum algorithm after Shor's is Grover's search algorithm [125]. It doesn't provide an exponential speed-up, only a quadratic speed-up, but it is much more widely applicable than Shor.

7.1 The problem

The search problem:

For $N = 2^n$, we are given an arbitrary $x \in \{0, 1\}^N$. The goal is to find an i such that $x_i = 1$ (and to output 'no solutions' if there are no such i). We denote the number of solutions in x by t (i.e., t is the Hamming weight of x).

This problem may be viewed as a simplification of the problem of searching an N -slot unordered database or search space, modeled by an N -bit string. Classically, a randomized algorithm would need $\Theta(N)$ queries to solve the search problem. Grover's algorithm solves it in $O(\sqrt{N})$ queries, and $O(\sqrt{N} \log N)$ other gates (the number of gates can be reduced a bit further, see Exercise 9).

7.2 Grover's algorithm

Let $O_{x,\pm}|i\rangle = (-1)^{x_i}|i\rangle$ denote the $\pm$ -type oracle for the input x (i.e., a phase-query), and R_0 be the unitary transformation that puts a -1 in front of all basis states $|i\rangle$ where $i \neq 0^n$, and that does nothing to the basis state $|0^n\rangle$.¹ The *Grover iterate* is

$$\mathcal{G} = H^{\otimes n} R_0 H^{\otimes n} O_{x,\pm}. \quad (7.1)$$

Note that 1 Grover iterate makes 1 query, and uses $O(\log N)$ other gates.

Grover's algorithm starts in the n -bit state $|0^n\rangle$, applies a Hadamard transformation to each qubit to get the uniform superposition $|U\rangle = \frac{1}{\sqrt{N}} \sum_i |i\rangle$ of all N indices, applies $\mathcal{G}$ to this state k times (for some k to be chosen later), and then measures the final state. Intuitively, what happens is that in each iteration some amplitude is moved from the indices of the 0-bits to the indices of the 1-bits. The algorithm stops when almost all of the amplitude is on the 1-bits, in which case a measurement of the final state will probably give the index of a 1-bit. Figure 7.1 illustrates this.

¹This unitary R_0 is independent of x , and can be implemented using $O(n)$ elementary gates (Exercise 2.8.a).

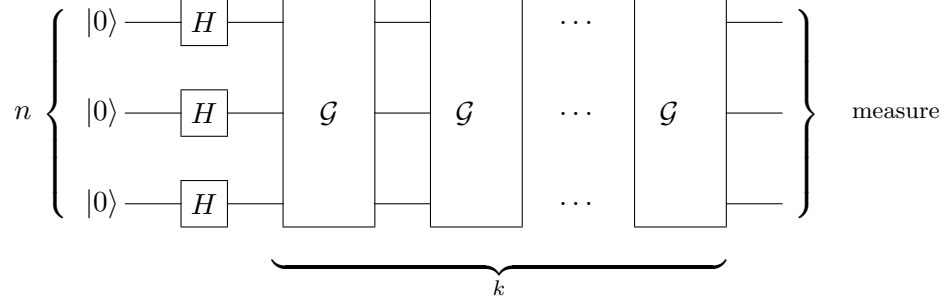


Figure 7.1: Grover's algorithm, with k Grover iterates

In order to analyze this, define the following “good” and “bad” states, corresponding to the solutions and non-solutions, respectively:

$$|G\rangle = \frac{1}{\sqrt{t}} \sum_{i:x_i=1} |i\rangle \text{ and } |B\rangle = \frac{1}{\sqrt{N-t}} \sum_{i:x_i=0} |i\rangle.$$

Then the uniform state over all indices can be written as

$$|U\rangle = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |i\rangle = \sin(\theta)|G\rangle + \cos(\theta)|B\rangle, \quad \text{for } \theta = \arcsin(\sqrt{t/N}).$$

The Grover iterate $\mathcal{G}$ is actually the product of two *reflections*.² Firstly, $O_{x,\pm}$ is a reflection through the subspace V spanned by the basis states that are not solutions; restricted to the 2-dimensional space spanned by $|G\rangle$ and $|B\rangle$ this is in fact just a reflection through the state $|B\rangle$. Secondly,

$$H^{\otimes n} R_0 H^{\otimes n} = H^{\otimes n} (2|0^n\rangle\langle 0^n| - I) H^{\otimes n} = 2H^{\otimes n} |0^n\rangle\langle 0^n| H^{\otimes n} - H^{\otimes n} I H^{\otimes n} = 2|U\rangle\langle U| - I$$

is a reflection through $|U\rangle$.

Here is Grover's algorithm restated, assuming we know the fraction of solutions is $\varepsilon = t/N$:

1. Set up the starting state $|U\rangle = H^{\otimes n}|0\rangle$
2. Repeat the following $k = O(1/\sqrt{\varepsilon})$ times:
 - (a) Reflect through $|B\rangle$ (i.e., apply $O_{x,\pm}$)
 - (b) Reflect through $|U\rangle$ (i.e., apply $H^{\otimes n} R_0 H^{\otimes n}$)
3. Measure the first register and check that the resulting i is a solution

²A reflection through a subspace V is a unitary A such that $Av = v$ for all vectors $v \in V$, and $Aw = -w$ for all w orthogonal to V . Note that we can write $A = 2P_V - I$, where P_V is the projector onto subspace V . If V is 1-dimensional, spanned by vector u , we also call this a reflection through u .

Geometric argument: There is a fairly simple geometric argument why the algorithm works. The analysis is in the 2-dimensional real plane spanned by $|B\rangle$ and $|G\rangle$. We start with

$$|U\rangle = \sin(\theta)|G\rangle + \cos(\theta)|B\rangle.$$

The two reflections (a) and (b) increase the angle from θ to 3θ , moving us towards the good state as illustrated in Figure 7.2.

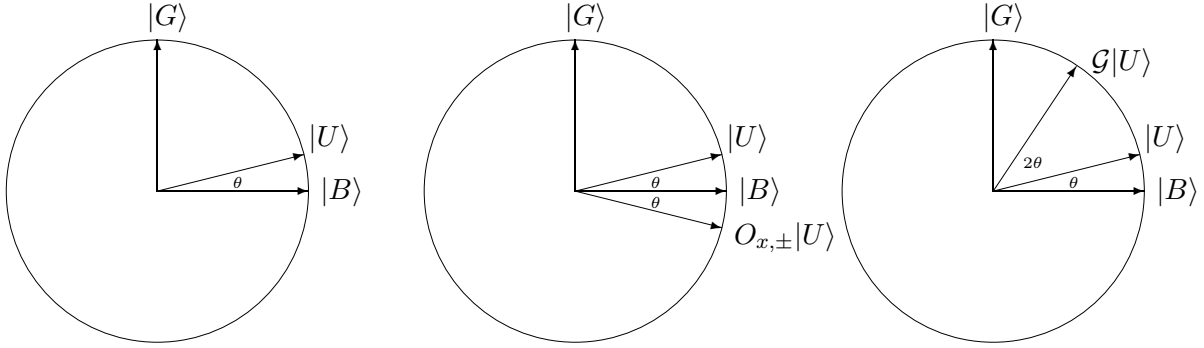


Figure 7.2: The first iteration of Grover: (picture on the left) start with $|U\rangle$; (middle) reflect through $|B\rangle$ to get $O_{x,\pm}|U\rangle$; (right) reflect through $|U\rangle$ to get $\mathcal{G}|U\rangle$

The next two reflections (a) and (b) increase the angle with another 2θ , etc. More generally, after k applications of (a) and (b) our state has become

$$\sin((2k+1)\theta)|G\rangle + \cos((2k+1)\theta)|B\rangle.$$

If we now measure, the probability of seeing a solution is $P_k = \sin^2((2k+1)\theta)$. We want P_k to be as close to 1 as possible. Note that if we can choose $\tilde{k} = \frac{\pi}{4\theta} - 1/2$, then $(2\tilde{k}+1)\theta = \pi/2$ and hence $P_{\tilde{k}} = \sin^2(\pi/2) = 1$. An example where this works is if $t = N/4$, for then $\theta = \pi/6$ and $\tilde{k} = 1$.

Unfortunately $\tilde{k} = \frac{\pi}{4\theta} - 1/2$ will usually not be an integer, and we can only do an integer number of Grover iterations. However, if we choose k to be the integer closest to $\tilde{k}$, then our final state will still be close to $|G\rangle$ and the failure probability will still be small (assuming $t \ll N$):

$$\begin{aligned} 1 - P_k &= \cos^2((2k+1)\theta) = \cos^2((2\tilde{k}+1)\theta + 2(k-\tilde{k})\theta) \\ &= \cos^2(\pi/2 + 2(k-\tilde{k})\theta) = \sin^2(2(k-\tilde{k})\theta) \leq \sin^2(\theta) = \frac{t}{N}, \end{aligned}$$

where we used $|k - \tilde{k}| \leq 1/2$. Since $\arcsin(\theta) \geq \theta$, the number of queries is $k \leq \frac{\pi}{4\theta} \leq \frac{\pi}{4} \sqrt{\frac{N}{t}}$.

Algebraic argument: For those who don't like geometry, here's an alternative (but equivalent) algebraic argument. Let a_k denote the amplitude of the indices of the t 1-bits after k Grover iterates, and b_k the amplitude of the indices of the 0-bits. Initially, for the uniform superposition $|U\rangle$ we have $a_0 = b_0 = 1/\sqrt{N}$. Using that $H^{\otimes n} R_0 H^{\otimes n} = \frac{2}{N} J - I$, where J is the $N \times N$ all-1s matrix, we find the following recursion:

$$\begin{aligned} a_{k+1} &= \frac{N-2t}{N} a_k + \frac{2(N-t)}{N} b_k \\ b_{k+1} &= \frac{-2t}{N} a_k + \frac{N-2t}{N} b_k \end{aligned}$$

The following formulas, due to Boyer et al. [62], provide a closed form for a_k and b_k (which may be verified by substituting them into the recursion). With $\theta = \arcsin(\sqrt{t/N})$ as before, define

$$\begin{aligned} a_k &= \frac{1}{\sqrt{t}} \sin((2k+1)\theta) \\ b_k &= \frac{1}{\sqrt{N-t}} \cos((2k+1)\theta) \end{aligned}$$

Accordingly, after k iterations the success probability (the sum of squares of the amplitudes of the locations of the t 1-bits) is the same as in the geometric analysis

$$P_k = t \cdot a_k^2 = (\sin((2k+1)\theta))^2.$$

Thus we have a bounded-error quantum search algorithm with $O(\sqrt{N/t})$ queries, *assuming we know t* . We now list (without full proofs) a number of useful variants of Grover:

- If we know t exactly, then the algorithm can be tweaked to end up in *exactly* the good state. Roughly speaking, you can make the angle θ slightly smaller, such that $\tilde{k} = \pi/4\theta - 1/2$ becomes an integer (see Exercise 5).
- If we do *not* know t , then there is a problem: we do not know which k to use, so we do not know when to stop doing the Grover iterates. Note that if k gets too big, the success probability $P_k = (\sin((2k+1)\theta))^2$ goes down again! However, a slightly more complicated algorithm due to [62] (basically running the above algorithm with exponentially increasing guesses for k) shows that an *expected* number of $O(\sqrt{N/t})$ queries still suffices to find a solution if there are t solutions. If there is no solution ($t = 0$), then we can easily detect that by checking x_i for the i that the algorithm outputs.
- If we know a lower bound τ on the actual (possibly unknown) number of solutions t , then the above algorithm uses an *expected* number of $O(\sqrt{N/\tau})$ queries. If we run this algorithm for up to three times its expected number of queries, then (by Markov's inequality) with probability at least $2/3$ it will have found a solution. This way we can turn an *expected* runtime into a *worst-case* runtime.
- If we do not know t but would like to reduce the probability of not finding a solution to some small $\varepsilon > 0$, then we can do this using $O(\sqrt{N \log(1/\varepsilon)})$ queries (see Exercise 6).
NB: The important part here is that the $\log(1/\varepsilon)$ is inside the square-root; usual error-reduction by $O(\log(1/\varepsilon))$ repetitions of basic Grover would give the worse upper bound of $O(\sqrt{N} \log(1/\varepsilon))$ queries.

7.3 Amplitude amplification

The analysis that worked for Grover's algorithm is actually much more generally applicable (we will also see it again in the next chapter). In this section we describe a very similar procedure that allows us to amplify the “good” part of the outcome of an algorithm. Quite abstractly, suppose we have a quantum circuit A (without measurements) that acts on m qubits, such that

$$A|0^m\rangle = \sqrt{p}|\psi_1\rangle + \sqrt{1-p}|\psi_0\rangle,$$

where $|\psi_1\rangle$ and $|\psi_0\rangle$ are normalized m -qubit states that are orthogonal to each other (it could for instance be that the last qubit of $|\psi_1\rangle$ is $|1\rangle$ and the last qubit of $|\psi_0\rangle$ is $|0\rangle$). For some reason we like the state $|\psi_1\rangle$ and we want to increase its “weight” $\sqrt{p}$ in the superposition. The following procedure achieves this.

In analogy with the analysis of Grover, think of $|\psi_1\rangle$ as the “good state” and $|\psi_0\rangle$ as the “bad state,” and view these two states as the vertical and horizontal axes in a 2-dimensional picture. Our starting state will be $|U\rangle = A|0^m\rangle$, which plays the role that the uniform state played in Grover, and which is of course easy to obtain by applying A once to basis state $|0^m\rangle$. The angle between $|U\rangle$ and the horizontal axis is $\theta = \arcsin \sqrt{p}$. We would like to rotate this initial state towards the good state, i.e., towards the vertical axis. As for Grover, we could implement the desired rotation as a product of two reflections: a reflection through the bad state and a reflection through $|U\rangle$.

For the first reflection, suppose we have a circuit R_G that can somehow distinguish the good state from the bad state by putting a “−” in front of $|\psi_1\rangle$ and leaving $|\psi_0\rangle$ alone. For example, if the last qubit of $|\psi_1\rangle$ is $|1\rangle$ and the last qubit of $|\psi_0\rangle$ is $|0\rangle$, then R_G would be extremely easy: it would just apply a Z -gate to the last qubit. The second reflection can be implemented as

$$AR_0A^{-1}.$$

It is easy to check that this maps the state $|U\rangle = A|0^m\rangle$ to itself, while every state orthogonal to $|U\rangle$ gets a “−” in front of it, so indeed this reflects through $|U\rangle$. Like before, the product of these two reflections corresponds to a rotation by an angle of 2θ in the 2-dimensional picture.

The following *amplitude amplification* procedure from [66] increases the amplitude of the good state to be close to 1:

1. Setup the starting state $|U\rangle = A|0^m\rangle$
2. Repeat the following $O(1/\sqrt{p})$ times:
 - (a) Reflect through the bad state $|\psi_0\rangle$ (i.e., apply R_G)
 - (b) Reflect through $|U\rangle$ (i.e., apply AR_0A^{-1})

The analysis is the same as for Grover: the initial angle between our algorithm’s state and the horizontal axis is θ and every iteration increases this angle by 2θ , so after k iterations our state is

$$\sin((2k+1)\theta)|\psi_1\rangle + \cos((2k+1)\theta)|\psi_0\rangle.$$

We would like to end up with angle $(2k+1)\theta \approx \pi/2$, because then the amplitude of the good state $|\psi_1\rangle$ would be close to $\sin(\pi/2) = 1$. Hence, like before, we choose k to be $\frac{\pi}{4\theta} - 1/2$ rounded to the nearest integer. This is $O(1/\sqrt{p})$. If we do not know in advance what p is, then we can try out exponentially decreasing guesses for its value, similar to how we handle the case of Grover with unknown number of solutions.

Note that the Hadamard transform $H^{\otimes n}$ can be viewed as an algorithm with success probability $p = t/N$ for a search problem of size N with t solutions, because $H^{\otimes n}|0^n\rangle$ is the uniform superposition over all N locations. Hence Grover’s algorithm is a special case of amplitude amplification, where $m = n$, $A = A^{-1} = H^{\otimes n}$, and R_G corresponds to a phase-query to x .

Amplitude amplification allows to speed up a very large class of classical algorithms: any algorithm A that has some non-trivial probability p of finding a solution to whatever problem we’re trying to solve, can be amplified to success probability nearly 1 by $O(1/\sqrt{p})$ runs of A and A^{-1} provided we can efficiently “recognize” solutions, i.e., implement R_G . In contrast, classically we would need to repeat A $O(1/p)$ times before we have success probability close to 1.

7.4 Application: satisfiability

Grover’s algorithm has many applications: basically any classical algorithm that has some search-component can be improved using Grover’s algorithm as a subroutine. This includes many basic computer applications such as finding shortest paths and minimum spanning trees, various other graph algorithms, etc.

We can also use it to speed up the computation of **NP**-complete problems (see Chapter 13 for the complexity class **NP**), albeit only quadratically, not exponentially. As an example, consider the satisfiability problem: we are given a Boolean formula $\phi(i_1, \dots, i_n)$ and want to know if it has a satisfying assignment, i.e., a setting of the bits $i_1, \dots, i_n$ that makes $\phi(i_1, \dots, i_n) = 1$. A classical brute-force search along all 2^n possible assignments takes time roughly 2^n .

To find a satisfying assignment faster, define the $N = 2^n$ -bit input to Grover’s algorithm by $x_i = \phi(i)$, where $i \in \{0, 1\}^n$. For a given assignment $i = i_1 \dots i_n$ it is easy to compute $\phi(i)$ classically in polynomial time. We can write that computation as a reversible circuit (using only Toffoli gates), corresponding to a unitary U_ϕ that maps $|i, 0, 0\rangle \mapsto |i, \phi(i), w_i\rangle$, where the third register holds some classical workspace the computation may have needed. To apply Grover we need an oracle that puts the answer in the phase and doesn’t leave workspace around (as that could mess up the interference effects, see Exercise 2.10 for an example). Define O_x as the unitary that first applies U_ϕ , then applies a Z -gate to the second register, and then applies U_ϕ^{-1} to “clean up” the workspace again. This has the form we need for Grover: $O_{x,\pm}|i\rangle = (-1)^{x_i}|i\rangle$; here we did not explicitly write the workspace qubits, which start and end in $|0\rangle$. Now we can run Grover and find a satisfying assignment with high probability if there is one, using a number of elementary operations that is $\sqrt{2^n}$ times some polynomial factor.

If brute-force search is basically the best thing we can do classically to solve some particular **NP**-hard problem, then that computation can be sped up quadratically on a quantum computer using Grover search like above. However, there are also **NP**-hard problems where we know algorithms that still run in exponential time, but that are much faster than brute-force search. For example, consider the famous Traveling Salesman Problem (TSP): given an n -vertex graph with weights (distances) on the edges, find the shortest tour in this graph that visits every node exactly once. Since there are $(n-1)!$ many different tours, classical brute-force search would take time $(n-1)!$, times some polynomial in n . Grover’s algorithm could speed this up quadratically. However, there are much more clever classical algorithms for TSP. In particular, the Bellman-Held-Karp dynamic programming algorithm solves TSP in time 2^n , times a polynomial in n . This algorithm is *much* faster than $O(\sqrt{n}!)$ (which is roughly $(n/e)^{n/2}$), and is not amenable to a straightforward speed-up using Grover. Nevertheless, it turns out quantum computers can still solve TSP polynomially faster than the best known classical algorithms, albeit in a much more complicated way than by just applying Grover [16].

Exercises

1. (a) Suppose $n = 2$, and $x = x_{00}x_{01}x_{10}x_{11} = 0001$. Give the specific initial state, three intermediate states, and final state in Grover’s algorithm, for $k = 1$ iterations (using the decomposition of one Grover iterate into a product of four unitaries from Eq. (7.1)). What is the success probability?
- (b) Give the final state after $k = 2$ iterations. What is now the success probability?

2. (a) Suppose you have a quantum algorithm for some computational problem that takes $\sqrt{N}$ operations on inputs of size N , each operation of constant cost C . And the best-possible classical algorithm for the same computational problem takes N operations, each of constant cost c . Suppose C is much larger than c (which is certainly the case in the current state of quantum technology: doing one elementary quantum gate is much more expensive than one doing classical logic gate). How large does the input-size N have to be before the quantum algorithm has lower cost than the best-possible classical algorithm?
- (b) Suppose you have a quantum algorithm of cost $C\sqrt{2^n}$ for satisfiability of n -variable Boolean formulas, where the best-possible classical algorithm has cost $c\sqrt{2^n}$, and again C is much larger than c . How large does n have to be before the quantum algorithm has lower cost than the best-possible classical algorithm?
3. Show that if the number of solutions is $t = N/4$, then Grover's algorithm always finds a solution with certainty after just one query. How many queries would a classical algorithm need to find a solution with certainty if $t = N/4$? And if we allow the classical algorithm error probability $1/10$?
4. Suppose we have a string of $N = 2^n$ bits, containing t ones (solutions) and $N - t$ zeroes. You may assume you know the number t .
 - (a) Show that we can use Grover's algorithm to find the positions of *all* t ones, using an expected number of $O(t\sqrt{N})$ queries. You can argue on a high level, no need to draw actual quantum circuits.
 - (b) (H) Show that this can be improved to an expected number of $O(\sqrt{tN})$ queries.
5. At the end of Section 7.2 we claimed without proof that Grover's algorithm can be tweaked to work *with probability 1* if we know the number of solutions exactly. For $N = 2^n$, this question asks you to provide such an exact algorithm for an $x \in \{0, 1\}^N$ with a unique solution (so we are promised that there is exactly one $i \in \{0, 1\}^n$ with $x_i = 1$, and our goal is to find this i).
 - (a) Give the success probability of the basic version of Grover's algorithm after k iterations.
 - (b) Suppose the optimal number of iterations $\tilde{k} = \frac{\pi}{4 \arcsin(1/\sqrt{N})} - \frac{1}{2}$ is not an integer. Show that if we round $\tilde{k}$ up to the nearest integer, doing $\lceil \tilde{k} \rceil$ iterations, then the algorithm will have success probability strictly less than 1.
 - (c) Define a new $2N$ -bit string $y \in \{0, 1\}^{2N}$, indexed by $(n+1)$ -bit strings $j = j_1 \dots j_n j_{n+1}$, by setting

$$y_j = \begin{cases} 1 & \text{if } x_{j_1 \dots j_n} = 1 \text{ and } j_{n+1} = 0, \\ 0 & \text{otherwise.} \end{cases}$$

Show how you can implement the following $(n+1)$ -qubit unitary

$$S_y : |j\rangle \mapsto (-1)^{y_j} |j\rangle,$$

using one query to x (of the usual form $O_x : |i, b\rangle \mapsto |i, b \oplus x_i\rangle$) and a few elementary gates.

- (d) Let $\gamma \in [0, 2\pi)$ and let $U_\gamma = \begin{pmatrix} \cos \gamma & -\sin \gamma \\ \sin \gamma & \cos \gamma \end{pmatrix}$ be the corresponding rotation matrix. Let $\mathcal{A} = H^{\otimes n} \otimes U_\gamma$ be an $(n+1)$ -qubit unitary. What is the probability (as a function of γ) that measuring the state $\mathcal{A}|0^{n+1}\rangle$ in the computational basis gives a solution $j \in \{0, 1\}^{n+1}$ for y (i.e., such that $y_j = 1$)?
- (e) (H) Give a quantum algorithm that finds the unique solution in string x with probability 1 using $O(\sqrt{N})$ queries to x .
6. Given query access to $x \in \{0, 1\}^N$, with unknown Hamming weight $t = |x|$, we want to find a solution, i.e., an index $i \in \{0, \dots, N-1\}$ such that $x_i = 1$. If $x = 0^N$ then our search algorithm should output “no solution.”
- (a) (H) Suppose we know an integer s such that $t \in \{1, \dots, s\}$. Give a quantum algorithm that finds a solution with probability 1, using $O(\sqrt{sN})$ queries to x .
- (b) Suppose we know that $t \in \{s+1, \dots, N\}$. Give a quantum algorithm that finds a solution with probability at least $1 - 2^{-s}$, using $O(\sqrt{sN})$ queries to x .
- (c) For given $\varepsilon > 2^{-N}$, give a quantum algorithm that solves the search problem with probability $\geq 1 - \varepsilon$ using $O(\sqrt{N \log(1/\varepsilon)})$ queries, without assuming anything about t .
7. (H) Here we will *approximately count* the number of 1s in a string $x \in \{0, 1\}^N$. Let $t = |x|$ denote that (unknown) number.
- (a) Given an integer $m \in \{1, \dots, N\}$, describe a quantum algorithm that makes $O(\sqrt{N/m})$ queries to x and decides between the cases $t \leq m/2$ and $t \in [m, 2m]$ with probability at least $2/3$. That is, the algorithm has to output 0 with probability $\geq 2/3$ whenever $t \leq m/2$, has to output 1 with probability $\geq 2/3$ whenever $t \in [m, 2m]$, and can output whatever it wants for other values of t .
- (b) Give a quantum algorithm that uses $O(\sqrt{N} \log \log N)$ queries to x and that outputs an integer m such that, with probability $\geq 2/3$, the unknown t lies between $m/2$ and $2m$.
8. Suppose we have a quantum circuit A acting on m qubits, such that

$$A|0^m\rangle = \sqrt{a}|\phi_1\rangle|1\rangle + \sqrt{1-a}|\phi_0\rangle|0\rangle,$$

where $|\phi_1\rangle$ and $|\phi_0\rangle$ are arbitrary normalized $(m-1)$ -qubit states, and $a \in [0, 1/4]$ is an unknown number. Our goal is to estimate a (this is known as *amplitude estimation*). Let $\mathcal{S}$ be the 2-dimensional subspace spanned by $|\phi_1\rangle|1\rangle$ and $|\phi_0\rangle|0\rangle$.

- (a) Show that in $\mathcal{S}$, the unitary $I \otimes Z$ (where I is the identity on $m-1$ qubits and Z is the phase-flip gate) is a reflection through $|\phi_0\rangle|0\rangle$.
- (b) Let $R_0 = 2|0^m\rangle\langle 0^m| - I$ be a reflection through $|0^m\rangle$. Show that in $\mathcal{S}$, the unitary AR_0A^{-1} is a reflection through $A|0^m\rangle$.
- (c) Show that in $\mathcal{S}$, the unitary $U = AR_0A^{-1} \cdot (I \otimes Z)$ is a rotation over angle 2θ , where $\theta = \arcsin \sqrt{a}$.
- (d) (H) Given some $\varepsilon \in (0, 1/2)$, show how you can use phase estimation (Section 4.6) with $O(1/\varepsilon)$ applications of U to find an approximation $\tilde{a}$ of a such that $|\sqrt{\tilde{a}} - \sqrt{a}| \leq \varepsilon$.

- (e) (H) Suppose we have query-access to a string $x \in \{0, 1\}^N$ of unknown Hamming weight $t = |x|$. Use (d) to compute an integer $\tilde{t}$ such that $|\tilde{t} - t| \leq \sqrt{N}$ with success probability $\geq 2/3$, using $O(\sqrt{N})$ queries to x .
- (f) Suppose $x \in \{0, 1\}^N$ has $|x| \in \{0, 1\}$. Use (d) to compute $|x|$ with success probability $\geq 2/3$, using $O(\sqrt{N})$ queries to x (you may *not* invoke Grover here).
9. Suppose you are given query access to $x \in \{0, 1\}^N$, where $|x| = 1$ and $N = 2^n$. You want to find the unique index $i = i_{n-1} \dots i_0 \in \{0, 1\}^n$ such that $x_i = 1$.
- (a) Let $k \in \{1, \dots, n-1\}$. Fix the first $n-k$ bits of the n -bit index $i_{n-1} \dots i_0$ to specific values $i_{n-1}^* \dots i_k^*$. Give a quantum algorithm to find a solution with success probability 1 (if one exists) among the 2^k indices $i \in \{0, 1\}^n$ that start with $i_{n-1}^* \dots i_k^*$, using $O(\sqrt{2^k})$ queries and $O(\sqrt{2^k} k)$ other gates.
- (b) Give a quantum algorithm that solves the search problem on x using $O(\sqrt{N})$ queries and $O(\sqrt{N} \log \log N)$ other gates.
 Comment: The $O(\sqrt{N} \log \log N)$ gates achieved here is better than standard Grover, which uses $O(\sqrt{N} \log N)$ gates. The $\log \log N$ can be reduced a bit further, to nearly-constant [28].
10. (H) Let $x = x_0 \dots x_{N-1}$ be a sequence of distinct real numbers, where $N = 2^n$, and each x_i can be written exactly using b bits. We can query these in the usual way, i.e., we can apply $(n+b)$ -qubit unitary $O_x : |i, 0^b\rangle \mapsto |i, x_i\rangle$, as well as its inverse. The *minimum* of x is defined as $\min\{x_i \mid i \in \{0, \dots, N-1\}\}$. Give a quantum algorithm that finds (with probability $\geq 2/3$) an index achieving the minimum, using at most $O(\sqrt{N} \log N)$ queries to the input, and prove that this algorithm works.
 Bonus: give a quantum algorithm that uses $O(\sqrt{N})$ queries.
11. Let $x = x_0 \dots x_{N-1}$, where $N = 2^n$ and $x_i \in \{0, 1\}^n$, be an input that we can query in the usual way. We are promised that this input is 2-to-1: for each i there is exactly one other j such that $x_i = x_j$.³ Such an (i, j) -pair is called a *collision*.
- (a) Suppose S is a uniformly randomly chosen set of $s \leq N/2$ elements of $\{0, \dots, N-1\}$. What is the probability that there exists a collision in S ?
- (b) (H) Give a classical randomized algorithm that finds a collision (with probability $\geq 2/3$) using $O(\sqrt{N})$ queries to x .
- (c) (H) Give a quantum algorithm that finds a collision (with probability $\geq 2/3$) using $O(N^{1/3})$ queries.
12. Consider an undirected graph $G = (V, E)$, with vertex set $V = \{1, \dots, n\}$ and edge-set E . We say G is *connected* if, for every pair of vertices $i, j \in V$, there is a path between i and j in the graph. The *adjacency matrix* of G is the $n \times n$ Boolean matrix M where $M_{ij} = 1$ iff $(i, j) \in E$ (note that M is a symmetric matrix because G is undirected). Suppose we are given input graph G in the form of a unitary that allows us to query whether an edge (i, j) is present in G or not:

$$O_M : |i, j, b\rangle \mapsto |i, j, b \oplus M_{ij}\rangle.$$

³The 2-to-1 inputs for Simon's algorithm are a very special case of this, where the collisions are determined by a secret string $s \in \{0, 1\}^n$: x_i equals x_j if $i = j \oplus s$.

- (a) Assume G is connected. Suppose we have a set A of edges which we already know to be in the graph (so $A \subseteq E$; you can think of A as given classically, you don't have to query it). Let $G_A = (V, A)$ be the subgraph induced by only these edges, and suppose G_A is not connected, so it consists of $c > 1$ connected components. Call an edge $(i, j) \in E$ "good" if it connects two of these components. Give a quantum algorithm that finds a good edge with an *expected* number of $O(n/\sqrt{c-1})$ queries to M .
- (b) Give a quantum algorithm that uses at most $O(n^{3/2})$ queries to M and decides (with success probability at least $2/3$) whether G is connected or not.
- (c) Show that classical algorithms for deciding (with success probability at least $2/3$) whether G is connected, need to make $\Omega(n^2)$ queries to M .

Chapter 8

Quantum Walk Algorithms

8.1 Classical random walks

Consider an undirected graph G with N vertices. Suppose at least an ε -fraction of the vertices are “marked,” and we would like to find a marked vertex. One way to do this is with a *random walk*:

Start at some specific vertex y of the graph.

Repeat the following a number of times: Check if y is marked, and if not then choose one of its neighbors at random and set y to be that neighbor.

This may seem like a stupid algorithm, but it has certain advantages. For instance, it only needs space $O(\log N)$, because you only need to keep track of the current vertex y , and maybe a counter that keeps track of how many steps you’ve already taken.¹ Such an algorithm can for example decide whether there is a path from a specific vertex y to a specific vertex x using $O(\log N)$ space. We’d start the walk at y and only x would be marked; one can show that if there exists a path from y to x in G , then we will reach x in $\text{poly}(N)$ steps.

Let us restrict attention to d -regular graphs without self-loops, so each vertex has exactly d neighbors. A random walk on such a graph G corresponds to an $N \times N$ symmetric matrix P , where $P_{x,y} = 1/d$ if (x,y) is an edge in G , and $P_{x,y} = 0$ otherwise. This P is the normalized adjacency matrix of G . If $v \in \mathbb{R}^N$ is a vector with a 1 at position y and 0s elsewhere, then Pv is a vector whose x -th entry is $(Pv)_x = 1/d$ if (x,y) is an edge, and $(Pv)_x = 0$ otherwise. In other words, Pv is the uniform probability distribution over the neighbors of y , which is what you get by taking one step of the random walk starting at y . More generally, if v is a probability distribution on the vertices, then Pv is the new probability distribution on vertices after taking one step of the random walk, and $P^k v$ is the probability distribution after taking k steps.

Suppose we start with some probability-distribution vector v (which may or may not be concentrated at one vertex y). We will assume G is connected and not bipartite. Then $P^k v$ will converge to the uniform distribution over all vertices, and the speed of convergence is determined by the “gap” between the first eigenvalue of P and all other eigenvalues. This can be seen as follows. Let $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_N$ be the eigenvalues of P , ordered by size, and $v_1, \dots, v_N$ be corresponding orthogonal eigenvectors.² The largest eigenvalue is $\lambda_1 = 1$, and corresponds to the

¹Here we’re assuming the neighbors of a given vertex are efficiently computable, so you don’t actually need to keep the whole graph in memory. This will be true for all graphs we consider here.

²Analyzing graphs by looking at the eigenvalues and eigenvectors of their adjacency matrix is called “algebraic graph theory” or “spectral graph theory,” see for instance [70].

eigenvector $v_1 = u = (1/N)$ which is the uniform distribution over all vertices. One can show that our assumption that G is connected implies $\lambda_2 < 1$, and our assumption that G is not bipartite implies $\lambda_N > -1$. Hence all eigenvalues λ_i for $i \in \{2, \dots, N\}$ will be in $(-1, 1)$; the corresponding eigenvector v_i will be orthogonal to the uniform vector u , so the sum of its entries is 0. Let $\delta > 0$ be the difference between $\lambda_1 = 1$ and $\max_{i \geq 2} |\lambda_i|$ (hence $|\lambda_i| \leq 1 - \delta$ for all $i \geq 2$). This δ is called the “spectral gap” of the graph.

Now decompose the starting distribution v as $v = \sum_{i=1}^N \alpha_i v_i$. Since the sum of v ’s entries is 1, and the sum of v_1 ’s entries is 1, while each other eigenvector v_i ($i \geq 2$) has entries summing to 0, it follows that $\alpha_1 = 1$. Now let us see what happens if we apply the random walk for k steps, starting from v :

$$P^k v = P^k \left(\sum_i \alpha_i v_i \right) = \sum_i \alpha_i \lambda_i^k v_i = u + \sum_{i \geq 2} \alpha_i \lambda_i^k v_i.$$

Consider the squared norm of the difference between $P^k v$ and u :

$$\|P^k v - u\|^2 = \left\| \sum_{i \geq 2} \alpha_i \lambda_i^k v_i \right\|^2 = \sum_{i \geq 2} |\alpha_i|^2 |\lambda_i|^{2k} \leq \|v\|^2 (1 - \delta)^{2k}.$$

Since v is a probability distribution, we have $\|v\|^2 \leq 1$. By choosing $k = \ln(1/\eta)/\delta$, we get $\|P^k v - u\| \leq \eta$. In particular, if δ is not too small, then we get quick convergence of the random walk to the uniform distribution u , no matter which distribution v we started with.³ Once we are close to the uniform distribution, we have probability roughly ε of hitting a marked vertex. Of course, the same happens if we just pick a vertex uniformly at random, but that may not always be an option if the graph is given implicitly.

Suppose it costs S to set up an initial state v ; it costs U to update the current vertex, i.e., to perform one step of the random walk; and it costs C to check whether a given vertex is marked. “Cost” is left undefined for now, but typically it will count number of queries to some input, or number of elementary operations. Consider a classical search algorithm that starts at v , and then repeats the following until it finds a marked vertex: check if the current vertex is marked, and if not run a random walk for roughly $1/\delta$ steps to get close to the uniform distribution. Ignoring constant factors, the expected cost before this procedure finds a marked item, is on the order of

$$S + \frac{1}{\varepsilon} \left(C + \frac{1}{\delta} U \right). \quad (8.1)$$

8.2 Quantum walks

We will now modify the classical random walk algorithm preceding Eq. (8.1) to a quantum algorithm, where the distribution-preserving matrix P is changed to a norm-preserving matrix $W(P)$ (i.e., a unitary). This is due to Magniez et al. [182], inspired by Szegedy [233]; our presentation is mostly based on Santha’s survey paper [216], to which we refer for more details and references.

While the basis state of a classical random walk is the current vertex we are at, a basis state of a quantum walk has *two* registers, the first corresponding to the *current* vertex and the second

³Convergence in total variation distance can be derived from this by Cauchy-Schwarz, choosing $\eta \ll 1/\sqrt{N}$.

corresponding to the *previous* vertex. Equivalently, a basis state of a quantum walk corresponds to an *edge* of the graph.

Our resulting quantum walk algorithm for search will actually be quite analogous to Grover's algorithm. We'll call a basis state $|x\rangle|y\rangle$ "good" if x is a marked vertex, and "bad" otherwise. Define $|p_x\rangle = \sum_y \sqrt{P_{xy}}|y\rangle$ to be the uniform superposition over the neighbors of x . As for Grover, define "good" and "bad" states as the superpositions over good and bad basis states:

$$|G\rangle = \frac{1}{\sqrt{|M|}} \sum_{x \in M} |x\rangle|p_x\rangle \text{ and } |B\rangle = \frac{1}{\sqrt{N - |M|}} \sum_{x \notin M} |x\rangle|p_x\rangle,$$

where M denotes the set of marked vertices. Note that $|G\rangle$ is just the uniform superposition over all edges (x, y) where the first coordinate is marked, and $|B\rangle$ is just the uniform superposition over all edges (x, y) where the first coordinate is not marked.

If $\varepsilon = |M|/N$ and $\theta := \arcsin(\sqrt{\varepsilon})$ then the uniform state over all edges can be written as

$$|U\rangle = \frac{1}{\sqrt{N}} \sum_x |x\rangle|p_x\rangle = \sin(\theta)|G\rangle + \cos(\theta)|B\rangle.$$

Here is the algorithm for searching a marked vertex if an ε -fraction is marked⁴:

1. Setup the starting state $|U\rangle$
2. Repeat the following $O(1/\sqrt{\varepsilon})$ times:
 - (a) Reflect through $|B\rangle$
 - (b) Reflect through $|U\rangle$
3. Measure the first register and check that the resulting vertex x is marked.

The description and analysis of this algorithm takes places in the 2-dimensional space spanned by $|G\rangle$ and $|B\rangle$. We'll explain in a moment how to implement (a) and (b). Assuming we know how to do that, the proof that this algorithm finds a marked vertex is the same as for Grover and for amplitude amplification (Chapter 7). We start with $|U\rangle = \sin(\theta)|G\rangle + \cos(\theta)|B\rangle$. The two reflections (a) and (b) increase the angle from θ to 3θ , moving us towards the good state (similarly to the analysis for Grover, you can draw a 2-dimensional picture with axes $|B\rangle$ and $|G\rangle$ to see this). More generally, after k applications of (a) and (b) our state has become

$$\sin((2k+1)\theta)|G\rangle + \cos((2k+1)\theta)|B\rangle.$$

Choosing $k \approx \frac{\pi}{4\theta} = O(1/\sqrt{\varepsilon})$, we will have $\sin((2k+1)\theta) \approx 1$, at which point measuring the first register will probably yield a vertex x that is marked. We now look more closely how to implement the two kinds of reflections.

(a) Reflect through $|B\rangle$. Reflecting through $|B\rangle$ is relatively straightforward: we just have to "recognize" whether the first register contains a marked x , and put a -1 if so (note that this is really a reflection through the subspace spanned by the bad basis states, but restricted to the 2-dimensional subspace spanned by $|G\rangle$ and $|B\rangle$ that's the same as a reflection through $|B\rangle$).

⁴As in Grover, if we don't know ε then we just run the algorithm repeatedly with exponentially decreasing guesses for ε ($1/2, 1/4, 1/8, \dots$). If at the end we still haven't found a marked item, we'll conclude that probably none exists.

(b) Reflect through $|U\rangle$. This is where the quantum walk comes in. Let $\mathcal{A}$ be the subspace $\text{span}\{|x\rangle|p_x\rangle\}$ and $\mathcal{B}$ be $\text{span}\{|p_y\rangle|y\rangle\}$. Let $\text{ref}(\mathcal{A})$ and $\text{ref}(\mathcal{B})$ denote reflections through $\mathcal{A}$ and $\mathcal{B}$, respectively. Define $W(P) = \text{ref}(\mathcal{B})\text{ref}(\mathcal{A})$ to be the product of these two reflections. This is the unitary analogue of P . Suppose we are able to implement the following two operations (even in a controlled manner):

$$\begin{aligned} (1) \quad & |x\rangle|0\rangle \mapsto |x\rangle|p_x\rangle \\ (2) \quad & |0\rangle|y\rangle \mapsto |p_y\rangle|y\rangle \end{aligned}$$

Since (1) and (2) prepare a uniform superposition over the neighbors of x and y , respectively, one can think of them as taking one classical walk step “in superposition.” Note that $\text{ref}(\mathcal{A})$ can be implemented by applying the inverse of (1), putting a minus if the second register is not $|0\rangle$, and applying (1). We can similarly implement $\text{ref}(\mathcal{B})$ using (2) and its inverse. Hence we can think of $W(P) = \text{ref}(\mathcal{B})\text{ref}(\mathcal{A})$ as corresponding to *four* steps of the classical walk in superposition.

To see how to implement the reflection through $|U\rangle$, let us consider the eigenvalues and eigenvectors of $W(P)$. The eigenvalues of $W(P)$ can be related to the eigenvalues $\lambda_1, \lambda_2, \dots$ of P as follows. Let $\theta_j \in [0, \pi/2]$ be such that $|\lambda_j| = \cos(\theta_j)$. We won’t prove it here, but it turns out that the eigenvalues of $W(P)$ are of the form $e^{\pm 2i\theta_j}$. The state $|U\rangle$ is an eigenvalue-1 eigenvector of $W(P)$, corresponding to $\theta_1 = 0$. The spectral gap of P is δ , so all eigenvectors of $W(P)$ that do not have eigenvalue 1, have eigenvalue $e^{\pm 2i\theta_j}$ where $\theta_j \geq \sqrt{2\delta}$, because $1 - \delta \geq |\lambda_j| = \cos(\theta_j) \geq 1 - \theta_j^2/2$. We now want to implement a reflection $R(P)$ through the subspace spanned by the eigenvalue-1 eigenvectors of $W(P)$; restricted to the 2-dimensional subspace spanned by $|G\rangle$ and $|B\rangle$ this will be the desired reflection through $|U\rangle$.

We will implement $R(P)$ by using *phase estimation* (see Section 4.6) with precision $\sqrt{\delta}/2$ on $W(P)$ to distinguish the eigenvalue-1 eigenvectors from the other eigenvectors. This precision requires $O(1/\sqrt{\delta})$ applications of $W(P)$, and $O(\log(1/\delta))$ auxiliary qubits that start in $|0\rangle$ and where the estimate will be stored. Assume for simplicity that phase estimation always gives an estimate $\tilde{\theta}_j$ of θ_j that is within precision $\sqrt{\delta}/2$.⁵ Because the nonzero θ_j are at least $\sqrt{2\delta}$, approximating them within additive error $\sqrt{\delta}/2$ is good enough to determine whether the actual value θ_j itself is 0 or not. We then multiply the state with a -1 if the estimate is sufficiently far from 0, and finally reverse the phase estimation to put the auxiliary qubits back to 0. Applied to some eigenvector $|w\rangle$ of $W(P)$ with corresponding eigenvalue $e^{\pm 2i\theta_j}$, $R(P)$ maps

$$R(P) : |w\rangle|0\rangle \xrightarrow{\text{PE}} |w\rangle|\tilde{\theta}_j\rangle \mapsto (-1)^{[\theta_j \neq 0]} |w\rangle|\tilde{\theta}_j\rangle \xrightarrow{\text{PE}^{-1}} (-1)^{[\theta_j \neq 0]} |w\rangle|0\rangle.$$

This has the desired effect: ignoring the auxiliary qubits (which start and end in 0), $R(P)$ maps eigenvalue-1 eigenvectors of $W(P)$ to themselves, and puts a -1 in front of the other eigenvectors.

Now that we know how to implement the algorithm, let us look at its complexity. Consider the following setup, update, and checking costs:

- Setup cost $\mathcal{S}$: the cost of constructing $|U\rangle$
- Checking cost $\mathcal{C}$: the cost of the unitary map $|x\rangle|y\rangle \mapsto m_x|x\rangle|y\rangle$, where $m_x = -1$ if x is marked, and $m_x = 1$ otherwise

⁵Phase estimation will actually give a superposition over estimates $\tilde{\theta}_j$, with small but nonzero amplitudes on bad estimates, but we’ll skip the technical details that are needed to deal with this.

- Update cost U : the cost of one step of the quantum walk, i.e., of $W(P)$

The cost of part (a) of the algorithm is C . Since $R(P)$ uses $O(1/\sqrt{\delta})$ applications of $W(P)$, and a few other gates, the cost of part (b) of the algorithm is essentially $O(U/\sqrt{\delta})$. Ignoring constant factors, the total cost of the algorithm is then

$$S + \frac{1}{\sqrt{\varepsilon}} \left(C + \frac{1}{\sqrt{\delta}} U \right). \quad (8.2)$$

Compare this with the classical cost of Eq. (8.1): quantum search square-roots both ε and δ .

8.3 Applications

There are a number of interesting quantum walk algorithms that beat the best classical algorithms. We'll give three examples here. More can be found in [216].

8.3.1 Grover search

Let us first derive a quantum algorithm for search. Suppose we have an N -bit string x of weight t , and we know $t/N \geq \varepsilon$. Consider the complete graph G on N vertices. Then the matrix P for the random walk on G has 0s on its diagonal, and its off-diagonal entries are all equal to $1/(N-1)$. This can be written as $P = \frac{1}{N-1}J - \frac{1}{N-1}I$, where J is the all-1 matrix and I is the identity. The only nonzero eigenvalues of J is N , and adding a multiple of I just shifts the eigenvalues of a matrix, hence the largest eigenvalue of P is $\lambda_1 = N/(N-1) - 1/(N-1) = 1$ (corresponding to the uniform vector) and all its other eigenvalues are $-1/(N-1)$. Note that the spectral gap δ is very large here: $\delta = 1 - 1/(N-1) \approx 1$. We'll mark a vertex i iff $x_i = 1$. Then, measuring cost by number of queries, a quantum walk on G will have $S = U = 0$ and $C = 1$. Plugging this into Eq. (8.2), the quantum walk will find a marked vertex (with high probability) using $O(1/\sqrt{\varepsilon})$ queries. The worst case is $\varepsilon = 1/N$, in which case we'll use $O(\sqrt{N})$ queries. Not surprisingly, we've essentially rederived Grover's algorithm.

8.3.2 Collision problem

Consider the following collision problem:

Input: $x = x_0, \dots, x_{n-1}$, where each x_i is an integer.⁶

Goal: find distinct i and j such that $x_i = x_j$ if these exist, otherwise output "all elements are distinct."

The decision version of this problem (deciding if there exists at least one collision) is also known as *element distinctness*.

Consider the graph whose vertices correspond to the sets $R \subseteq \{0, \dots, n-1\}$ of r elements. The total number of vertices is $N = \binom{n}{r}$. We'll put an edge between the vertices for R and R' iff these two sets differ in exactly two elements; in other words, you can get from R to R' by removing one element i from R and replacing it by a new element j . The resulting graph $J(n, r)$ is known as the *Johnson graph*. It is $r(n-r)$ -regular, since every R has $r(n-r)$ different neighbors R' . Its

⁶Say, all $x_i \leq n^2$ to avoid having to use too much space to store these numbers.

spectral gap is known to be $\delta = \frac{n}{r(n-r)}$ [70, Sec. 12.3.2]; we won't prove that here, just note that if $r \ll n$, then $\delta \approx 1/r$. For each set R we also keep track of the corresponding sequence of x -values, $x_R = (x_i)_{i \in R}$. Hence the full "name" of a vertex is the pair (R, x_R) .

We'll call a vertex in $J(n, r)$ *marked* if it contains a collision, i.e., the corresponding set R contains distinct i, j such that $x_i = x_j$. In the worst case there is exactly one colliding pair i, j (more collisions only make the problem easier). The probability that i and j are both in a random r -set R , is $\varepsilon = \frac{r}{n} \frac{r-1}{n-1}$. Hence the fraction of marked vertices is at least $\varepsilon \approx (r/n)^2$.

We will now determine the setup, checking, and update costs. The setup cost (measured in terms of queries) is $S = r + 1$: we have to create a uniform superposition $|U\rangle$ over all edges R, R' , and for each such basis state query all $r + 1$ elements of $R \cup R'$ to add the information x_R and $x_{R'}$. Checking whether a given vertex R, x_R contains a collision doesn't take any queries because we already have x_R , hence $C = 0$. To determine the update cost, note that mapping the second register of $|R, x_R\rangle|0\rangle$ to a superposition of all neighbors $R', x_{R'}$ requires querying (in superposition for all neighbors R') the value x_j of the element j that was added to get R' . Hence $U = O(1)$. Plugging this into Eq. (8.2), the cost of a quantum walk algorithm for collision-finding is

$$S + \frac{1}{\sqrt{\varepsilon}} \left(C + \frac{1}{\sqrt{\delta}} U \right) = O(r + n/\sqrt{r}).$$

This cost is $O(n^{2/3})$ if we choose to set $r = n^{2/3}$ (rounded to an integer). What the quantum walk produces at the end is a superposition where, if we measure the first register, with high probability we'll see a marked vertex. That way we obtain a set R that contains a collision; and because R is small, we can now cheaply find the colliding indices $i, j \in R$.

This query complexity $O(n^{2/3})$ turns out to be the optimal quantum query complexity for the collision problem [4]. By some more work involving efficient data structures, using a quantum-accessible classical RAM, the *time* complexity (= total number of elementary quantum gates plus total number of queries) can be brought down to $n^{2/3}(\log n)^{O(1)}$ [14].

8.3.3 Finding a triangle in a graph

Consider the following triangle-finding problem:

Input: the adjacency matrix of a graph H on n vertices.

Goal: find vertices u, v, w that form a triangle (i.e., $(u, v), (v, w), (w, u)$ are all edges in the graph), if they exist.

We'll assume we have query access to the entries of the adjacency matrix of H , which tells us whether (u, v) is an edge or not. There are $\binom{n}{2}$ bits in this oracle, one for each potential edge of H . It is not hard to see that a classical algorithm needs $\Omega(n^2)$ queries before it can decide with good probability whether a graph contains a triangle or not. For example, take a bipartite graph consisting of 2 sets of $n/2$ vertices each, such that any pair of vertices from different sets is connected by an edge. Such a graph is triangle-free, but adding any one edge will create a triangle. A classical algorithm would have to query all those edges separately.

Let us try a quantum walk approach. Again consider the Johnson graph $J(n, r)$. Each vertex will correspond to a set $R \subseteq \{0, \dots, n-1\}$ of r vertices, annotated with the result of querying all possible $\binom{r}{2}$ edges having both endpoints in R . We will call the vertex for set R *marked* if it contains *one edge* of a triangle. If there is at least one triangle in the graph, then the fraction of marked vertices is at least $\varepsilon \approx (r/n)^2$.

The setup cost will be $\mathbf{S} = \binom{r+1}{2}$: for an edge (R, R') of the Johnson graph we query the $\binom{r+1}{2}$ possible edges induced by the $r+1$ H -vertices of $R \cup R'$. The update cost will be $\mathbf{U} = 2r - 2$, because if we remove one vertex i from R then we have to remove information about $r-1$ edges in H , and if we add a new j to R we have to query $r-1$ new edges in H .

Getting a good upper bound for the checking cost $\mathbf{C}$ requires some more work—namely Grover search plus another quantum walk! Suppose we are given a set R of r vertices. How do we decide whether R contains an edge of a triangle? If we can efficiently decide, for a given u and R , whether R contains vertices v, w such that u, v, w form a triangle in H , then we could combine this with a Grover search over all n possible vertices u of H . Given u and R , let us design a subroutine based on another quantum walk, this time on the Johnson graph $J(r, r^{2/3})$. Each vertex of this Johnson graph corresponds to a subset $R' \subseteq R$ of $r' = r^{2/3}$ vertices. Its spectral gap is $\delta' = r/r'(r - r') \approx 1/r^{2/3}$. We'll mark R' if it contains vertices v, w such that u, v, w form a triangle. If there is at least one triangle involving u and some $v, w \in R$, then the fraction of marked vertices R' in $J(r, r^{2/3})$ is at least $\varepsilon' \approx (r'/r)^2 = 1/r^{2/3}$. For this subroutine, the setup cost is $O(r^{2/3})$ (for each $v \in R$, query whether (u, v) is an edge in H); the update cost is $O(1)$ (if we replace v in R by w , then we need to “unquery” edge (u, v) and query edge (u, w)); and the checking cost is 0. Plugging this into Eq. (8.2), we can decide whether a fixed u forms a triangle with two vertices in R , using $O(r^{2/3})$ queries. Let's ignore the small error probability of the latter subroutine (it can be dealt with, but that's technical). Then we can combine it with Grover search over all n vertices u to get checking cost $\mathbf{C} = O(\sqrt{nr^{2/3}})$.

Plugging these $\mathbf{S}$, $\mathbf{U}$, and $\mathbf{C}$ into Eq. (8.2), the overall cost of a quantum walk algorithm for triangle-finding is

$$\mathbf{S} + \frac{1}{\sqrt{\varepsilon}} \left(\mathbf{C} + \frac{1}{\sqrt{\delta}} \mathbf{U} \right) = O \left(r^2 + \frac{n}{r} (\sqrt{nr^{2/3}} + r^{3/2}) \right).$$

This is $O(n^{13/10})$ if we set $r = n^{3/5}$ [183]. The quantum walk algorithm ends with a superposition where most of the amplitude sits on sets R containing one edge of a triangle (i.e., two vertices of H that are part of a triangle). Now a measurement of that final state gives us such a set R with high probability, and then it's relatively cheap to find the third vertex of the triangle by another Grover search over the $n - r$ vertices of H that are not in R .

The exponent 13/10 can be slightly improved further [41, 163, 145], and the current best exponent is 5/4 [162]. It is an open question what the optimal quantum query complexity for triangle-finding is; the best lower bound is only $\Omega(n)$. Also, the optimal quantum *time* complexity of this problem is still wide open.

Exercises

1. Let $d < n$, and P be the projector on a d -dimensional subspace $V \subseteq \mathbb{R}^n$ that is spanned by orthonormal vectors $v_1, \dots, v_d$. This means that $Pv = v$ for all $v \in V$, and $Pw = 0$ for all w that are orthogonal to V .
 - (a) Show that P can be written in Dirac notation as $P = \sum_{i=1}^d |v_i\rangle\langle v_i|$.
 - (b) Show that $R = 2P - I$ is a reflection through the subspace corresponding to P , i.e., $Rv = v$ for all v in the subspace, and $Rw = -w$ for all w that are orthogonal to the subspace.

2. Let G be a d -regular graph that is bipartite, so its vertex set $V = [N]$ can be partitioned into disjoint sets A and B , and all its edges are in $A \times B$. Give an eigenvector with eigenvalue 1 of the associated $N \times N$ normalized adjacency matrix P , and another eigenvector with eigenvalue -1 .
3. This exercise is about obtaining a quantum algorithm for the collision problem with a slightly different quantum walk. Consider the problem of Section 8.3.2: we can query elements of the sequence of integers $x_0, \dots, x_{n-1}$, and want to find distinct i and j such that $x_i = x_j$ (or report that there are no collisions). Again consider the Johnson graph $J(n, r)$, for some r to be optimized over later. Deviating from Section 8.3.2, now call a vertex R *marked* if there exist $i \in R$ and $j \in [n] \setminus R$ such that $x_i = x_j$. Show that we can find a marked vertex in this graph with high probability using $O(n^{2/3})$ queries to x . You may ignore small error probabilities, for example when using Grover's algorithm. Be explicit about what data you store about x at each vertex R .
4. (H) Let A , B , and C be $n \times n$ matrices with real entries. We'd like to decide whether or not $AB = C$. Of course, you could multiply A and B and compare the result with C , but matrix multiplication is expensive (the current best algorithm takes time roughly $O(n^{2.38})$).
 - (a) Give a classical randomized algorithm that verifies whether $AB = C$ (with success probability at least $2/3$) using $O(n^2)$ steps, using the fact that matrix-vector multiplication can be done in $O(n^2)$ steps.
 - (b) Show that if we have query-access to the entries of the matrices (i.e., oracles that map $i, j, 0 \mapsto i, j, A_{i,j}$ and similarly for B and C), then every classical algorithm needs at least $\Omega(n^2)$ queries to detect a difference between AB and C with error probability $\leq 1/3$.
 - (c) Give a quantum walk algorithm that verifies whether $AB = C$ (with success probability at least $2/3$) using $O(n^{5/3})$ queries to matrix-entries.
5. A 3-SAT instance ϕ over n Boolean variables $x_1, \dots, x_n$ is a formula which is the AND of a number of clauses, each of which is an OR of 3 variables or their negations. For example, $\phi(x_1, \dots, x_4) = (x_1 \vee x_2 \vee \bar{x}_3) \wedge (x_2 \vee x_3 \vee \bar{x}_4)$ is a 3-SAT formula with 2 clauses. A satisfying assignment is a setting of the n variables such that $\phi(x_1, \dots, x_n) = 1$ (i.e., TRUE). You may assume the number of clauses is at most some polynomial in n . In general it is NP-hard to find a satisfying assignment to such a formula. Brute force would try out all 2^n possible truth-assignments, but something much better is possible: consider the following simple algorithm of Schönning [219], which is a classical random walk on the set of all $N = 2^n$ truth assignments:

Start with a uniformly random $x \in \{0, 1\}^n$.

Repeat the following at most $3n$ times: if $\phi(x) = 1$ then STOP, else find the leftmost clause that is false, randomly choose one of its 3 variables and flip its value.

One can show that this algorithm has probability at least $(3/4)^n / \sqrt{5n}$ of finding a satisfying assignment (if ϕ is satisfiable). You may assume this without proof.

- (a) Use the above to give a classical algorithm that finds a satisfying assignment with high probability in time $(4/3)^n \cdot p(n)$, where $p(n)$ is some polynomial factor.
- (b) (H) Give a quantum algorithm that finds a satisfying assignment (with high probability) in time $\sqrt{(4/3)^n} \cdot p(n)$.

Chapter 9

Hamiltonian Simulation

9.1 Hamiltonians

Thus far, we have viewed the dynamics of quantum systems from the perspective of *unitary transformations*: apart from measurement, the only way a quantum state (i.e., a vector of amplitudes) can change is by multiplication with a unitary matrix, for instance a 2-qubit gate tensored with identities on the other qubits. But which unitary will actually occur in a given physical system? This is determined by the *Hamiltonian* of the system, which is the observable H corresponding to the *total energy* in the system. The expectation value $\langle\psi|H|\psi\rangle$ is called the energy of state $|\psi\rangle$. Typically, this total energy is the sum of several different terms, corresponding to kinetic energy, potential energy, etc. Also typically, it is the sum of many *local* terms that each act on only a few of the particles (qubits) of the system, for example if all interactions are between pairs of particles.

One can think of the Hamiltonian H as describing the physical characteristics of the system. These do not determine the initial state $|\psi(0)\rangle$ of the system, but they do determine the *evolution* of the state in time, i.e., the state $|\psi(t)\rangle$ as a function of the time-parameter t , given initial state $|\psi(0)\rangle$. This is governed by the most important equation in quantum mechanics: the *Schrödinger equation*. It is a linear differential equation that relates the time-derivative of the current state to that state itself and to the Hamiltonian:

$$i\hbar \frac{d|\psi(t)\rangle}{dt} = H|\psi(t)\rangle.$$

Here $\hbar$ is a very small (at least in standard units) yet very important physical constant: Planck's constant divided by 2π . We can set it to 1 by choosing appropriate units, and hence will ignore it from now on. In general H may itself change with t , but for simplicity we will only consider here the case where H is time-independent. Then, if we start in some state $|\psi(0)\rangle$, the solution to this differential equation is the following unitary evolution of the state:¹

$$|\psi(t)\rangle = U|\psi(0)\rangle, \quad \text{where } U = e^{-iHt}.$$

So t time-steps of evolution induced by Hamiltonian H , corresponds to applying the unitary matrix e^{-iH} t times. Note, however, that t need not be integer here: this evolution is continuous in time, in contrast to the discrete picture one gets from the circuit model with elementary quantum gates.

¹Applying a function, for instance $f(x) = e^{-ix}$, to a normal matrix means applying f to its eigenvalues: if A has diagonalization VDV^{-1} then $f(A) = Vf(D)V^{-1}$, where $f(D)$ is the diagonal matrix obtained by applying f to the diagonal entries of D . For example, if $A = \sum_j \lambda_j a_j a_j^T$ and $f(x) = e^{-ix}$, then $f(A) = \sum_j e^{-i\lambda_j} a_j a_j^T$. Note that if A is Hermitian, then e^{iA} is unitary.

In areas like quantum chemistry (i.e., the study of properties of molecules and their interaction) and material sciences, it is often important to figure out how a quantum system will evolve from some given initial state, for instance a basis state.² This is typically hard to do on classical computers, since the number of parameters (amplitudes) is exponential in the number of particles. However, a *quantum* computer is like a universal quantum system, and should be able to efficiently simulate every efficient quantum process, in the same way that a classical universal Turing machine can efficiently simulate other (classical) physical processes.³ In fact, this was the main reason why Feynman invented quantum computers: as a controllable quantum system that can be used to simulate other quantum systems. In order to realize that idea, we need methods to efficiently implement the unitary evolution that is induced by a given Hamiltonian. In other words, we need methods to implement $U = e^{-iHt}$ as a quantum circuit of gates (say, up to some small error ε ⁴), and to apply this to a given initial state $|\psi\rangle$. This is known as the problem of “Hamiltonian simulation.”

In this chapter we will cover several methods for Hamiltonian simulation. For simplicity we’ll ignore the minus sign in Hamiltonian simulation, implementing $U = e^{iHt}$ rather than e^{-iHt} . We will also assume that our quantum system consists of n qubits. Some physical systems, for instance electron spins, naturally correspond to qubits. More complicated Hilbert spaces, for instance with basis states labeled by the positions (x, y, z coordinates) of all particles involved, can be encoded (approximately) in binary to reduce them to the case of qubits. This encoding can be done in many ways; much of the art in quantum chemistry is in how best to do this for specific systems, but we won’t study that here (see for instance [77]).

Word of warning: this chapter is denser and more complicated than most of the other chapters in these notes. On the other hand, unlike those chapters it explains some very recent, cutting-edge results.

9.2 Method 1: Lie-Suzuki-Trotter methods

Note that an n -qubit Hamiltonian is a $2^n \times 2^n$ matrix, which is huge even for moderate n . Typically in Hamiltonian simulation we are dealing with very structured Hamiltonians that have a much shorter classical description. Suppose our Hamiltonian is of the form $H = \sum_{j=1}^m H_j$, where m is not too big (say, polynomial in n) and each H_j acts only on a few of the n qubits. For concreteness

²It is also very important in chemistry to be able to find out *global* properties of a given Hamiltonian like its lowest energy, a.k.a. *ground state energy*. Unfortunately this problem seems to be hard to solve (in fact it is so-called **QMA**-hard, see Chapter 14) even for a quantum computer, even for the special case of 2-local Hamiltonians [156, 149].

³In Chapter 13 we will see that it is actually possible to classically simulate quantum computers (and hence quantum systems more generally) with a polynomial amount of *space*, but our best methods still use an exponential amount of *time*. If factoring a large integer is a hard problem for classical computers (which is widely believed), then Shor’s efficient quantum factoring algorithm (Chapter 5) implies that it is *impossible* to simulate a quantum computer in polynomial time on a classical computer.

⁴If n -qubit unitary $\tilde{U}$ (e.g., a quantum circuit with not too many gates) is meant to approximate n -qubit unitary U , then we can measure the error by the operator norm of their difference $\|U - \tilde{U}\| = \max_{\psi} \|U|\psi\rangle - \tilde{U}|\psi\rangle\|$. However, we will also see simulation methods that allow some auxiliary qubits, say a of them, which start in state $|0^a\rangle$ and should end in something close to state $|0^a\rangle$. In this case $\tilde{U}$ acts on more qubits than U , so we cannot use the operator norm of their difference; instead we can measure the error on the subspace of $(n + a)$ -qubit states where the last a qubits are $|0\rangle$: $\max_{\psi} \|(U|\psi\rangle)|0^a\rangle - \tilde{U}(|\psi\rangle|0^a\rangle)\|$. This way of measuring error still allows you to analyze a sequence of approximate unitaries using triangle inequality (as in Ex 4.4) in a way that the errors add up at most linearly.

assume each H_j acts non-trivially on only *two* of the qubits.⁵ Such a Hamiltonian is called *2-local*. Note that, for fixed t , the unitary $e^{iH_j t}$ is really just a 2-qubit gate, acting like identity on the other $n - 2$ qubits; this 2-qubit gate could in turn be constructed from CNOTs and single-qubit gates.

Our goal is to implement $U = e^{iHt} = e^{i\sum_j H_j t}$. It is now tempting to view this exponential of a sum of matrices as a product $\prod_{j=1}^m e^{iH_j t}$, which is just a product of m 2-qubit gates. If all terms H_j are diagonal, or if there is some basis in which all terms are diagonal (equivalently, if all H_j commute), then this indeed works out. However, in general matrix exponentials do not work that way: e^{A+B} need not equal $e^A e^B$ if A and B do not commute (see Exercise 1). The Lie-Suzuki-Trotter decomposition gives us a way to handle this. It uses the fact that if A and B have small operator norm, then e^{A+B} and $e^A e^B$ are *approximately* equal: $e^{A+B} = e^A e^B + E$, where the error-term E is a matrix whose operator norm is $O(\|A\| \cdot \|B\|)$.⁶

How can we use this to approximate U by a circuit $\tilde{U}$ of 2-qubit gates? Assume each of the terms H_j has operator norm ≤ 1 (see Exercise 2 for why such normalization matters). First consider the simple case $m = 2$, so $H = H_1 + H_2$. We can now implement $U = e^{iHt}$ by doing a little bit of H_1 , a little bit of H_2 , a little bit of H_1 , etc. More precisely, for every integer $r \geq 1$ of our choice, we have

$$U = e^{iHt} = (e^{iHt/r})^r = (e^{iH_1 t/r + iH_2 t/r})^r = (e^{iH_1 t/r} e^{iH_2 t/r} + E)^r. \quad (9.1)$$

Here the error-term E has norm $\|E\| = O(\|iH_1 t/r\| \cdot \|iH_2 t/r\|) = O(\|H_1\| \cdot \|H_2\| t^2/r^2)$. Our approximating circuit will be $\tilde{U} = (e^{iH_1 t/r} e^{iH_2 t/r})^r$, which uses $2r = O(t^2/\varepsilon)$ 2-qubit gates. Since errors in a product of unitaries add at most linearly (see Exercise 4.4), we have approximation error

$$\|U - \tilde{U}\| \leq r\|E\| = O(\|H_1\| \cdot \|H_2\| t^2/r) = O(t^2/r).$$

Choosing $r = O(t^2/\varepsilon)$, we can make this error $\leq \varepsilon$.

The same idea works for the general case where we have $m > 2$ Hamiltonian terms:

$$U = e^{iHt} = (e^{iHt/r})^r = (e^{iH_1 t/r + \dots + iH_m t/r})^r = (e^{iH_1 t/r} \dots e^{iH_m t/r} + E)^r, \quad (9.2)$$

where $\|E\| = O(m^2 t^2/r^2)$ (Exercise 3). Choosing $r = O(m^2 t^2/\varepsilon)$, we have an approximating circuit $\tilde{U} = (e^{iH_1 t/r} \dots e^{iH_m t/r})^r$ with $mr = O(m^3 t^2/\varepsilon)$ 2-qubit gates, and error $\|U - \tilde{U}\| \leq r\|E\| \leq \varepsilon$.

This is the *first-order* Lie-Suzuki-Trotter approach to Hamiltonian simulation, due to Lloyd [172]. The number of gates of the circuit $\tilde{U}$ depends quadratically on the time t for which we want to simulate the evolution, which is not optimal. One can do fancier higher-order Lie-Suzuki-Trotter decompositions that make the dependence on t nearly linear, but we won't explain those here. Instead we will describe two methods below with linear t -dependence. The number of gates of $\tilde{U}$ depends polynomially on ε ; this can be very much improved as well, as we will see.⁷

⁵This means H can be described efficiently by m 4×4 matrices, rather than by a $2^n \times 2^n$ matrix. A different assumption that is often made on Hamiltonians and that we will see later, is that H is *s-sparse*, meaning each of the 2^n columns has at most s nonzero entries, and we have some efficient "sparse access" to these nonzero entries. Note that if $H = \sum_j H_j$ and each H_j acts on only 2 qubits, then H is $4m$ -sparse. Thus, roughly speaking, the locality assumption implies the sparsity assumption.

⁶A non-rigorous but reasonably convincing way to see this is to approximate term e^M by its first-order Taylor series $I + M$, which is a good approximation if M has small norm (the error of the approximation will be quadratic in that norm). Then $e^A e^B - e^{A+B} \approx (I + A)(I + B) - (I + A + B) = AB$. In case you ever need it: the so-called Baker-Campbell-Hausdorff formula gives a much more precise expression.

⁷While the upper bounds on the number of gates for Trotter methods are theoretically worse (in their dependence on t and ε) than the other two methods explained in this chapter, in practice Trotter is quite competitive [85]. Trotter also has the advantages of being relatively simple and of not requiring any auxiliary qubits.

9.3 Method 2: Linear combination of unitaries (LCU)

Here we will describe a method for Hamiltonian simulation whose complexity depends linearly on the time t for which we want to evolve the state, and only logarithmically on the desired error ε .

Let's start with a more general problem. Suppose we have a $2^n \times 2^n$ matrix M and an n -qubit state $|\psi\rangle$, and we would like to prepare the state $M|\psi\rangle/\|M|\psi\rangle\|$. Here M need not be unitary, but suppose we can write M as a linear combination of unitaries:⁸

$$M = \sum_{j=1}^m \alpha_j V_j,$$

with the α_j being nonnegative reals (we can always absorb complex phases into the V_j). Let $\|\alpha\|_1 = \sum_j \alpha_j$, and let W be a unitary acting on $\lceil \log m \rceil$ qubits that maps

$$W : |0\rangle \mapsto \frac{1}{\sqrt{\|\alpha\|_1}} \sum_j \sqrt{\alpha_j} |j\rangle.$$

Suppose each V_j is an “easy” unitary, for instance a 2-qubit gate tensored with identity on the other $n - 2$ qubits, or a small circuit. Also suppose we can implement these unitaries in a controlled way: we have access to a 2-register unitary $V = \sum_{j=1}^m |j\rangle\langle j| \otimes V_j$. This maps $|j\rangle|\phi\rangle \mapsto |j\rangle V_j|\phi\rangle$, and we can think of the first register as “selecting” which unitary V_j to apply to the second register.⁹

We want to use V and W to implement M on a given state $|\psi\rangle$. Consider the following algorithm:

1. Start with two-register state $|0\rangle|\psi\rangle$, where the first register has $\lceil \log m \rceil$ qubits.
2. Apply W to the first register.
3. Apply V to the whole state.
4. Apply W^{-1} to the first register.

A small calculation (see Exercise 6) shows that the resulting state can be written as

$$\frac{1}{\|\alpha\|_1} |0\rangle M|\psi\rangle + \sqrt{1 - \frac{\|M|\psi\rangle\|^2}{\|\alpha\|_1^2}} |\phi\rangle, \quad (9.3)$$

where $|\phi\rangle$ is some other normalized state that we don't care about, but that has no support on basis states where the first register is $|0\rangle$. Note that the state of (9.3) has norm 1, because the squared norm of the first term is $\|M|\psi\rangle\|^2/\|\alpha\|_1^2$.

If we were to measure the first register, the probability of outcome 0 is $p = \|M|\psi\rangle\|^2/\|\alpha\|_1^2$. In case of that measurement outcome, the second register would collapse to the normalized version of $M|\psi\rangle$, as desired. The success probability p may be small, but we could use $O(1/\sqrt{p}) =$

⁸In fact *every* M can be written in such a way, because the 4^n n -qubit Pauli matrices (each of which is unitary) form a basis for the linear space of all $2^n \times 2^n$ matrices. See Appendix A.9.

⁹In the literature, this V is often called “select- V .” One might expect the cost of V to be not much higher than the costliest V_j , just like the cost of a classical “if A then B , else C ” statement is not much bigger than the largest of the costs of B and C . However, if we measure circuit size, then the cost of V could be roughly the *sum* of the costs of the V_j s because circuits for each V_j should be “included” in the circuit for V .

$O(\|\alpha\|_1/\|M|\psi\rangle\|)$ rounds of amplitude amplification to amplify the part of the state that starts with $|0\rangle$.¹⁰ Thus we would prepare (the normalized version of) $M|\psi\rangle$ in the second register. Unfortunately this usage of amplitude amplification assumes the ability to implement a unitary (as well as its inverse) to prepare $|\psi\rangle$ from a known initial state, say $|0\rangle$. Regular amplitude amplification won't work if instead of a unitary that prepares $|\psi\rangle$ we just have one copy of the state $|\psi\rangle$ available, which is the typical situation in Hamiltonian simulation. However, Exercise 8 gives us a variant called *oblivious* amplitude amplification, which circumvents this problem: it works even with just one copy of $|\psi\rangle$, as long as M is proportional to a unitary (or close to that). Fortunately, this is the situation when we use LCU for Hamiltonian simulation, where $M \approx e^{iHt}$.

9.3.1 Hamiltonian simulation via LCU

Recall that our goal is to efficiently implement the unitary e^{iHt} that is induced by a given Hamiltonian H , normalized so that $\|H\| \leq 1$. The following approach is due to Berry et al. [54, 55, 56]. Suppose, somewhat paradoxically, that we can write out the *Hermitian* matrix H as a linear combination of *unitaries*: $H = \sum_j \alpha_j V_j$. For example, if H is the sum of m 2-local terms like before, then every 2-local term can be written as the sum of at most 16 n -qubit Pauli matrices (each of which is unitary and acts non-trivially on only two qubits). Thus we would decompose H as a sum of at most $16m$ unitaries, each acting non-trivially on only two of the n qubits. The sum of coefficients $\|\alpha\|_1$ will be $O(m)$.

Using the Taylor series $e^x = \sum_{k=0}^{\infty} x^k/k!$, we write the unitary we want to implement as

$$e^{iHt} = \sum_{k=0}^{\infty} \frac{(iHt)^k}{k!} = \sum_{k=0}^{\infty} \frac{(it)^k}{k!} \left(\sum_{j \in [m]} \alpha_j V_j \right)^k = \sum_{k=0}^{\infty} \frac{(it)^k}{k!} \sum_{j_1, \dots, j_k \in [m]} \alpha_{j_1} \cdots \alpha_{j_k} V_{j_1} \cdots V_{j_k}. \quad (9.4)$$

Note that if each V_j is easy to implement and k is not too big, then the unitary $V_{j_1} \cdots V_{j_k}$ is also not too hard to implement. Exercise 9 shows that if we truncate the Taylor series at $k = O(t + \log(1/\varepsilon))$, dropping the terms of higher order, then the induced error (i.e., the dropped part) has operator norm at most ε . Accordingly, we can take the part of the right-hand side of Eq. (9.4) for $k = O(t + \log(1/\varepsilon))$ and then use the linear combination of unitaries approach to approximately implement e^{iHt} . The unitaries in this decomposition are of the form $V_{j_1, \dots, j_k} = i^k V_{j_1} \cdots V_{j_k}$; let $\mathcal{V} = \sum_{j_1, \dots, j_k} |j_1, \dots, j_k\rangle\langle j_1, \dots, j_k| \otimes V_{j_1, \dots, j_k}$ denote the controlled operation of the $V_{j_1, \dots, j_k}$ unitaries, each of which involves k V_j 's. The corresponding nonnegative coefficients in this decomposition are

$$\beta_{j_1, \dots, j_k} = \frac{t^k}{k!} \alpha_{j_1} \cdots \alpha_{j_k}, \quad \text{for } k \leq O(t + \log(1/\varepsilon)).$$

These β -coefficients add up to

$$\|\beta\|_1 = \sum_{k=0}^{O(t + \log(1/\varepsilon))} \frac{t^k}{k!} \sum_{j_1, \dots, j_k} \alpha_{j_1} \cdots \alpha_{j_k} \leq \sum_{k=0}^{\infty} \frac{t^k}{k!} \sum_{j_1, \dots, j_k} \alpha_{j_1} \cdots \alpha_{j_k} = \sum_{k=0}^{\infty} \frac{(t\|\alpha\|_1)^k}{k!} = e^{t\|\alpha\|_1},$$

so straightforward application of the LCU method with oblivious amplitude amplification uses $O(\|\beta\|_1) = O(e^{t\|\alpha\|_1})$ applications of $\mathcal{V}$ and $\mathcal{V}^{-1}$.

¹⁰If we do not know the value of p in advance, then we can try out exponentially decreasing guesses for p , like we do for Grover's algorithm in Chapter 7 when we don't know the number of solutions.

The logarithmic error-dependence of the complexity of the above method is excellent. The exponential dependence on $t\|\alpha\|_1$ is quite terrible for large t , but not too bad for very small t . So what we'll do if we want to do a simulation for large t , is to divide that t into $b = t\|\alpha\|_1$ blocks of time $\tau = 1/\|\alpha\|_1$ each, run the above algorithm for time τ with error $\varepsilon' = \varepsilon/b$, and then glue b time- τ simulations together. This will simulate $(e^{iH\tau})^b = e^{iHt}$, with error $\leq b\varepsilon' = \varepsilon$. The cost of each time- τ simulation is $O(e^{\tau\|\alpha\|_1}) = O(1)$ applications of $\mathcal{V}$ and $\mathcal{V}^{-1}$, each of which involves $O(\tau + \log(1/\varepsilon')) = O(\log(t\|\alpha\|_1/\varepsilon))$ applications of the V_j 's. The overall cost will be b times that, since we'll run b subsequent time- τ simulations in order to implement a time- t simulation.

To give a more concrete example, consider again the special case where $H = \sum_i H_i$ consists of 2-local terms, so the unitaries V_j in the induced linear combination of unitaries $H = \sum_{j=1}^m \alpha_j V_j$ only act nontrivially on 2 qubits each. Then we approximate the time- τ unitary $e^{iH\tau}$ by a linear combination of unitaries

$$M = \sum_{k=0}^{O(\tau + \log(1/\varepsilon'))} \sum_{j_1, \dots, j_k \in [m]} \beta_{j_1, \dots, j_k} V_{j_1, \dots, j_k}, \quad (9.5)$$

where each $V_{j_1, \dots, j_k}$ is a product of $k = O(\tau + \log(1/\varepsilon')) = O(\log(t\|\alpha\|_1/\varepsilon))$ 2-qubit gates. We can implement this using the linear combination of unitaries approach, and repeat this $b = t\|\alpha\|_1$ times. The cost of the unitary W is typically relatively small (see Exercise 7), so we can ε -approximate the unitary e^{iHt} using a circuit of roughly $O(t\|\alpha\|_1 \log(t\|\alpha\|_1/\varepsilon)) = O(mt \log(mt/\varepsilon))$ applications of $\mathcal{V}$ and $\mathcal{V}^{-1}$, and slightly more other 2-qubit gates. Note the linear dependence of the cost on the evolution-time t , and the logarithmic dependence on the error ε , both of which are better than Lie-Suzuki-Trotter methods.

9.4 Method 3: Transforming block-encoded matrices

In this section we'll describe a recent approach that is very general and flexible. Suppose A is an n -qubit matrix with operator norm $\|A\| \leq 1$, and we can implement an $(n+1)$ -qubit unitary

$$U = \begin{pmatrix} A & \cdot \\ \cdot & \cdot \end{pmatrix}. \quad (9.6)$$

The ' $\cdot$'s are unspecified $2^n \times 2^n$ -dimensional matrices, the only constraint on which is that U is unitary. Such a U is called a unitary *block-encoding* of A . Note that

$$U : |0\rangle|\psi\rangle \mapsto |0\rangle A|\psi\rangle + |1\rangle|\phi\rangle,$$

where we can't say much about the (subnormalized) state $|\phi\rangle$. Written more technically, the defining property of such a block-encoding is $(\langle 0| \otimes I)U(|0\rangle \otimes I) = A$, where the first register is one qubit. More generally we can define an a -qubit block-encoding of A , which is an $(a+n)$ -qubit unitary U with the property that $(\langle 0^a| \otimes I)U(|0^a\rangle \otimes I) = A$.

Example 1: LCU does block-encoding. From Eq. (9.3) we can see that LCU (without the final amplitude amplification) implements a $\lceil \log m \rceil$ -qubit block-encoding of the matrix $A = M/\|\alpha\|_1$.

Example 2: Block-encoding a sparse Hermitian matrix. Let A be a $2^n \times 2^n$ Hermitian matrix of operator norm $\|A\| \leq 1$ that is s -sparse, so each row and column of A have at most s nonzero entries (for simplicity assume *exactly* s nonzero entries). Since this matrix A is still an exponentially large object, we have to be careful how we can access such sparse matrices. First, we assume we can query the entries of A in the usual way: we have an oracle

$$O_A : |i, j\rangle|0\rangle \mapsto |i, j\rangle|A_{ij}\rangle,$$

where we assume the last register has sufficiently many qubits to write down the complex entry A_{ij} either exactly or with sufficient precision. Of course, since A is sparse, A_{ij} will actually be 0 for most (i, j) . Let $\nu(j, \ell) \in \{0, \dots, N-1\}$ denote the location of the ℓ -th nonzero entry of the j -th column of A ; so the s nonzero entries in the j -th column are at positions $\nu(j, 0), \dots, \nu(j, s-1)$. We also assume we have another oracle that allows us to find these locations:

$$O_{A,loc} : |j, \ell\rangle \mapsto |j, \nu(j, \ell)\rangle.$$

We also assume we can run O_A^{-1} and $O_{A,loc}^{-1}$. Together these assumptions are called having “sparse access” to A .

We will now show how to implement a block-encoding of the matrix A/s . Exercise 10 shows how we can implement two $(2n+1)$ -qubit unitaries that create superpositions over the locations of the nonzero entries in the j -th column and i -th row of A , respectively:

$$W_1 : |0\rangle|0^n\rangle|j\rangle \mapsto \frac{1}{\sqrt{s}}|0\rangle \sum_{k:A_{kj} \neq 0} |k, j\rangle, \quad W_3 : |0\rangle|0^n\rangle|i\rangle \mapsto \frac{1}{\sqrt{s}} \sum_{\ell:A_{i\ell} \neq 0} |0\rangle|i, \ell\rangle,$$

using one $O_{A,loc}$ -query and a few other A -independent gates. We can also implement the following unitary using one query to each of O_A and O_A^{-1} , and a few other A -independent gates (and some auxiliary qubits that start and end in $|0\rangle$):

$$W_2 : |0\rangle|k, j\rangle \mapsto A_{kj}|0\rangle|k, j\rangle + \sqrt{1 - |A_{kj}|^2}|1\rangle|k, j\rangle.$$

By following the action on initial state $|0^{n+1}j\rangle$ step-by-step (Exercise 10), one can show that the $(0^{n+1}i, 0^{n+1}j)$ -entry of $U = W_3^{-1}W_2W_1$ equals A_{ij}/s . In other words, U is an $(n+a)$ -qubit block-encoding of matrix A/s for some a (depending on how many auxiliary qubits are actually used).

How can we use a given block-encoding U of A ? Suppose that for some function $f : \mathbb{R} \rightarrow \mathbb{R}$ we want to implement a unitary V that looks like

$$V = \begin{pmatrix} f(A) & \cdot \\ \cdot & \cdot \end{pmatrix},$$

using a small number of applications of the block-encoding of A . Here we don’t care what submatrices sit at the ‘ $\cdot$ ’ entries of U or V , as long as the upper-left block of V is $f(A)$ and V as a whole is unitary. For example, in Hamiltonian simulation A would be the Hamiltonian H and $f(x)$ would be e^{ixt} , so that we are effectively implementing $f(H) = e^{iHt}$. In the HHL algorithm in the next chapter, $f(x)$ will be $1/x$, so that we effectively implement A^{-1} .

It turns out that we can implement a good approximation of V efficiently if we have a *low-degree polynomial* P approximating f . The idea is that we can let P act on the eigenvalues of A , thus transforming a block-encoding of A into one of $P(A)$. We state without proof the following theorem by Gilyén et al. [122, follows by applying their Theorem 56 to the real and to the imaginary part of the polynomial], which extends work of Low et al. [177, 178, 176, 179, 178].

Theorem 1 Let $P : [-1, 1] \rightarrow \{c \in \mathbb{C} \mid |c| \leq 1/4\}$ be a degree- d polynomial, and let U be a unitary a -qubit block-encoding of Hermitian matrix A . We can implement a unitary $O(a)$ -qubit block-encoding V of $P(A)$ using d applications of U and U^{-1} , one controlled application of U , and $O(ad)$ other 2-qubit gates.

This theorem can be generalized to a powerful technique called “singular-value transformation” [122], where A can be an arbitrary matrix, non-Hermitian and even non-square.

9.4.1 Hamiltonian simulation via transforming block-encoded matrices

Let’s see how we can use Theorem 1 for Hamiltonian simulation for a given sparse Hamiltonian H . We again approximate the function $f(x) = e^{ixt}$ using a polynomial P degree $d = O(t + \log(1/\varepsilon))$, which is the first d terms of the Taylor series of f (see Exercise 9), divided by 4 to ensure that its range satisfies the condition of Theorem 1. If H is s -sparse and we have sparse access to it, then Example 2 of Section 9.4 shows how to efficiently implement a block-encoding U of the scaled Hamiltonian H/s , using $O(1)$ queries to H and $O(n)$ other gates. Note that evolving Hamiltonian H for time t is the same as evolving H/s for time st . Theorem 1 now gives us a block-encoding V of $P(H) \approx \frac{1}{4}e^{iHt}$. This V invokes U and U^{-1} $O(st + \log(1/\varepsilon))$ times, and maps:

$$V : |0\rangle|\psi\rangle \mapsto |0\rangle P(H)|\psi\rangle + |\phi\rangle,$$

where $|\phi\rangle$ has no support on basis states starting with $|0\rangle$. Since $P(H) \approx \frac{1}{4}e^{iHt}$ is essentially proportional to a unitary, we can now apply $O(1)$ rounds of oblivious amplitude amplification to boost the factor $\frac{1}{4}$ to essentially 1, using only one copy of $|\psi\rangle$.

This implements the desired unitary e^{iHt} on one copy of $|\psi\rangle$, up to small error. The complexity of ε -precise Hamiltonian simulation of an s -sparse Hamiltonian H of operator norm ≤ 1 then becomes $O(st + \log(1/\varepsilon))$ queries to H and $O(n(st + \log(1/\varepsilon)))$ 2-qubit gates.

Exercises

1. Compute the following five 2×2 unitaries: $e^{i\pi Z}$, $e^{i\pi X}$, $e^{i\pi X}e^{i\pi Z}$, $e^{i\pi Z}e^{i\pi X}$, and $e^{i\pi(X+Z)}$. Here X and Z are the usual Pauli matrices.
2. Suppose we want to implement a certain unitary U , and we can do that by switching on a Hamiltonian H for some time t : $U = e^{-iHt}$. Now suppose H' is another Hamiltonian, with 100 times as much energy as H : $H' = 100H$. Show that using H' we can implement U a 100 times faster than with H .

Comment: This exercise is about time in the physical sense of the word, not about “time complexity” in the sense of circuit size. It shows why some kind of normalization of H is needed if we want to talk about the time it takes to implement something. We can always “speed up” a computation by a factor k if we can multiply our Hamiltonian with a factor k .

3. (H) This exercise justifies the error bound of Eq. (9.2). Let $\delta > 0$ be small (in Eq. (9.2) we’d set $\delta = t/r$). Show that there is a constant c , independent of δ and $H_1, \dots, H_m$, such that

$$e^{iH_1\delta + \dots + iH_m\delta} = e^{iH_1\delta} \dots e^{iH_m\delta} + E,$$

for some E of norm $\|E\| \leq c\delta^2 \sum_{j=1}^{m-1} j$.

4. Consider the simple case of the linear-combination-of-unitaries trick where $m = 2$ and $M = V_1 + V_2$. Describe the unitaries V and W , and track the initial state $|0\rangle|\psi\rangle$ through the 4-step algorithm in Section 9.3.
5. (a) Write the gate $G = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -i \\ i & -1 \end{pmatrix}$ as a linear combination of the Pauli matrices (see Appendix A.9).
 (b) Suppose you want to implement G via LCU, using the linear combination of (a). What are $W|0\rangle$ and V ?
 (c) Consider the final state after the 4-step algorithm of Section 9.3. Calculate the part of that state that starts with $|0\rangle$ (without writing out that final state fully!).
6. (H) Give a calculation to justify that the 4-step algorithm in Section 9.3 indeed always produces a state of the form of Eq. (9.3).
7. Let $v \in [-1, 1]^N$ be a vector with real entries, of dimension $N = 2^n$, indexed by $i \in \{0, 1\}^n$. Suppose we can query the entries of this vector by a unitary that maps

$$O_v : |i\rangle|0^p\rangle \mapsto |i\rangle|v_i\rangle,$$

so where the binary representation of the i -th entry of v is written into the second register. We assume this second register has p qubits, and the numbers v_i can all be written exactly with p bits of precision (it doesn't matter how, but for concreteness say that the first bit indicates the sign of the number, followed by the $p - 1$ most significant bits after the decimal dot). Our goal is to prepare the n -qubit quantum state $|\psi\rangle = \frac{1}{\|v\|} \sum_{i \in \{0,1\}^n} v_i |i\rangle$.

- (a) Show how you can implement the following 3-register map (where the third register is one qubit) using one application of O_v and one of O_v^{-1} , and some v -independent unitaries (you don't need to draw detailed circuits for these unitaries, nor worry about how to write those in terms of elementary gates).

$$|i\rangle|0^p\rangle|0\rangle \mapsto |i\rangle|0^p\rangle(v_i|0\rangle + \sqrt{1 - v_i^2}|1\rangle).$$

- (b) Suppose you apply the map of (a) to a uniform superposition over all $i \in \{0, 1\}^n$. Write the resulting state, and calculate the probability that measuring the last qubit in the computational basis gives outcome 0.
- (c) What is the resulting 3-register state if the previous measurement gave outcome 0?
- (d) Assume you know $\|v\|$ exactly. Give an algorithm that prepares $|\psi\rangle$ exactly, using $O\left(\frac{\sqrt{N}}{\|v\|}\right)$ applications of O_v and O_v^{-1} , and some v -independent unitaries.
8. (H) This exercise explains *oblivious* amplitude amplification. Let M be an n -qubit unitary. We start from $|\Psi\rangle = |0^a\rangle|\psi\rangle$ for unknown n -qubit state $|\psi\rangle$. Our goal is to prepare the state $|\Phi\rangle = |0^a\rangle M|\psi\rangle$ (this $|\Phi\rangle$ is the analogue of the “good state” in amplitude amplification). Let U be an $(a + n)$ -qubit unitary, independent of $|\psi\rangle$, such that

$$U|\Psi\rangle = \sin(\theta)|\Phi\rangle + \cos(\theta)|\Phi^\perp\rangle,$$

where θ is some angle that's independent of $|\psi\rangle$, while $|\Phi^\perp\rangle$ is some normalized state that depends on $|\psi\rangle$ and has no support on basis states starting with 0^a (this $|\Phi^\perp\rangle$ is the analogue of the “bad state”). If θ is close to $\pi/2$, then we can just apply U to our starting state $|\Psi\rangle$ and measure the first register; we'll see 0^a with probability $\sin(\theta)^2 \approx 1$ and in that case end up with the desired state $|\Phi\rangle$. But suppose θ is quite small. Here we will see how we can amplify the angle θ to roughly $\pi/2$, without assuming a unitary to prepare $|\Psi\rangle$.

(a) Let $\mathcal{S}$ be the 2-dimensional space spanned by $|\Phi\rangle$ and $|\Phi^\perp\rangle$. Let $R = (I - 2|0^a\rangle\langle 0^a|) \otimes I$ be a unitary that puts a ‘−’ in front of every basis state that starts with 0^a . Show that R , restricted to $\mathcal{S}$, is a reflection through $|\Phi^\perp\rangle$.

(b) Define $|\Psi^\perp\rangle = U^{-1}(\cos(\theta)|\Phi\rangle - \sin(\theta)|\Phi^\perp\rangle)$. Show $U|\Psi\rangle$ and $U|\Psi^\perp\rangle$ are orthogonal.

One can also show with a bit more work [54, Lemma 3.7] the stronger statement that $|\Psi^\perp\rangle$ has no support on basis states starting with 0^a . You may assume this fact without proof in the remainder of this exercise.

(c) Show that $-URU^{-1}$, restricted to $\mathcal{S}$, is a reflection through $U|\Psi\rangle$ (note the minus sign!)

(d) Show that $(-URU^{-1}R)^k U|0^a\rangle|\psi\rangle = \sin((2k+1)\theta)|\Phi\rangle + \cos((2k+1)\theta)|\Phi^\perp\rangle$.

(e) How large should we take k in order to end up with (approximately) the state $|\Phi\rangle$?

NB: If you know θ exactly, then you can even exactly prepare $|\Phi\rangle$ (along the lines of Exercise 7.5) but you don't need to show that.

9. (H) Let $\varepsilon \in (0, 0.99)$. Show that you can choose a sufficiently large constant c (independent of t and ε) such that for all Hermitian H with operator norm $\|H\| \leq 1$, we have

$$\left\| e^{iHt} - \sum_{k=0}^{c(t+\log(1/\varepsilon))-1} \frac{(iHt)^k}{k!} \right\| = \left\| \sum_{k=c(t+\log(1/\varepsilon))}^{\infty} \frac{(iHt)^k}{k!} \right\| \leq \varepsilon.$$

10. This exercise looks at the details of block-encoding an s -sparse matrix A with $\|A\| \leq 1$ from Section 9.4. Assume for simplicity that the entries of A are real.

(a) Show how to implement W_1 using an $O_{A,loc}$ -query and a few other A -independent gates. For simplicity you may assume s is a power of 2 here, and you can use arbitrary single-qubit gates, possibly controlled by another qubit.

(Note that the same method allows to implement W_3 .)

(b) (H) Show how to implement W_2 using an O_A -query, an O_A^{-1} -query, and a few other A -independent gates (you may use auxiliary qubits as long as those start and end in $|0\rangle$).

(c) Show that the $(0^{n+1}i, 0^{n+1}j)$ -entry of $W_3^{-1}W_1$ is $1/s$ if $A_{ij} \neq 0$, and is 0 if $A_{ij} = 0$.

(d) Show that the $(0^{n+1}i, 0^{n+1}j)$ -entry of $W_3^{-1}W_2W_1$ is exactly A_{ij}/s .

11. (a) Give a quantum circuit on $n+1$ qubits that uses $O(n)$ gates, no auxiliary qubits, and computes the parity of the first n bits, in the sense that it maps

$$|x\rangle|0\rangle \mapsto |x\rangle \left| \sum_{i=1}^n x_i \bmod 2 \right\rangle \text{ for all } x \in \{0,1\}^n.$$

- (b) Let $P = Z \otimes Z \otimes \cdots \otimes Z$ be an n -qubit Hamiltonian, where Z is the usual phase-flip Pauli matrix. What is the result of applying matrix P to an n -qubit basis state $|x\rangle$?
 - (c) Fix some positive real number t . Let $U = e^{iPt}$ be the n -qubit unitary induced by applying the above Hamiltonian for time t (via the Schrödinger equation, dropping the minus sign in the exponent like in Section 9.1). What is the result of applying U to an n -qubit basis state $|x\rangle$?
 - (d) (H) Give a quantum circuit with $O(n)$ gates that implements U exactly using one auxiliary qubit that starts and ends in $|0\rangle$. You may use arbitrary single-qubit gates, which may be controlled by another qubit.
 - (e) (H) Now suppose P is a product of n arbitrary Pauli matrices, not necessarily all- Z . Show how to implement $U = e^{iPt}$.
12. Suppose you have a classical description of an n -qubit Hamiltonian H that is the sum of $m = n^2$ 2-local terms. Assume the eigenvalues of the Hermitian matrix H lie in $[0, 1)$, and can all be written exactly with $2 \log n$ bits of precision. You would like to exactly determine the smallest eigenvalue $\lambda_{\min}$ of H , corresponding to unknown n -qubit eigenstate $|\psi_{\min}\rangle$. You're given (as a quantum state) an n -qubit state $|\psi\rangle$ that has a significant overlap with $|\psi_{\min}\rangle$: $|\langle\psi|\psi_{\min}\rangle|^2 \geq 0.7$. Give a polynomial-size quantum circuit that, with probability $\geq 2/3$, outputs $\lambda_{\min}$ exactly.

NB: You don't need to write down the circuit to the last detail; a clear description of the different parts of the circuit (possibly with some reference to details in the lecture notes) suffices.

Chapter 10

The HHL Algorithm

10.1 The linear-system problem

In this chapter we present the Harrow-Hassidim-Lloyd (HHL [134]) algorithm for solving large systems of linear equations. Such a system is given by an $N \times N$ matrix A with real or complex entries, and an N -dimensional nonzero vector b . Assume for simplicity that $N = 2^n$. The linear-system problem is

LSP: find an N -dimensional vector x such that $Ax = b$.

Solving large systems of linear equations is extremely important in many computational problems in industry, in science, in optimization, in machine learning, etc. In many applications it suffices to find a vector $\tilde{x}$ that is close to the actual solution x .

We will assume A is invertible (equivalently, has rank N) in order to guarantee the existence of a unique solution vector x , which is then just $A^{-1}b$. This assumption is just for simplicity: if A does not have full rank, then the methods below would still allow to invert it on its support, replacing A^{-1} by the “Moore-Penrose pseudoinverse.”

The HHL algorithm can solve “well-behaved” large linear systems very fast (under certain assumptions), but in a rather weak sense: instead of outputting the N -dimensional solution vector x itself, its goal is to output the n -qubit state

$$|x\rangle := \frac{1}{\|x\|} \sum_{i=0}^{N-1} x_i |i\rangle,$$

or some other n -qubit state close to $|x\rangle$. This state $|x\rangle$ has the solution vector *as its vector of amplitudes*, up to normalization. This is called the *quantum* linear-system problem:

QLSP: find an n -qubit state $|\tilde{x}\rangle$ such that $\| |x\rangle - |\tilde{x}\rangle \| \leq \varepsilon$ and $Ax = b$.

Note that the QLSP is an inherently quantum problem, since the goal is to produce an n -qubit state whose amplitude-vector (up to normalization and up to ε -error) is a solution to the linear system. In general this is not as useful as just having the N -dimensional vector x written out on a piece of paper, but in some cases where we only want some partial information about x , it may suffice to just (approximately) construct $|x\rangle$.

We will assume without loss of generality that A is Hermitian (see Exercise 1). Let us state the more restrictive assumptions that will make the linear system “well-behaved” and suitable for the HHL algorithm:

1. We have a unitary that can prepare the vector b as an n -qubit quantum state $|b\rangle = \frac{1}{\|b\|} \sum_i b_i |i\rangle$ using a circuit of B 2-qubit gates. We also assume for simplicity that $\|b\| = 1$.
2. The matrix A is *s-sparse* and we have sparse access to it, like in Section 9.4. Such sparsity is not essential to the algorithm, and could be replaced by other properties that enable an efficient block-encoding of A .
3. The matrix A is *well-conditioned*: the ratio between its largest and smallest singular value is at most some κ .¹ For simplicity, assume the smallest singular value is $\geq 1/\kappa$ while the largest is ≤ 1 . In other words, all eigenvalues of A lie in the interval $[-1, -1/\kappa] \cup [1/\kappa, 1]$. The smaller the “condition number” κ is, the better it will be for the algorithm. Let’s assume our algorithm knows κ , or at least knows a reasonable upper bound on κ .

10.2 The basic HHL algorithm for linear systems

Let us start with some intuition. The solution vector x that we are looking for is $A^{-1}b$, so we would like to apply A^{-1} to b . Because A is assumed to be Hermitian, it has spectral decomposition $A = \sum_{j=0}^{N-1} \lambda_j a_j a_j^*$, where the vectors a_j are an orthonormal basis of eigenvectors for the whole N -dimensional space, and $\lambda_j \in \mathbb{R}$ are the corresponding eigenvalues. Then the map A^{-1} is the same as the map $a_j \mapsto \frac{1}{\lambda_j} a_j$: we just want to multiply the eigenvector a_j with the scalar $1/\lambda_j$. The vector b can also be written as a linear combination of the eigenvectors a_j : $b = \sum_j \beta_j a_j$ (we don’t need to know the coefficients β_j for what follows). We want to apply A^{-1} to b to obtain $A^{-1}b = \sum_j \beta_j \frac{1}{\lambda_j} a_j$, normalized, as an n -qubit quantum state.

Unfortunately the maps A and A^{-1} are not unitary (unless $|\lambda_j| = 1$ for all j), so we cannot just apply A^{-1} as a quantum operation to state $|b\rangle$ to get state $|x\rangle$. Fortunately $U = e^{iA} = \sum_j e^{i\lambda_j} a_j a_j^T$ is unitary, and has the same eigenvectors as A and A^{-1} . We can implement U and powers of U by Hamiltonian simulation, and then use phase estimation (Section 4.6) to estimate the λ_j associated with eigenvector $|a_j\rangle$ with some small approximation error (for this sketch, assume for simplicity that the error is 0). Conditioned on our estimate of λ_j we can then rotate an auxiliary $|0\rangle$ -qubit to $\frac{1}{\kappa\lambda_j}|0\rangle + \sqrt{1 - \frac{1}{(\kappa\lambda_j)^2}}|1\rangle$ (this is a valid state because $|\kappa\lambda_j| \geq 1$). Next we undo the phase estimation to set the register that contained the estimate back to $|0\rangle$. Suppressing the auxiliary qubits containing the temporary results of the phase estimation (these qubits start and end in state $|0\rangle$), we have now unitarily mapped

$$|a_j\rangle|0\rangle \mapsto |a_j\rangle \left(\frac{1}{\kappa\lambda_j}|0\rangle + \sqrt{1 - \frac{1}{(\kappa\lambda_j)^2}}|1\rangle \right).$$

If we prepare a copy of $|b\rangle|0\rangle = \sum_j \beta_j |a_j\rangle|0\rangle$ and apply the above unitary map to it, then we obtain

$$\sum_j \beta_j |a_j\rangle \left(\frac{1}{\kappa\lambda_j}|0\rangle + \sqrt{1 - \frac{1}{(\kappa\lambda_j)^2}}|1\rangle \right) = \frac{1}{\kappa} \underbrace{\sum_j \beta_j \frac{1}{\lambda_j} |a_j\rangle}_{\propto |x\rangle} |0\rangle + |\phi\rangle|1\rangle,$$

¹This ratio is called the *condition number*. Note that the assumption that A is invertible is equivalent to having a finite condition number. The stronger assumption that the condition number is small, intuitively says that A is invertible in a stable or robust way, so that small errors (say due to noise or to finite-precision rounding) don’t lead to massive errors in the solution vector x .

where we don't care about the (subnormalized) state $|\phi\rangle$. Note that because $\sum_j |\beta_j/\lambda_j|^2 \geq \sum_j |\beta_j|^2 = 1$, the norm of the part of the state ending in qubit $|0\rangle$ is at least $1/\kappa$. Accordingly, we can now apply $O(\kappa)$ rounds of amplitude amplification to amplify this part of the state to have amplitude essentially 1. This prepares state $|x\rangle$ to good approximation, as intended.

This rough sketch (which Exercise 2 asks you to make more precise) is the basic idea of HHL. It leads to an algorithm that produces a state $|\tilde{x}\rangle$ that is ε -close to $|x\rangle$, using roughly $\kappa^2 s/\varepsilon$ queries to A and roughly $\kappa s(\kappa n/\varepsilon + B)$ other 2-qubit gates.

10.3 Improving the efficiency of the HHL algorithm

The complexity of the above basic HHL algorithm can be improved further. Gilyén et al. [122] used the singular-value transformation technique of Section 9.4 to implement A^{-1} , improving on an LCU construction due to Childs et al. [83]. We would like to apply the function $f(x) = 1/x$ to a block-encoding of A in order to get a block-encoding of A^{-1} (up to normalization) that we can then apply to $|b\rangle|0\rangle$.

The function $f(x) = 1/x$ is not itself a polynomial, so we need to approximate it by a low-degree polynomial to be able to apply Theorem 1 of Chapter 9. Childs et al. [83, Lemmas 17-19] started from the following polynomial of degree $D = 2b - 1$ for $b = O(\kappa^2 \log(\kappa/\varepsilon))$:

$$\frac{1 - (1 - x^2)^b}{x}.$$

This is indeed a polynomial because all terms in the numerator have degree ≥ 1 , so we can divide out the x of the denominator. Since $(1 - x^2)^b$ is close to 0 (unless $|x|$ is small), this polynomial is indeed close to $1/x$ (unless $|x|$ is small, but we won't care about that because we'll apply this polynomial to a matrix whose eigenvalues aren't close to 0). More precisely, this polynomial is $\varepsilon/2$ -close to $1/x$ whenever x lies in the interval $E_\kappa = [-1, -1/\kappa] \cup [1/\kappa, 1]$. Its range on this domain is $[-\kappa, -1] \cup [1, \kappa]$ (ignoring the small ε for simplicity). Like every degree- D polynomial, f can be written exactly as a sum of the first $D + 1$ Chebyshev polynomials of the first kind.² Childs et al. show that the coefficients in this sum decrease quickly for larger degree, and that dropping the Chebyshev polynomials of degree higher than $d = O(\kappa \log(\kappa/\varepsilon))$ incurs only small error $\varepsilon/2$. The resulting degree- d polynomial p ε -approximates $1/x$ on the interval E_κ , and its largest value (in absolute value) on this domain is κ . Now define the polynomial $P = p/(4\kappa)$. This has the same degree d as p , but a range $[-1/4, 1/4]$ that fits the assumption of Theorem 1 of Chapter 9 (there's a trick to ensure the values of P are within that range even for $x \approx 0$, which we'll skip here).

As we saw in Section 9.4, we can implement a block-encoding of the s -sparse matrix A/s using $O(1)$ sparse-access queries to A and $O(n)$ other gates. Using a factor $O(s)$ more work, we can turn this into a block-encoding of A itself (alternatively, we could directly invert the matrix A/s , whose singular values are $\geq 1/(\kappa s)$). We now apply Theorem 1 with this block-encoding of A , and the polynomial $P = p/(4\kappa)$, of degree $d = O(\kappa \log(\kappa/\varepsilon))$. Note that all eigenvalues of A lie in the interval E_κ , where $p(x) \approx 1/x$, hence $p(A) \approx A^{-1}$ and $P(A) \approx \frac{1}{4\kappa} A^{-1}$. Theorem 1 then gives us a block-encoding of $P(A)$, at the expense of running the block-encoding of A $O(d)$ times. Using $O(\kappa)$ rounds of amplitude amplification on top of this, we can get rid of the $1/(4\kappa)$ factor

²These univariate polynomials are defined recursively as follows: $T_0(x) = 1$, $T_1(x) = x$, and $T_{d+1} = 2xT_d(x) - T_{d-1}(x)$. Note that T_d has degree d , and maps $[-1, 1]$ to $[-1, 1]$. The polynomials $T_0, \dots, T_D$ are linearly independent (even orthonormal in a certain way) and hence span the set of all univariate polynomials of degree $\leq D$.

and end up with essentially the state $A^{-1}|b\rangle$, normalized.³ This gives a quantum algorithm that solves the QLSP using $O(d\kappa s) = O(\kappa^2 s \log(\kappa/\varepsilon))$ queries to A , and $O(\kappa s(\kappa n \log(\kappa/\varepsilon) + B))$ 2-qubit gates. Note that compared to basic HHL, the dependence on $1/\varepsilon$ has been improved from linear to logarithmic. The dependence on κ can also be improved from quadratic to linear, using a technique called “variable-time amplitude amplification” [15, 83, 79, 169] that we won’t explain here.

The HHL algorithm can in some cases solve the QLSP exponentially faster than classical algorithms can solve the LSP. In particular, if the sparsity s , the condition number κ , and the cost B of preparing $|b\rangle$ are all $\leq \text{polylog}(N)$, and the allowed error is $\varepsilon \geq 2^{-\text{polylog}(N)}$, then this improved version of the HHL algorithm uses $\text{polylog}(N)$ queries and gates to solve (in a quantum way) an N -dimensional linear system. It can also be used for other tasks, for instance approximately solving differential equations and other applications in scientific computing, see the lecture notes of Lin Lin [168] and references therein.

Exercises

1. Suppose we are given an arbitrary invertible $N \times N$ matrix A and an N -dimensional vector b .
 - (a) Give a *Hermitian* $2N \times 2N$ matrix A' (depending on A but not on b) and $2N$ -dimensional vector b' (depending on b but not on A), such that a solution x to the linear system $Ax = b$ can be read off from a solution to the system $A'x' = b'$.
 - (b) How does the condition number of your A' relate to that of A ?
2. This exercise asks you to add more details to the sketch of the basic HHL algorithm given at the start of Section 10.2. For simplicity we will only count queries, not gates.

- (a) Use Hamiltonian simulation and phase estimation to implement the following unitary map:

$$|a_j\rangle|0\rangle \mapsto |a_j\rangle|\tilde{\lambda}_j\rangle,$$

where $|\tilde{\lambda}_j\rangle$ is a superposition over estimates of λ_j , which (if measured) gives with probability ≥ 0.99 an estimator $\ell \in [-1, 1]$ such that $|\lambda_j - \ell| \leq \varepsilon/\kappa$. Your implementation is allowed to use $O(\kappa s/\varepsilon + \log(\kappa/\varepsilon))$ queries to the sparse matrix A . You may invoke the best Hamiltonian simulator for sparse matrices from Section 9.4.

- (b) Show that the basic HHL algorithm can be implemented using $O(\kappa^2 s/\varepsilon + \kappa \log(\kappa/\varepsilon))$ sparse-access queries to A . To make your life easier, you may assume that $|\tilde{\lambda}_j\rangle$ is just one basis state, so one estimator which is close to λ_j rather than a superposition over estimators (and hence the success probability 0.99 is actually 1). You may also assume for simplicity that the amplitude amplification at the end works perfectly.
3. Suppose A and B are sparse, well-conditioned $N \times N$ matrices, and we can efficiently generate vector $b \in \mathbb{R}^N$ as a quantum state $|b\rangle$. Here we will see how we can efficiently find the solution to the linear system $ABx = b$ as a quantum state $|x\rangle$.
More precisely, assume $N = 2^n$. Assume we have a unitary circuit to produce the n -qubit state $|b\rangle$ with a number of elementary gates that’s polynomial in n . Let s_A and s_B be the

³Note that we need to assume a unitary to prepare $|b\rangle$ here, having just one copy of $|b\rangle$ is not enough. We cannot use *oblivious* amplitude amplification because that assumes we have a block-encoding of a matrix that is proportional to a unitary (or close to that), which A^{-1} is not.

sparsities of the matrices A and B , respectively, and κ_A and κ_B be their condition numbers (ratio of largest over smallest singular value). Let $x \in \mathbb{C}^N$ be $B^{-1}A^{-1}b$, which is the unique solution to the linear system $ABx = b$. Show how you can produce an n -qubit state $|\tilde{x}\rangle$ that is ε -close (in the usual Euclidean distance) to the n -qubit state $|x\rangle = \frac{1}{\|x\|} \sum_{i \in \{0,1\}^n} x_i |i\rangle$, using a number of queries (to matrix entries) and elementary gates that is polynomial in s_A , s_B , κ_A , κ_B , $1/\varepsilon$, and n .

Comment: Again, the point here is to avoid the polynomial dependence on the dimension N that classical linear solvers would have, and replace that by a polynomial dependence on $n = \log N$.

Chapter 11

Quantum Query Lower Bounds

11.1 Introduction

Most of the algorithms we have seen so far worked in the *query* model. Here the goal usually is to compute some function $f : \{0, 1\}^N \rightarrow \{0, 1\}$ on a given input $x = x_0 \dots x_{N-1} \in \{0, 1\}^N$. The distinguishing feature of the query model is the way x is accessed: x is not given explicitly, but is stored in a random access memory, and we're being charged unit cost for each *query* that we make to this memory. Informally, a query asks for and receives the i -th element x_i of the input. Formally, we model a query unitarily as the following 2-register quantum operation O_x , where the first register is N -dimensional and the second is 2-dimensional¹:

$$O_x : |i, b\rangle \mapsto |i, b \oplus x_i\rangle.$$

In particular, $|i, 0\rangle \mapsto |i, x_i\rangle$. This only states what O_x does on basis states, but by linearity this determines the full unitary. Note that a quantum algorithm can apply O_x to a superposition of basis states, gaining some sort of access to several input bits x_i at the same time.

A T -query quantum algorithm starts in a fixed state, say the all-0 state $|0 \dots 0\rangle$, and then interleaves fixed unitary transformations $U_0, U_1, \dots, U_T$ with queries. The algorithm's fixed unitaries may act on a workspace-register, in addition to the two registers on which O_x acts. In this case we implicitly extend O_x by tensoring it with the identity operation on this extra register, so it maps

$$O_x : |i, b, w\rangle \mapsto |i, b \oplus x_i, w\rangle.$$

Hence the final state of the algorithm can be written as the following matrix-vector product:

$$U_T O_x U_{T-1} O_x \dots O_x U_1 O_x U_0 |0 \dots 0\rangle.$$

This state depends on the input x only via the T queries. The output of the algorithm is obtained by a measurement of the final state. For instance, if the output is Boolean, the algorithm could just measure the final state in the computational basis and output the first bit of the result.

The query complexity of some function f is now the minimal number of queries needed for an algorithm that outputs the correct value $f(x)$ for every x in the domain of f (with error probability

¹If the input x consists of non-binary items x_i (as is the case for instance with the input for Simon's algorithm) then those can be simulated by querying individual bits of each x_i .

at most $1/3$, say). Note that we just count queries to measure the complexity of the algorithm², while the intermediate fixed unitaries are treated as costless.

In many cases, the overall computation time of quantum query algorithms (as measured by the total number of elementary gates, say) is not much bigger than the query complexity. This justifies analyzing the latter as a proxy for the former. This is the model in which essentially all the quantum algorithm we've seen work: Deutsch-Jozsa, Simon, Grover, the various random walk algorithms. Even the period-finding algorithm that is the quantum core of Shor's algorithm works because it needs only few queries to the periodic function.

11.2 The polynomial method

From quantum query algorithms to polynomials. An N -variate multilinear polynomial p is a function $p : \mathbb{C}^N \rightarrow \mathbb{C}$ that can be written as

$$p(x_0, \dots, x_{N-1}) = \sum_{S \subseteq \{0, \dots, N-1\}} a_S \prod_{i \in S} x_i,$$

for some complex numbers a_S . The *degree* of p is $\deg(p) = \max\{|S| : a_S \neq 0\}$. It is easy to show that every function $f : \{0, 1\}^N \rightarrow \mathbb{C}$ has a unique representation as such a polynomial; $\deg(f)$ is defined as the degree of that polynomial (see Exercise 1). For example, the 2-bit AND function is $p(x_0, x_1) = x_0 x_1$, and the 2-bit Parity function is $p(x_0, x_1) = x_0 + x_1 - 2x_0 x_1$. Both polynomials have degree 2. Sometimes a lower degree suffices for a polynomial to *approximate* the function. For example, $p(x_0, x_1) = \frac{1}{3}(x_0 + x_1)$ approximates the 2-bit AND function up to error $1/3$ for all inputs, using degree 1.

A very useful property of T -query algorithms is that the amplitudes of their final state are degree- T N -variate polynomials of x [113, 37]. More precisely: consider a T -query algorithm with input $x \in \{0, 1\}^N$ acting on an m -qubit space. Then its final state can be written

$$\sum_{z \in \{0, 1\}^m} \alpha_z(x) |z\rangle,$$

where each α_z is a multilinear complex-valued polynomial in x of degree at most T .

Proof. The proof is by induction on T . The base case ($T = 0$) trivially holds: the algorithm's state $U_0|0 \dots 0\rangle$ is independent of x , so its amplitudes are constants.

For the induction step, suppose we have already done T queries. Then by the induction hypothesis the state after U_T can be written as

$$\sum_{z \in \{0, 1\}^m} \alpha_z(x) |z\rangle,$$

where each α_z is a multilinear polynomial in x of degree at most T . Each basis state $|z\rangle = |i, b, w\rangle$ consists of 3 registers: the two registers $|i, b\rangle$ of the query, and a workspace register containing basis state $|w\rangle$. The algorithm now makes another query O_x followed by a unitary U_{T+1} . The query

²Clearly, N queries always suffice since we can just query each of the N input bits separately, thus learning x completely, and then look up and output whatever the correct value is for that input.

swaps basis states $|i, 0, w\rangle$ and $|i, 1, w\rangle$ if $x_i = 1$, and doesn't do anything to these basis states if $x_i = 0$. This changes amplitudes as follows:

$$\begin{aligned} & \alpha_{i,0,w}(x)|i, 0, w\rangle + \alpha_{i,1,w}(x)|i, 1, w\rangle \mapsto \\ & ((1 - x_i)\alpha_{i,0,w}(x) + x_i\alpha_{i,1,w}(x))|i, 0, w\rangle + (x_i\alpha_{i,0,w}(x) + (1 - x_i)\alpha_{i,1,w}(x))|i, 1, w\rangle. \end{aligned}$$

Now the new amplitudes are of the form $(1 - x_i)\alpha_{i,0,w}(x) + x_i\alpha_{i,1,w}(x)$ or $x_i\alpha_{i,0,w}(x) + (1 - x_i)\alpha_{i,1,w}(x)$. The new amplitudes are still polynomials in $x_0, \dots, x_{N-1}$. Their degree is at most 1 more than the degree of the old amplitudes, so at most $T + 1$. Finally, since U_{T+1} is a linear map that is independent of x , it does not increase the degree of the amplitudes further (the amplitudes after U_{T+1} are linear combinations of the amplitudes before U_{T+1}). This concludes the induction step.

Note that this construction could introduce degrees higher than 1, e.g., terms of the form x_i^2 . However, our inputs x_i are 0/1-valued, so we have $x_i^k = x_i$ for all integers $k \geq 1$. Accordingly, we can reduce higher degrees to 1, making the polynomials multilinear without increasing degree. $\square$

Suppose our algorithm acts on an m -qubit state. If we measure the first qubit of the final state and output the resulting bit, then the probability of output 1 is given by

$$p(x) = \sum_{z \in \{1\} \times \{0,1\}^{m-1}} |\alpha_z(x)|^2.$$

This is a real-valued polynomial of x of degree at most $2T$, because $|\alpha_z(x)|^2$ is the sum of the squares of the real and imaginary parts of the amplitude $\alpha_z(x)$, and each of those two parts is a polynomial of degree $\leq T$. Note that if the algorithm computes f with error $\leq 1/3$, then p is an approximating polynomial for f : if $f(x) = 0$ then $p(x) \in [0, 1/3]$ and if $f(x) = 1$ then $p(x) \in [2/3, 1]$. This gives a method to lower bound the minimal number of queries needed to compute f : if one can show that every polynomial that approximates f has degree at least d , then every quantum algorithm computing f with error $\leq 1/3$ must use at least $d/2$ queries.

Applications of the polynomial method. For our examples we will restrict attention to *symmetric* functions.³ Those are the ones where the function value $f(x)$ only depends on the Hamming weight (number of 1s) in the input x . Examples are N -bit OR, AND, Parity, Majority, etc.

Suppose we have a polynomial $p(x_0, \dots, x_{N-1})$ that approximates f with error $\leq 1/3$. Then it is easy to see that a polynomial that averages over all permutations π of the N input bits $x_0, \dots, x_{N-1}$:

$$q(x) = \frac{1}{N!} \sum_{\pi \in S_N} p(\pi(x)),$$

still approximates f . As it turns out, we can define a single-variate polynomial $r(z)$ of the same degree as q , such that $q(x) = r(|x|)$.⁴ This r is defined on all real numbers, and we know something

³One can also use the polynomial method for non-symmetric functions, for instance to prove a tight lower bound of $\Omega(N^{2/3})$ queries for the general problem of collision-finding; this matches the quantum walk algorithm of Section 8.3.2. However, that lower bound proof is substantially more complicated and we won't give it here (see [4]).

⁴To see why this is the case, note that for every degree i , all degree- i monomials in the symmetrized polynomial q have the same coefficient a_i . Moreover, on input $x \in \{0,1\}^N$ of Hamming weight z , exactly $\binom{z}{i}$ of the degree- i monomials are 1, while the others are 0. Hence $q(x) = \sum_{i=0}^d a_i \binom{z}{i}$. Since $\binom{z}{d} = z(z-1)\cdots(z-d+1)/d!$ is a single-variate polynomial in z of degree d , we can define $r(z) = \sum_{i=0}^d a_i \binom{z}{i}$. For example, if $q(x_0, x_1) = x_0 x_1$ then r would be the unique univariate polynomials such that $r(0) = 0$, $r(1) = 0$ and $r(2) = 1$, i.e., $r(z) = z(z-1)/2 = \binom{z}{2}$.

about its behavior on integer points $\{0, \dots, N\}$. Thus it suffices to lower bound the degree of single-variate polynomials with the appropriate behavior.

For an important example, consider the N -bit OR function. Grover’s algorithm can find an i such that $x_i = 1$ (if such an i exists) and hence can compute the OR function with error probability $\leq 1/3$ using $O(\sqrt{N})$ queries. By the above reasoning, any T -query quantum algorithm that computes the OR with error $\leq 1/3$ induces a single-variate polynomial r satisfying

$$r(0) \in [0, 1/3], \text{ and } r(t) \in [2/3, 1] \text{ for all integers } t \in \{1, \dots, N\}.$$

This polynomial $r(x)$ “jumps” between $x = 0$ and $x = 1$ (i.e., it has a derivative $r'(x) \geq 1/3$ for some $x \in [0, 1]$), while it remains fairly constant on the domain $\{1, \dots, N\}$. By a classical theorem from approximation theory (proved independently around the same time by Ehlich and Zeller [105], and by Rivlin and Cheney [212]), such polynomials must have degree $d \geq \Omega(\sqrt{N})$. Hence $T \geq \Omega(\sqrt{N})$ as well. Accordingly, Grover’s algorithm is optimal (up to a constant factor) in terms of number of queries.

What about *exact* algorithms for OR? Could we tweak Grover’s algorithm so that it always finds a solution with probability 1 (if one exists), using $O(\sqrt{N})$ queries? This turns out not to be the case: a T -query exact algorithm for OR induces a polynomial r of degree $\leq 2T$ that satisfies

$$r(0) = 0, \text{ and } r(t) = 1 \text{ for all integers } t \in \{1, \dots, N\}.$$

It is not hard to see that such a polynomial needs degree at least N : observe that $r(x) - 1$ is a non-constant polynomial with at least N roots.⁵ Hence $T \geq N/2$ (this can be improved to $T \geq N$, see Exercise 5). Accordingly, Grover cannot be made exact without losing the square-root speed-up!

Using the polynomial method, one can in fact show for *every symmetric function* f that is defined on all 2^N inputs, that quantum algorithms cannot provide a more-than-quadratic speed-up over classical algorithms. More generally, for *every* function f (symmetric or non-symmetric) that is defined on all inputs⁶, quantum algorithms cannot provide a more-than-6th-root speed-up over classical algorithms (see Exercise 11). The polynomial method has recently been strengthened by Arunachalam et al. [26] to an *optimal* lower bound method, by imposing more constraints on the polynomial (which can increase the required degree, while still giving a lower bound on quantum query complexity).

11.3 The quantum adversary method

The polynomial method has a strength which is also a weakness: it applies even to a stronger (and less physically meaningful) model of computation where we allow *any linear transformation* on the state space, not just unitary ones. As a result, it does not always provide the strongest possible lower bound for quantum query algorithms.

Ambainis [12, 13] provided an alternative method for quantum lower bounds, the *quantum adversary*. This exploits unitarity in a crucial way and in certain cases yields a provably better bound than the polynomial method [13]. We will present a very simple version of the adversary method here, a much stronger (in fact optimal!) version is given in Chapter 12.

⁵A “root” is an x such that $r(x) = 0$. It is a well-known fact from algebra that every univariate non-constant polynomial of degree d has at most d roots (over any field). Note that this is not true for *multivariate* polynomials; for example the polynomial $x_0 \cdots x_{N-1}$ has $2^N - 1$ roots in $\{0, 1\}^N$ but its degree is only N .

⁶Note that this doesn’t include functions where the input has to satisfy a certain promise, such as Deutsch-Jozsa and Simon’s problem.

Recall that a quantum query algorithm is a sequence

$$U_T O_x U_{T-1} O_x \cdots O_x U_1 O_x U_0,$$

applied to the fixed starting state $|0 \dots 0\rangle$, where the basic “query transformation” O_x depends on the input x , and $U_0, U_1, \dots, U_T$ are arbitrary unitaries that don’t depend on x . Consider the evolution of our quantum state under all possible choices of x . Let $|\psi_x^t\rangle$ denote the state after applying U_t when the input is x . In particular, $|\psi_x^0\rangle = U_0|0 \dots 0\rangle$ for all x (and hence $\langle \psi_x^0 | \psi_y^0 \rangle = 1$ for each x, y), and $|\psi_x^T\rangle$ is the final state of the algorithm on input x before the final measurement. Now if the algorithm computes the Boolean function f with success probability $2/3$ on every input, then the final measurement must accept (i.e., output 1) every $x \in f^{-1}(0)$ with probability $\leq 1/3$, and must accept every $y \in f^{-1}(1)$ with probability $\geq 2/3$. This means the two states $|\psi_x^T\rangle$ and $|\psi_y^T\rangle$ cannot be too close together, or equivalently their inner product cannot be too close to 1. Specifically, we must have $|\langle \psi_x^T | \psi_y^T \rangle| \leq \frac{17}{18}$.⁷ This suggests that we find a set $R \subseteq f^{-1}(0) \times f^{-1}(1)$ of hard-to-distinguish (x, y) -pairs, and consider the following *progress measure*

$$S_t = \sum_{(x,y) \in R} |\langle \psi_x^t | \psi_y^t \rangle|$$

as a function of t . By our observations, initially we have $S_0 = |R|$, and in the end we must have $S_T \leq \frac{17}{18}|R|$. Also, crucially, the progress measure is *unaffected* by each application of a unitary U_t , since each U_t is independent of the input and unitary transformations preserve inner products.

If we can determine an upper bound Δ on the change $|S_{t+1} - S_t|$ in the progress measure at each step, we can conclude that the number T of queries is at least $\frac{|R|}{18\Delta}$. Ambainis proved the following. Suppose that

- (i) each $x \in f^{-1}(0)$ appearing in R , appears at least m_0 times in pairs (x, y) in R ;
- (ii) each $y \in f^{-1}(1)$ appearing in R , appears at least m_1 times in pairs (x, y) in R ;
- (iii) for each $x \in f^{-1}(0)$ and $i \in \{0, \dots, N-1\}$, there are at most ℓ_0 inputs $y \in f^{-1}(1)$ such that $(x, y) \in R$ and $x_i \neq y_i$;
- (iv) for each $y \in f^{-1}(1)$ and $i \in \{0, \dots, N-1\}$, there are at most ℓ_1 inputs $x \in f^{-1}(0)$ such that $(x, y) \in R$ and $x_i \neq y_i$.

Then for all $t \geq 0$, $|S_{t+1} - S_t| = O\left(\sqrt{\frac{\ell_0}{m_0} \cdot \frac{\ell_1}{m_1}} \cdot |R|\right) =: \Delta$. We will not prove this inequality here, though it is a reasonably straightforward generalization of the answer to Exercise 12, and we will see a stronger result in the next chapter. This upper bound Δ on the progress-per-query immediately implies a lower bound on the number of queries:

$$T = \Omega\left(\sqrt{\frac{m_0}{\ell_0} \cdot \frac{m_1}{\ell_1}}\right). \quad (11.1)$$

⁷Remember Exercise 3 from Chapter 4 for states $|\phi\rangle$ and $|\psi\rangle$: if $\|\phi - \psi\| = \varepsilon$, then the total variation distance between the probability distributions you get from measuring $|\phi\rangle$ and $|\psi\rangle$, respectively, is at most ε . Hence, if we know there is a two-outcome measurement that accepts $|\phi\rangle$ with probability $\leq 1/3$ and accepts $|\psi\rangle$ with probability $\geq 2/3$, then we must have total variation distance at least $1/3$ and hence $\varepsilon \geq 1/3$. Assume for simplicity that the inner product $\langle \phi | \psi \rangle$ is real. Via the equation $\varepsilon^2 = \|\phi - \psi\|^2 = 2 - 2\langle \phi | \psi \rangle$, this translates into an upper bound $|\langle \phi | \psi \rangle| \leq 1 - \varepsilon^2/2 \leq 17/18$ (this upper bound can be improved to $2\sqrt{2}/3$ with more careful analysis).

Intuitively, conditions (i)-(iv) imply that $|S_{t+1} - S_t|$ is small relative to $|R|$ by bounding the “distinguishing ability” of any query. The art in applying this technique lies in choosing the relation R carefully to maximize this quantity, i.e., make m_0 and/or m_1 large, while keeping ℓ_0 and ℓ_1 small.

Note that for the N -bit OR function this method easily gives the optimal $\Omega(\sqrt{N})$ lower bound, as follows. Choose $R = \{(x, y) : x = 0^N, y \text{ has Hamming weight } 1\}$. Then $m_0 = N$ while $m_1 = \ell_0 = \ell_1 = 1$. Plugging this into Eq. (11.1) gives the right $\Omega(\sqrt{N})$ bound.

Let us give another application, a lower bound that is much harder to prove using the polynomial method. Suppose $f : \{0, 1\}^N \rightarrow \{0, 1\}$ is a 2-level AND-OR tree, with $N = k^2$ input bits: f is the AND of k ORs, each of which has its own set of k inputs bits. By carefully doing 2 levels of Grover search (search for a subtree which is 0^k), one can construct a quantum algorithm that computes f with small error probability and $O(\sqrt{k} \cdot \sqrt{k}) = O(\sqrt{N})$ queries. It was long an open problem to give a matching lower bound on the approximate degree, and this was proved only in 2013 [226, 76]. In contrast, the adversary method gives the optimal lower bound on the quantum query complexity quite easily: choose the relation R as follows

R consists of those pairs (x, y) where
 x has one subtree with input 0^k and the other $k - 1$ subtrees have an arbitrary k -bit input of Hamming weight 1 (note $f(x) = 0$)
 y is obtained from x by changing one of the bits of the 0^k -subtree to 1 (note $f(y) = 1$).

Then $m_0 = m_1 = k$ and $\ell_0 = \ell_1 = 1$, and we get a lower bound of $\Omega\left(\sqrt{\frac{m_0 m_1}{\ell_0 \ell_1}}\right) = \Omega(k) = \Omega(\sqrt{N})$. Another lower bound one can prove fairly easily using a strengthened version of the adversary method is for inverting a permutation, see Exercise 9.

Exercises

1. Consider a function $f : \{0, 1\}^N \rightarrow \mathbb{R}$. Show that this function can be represented by an N -variate multilinear polynomial of degree $\leq N$, and that this representation is unique.
2. Consider a 2-bit input $x = x_0 x_1$ with phase-oracle $O_{x, \pm} : |i\rangle \mapsto (-1)^{x_i} |i\rangle$. Write out the final state of the following 1-query quantum algorithm: $H O_{x, \pm} H |0\rangle$. Give a degree-2 polynomial $p(x_0, x_1)$ that equals the probability that this algorithm outputs 1 on input x . What function does this algorithm compute?
3. Consider polynomial $p(x_0, x_1) = 0.3 + 0.4x_0 + 0.5x_1$, which approximates the 2-bit OR function. Write down the symmetrized polynomial $q(x_0, x_1) = \frac{1}{2}(p(x_0, x_1) + p(x_1, x_0))$. Give a single-variate polynomial r such that $q(x) = r(|x|)$ for all $x \in \{0, 1\}^2$.
4. (H) Let f be the N -bit Parity function, which is 1 if its input $x \in \{0, 1\}^N$ has odd Hamming weight, and 0 if the input has even Hamming weight (assume N is an even number).
 - (a) Give a quantum algorithm that computes Parity with success probability 1 on every input x , using $N/2$ queries.
 - (b) Show that this is optimal, even for quantum algorithms that have error probability $\leq 1/3$ on every input

5. Suppose we have a T -query quantum algorithm that computes the N -bit AND function with success probability 1 on all inputs $x \in \{0,1\}^N$. In Section 11.2 we showed that such an algorithm has $T \geq N/2$ (we showed it for OR, but the same argument works for AND). Improve this lower bound to $T \geq N$.
6. Consider the following 3-bit function $f : \{0,1\}^3 \rightarrow \{0,1\}$:
 $f(x_0, x_1, x_2) = 1$ if $x_0 = x_1 = x_2$, and $f(x_0, x_1, x_2) = 0$ otherwise
 - (a) How many queries does a classical deterministic algorithm need to compute f ? Explain your answer.
 - (b) Give a quantum algorithm that computes f with success probability 1 using 2 queries.
 - (c) (H) Show that 2 queries is optimal: there is no quantum algorithm that computes f with success probability 1 using only 1 query.
7. Let f be the N -bit Majority function, which is 1 if its input $x \in \{0,1\}^N$ has Hamming weight $> N/2$, and 0 if the input has Hamming weight $\leq N/2$ (assume N is even).
 - (a) Prove that $\deg(f) \geq N/2$. What does this imply for the query complexity of exact quantum algorithms that compute majority?
 - (b) (H) Use the adversary method to show that every bounded-error quantum algorithm for computing Majority, needs $\Omega(N)$ queries. Be explicit about what relation R you're using, and about the values of the parameters m_0, m_1, ℓ_0, ℓ_1 .
8. Let k be an odd natural number, $N = k^2$, and define the Boolean function $f : \{0,1\}^N \rightarrow \{0,1\}$ as the k -bit majority of k separate k -bit OR functions. In other words, the N -bit input is $x = x^{(1)} \dots x^{(k)}$ with $x^{(i)} \in \{0,1\}^k$ for each $i \in [k]$, and $f(x)$ is the majority value of the k bits $\text{OR}(x^{(1)}), \dots, \text{OR}(x^{(k)})$. Use the adversary method to prove that computing this f with error probability $\leq 1/3$ requires $\Omega(N^{3/4})$ quantum queries. Be explicit about what relation R you're using, and about the values of the parameters m_0, m_1, ℓ_0, ℓ_1 .
9. This question is about the quantum complexity of inverting a permutation, which is an important problem in cryptography. Let N be a power of 2 and $S = \{0, \dots, N-1\}$. Let $x \in S^N$ correspond to a permutation on S , meaning that each $j \in S$ occurs exactly once as an entry of x (so the map $i \mapsto x_i$ is a permutation). Suppose we can query x , i.e., we have a unitary O_x that maps $|i, j\rangle \rightarrow |i, x_i + j \bmod N\rangle$ for all $i, j \in S$, and we can also apply O_x^{-1} .
 - (a) Show how we can find the unique index $i \in S$ for which $x_i = 0$, with success probability $\geq 2/3$, using $O(\sqrt{N})$ queries to O_x and O_x^{-1} .
 - (b) (H) The adversary lower bound of Section 11.3 still works with the following modifications:
 - (1) the x 's and y 's are not binary strings, but strings over a larger alphabet, such as S , and
 - (2) let $\ell_{x,i}$ be the number of $y \in Y$ such that $(x, y) \in R$ and $x_i \neq y_i$; $\ell_{y,i}$ be the number of $x \in X$ such that $(x, y) \in R$ and $x_i \neq y_i$; and $\ell_{\max} = \max\{\ell_{x,i} \cdot \ell_{y,i} : (x, y) \in R, i \in \{0, \dots, N-1\}, x_i \neq y_i\}$.
 In this case the quantum query lower bound is $\Omega(\sqrt{m_0 m_1 / \ell_{\max}})$. You may assume this without proof.
 Use this strengthened adversary bound to show a lower bound of $\Omega(\sqrt{N})$ quantum queries for computing the task of part (a).

10. (H) Consider the *sorting* problem: there are N numbers $a_1, \dots, a_N$ and we want to sort these. We can only access the numbers by making *comparisons*. A comparison is similar to a black-box query: it takes 2 indices i, j as input and outputs whether $a_i < a_j$ or not. The output of a sorting algorithm should be the list of N indices, sorted in increasing order. It is known that for classical computers, $N \log(N) + O(N)$ comparisons are necessary and sufficient for sorting. Prove that a quantum algorithm needs at least $\Omega(N)$ comparisons for sorting, even if it is allowed an error probability $\leq 1/3$.
11. Consider a total Boolean function $f : \{0, 1\}^N \rightarrow \{0, 1\}$. Given an input $x \in \{0, 1\}^N$ and subset $B \subseteq \{0, \dots, N-1\}$ of indices of variables, let x^B denote the N -bit input obtained from x by flipping all bits x_i whose index i is in B . The *block sensitivity* $bs(f, x)$ of f at input x , is the maximal integer k such that there exist disjoint sets $B_1, \dots, B_k$ satisfying $f(x) \neq f(x^{B_i})$ for all $i \in [k]$. The *block sensitivity* $bs(f)$ of f is $\max_x bs(f, x)$.
- (a) (H) Show that the bounded-error quantum query complexity of f is $\Omega(\sqrt{bs(f)})$.
- (b) It is known that for every total Boolean function f , there is a classical deterministic algorithm that computes it using $O(bs(f)^3)$ many queries. What can you conclude from this and part (a) about the relation between deterministic and quantum query complexity for total functions?
12. (H) In this exercise we will derive the quantum lower bound for the search problem in a self-contained way, without using the polynomial or adversary method (this exercise uses what is called the “hybrid method”).

Let $N = 2^n$. Consider an input $x \in \{0, 1\}^N$ that we can query. Assume x has Hamming weight 0 or 1, and suppose we would like to find the unique solution to the search problem (if a solution exists). Let $\mathcal{A}$ be any T -query quantum algorithm for this. Suppose for simplicity that the algorithm acts on only n qubits (so there are no auxiliary qubits), and $\mathcal{A} = U_T O_{x,\pm} U_{T-1} O_{x,\pm} \dots U_1 O_{x,\pm} U_0$, so $\mathcal{A}$ interleaves phase-queries to x and unitaries that are independent of x . The initial state is $|0^n\rangle$. Let $|\psi_x^t\rangle$ denote the n -qubit state right after applying U_t , when we run $\mathcal{A}$ on input x , so the final state is $|\psi_x^T\rangle$. Let $e_i \in \{0, 1\}^N$ be the input that has a 1 only at position i . Assume the algorithm $\mathcal{A}$ is successful in finding the right solution i after T queries in the following sense: $\| |\psi_{e_i}^T\rangle - |i\rangle \| \leq 1/4$ and $\| |\psi_{0_N}^T\rangle - |i\rangle \| \geq 3/4$ for all $i \in \{0, \dots, N-1\}$ (note that the basic Grover algorithm is an example of such an $\mathcal{A}$).

- (a) Consider the run of algorithm $\mathcal{A}$ on input $x = 0^N$, and for $t \in \{0, \dots, T-1\}$ let the amplitudes $\alpha_{t,i}$ be such that $|\psi_{0_N}^t\rangle = \sum_{i=0}^{N-1} \alpha_{t,i} |i\rangle$.
Prove that $\| |\psi_{0_N}^1\rangle - |\psi_{e_i}^1\rangle \| \leq 2|\alpha_{0,i}|$, for all $i \in \{0, \dots, N-1\}$.
- (b) Prove that $\| |\psi_{0_N}^T\rangle - |\psi_{e_i}^T\rangle \| \leq 2 \sum_{t=0}^{T-1} |\alpha_{t,i}|$, for all $i \in \{0, \dots, N-1\}$.
- (c) Prove that $1/2 \leq \| |\psi_{0_N}^T\rangle - |\psi_{e_i}^T\rangle \|$, for all $i \in \{0, \dots, N-1\}$.
- (d) Prove that $T \geq \sqrt{N}/4$.
13. Consider a standard quantum query algorithm: it makes T queries to a string $x \in \{0, 1\}^N$, with arbitrary unitaries $U_0, U_1, \dots, U_T$ (that are independent of x) around the queries, and then measures a POVM $\{M, I - M\}$ on the final m -qubit state $|\psi_x\rangle$.

- (a) Show that the probability $P(x)$ of getting the first measurement outcome (on input x) is $\langle \psi_x | M | \psi_x \rangle$, and that this can be written as an N -variate multilinear polynomial in the bits of x of degree $\leq 2T$.
- (b) (H) A *k-wise independent distribution* D is a probability distribution over $\{0, 1\}^N$, such that for each set $S \subseteq [N]$ of at most k coordinates, the distribution on the k -bit substring $x_S = (x_i)_{i \in S}$ is uniformly random (i.e, for each $z \in \{0, 1\}^k$, the probability under distribution D of the event that $x_S = z$, is $1/2^k$).

Show that a T -query quantum algorithm cannot distinguish the uniform distribution U on its input x from a $2T$ -wise independent distribution D on x , in the sense that no matter what binary measurement the algorithm does at the end, the probability of output 1 is the same under U and under D .

Chapter 12

Quantum Algorithms from the Generalized Adversary Bound

12.1 The generalized adversary bound

In the previous chapter we saw two different lower bound methods for the quantum query complexity of a given function f : the polynomial method and the adversary method. Neither is optimal for every possible f . For example, the polynomial method doesn't give optimal lower bounds for iterations of some small functions [13] (see Exercise 1), while the adversary bound of Section 11.3 cannot prove optimal lower bounds for instance for distinguishing 2-to-1 from 1-to-1 inputs.

In this chapter we will look at a stronger version of the adversary bound, which turns out to give *optimal* quantum query complexity lower bounds for all Boolean functions. The beauty of an optimal lower bound method is that it can also produce algorithms: if the best-possible lower bound on the query complexity of f is T , then there must actually exist a T -query algorithm for f !

Suppose $f : \mathcal{D} \rightarrow \{0, 1\}$, with $\mathcal{D} \subseteq \{0, 1\}^N$, is a Boolean function whose quantum query complexity we'd like to determine.¹ Consider a T -query quantum algorithm $\mathcal{A} = U_T O_x U_{T-1} \cdots U_1 O_x U_0$, with initial state $|0^m\rangle$, that computes f with error probability $\leq \varepsilon < 1/2$ for each $x \in \mathcal{D}$. Let $|\psi_x^t\rangle$ denote the algorithm's state after U_t has been applied, given input x . Note that $|\psi_x^0\rangle = U_0|0^m\rangle$ is independent of x . The crucial property, already used in the earlier version of the adversary bound, is that $\langle \psi_x^t | \psi_y^t \rangle$ is 1 at the start ($t = 0$), but has to be small at the end ($t = T$) for every (x, y) -pair with different function values $f(x) \neq f(y)$.

The generalized adversary matrix puts weights $\alpha_x \in \mathbb{C}$ on the inputs $x \in \mathcal{D}$, with the constraint $\sum_{x \in \mathcal{D}} |\alpha_x|^2$ for normalization. It also puts real (but possibly negative!) weights Γ_{xy} on (x, y) -pairs with different function values. We impose the constraint that $\Gamma_{xy} = \Gamma_{yx}$, and $\Gamma_{xy} = 0$ whenever $f(x) = f(y)$. A $|\mathcal{D}| \times |\mathcal{D}|$ matrix Γ with these properties is called an *adversary matrix*.

Let us use these weights to define a progress measure:²

$$S_t = \sum_{x, y \in \mathcal{D}} \Gamma_{xy} \alpha_x^* \alpha_y \langle \psi_x^t | \psi_y^t \rangle.$$

In the same spirit as Section 11.3, we will show that $|S_0|$ is large, that $|S_T|$ is much smaller, and

¹If the domain is $\mathcal{D} = \{0, 1\}^N$ then f is called a *total* function, otherwise it's a *partial* function.

²We could have just absorbed the $\alpha_x^* \alpha_y$ into the Γ_{xy} and dispensed with the α_x 's altogether, but it will be cleaner to have separate weights on the x 's and separate weights Γ_{xy} on the pairs.

that S_{t+1} can't be too different from S_t (i.e., S_t can't change very fast if we spend one more query). This will give the lower bound on T .

At the start of the algorithm ($t = 0$), before any queries have been made, we have $\langle \psi_x^0 | \psi_y^0 \rangle = 1$ for all x, y and hence $S_0 = \sum_{x,y} \Gamma_{xy} \alpha_x^* \alpha_y = \alpha^* \Gamma \alpha$. Since α is restricted to a unit vector, the biggest we can make $|S_0|$ is

$$|S_0| = \|\Gamma\|,$$

the operator norm (largest singular value) of Γ , by choosing α to be an eigenvector of Γ corresponding to the largest eigenvalue in absolute value.

At the end of the algorithm ($t = T$), the final states $|\psi_x^T\rangle$ and $|\psi_y^T\rangle$ must be distinguishable with success probability $\geq 1 - \varepsilon$ whenever $f(x) \neq f(y)$. The following claim, proved in Exercise 2, shows that this forces $|S_T|$ to be significantly smaller than $|S_0|$.

Claim 1 $|S_T| \leq 2\sqrt{\varepsilon(1-\varepsilon)} \|\Gamma\|$.

For example, if our algorithm has error probability $\varepsilon = 1/3$, then $|S_T| < 0.95 \|\Gamma\|$. Accordingly, the progress measure has to change significantly in the course of the T -query algorithm. How much can one more query change S_t ? This is upper bounded by the following claim, proved in Exercise 3.

Claim 2 Let Γ_i denote the $|\mathcal{D}| \times |\mathcal{D}|$ matrix obtained from Γ by setting Γ_{xy} to 0 if $x_i = y_i$. For all $t \in \{0, \dots, T-1\}$ we have $|S_t - S_{t+1}| \leq 2 \max_{i \in [N]} \|\Gamma_i\|$.

These two claims, together with our value for the initial $|S_0|$, imply

$$\left(1 - 2\sqrt{\varepsilon(1-\varepsilon)}\right) \|\Gamma\| \leq |S_0| - |S_T| \leq |S_0 - S_T| = \left| \sum_{t=0}^{T-1} S_t - S_{t+1} \right| \leq \sum_{t=0}^{T-1} |S_t - S_{t+1}| \leq 2T \max_{i \in [N]} \|\Gamma_i\|.$$

We get the following lower bound on quantum query complexity, due to Høyer, Lee, and Špalek [139]:

Theorem 2 (Generalized adversary bound) Let $f : \mathcal{D} \rightarrow \{0, 1\}$, with $\mathcal{D} \subseteq \{0, 1\}^N$, and Γ be an adversary matrix for f . Every quantum algorithm that computes f with worst-case error probability $\leq \varepsilon$, needs at least $\left(\frac{1}{2} - \sqrt{\varepsilon(1-\varepsilon)}\right) \frac{\|\Gamma\|}{\max_{i \in [N]} \|\Gamma_i\|}$ queries.

As an example, let us (again) prove the $\Omega(\sqrt{N})$ lower bound for search. Consider the domain $\mathcal{D} = \{0^N, e_1, \dots, e_N\}$ of inputs of weight 0 or 1 (e_j is the N -bit string that has a 1 only at position j). Define $(N+1) \times (N+1)$ adversary matrix

$$\Gamma = \begin{pmatrix} 0 & 1 & \dots & 1 \\ 1 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 0 & \dots & 0 \end{pmatrix}.$$

Then $\|\Gamma\| \geq \|\Gamma(1, 0, \dots, 0)^T\| = \sqrt{N}$. Each Γ_i is the 2×2 X -matrix padded with extra rows and columns of 0s, so $\|\Gamma_i\| = 1$. Hence we obtain the familiar lower bound of $\Omega(\sqrt{N})$.

More generally, we can recover the lower bound of Section 11.3 by constructing an appropriate adversary matrix Γ based on the relation R . The rows and columns of Γ are indexed by $\mathcal{D} =$

$f^{-1}(0) \cup f^{-1}(1)$. Define $d_x = |\{y : (x, y) \in R\}|$ and similarly define d_y . Define $\Gamma_{xy} = 1/\sqrt{d_x d_y}$ if $(x, y) \in R$ or $(y, x) \in R$, and $\Gamma_{xy} = 0$ otherwise. Let v be the vector with entries $\sqrt{d_x}$, and note that $\|v\|^2 = |R|$. We have $\|\Gamma\| \geq \frac{1}{\|v\|^2} v^T \Gamma v = \frac{1}{|R|} \sum_{(x,y) \in R} 1 = 1$. We can also show $\|\Gamma_i\| \leq \sqrt{\frac{\ell_0 \ell_1}{m_0 m_1}}$ for all i .³ Now Theorem 2 gives the $\Omega\left(\sqrt{\frac{m_0 m_1}{\ell_0 \ell_1}}\right)$ bound of Eq. (11.1).

12.2 The dual of the generalized adversary bound

Consider the best-possible lower bound that we can obtain by optimizing over adversary matrix Γ :

$$\begin{aligned} \max \quad & \|\Gamma\| \\ \text{s.t.} \quad & \|\Gamma_i\| \leq 1 \quad \forall i \in [N] \\ & \Gamma \text{ is symmetric} \\ & \Gamma_{xy} = 0 \quad \forall x, y \in \mathcal{D} \text{ with } f(x) = f(y) \end{aligned}$$

We will call the optimal value the *generalized adversary bound* for f (a.k.a. the *negative-weights adversary bound*), and denote it by $\text{ADV}^\pm(f)$. Because the statement $\|\Gamma_i\| \leq 1$ is equivalent to the matrix inequality $-I \preceq \Gamma_i \preceq I$, the above maximization problem can be written in the form of a so-called *semidefinite program*: an optimization problem over real-valued variables, typically arranged in one or more matrices, with an objective function that's linear in the variables, and psd constraints that are linear in the variables as well. Every maximization-SDP has an associated minimization-SDP which (under mild assumptions that hold in our case) has the *same optimal value*. The first SDP is called the *primal* SDP, the second is called the *dual* SDP.⁴ The equality of the optimal values of these two SDPs is called *strong duality* (see Exercise 4 for a proof of the easy half of this equality). This generalizes the better-known strong duality of linear programs, which correspond to SDPs with diagonal matrices.

With some effort that we will skip here, one can show that the dual of the above maximization-SDP can be written as the following minimization-SDP:

$$\begin{aligned} \min \quad & \max_{x \in \mathcal{D}} \max \left(\sum_{j \in [N]} \|u_{xj}\|^2, \sum_{j \in [N]} \|v_{xj}\|^2 \right) \\ \text{s.t.} \quad & \sum_{j: x_j \neq y_j} \langle u_{xj} | v_{yj} \rangle = [f(x) \neq f(y)] \quad \forall x, y \in \mathcal{D} \end{aligned}$$

The truth-value $[f(x) \neq f(y)]$ is 1 if $f(x) \neq f(y)$, and 0 if $f(x) = f(y)$. This SDP associates with every $x \in \mathcal{D}$ and every $j \in [N]$ two vectors u_{xj} and v_{xj} (of some dimension d that will implicitly be optimized over). We can write this more explicitly as an optimization problem over psd matrices

³Define two $|\mathcal{D}| \times |\mathcal{D}|$ matrices A, B by $A_{xy} = 1/\sqrt{d_x}$ if $x_i \neq y_i$ and $((x, y) \in R \text{ or } (y, x) \in R)$, and $A_{xy} = 0$ otherwise; and $B_{xy} = 1/\sqrt{d_y}$ if $x_i \neq y_i$ and $((x, y) \in R \text{ or } (y, x) \in R)$, and $B_{xy} = 0$ otherwise. Note that $\Gamma_i = A \circ B$, where ' $\circ$ ' denotes the entrywise product. For matrices with nonnegative entries, it is known [231, Appendix A] that $\|A \circ B\|$ is at most the largest norm among the rows of A (which is $\leq \sqrt{\ell_0/m_0}$, because each row of A has at most ℓ_0 nonzero entries, each of which is at most $1/\sqrt{m_0}$) times the largest norm among the columns of B (which is $\leq \sqrt{\ell_1/m_1}$). Hence $\|\Gamma_i\| \leq \sqrt{\frac{\ell_0 \ell_1}{m_0 m_1}}$.

⁴Strictly speaking the maximum in the primal should be a supremum and the minimum in the dual should be an infimum, because the optimal value need not always be attained; it could be that there's only an infinite sequence of feasible solutions whose objective values converge to the optimum without ever reaching it.

by defining, for each $j \in [N]$, a $2|\mathcal{D}| \times 2|\mathcal{D}|$ psd matrix Z_j whose entries are given by the pairwise inner products of the $2|\mathcal{D}|$ vectors u_{xj}, v_{xj} (these vectors are the “Gram vectors” of Z_j). Then the optimization is over psd matrices $Z_1, \dots, Z_N$, the objective function is the largest diagonal entry of the matrix $\sum_{j \in [N]} Z_j$, and the constraints are linear functions of the entries of the Z_j ’s.

By strong duality, the optimal value of this minimization-SDP is $\text{ADV}^\pm(f)$ as well. A feasible solution Γ to the primal gives a lower bound on $\text{ADV}^\pm(f)$, while a feasible solution $\{u_{xj}, v_{xj}\}_{x \in \mathcal{D}, j \in [N]}$ for the dual gives an upper bound on $\text{ADV}^\pm(f)$. The central result of this chapter is that $\text{ADV}^\pm(f)$ is not only a lower bound on the quantum query complexity of f (which follows from Theorem 2) but also an *upper* bound, as we will see in the next section. This means that *a feasible solution to the dual SDP actually gives us an algorithm for f !*

12.3 $\text{ADV}^\pm$ is an upper bound on quantum query complexity

In this section we will construct a bounded-error quantum algorithm for computing f , derived from a feasible solution $\{u_{xj}, v_{xj}\}_{x \in \mathcal{D}, j \in [N]}$ of the dual SDP for $\text{ADV}^\pm(f)$.⁵ Let’s say the objective value of this feasible solution is A ; the query complexity of our algorithm will turn out to be $O(A)$. Below x, y always range over $\mathcal{D}$, and j always ranges over $[N]$.

Our algorithm will act on 3 registers. The first register is spanned by $|j\rangle, j \in [N]$, the second is 1 qubit, and the third contains the states $|v_{xj}\rangle \in \text{span}\{|1\rangle, \dots, |d\rangle\}$ and the special state $|0\rangle$ (so the third register has $\lceil \log(d+1) \rceil$ qubits). For each x , define the following two 3-register states:

$$|t_x^+\rangle = \frac{1}{\sqrt{2}}(|0\rangle|0\rangle|0\rangle + |1\rangle|f(x)\rangle|0\rangle) \quad \text{and} \quad |t_x^-\rangle = \frac{1}{\sqrt{2}}(|0\rangle|0\rangle|0\rangle - |1\rangle|f(x)\rangle|0\rangle).$$

The algorithm starts with the all-0 state

$$|0\rangle|0\rangle|0\rangle = \frac{1}{\sqrt{2}}(|t_x^+\rangle + |t_x^-\rangle).$$

The goal of the algorithm is to (approximately) multiply $|t_x^+\rangle$ with $+1$ and $|t_x^-\rangle$ with -1 , which rather magically gives a final state that tells us $f(x)$:

$$\frac{1}{\sqrt{2}}(|t_x^+\rangle - |t_x^-\rangle) = |1\rangle|f(x)\rangle|0\rangle.$$

The key will be to use phase estimation (Section 4.6) with a well-chosen unitary U_x that depends on x , to distinguish $|t_x^+\rangle$ and $|t_x^-\rangle$. Define $U_x = (2\Pi_x - I)(2\Lambda - I)$ as the product of two reflections:

- Consider (unnormalized) states $|\psi_y\rangle = \frac{0.02}{\sqrt{A}}|t_y^-\rangle - \sum_j |j\rangle|\bar{y}_j\rangle|v_{yj}\rangle$, where $\bar{y}_j = 1 - y_j$.

Note that $\| |\psi_y\rangle \| \leq \frac{0.02}{\sqrt{A}} + \sqrt{A}$, using triangle inequality and the fact that $\sum_j \|v_{yj}\|^2 \leq A$. Let Λ be the projector on the subspace that is orthogonal to the span of these $|\psi_y\rangle$ ’s, and $2\Lambda - I$ be the reflection through this subspace. In other words, the unitary $2\Lambda - I$ puts a $-$ in front of all $|\psi_y\rangle$ ’s and leaves states alone if they are orthogonal to all $|\psi_y\rangle$ ’s. This reflection costs no queries to implement, since it doesn’t depend on the actual input x .

⁵Our presentation follows the approach of [164], modified and simplified for our special case of computing Boolean functions rather than their more general case of state-transformation. For other generalizations of the adversary bound to different scenarios, see [44]. Reichardt’s first proof of the optimality of the generalized adversary bound [205, 207] went through so-called “span programs,” but we won’t need those here.

- Let Π_x be the projector on the subspace spanned by states that have $|j\rangle|x_j\rangle$ in their first two registers (with arbitrary states in the third register) and by states having $|0\rangle$ in their third register. Then the reflection $2\Pi_x - I$ through this subspace puts a $-$ in front of states $|j\rangle|\bar{x}_j\rangle|v\rangle$ if $\langle v|0\rangle = 0$, and leaves the states alone that are in the subspace of Π_x . This reflection can be implemented with 1 query to x (Exercise 5).

We now relate $|t_x^+\rangle$ and $|t_x^-\rangle$ to the eigenstates of this unitary U_x . In the next two claims, the informal “close” should be read as “within small constant Euclidean distance.”

Claim 3 $|t_x^+\rangle$ is close to an eigenstate $|\phi\rangle$ of U_x that has eigenvalue 1 (i.e., phase 0).

Proof. Define

$$|\phi\rangle = |t_x^+\rangle + \frac{0.01}{\sqrt{A}} \sum_j |j\rangle|x_j\rangle|u_{xj}\rangle.$$

The second term on the right-hand side has norm ≤ 0.01 because $\sum_j \|u_{xj}\|^2 \leq A$, so $|t_x^+\rangle$ is indeed close to $|\phi\rangle$ ($|\phi\rangle$'s norm is close to but not equal to 1, but this doesn't matter).

Note that $\langle \phi|\psi_y\rangle = 0$ for all y , because $\langle t_x^+|t_y^-\rangle = \frac{1}{2}[f(x) \neq f(y)]$ and

$$\left(\sum_j |j\rangle|x_j\rangle|u_{xj}\rangle \right)^* \sum_j |j\rangle|\bar{y}_j\rangle|v_{yj}\rangle = \sum_j \langle x_j|\bar{y}_j\rangle \cdot \langle u_{xj}|v_{yj}\rangle = \sum_{j:x_j \neq y_j} \langle u_{xj}|v_{yj}\rangle = [f(x) \neq f(y)],$$

and because $|t_x^+\rangle, |t_x^-\rangle$ have $|0\rangle$ in the third register and so are orthogonal to all $|j\rangle|b\rangle|u_{xj}\rangle, |j\rangle|b\rangle|v_{xj}\rangle$, $b \in \{0, 1\}$. This shows that $|\phi\rangle$ lies in the subspace of Λ , so it is an eigenvalue-1 eigenvector of $2\Lambda - I$. Also, $|\phi\rangle$ is a linear combination of $|t_x^+\rangle$ (which has $|0\rangle$ in its third register) and states that have $|j\rangle|x_j\rangle$ in their first two registers, so $|\phi\rangle$ lies in the subspace of Π_x and hence is also an eigenvalue-1 eigenvector of $2\Pi_x - I$. Hence $|\phi\rangle$ is an eigenvalue-1 eigenvector of $U_x = (2\Pi_x - I)(2\Lambda - I)$. $\square$

Claim 4 $|t_x^-\rangle$ is close to a superposition of eigenstates of U_x with eigenvalues of the form $e^{i\theta}$ with $\theta \in (-\pi, \pi]$ and $|\theta| > \Theta = 1/(1000A)$.

Proof. Let $\{|\beta\rangle\}$ be a complete orthonormal set of eigenvectors of U_x , with respective eigenvalues $e^{i\theta_\beta}$, $\theta_\beta \in (-\pi, \pi]$. Let $P_\Theta = \sum_{\beta: |\theta_\beta| \leq \Theta} |\beta\rangle\langle\beta|$ be the projector on the eigenvectors with small eigenphase. Define vectors $w = 100\sqrt{A}|\psi_x\rangle$ and $v = P_\Theta \Pi_x w = P_\Theta |t_x^-\rangle$. Our goal is to show that v has small norm.

To that end, define $v' = (2\Lambda - I)v$ and $v'' = (2\Pi_x - I)v' = U_x v$, and note that

$$\|v - v''\|^2 = \left\| \sum_{\beta: |\theta_\beta| \leq \Theta} (1 - e^{i\theta_\beta}) \langle \beta|v \rangle |\beta\rangle \right\|^2 = \sum_{\beta: |\theta_\beta| \leq \Theta} |1 - e^{i\theta_\beta}|^2 \cdot |\langle \beta|v \rangle|^2 \leq 2(1 - \cos \Theta) \|v\|^2 \leq \Theta^2 \|v\|^2.$$

Because $v + v' = 2\Lambda v$, the vector $v + v'$ lies in the subspace corresponding to Λ and hence is orthogonal to $|\psi_x\rangle$ and to $w = 100\sqrt{A}|\psi_x\rangle$. We then have

$$0 = \langle v + v'|w \rangle = \langle v + v'|\Pi_x|w \rangle + \langle v + v'|(I - \Pi_x)|w \rangle = \langle v + v''|\Pi_x|w \rangle + \langle v - v''|(I - \Pi_x)|w \rangle,$$

where the last equality used that $v' + v'' = 2\Pi_x v'$ (hence $\Pi_x(v' + v'') = 2\Pi_x v'$ and so $\Pi_x v' = \Pi_x v''$) and $v' - v'' = 2(I - \Pi_x)v'$ (hence $(I - \Pi_x)v' = -(I - \Pi_x)v''$). We can now upper bound $\|v\|$ by

$$\begin{aligned}\|v\|^2 &= |\langle v | P_\Theta \Pi_x | w \rangle| = |\langle v | \Pi_x | w \rangle| \\ &= \frac{1}{2} |\langle v - v'' | \Pi_x | w \rangle + \langle v + v'' | \Pi_x | w \rangle| = \frac{1}{2} |\langle v - v'' | (2\Pi_x - I) | w \rangle| \\ &\leq \frac{1}{2} \|v - v''\| \cdot \|w\| \leq \frac{1}{2} \Theta \|v\| \cdot 100\sqrt{A} \|\psi_x\| \\ &\leq \frac{1}{2} \frac{1}{1000A} \|v\| \cdot 100\sqrt{A} \left(\frac{0.02}{\sqrt{A}} + \sqrt{A} \right) = \left(\frac{1}{20} + \frac{1}{1000A} \right) \|v\|,\end{aligned}$$

where the first inequality is Cauchy-Schwarz. This implies $\|v\| \leq 1/20 + 1/(1000A)$, which is small (we may assume $A \geq 1$). $\square$

Phase estimation with precision $\Theta/2$ can distinguish between these two cases (eigenphase 0 vs $\geq \Theta$). This allows us to put a $+$ in front of $|t_x^+\rangle$ and a $-$ in front of $|t_x^-\rangle$: run phase estimation, multiply with -1 whenever the absolute value of the phase estimate is $> \Theta/2$, and then invert the phase estimation. Phase estimation uses $O(1/\Theta) = O(A)$ applications of U_x , and hence $O(A)$ queries to x , as promised. There are small errors in this process due to the fact that Claims 3 and 4 say “close to” rather than “equal,” and due to the small approximation errors of phase estimation. Accordingly, our final state will be close to $|1\rangle|f(x)\rangle|0\rangle$ but necessarily equal to it, and we end up with an $O(A)$ -query quantum algorithm for f that has a small error probability.

It should be noted that the upper bound is on the algorithm’s query complexity, not on its gate complexity. The number of gates of the algorithm is $O(A)$ times the number of gates needed to implement the reflection $2\Lambda - I$ (the reflection $2\Pi_x - I$ is relatively easy to implement, see Exercise 5). In general this number of gates could be very large, though in some cases it can be made quite small, for instance [17].

12.4 Applications

Let us see how we can derive an $O(\sqrt{N})$ -query quantum algorithm for the N -bit OR function from the dual adversary, for the special case where the N -bit input x is promised to have at most one 1-bit. Consider the set $\mathcal{D} = \{0^N, e_1, \dots, e_N\}$ of possible inputs. For the dual adversary bound we need to choose vectors u_{xj}, v_{xj} for each $x \in \mathcal{D}$ and $j \in [N]$. Here vectors of dimension 1 (i.e., numbers) already suffice: we define $u_{0^N j} = v_{0^N j} = 1/N^{1/4}$ for all j , and $u_{e_k j} = v_{e_k j} = N^{1/4}$ if $j = k$ and $u_{e_k j} = v_{e_k j} = 0$ otherwise. For the objective function, note that for each x , $\sum_j u_{e_k j}^2 = \sum_j v_{e_k j}^2 = \sqrt{N}$; for $x = 0^N$ this is because each of the N j ’s contributes $(1/N^{1/4})^2 = 1/\sqrt{N}$ to the sum, while for $x = e_k$ there is only one nonzero contribution, namely $(N^{1/4})^2 = \sqrt{N}$ for $j = k$. It is also easy to verify that $\sum_{j: x_j \neq y_j} u_{xj} v_{yj} = [f(x) \neq f(y)]$ for all $x, y \in \mathcal{D}$.

A number of new quantum algorithms have been derived from the dual SDP for $\text{ADV}^\pm$, for instance for finding k -collisions [40], learning and testing “juntas” (functions that only depend on few coordinates) [43, 17], st -connectivity in graphs [46], and formula evaluation [208, 18]. In general it is often quite hard and non-intuitive to come up with a feasible solution $\{u_{xj}, v_{xj}\}$ for the dual SDP with a small objective value, but Belovs’s learning graphs [41] can sometimes help with more intuitive constructions of feasible solutions.

12.5 Perfect composition and AND-OR trees

If we have two Boolean functions $f : \{0, 1\}^n \rightarrow \{0, 1\}$ and $g : \{0, 1\}^m \rightarrow \{0, 1\}$, then we can define the function $F = f \circ g^n$ (on $N = nm$ bits) which is their composition, as

$$F(x^1, \dots, x^n) = f(g(x^1), \dots, g(x^n))$$

where each $x^i \in \{0, 1\}^m$. One beautiful property of the adversary bound is *perfect composition*:

$$\text{ADV}^\pm(F) = \text{ADV}^\pm(f)\text{ADV}^\pm(g).$$

There are no hidden constant factors here! The upper bound $\text{ADV}^\pm(F) \leq \text{ADV}^\pm(f)\text{ADV}^\pm(g)$ of this composition property can be proved by combining feasible solutions for the dual SDPs for f and g to a feasible solution for the dual SDP for F . Similarly, the lower bound $\text{ADV}^\pm(F) \geq \text{ADV}^\pm(f)\text{ADV}^\pm(g)$ can be proved by combining feasible solutions for the primal SDPs for f and g . We will skip the rather technical details, see [45] for proof of a stronger and more general result.

Because of the optimality of the generalized adversary bound, it follows from the composition property that the quantum query complexity of F equals the product of the query complexities of f and g , up to a constant factor. For example, for the 2-level AND-OR tree on $N = k^2$ bits mentioned at the end of Chapter 11, we immediately get an optimal $O(\sqrt{N})$ -query quantum algorithm from the fact that the k -bit AND and OR functions each have quantum query complexity (and hence $\text{ADV}^\pm$) equal to $\Theta(\sqrt{k})$. Note that we are not directly composing bounded-error algorithms here (e.g., trying to put Grover on top of another Grover to compute the AND-OR tree): reasoning about the composed function at the level of the adversary bound and then only translating to quantum query algorithms at the end, cleanly circumvents the problem of how the error probabilities of composed bounded-error algorithms for f and g affect the error probability of the resulting algorithm for F .

This composition result, used in a more subtle way, can also give a quantum speed-up for evaluating *game trees*. Imagine a two-player game, such as chess. First white chooses one of several possible moves, then black chooses one of several moves, etc. We can picture this as a tree where the root is the initial position and the leaves are the final positions (which are win, lose, or draw). If we assign a binary value to each leaf indicating whether white wins, then the evaluation of the game as a whole is a large, multilevel, unbalanced, AND-OR tree. If it's white's turn and at least one subtree evaluates to 1, then the current position is a 1 as well: there is a winning move for white (this corresponds to an OR function). If it's black's turn and one of the subtrees from the current position is labeled 0 then the current position is also labeled 0 because black has a non-losing move (this is an AND function). The value at the root of the tree indicates whether white has a sequence of moves guaranteed to win or not. Using the adversary bound to do a more subtle AND-OR composition, there is a quantum algorithm that evaluates this tree using roughly $\sqrt{N}$ queries to the binary values at the leaves [18, 206]. In contrast, a classical algorithm has to evaluate nearly all N leaves in the worst case unless the fan-out of the tree is very small [215].

Exercises

1. Consider the symmetric 4-bit Boolean function $g(x_0, x_1, x_2, x_3)$ which is 1 iff the 4-bit input x is increasing or decreasing, i.e., if $x \in \{0000, 0001, 0011, 0111, 1111, 1110, 1100, 1000\}$. Let f be the function on $N = 4^d$ input bits obtained by composing g with itself d times, in a tree of depth d , where the value of each internal node is obtained by applying g to the values of its 4 children, and with the input bits at the N leaves.

- (a) Show that $\deg(g) \leq 2$ and $\deg(f) \leq 2^d$.
- (b) Show that the polynomial method cannot prove a lower bound better than $O(\sqrt{N})$ on the bounded-error quantum query complexity of f .
- (c) It is known that $\text{ADV}^\pm(g) > 2.51 \dots$ (see [42, Example 3.3] for a proof, or you could use an SDP-solver). Use this to show that $\text{ADV}^\pm(f) \geq N^c$ for some $c > 1/2$.

Comment: This exercise shows that the generalized adversary bound can sometimes prove substantially stronger lower bounds than the polynomial method. Incidentally, [42, Example 3.3] also shows that if we restrict the adversary matrix to have nonnegative entries, then the best adversary lower bound we can prove for g is 2.5, and hence the best lower bound we can prove for the quantum query complexity f with such restricted Γ , is at most $\Omega(2.5^d)$. So this example also shows that the “negative weights” $\text{ADV}^\pm$ can give substantially better bounds than the “nonnegative weights” version of the adversary bound.

2. (H) This exercise justifies Claim 1. Below, the x, y always range over $\mathcal{D}$.
 - (a) Let P_1, P_0 denote the projectors on the subspaces corresponding to outputs 1 and 0, respectively. Suppose inputs x, y have $f(x) \neq f(y)$.
Show that $\langle \psi_x^t | \psi_y^t \rangle = \langle \psi_x^t | P_{f(x)} \cdot P_{1-f(y)} | \psi_y^t \rangle + \langle \psi_x^t | P_{1-f(x)} \cdot P_{f(y)} | \psi_y^t \rangle$ for all t, x, y .
 - (b) Define unnormalized states $|\phi\rangle = \sum_x \alpha_x P_{f(x)} |\psi_x^T\rangle |x\rangle$ and $|\phi^\perp\rangle = \sum_x \alpha_x P_{1-f(x)} |\psi_x^T\rangle |x\rangle$.
Show that $\| |\phi\rangle \|^2 + \| |\phi^\perp\rangle \|^2 = 1$, $\| |\phi^\perp\rangle \|^2 \leq \varepsilon$, and $\| |\phi\rangle \| \cdot \| |\phi^\perp\rangle \| \leq \sqrt{\varepsilon(1-\varepsilon)}$.
 - (c) Show that $|S_T| \leq 2|\langle \phi | (I \otimes \Gamma) | \phi^\perp \rangle|$.
 - (d) Show that $|S_T| \leq 2\sqrt{\varepsilon(1-\varepsilon)} \|\Gamma\|$.
3. (H) This exercise justifies Claim 2. It will be convenient for the proof to assume a phase-oracle, so the T -query algorithm will be of the form $U_T O_{x,\pm} U_{T-1} \cdots U_1 O_{x,\pm} U_0$, applied to initial state $|0^m\rangle$ and followed by a measurement of the first qubit to produce the output bit. Below, the x, y always range over $\mathcal{D}$ and the i ranges over $[N]$.
 - (a) Show that $S_t - S_{t+1} = \sum_{x,y} \Gamma_{xy} \alpha_x^* \alpha_y \langle \psi_x^t | (I - O_{x,\pm} O_{y,\pm}) | \psi_y^t \rangle$.
 - (b) Let $P_i = |i\rangle\langle i| \otimes I$ be the projector on the space where the query register is $|i\rangle$. Define $|\phi_i^t\rangle = \sum_x \alpha_x P_i |\psi_x^t\rangle |x\rangle$. Show that $\sum_i \| |\phi_i^t\rangle \|^2 = 1$.
 - (c) Show that $S_t - S_{t+1} = 2 \sum_i \langle \phi_i | (I \otimes \Gamma_i) | \phi_i \rangle$.
 - (d) Show that $|S_t - S_{t+1}| \leq 2 \max_i \|\Gamma_i\|$.
4. (H) The following is a primal-dual pair of SDPs in so-called standard form:

$$\begin{array}{ll}
 \min & \text{Tr}(CX) \\
 \text{s.t.} & \text{Tr}(A_i X) = b_i \quad \forall i \in [m] \\
 & X \succeq 0
 \end{array}
 \qquad
 \begin{array}{ll}
 \max & b^T y \\
 \text{s.t.} & \sum_{i=1}^m y_i A_i \preceq C
 \end{array}$$

The input here consists of Hermitian $n \times n$ matrices $C, A_1, \dots, A_m$ and vector $b \in \mathbb{R}^m$. The $n \times n$ matrix X is the variable of the primal, and the vector $y \in \mathbb{R}^m$ is the variable of the dual. Prove that “weak duality” always holds: for every feasible solution X for the primal and every feasible solution y for the dual, we have $\text{Tr}(CX) \geq b^T y$.

5. Show how the reflection $2\Pi_x - I$ can be implemented with 1 phase-query to x , a Z -gate, and a circuit that decides if the third register is $|0\rangle$.

Chapter 13

Quantum Complexity Theory

13.1 Most functions need exponentially many gates

As we have seen, quantum computers seem to provide enormous speed-ups for problems like factoring, and square-root speed-ups for various search-related problems. Could they be used to significantly speed up all or almost all problems? Here we will show that this is not the case: quantum computers are not significantly better than classical computers for most problems.

Consider the problem of computing a Boolean function $f : \{0,1\}^n \rightarrow \{0,1\}$ by means of a quantum circuit. Ideally, most such functions would be computable by efficient quantum circuits (i.e., using at most $\text{poly}(n)$ elementary gates). Instead, we will show by means of a simple counting argument that almost all such functions f have circuit complexity nearly 2^n . This is a variant of a well-known counting argument for classical Boolean circuits due to Riordan and Shannon [209].

Let us fix some finite set of elementary gates, for instance the Shor basis $\{H, T, \text{CNOT}\}$ or $\{H, \text{Toffoli}\}$. Suppose this set has k types of gates, of maximal fanout 3. Let us try to count the number of distinct circuits that have at most C elementary gates. For simplicity we include the initial qubits (the n input bits as well as workspace qubits, which are initially $|0\rangle$) as a $(k+1)$ st type among those C gates. First we need to choose which type of elementary gate each of the C gates is; this can be done in $(k+1)^C$ ways. Now every gate has at most 3 ingoing and 3 outgoing wires. For each of its 3 outgoing wires we can choose an ingoing wire into one of the gates in the following level; this can be done in at most $(3C)^3$ ways. Hence the total number of circuits with up to C elementary gates is at most $(k+1)^C (3C)^{3C} = C^{O(C)}$. We are clearly overcounting here, but that's OK because we want an upper bound on the number of circuits.

We'll say that a specific circuit *computes* a Boolean function $f : \{0,1\}^n \rightarrow \{0,1\}$ if for every input $x \in \{0,1\}^n$, a measurement of the first qubit of the final state (obtained by applying the circuit to initial state $|x, 0\rangle$) gives value $f(x)$ with probability at least $2/3$. Each of our $C^{O(C)}$ circuits can compute at most one f (in fact some of those circuits don't compute any Boolean function at all). Accordingly, with C gates we can compute at most $C^{O(C)}$ distinct Boolean functions $f : \{0,1\}^n \rightarrow \{0,1\}$. Hence even if we just want to be able to compute 1% of all 2^{2^n} Boolean functions, then we already need

$$C^{O(C)} \geq \frac{1}{100} 2^{2^n}, \text{ which implies } C \geq \Omega(2^n/n).$$

Accordingly, very few computational problems will be efficiently solvable on a quantum computer. Below we will try to classify those using the tools of complexity theory.

13.2 Classical and quantum complexity classes

A computational decision problem on binary strings corresponds to what is often called a “language” in complexity theory: a language $L \subseteq \{0, 1\}^*$ is a set of binary strings of arbitrary lengths, and the corresponding decision problem is to determine whether a given string is an element of L or not. For example, L could be the set of prime numbers encoded in binary (corresponding to the problem of deciding whether a given number is prime or not) or the set of satisfiable Boolean formulas. It is often convenient to think of such a decision problem as corresponding to a sequence of Boolean functions $f_n : \{0, 1\}^n \rightarrow \{0, 1\}$, one for each input length n , where f_n takes value 1 exactly on the n -bit strings that are in L .

A “complexity class” is a set of decision problems (i.e., languages) that all have similar complexity in some sense, for instance the ones that can be solved with polynomial time or polynomial space. Let us first mention four of the most important *classical* complexity classes:

- **P**. The class of problems that can be solved by classical deterministic computers using polynomial time.
- **BPP**. The problems that can be solved by classical randomized computers using polynomial time (and with error probability $\leq 1/3$ on every input).
- **NP**. The problems where the ‘yes’-instances can be verified in polynomial time if some prover gives us a polynomial-length “witness.” More precisely, a language L is in **NP** iff there exists a deterministic polynomial-time algorithm A , with two inputs x, y (where y is at most polynomially longer than x), such that $x \in L$ iff there is a y such that $A(x, y)$ outputs 1. Some problems L in this class are **NP-complete**, meaning that any other problem $L' \in \mathbf{NP}$ can be reduced to L in polynomial time: there exists a polynomial-time computable function f such that $x \in L'$ iff $f(x) \in L$. Hence the **NP**-complete problems are the hardest problems in **NP**. An example is the problem of satisfiability: we can verify that a given Boolean formula is satisfiable if a prover gives us a satisfying assignment y , so the satisfiability-problem is in **NP**, but one can even show that it is **NP**-complete. Other examples of **NP**-complete problems are integer linear programming, travelling salesman, graph-colorability, etc.
- **PSPACE**. The problems that can be solved by classical deterministic computers using polynomial space.

We can consider quantum analogues of all such classes, an enterprise that was started by Bernstein and Vazirani [53]:

- **EQP**. The class of problems that can be solved exactly by quantum computers using polynomial time. This class depends on the set of elementary gates one allows, and therefore is not so interesting.
- **BQP**. The class of problems that can be solved by quantum computers using polynomial time (and with error probability $\leq 1/3$ on every input). This class is the accepted formalization of “efficiently solvable by quantum computers.”
- “quantum **NP**”. In analogy with the above definition of **NP**, one could define quantum **NP** as the class of problems where the ‘yes’-instances can be verified efficiently if some prover gives us a “quantum witness” of a polynomial number of qubits. For every ‘yes’-instance

there should be a quantum witness that passes the verification with probability 1, while for ‘no’-instances every quantum witness should be rejected with probability 1. This class is again dependent on the elementary gates one allows, and not so interesting.

Allowing error probability $\leq 1/3$ on every input, we get a class called **QMA** (“quantum Merlin-Arthur”). This is a more robust and more interesting quantum version of **NP**. In particular, like **NP**, **QMA** has *complete* problems: problems in **QMA** to which every other **QMA**-problem can be efficiently reduced. The most famous example of such a problem is deciding whether the *ground state energy* (i.e., lowest eigenvalue) of a given *k-local Hamiltonian* (see Chapter 9) is at most some given number a or at least $a + 1/\text{poly}(n)$. Determining the ground state energy of a given physical system is extremely important in physics and chemistry. It is not hard to see that the problem is *in QMA*: we can just let the quantum witness be the ground state (i.e., an eigenstate for the lowest eigenvalue) and measure its energy using the Hamiltonian, which is the observable corresponding to total energy. The problem turns out to be **QMA**-complete already for $k = 2$ [156, 149]. We will devote Chapter 14 to this.

- **QPSpace**. The problems that can be solved by quantum computers using polynomial space. This turns out to be the same as classical **PSpace**.

As explained in Appendix B.2, in all the above cases the error probability $1/3$ can be reduced efficiently to much smaller constant $\varepsilon > 0$: just run the computation $O(\log(1/\varepsilon))$ times and take the majority of the answers given by these runs.

We should be a bit careful about what we mean by a “polynomial-time [or space] quantum algorithm.” Our model for computation has been quantum circuits, and we need a separate quantum circuit for each new input length. So a quantum algorithm of time $p(n)$ would correspond to a *family* of quantum circuits $\{C_n\}$, where C_n is the circuit that is used for inputs of length n ; it should have at most $p(n)$ elementary gates.¹

We have **BPP** $\subseteq$ **BQP**, because a **BPP**-machine on a fixed input length n can be written as a polynomial-size reversible circuit (i.e., consisting of Toffoli gates) that starts from a state that involves some coin flips. Quantum computers can generate those coin flips using Hadamard transforms, then run the reversible circuit, and measure the final answer bit. It is believed that **BQP** contains problems that aren’t in **BPP**, for example factoring large integers: this problem (or rather the decision-version thereof) is in **BQP** because of Shor’s algorithm, and is generally believed not to be in **BPP**. In the next section we will prove that **BQP** $\subseteq$ **PSpace**. Thus we get the following sequence of inclusions:

$$\mathbf{P} \subseteq \mathbf{BPP} \subseteq \mathbf{BQP} \subseteq \mathbf{PSpace}.$$

It is generally believed that **P** = **BPP** [140], while the other inclusions are believed to be strict. Note that a *proof* that **BQP** is strictly greater than **BPP** (for instance, a proof that factoring cannot be solved efficiently by classical randomized computers) would imply that **P** $\neq$ **PSpace**, solving what has been one of the main open problems in computer science since the 1960s. Hence such a proof—if it exists at all—will probably be very hard.

¹To avoid smuggling loads of hard-to-compute information into this definition (e.g., C_n could contain information about whether the n -th Turing machine halts or not), we will require this family to be efficiently describable: there should be a classical Turing machine which, on input n and j , outputs (in time polynomial in n) the j -th elementary gate of C_n , with information about where its incoming and outgoing wires go.

What about the relation between **BQP** and **NP**? It’s generally believed that **NP**-complete problems are probably not in **BQP**. The main evidence for this is the lower bound for Grover search: a quantum brute-force search on all 2^n possible assignments to an n -variable formula gives a square-root speed-up, but not more. This is of course not a proof, since there might be some more clever, non-brute-force methods that exploit the structure of the problem to solve satisfiability. However, neither in the classical nor in the quantum case do we know clever methods that solve the general satisfiability problem much faster than brute-force search.

Finally, there could also be problems in **BQP** that are not in **NP**, so it may well be that **BQP** and **NP** are incomparable. Much more can be said about quantum complexity classes; see for instance Watrous’s survey [246].

13.3 Classically simulating quantum computers in polynomial space

When Richard Feynman first came up with quantum computers [111], he motivated them by

“the full description of quantum mechanics for a large system with R particles is given by a function $q(x_1, x_2, \dots, x_R, t)$ which we call the amplitude to find the particles $x_1, \dots, x_R$ [RdW: think of x_i as one qubit], and therefore, because it has too many variables, it cannot be simulated with a normal computer with a number of elements proportional to R or proportional to N .” [...]

“Can a quantum system be probabilistically simulated by a classical (probabilistic, I’d assume) universal computer? In other words, a computer which will give the same probabilities as the quantum system does. If you take the computer to be the classical kind I’ve described so far (not the quantum kind described in the last section) and there are no changes in any laws, and there’s no hocus-pocus, the answer is certainly, No!”

The suggestion to devise a *quantum* computer to simulate quantum physics is of course a brilliant one, but the main motivation is not quite accurate. As it turns out, it is not necessary to keep track of all (exponentially many) amplitudes in the state to classically simulate a quantum system. Here will prove the result of Bernstein and Vazirani [53] that quantum computers can actually be simulated efficiently in terms of *space* (though not necessarily in terms of *time*).

Consider a circuit with $T = \text{poly}(n)$ gates that acts on S qubits, where the first n of those S qubits give the classical input string. Assume for simplicity that all gates are either the 1-qubit Hadamard or the 3-qubit Toffoli gate (as mentioned in Section 2.2, these two gates together suffice for universal quantum computation), and that the classical output (0 or 1) of the algorithm is determined by a measurement of the first qubit of the final state. Without loss of generality $S \leq 3T$, because T Toffoli gates won’t affect more than $3T$ qubits. Let U_j be the unitary that applies the j -th gate to its (1 or 3) qubits, and applies identity to all other qubits. The entries of this matrix are of a simple form (0, $1/\sqrt{2}$, or $-1/\sqrt{2}$ for Hadamard; 0 or 1 for Toffoli) and easy to compute. Let $|i_0\rangle = |x\rangle|0^{S-n}\rangle$ be the starting state, where $x \in \{0, 1\}^n$ is the classical input, and the second register contains the workspace qubits the algorithm uses. The final state will be

$$|\psi_x\rangle = U_T U_{T-1} \cdots U_2 U_1 |i_0\rangle.$$

The amplitude of basis state $|i_T\rangle$ in this final state is

$$\langle i_T | \psi_x \rangle = \langle i_T | U_T U_{T-1} U_{T-2} \cdots U_2 U_1 | i_0 \rangle.$$

Inserting an identity matrix $I = \sum_{i \in \{0,1\}^S} |i\rangle\langle i|$ between the gates, we can rewrite this as²

$$\begin{aligned} \langle i_T | \psi_x \rangle &= \langle i_T | U_T \left(\sum_{i_{T-1} \in \{0,1\}^S} |i_{T-1}\rangle\langle i_{T-1}| \right) U_{T-1} \left(\sum_{i_{T-2}} |i_{T-2}\rangle\langle i_{T-2}| \right) U_{T-2} \cdots U_2 \left(\sum_{i_1} |i_1\rangle\langle i_1| \right) U_1 |x, 0\rangle \\ &= \sum_{i_{T-1} \in \{0,1\}^S, \dots, i_1 \in \{0,1\}^S} \prod_{j=1}^T \langle i_j | U_j | i_{j-1} \rangle. \end{aligned}$$

The term $\langle i_j | U_j | i_{j-1} \rangle$ is just one entry of the matrix U_j and hence easy to calculate because U_j acts non-trivially on only 1 or 3 qubits (see Exercise 2). Then $\prod_{j=1}^T \langle i_j | U_j | i_{j-1} \rangle$ is also easy to compute, in polynomial space (and even in polynomial *time*). If ℓ of the T gates are Hadamards, then each such term is either 0 or $\pm 1/\sqrt{2}^\ell$.

Adding up $\prod_{j=1}^T \langle i_j | U_j | i_{j-1} \rangle$ for all $i_{T-1}, \dots, i_1$ is also easy to do in polynomial space if we reuse space for each new $i_{T-1}, \dots, i_1$. Hence the amplitude $\langle i_T | \psi_x \rangle$ can be computed exactly using polynomial space.³ We assume that the **BQP** machine's answer is obtained by measuring the first qubit of the final state. Then its acceptance probability is the sum of squares of all amplitudes of basis states starting with a 1: $\sum_{i_T: (i_T)_1=1} |\langle i_T | \psi_x \rangle|^2$. Since we can compute each amplitude $\langle i_T | \psi_x \rangle$ in polynomial space, and we can loop over all $i_T \in \{0,1\}^S$ whose first bit is 1 to sum their squared amplitudes, the acceptance probability of a **BQP**-circuit on classical input x can also be computed in polynomial space. This proves the inclusion **BQP** $\subseteq$ **PSPACE**.

Exercises

1. (H) The following problem is a decision version of the factoring problem:

Given positive integers N and k , decide if N has a prime factor $p \in \{k, \dots, N-1\}$.

Show that if you can solve this decision problem efficiently (i.e., in time polynomial in the input length $n = \lceil \log N \rceil$), then you can also find the prime factors of N efficiently.

2. (a) Let U be an S -qubit unitary which applies a Hadamard gate to the k -th qubit, and identity gates to the other $S-1$ qubits. Let $i, j \in \{0,1\}^S$. Show an efficient way (i.e., using time polynomial in S) to classically calculate the matrix-entry $U_{i,j} = \langle i | U | j \rangle$ (note: even though U is a tensor product of 2×2 matrices, it's still a $2^S \times 2^S$ matrix, so calculating U completely isn't efficient).
- (b) Let U be an S -qubit unitary which applies a CNOT gate to the k -th and ℓ -th qubits, and identity gates to the other $S-2$ qubits. Let $i, j \in \{0,1\}^S$. Show an efficient way to classically calculate the matrix-entry $U_{i,j} = \langle i | U | j \rangle$. Here k and ℓ need not be adjacent, but you may assume that they are in order to simplify your notation.

3. This exercise shows how to use **BQP**-algorithms as subroutines in other **BQP**-algorithms.

²For the physicists: this is very similar to a path integral.

³Of course, the calculation will take exponential *time*, because there are $2^{S(T-1)}$ different sequences $i_{T-1}, \dots, i_1$ that we need to go over sequentially.

- (a) (H) Suppose L is a language in **BQP**. Let f be the corresponding Boolean function, so $f(x) = 1$ iff $x \in L$. Show that there is a $w \leq \text{poly}(n)$ and a polynomial-size quantum circuit U that implements the following map for all $x \in \{0, 1\}^n$:

$$|x, 0^{w+1}\rangle \mapsto \sqrt{p}|x, f(x)\rangle|\phi(x)\rangle + \sqrt{1-p}|x, 1-f(x)\rangle|\psi(x)\rangle,$$

where $p \geq 1 - \exp(-n)$, and $|\phi(x)\rangle$ and $|\psi(x)\rangle$ are states of the w -qubit workspace.

- (b) Show that there is a polynomial-size quantum circuit V that (when restricted to the subspace where the workspace qubits are $|0\rangle$) is $\exp(-n)$ -close in operator norm to the following unitary:

$$O_f : |x, b, 0^w\rangle \mapsto |x, b \oplus f(x), 0^w\rangle,$$

for all $x \in \{0, 1\}^n$ and $b \in \{0, 1\}$.

- (c) (H) Suppose L is a language in **BQP**, and you have a polynomial-size quantum circuit for another language L' that uses queries to the language L (i.e., applications of the unitary O_f). Show that the language L' is also in **BQP**: there is a polynomial-size quantum circuit for L' that doesn't need queries to L .
4. (H) Consider a circuit C with $T = \text{poly}(n)$ elementary gates (only Hadamards and Toffolis) acting on $S = \text{poly}(n)$ qubits. Suppose this circuit computes $f : \{0, 1\}^n \rightarrow \{0, 1\}$ with bounded error probability: for every $x \in \{0, 1\}^n$, when we start with basis state $|x, 0^{S-n}\rangle$, run the circuit and measure the first qubit, then the result equals $f(x)$ with probability at least $2/3$.
- (a) Consider the following quantum algorithm: start with basis state $|x, 0^{S-n}\rangle$, run the above circuit C without the final measurement, apply a Z gate to the first qubit, and reverse the circuit C . Denote the resulting final state by $|\psi_x\rangle$. Show that if $f(x) = 0$ then the amplitude of basis state $|x, 0^{S-n}\rangle$ in $|\psi_x\rangle$ is in the interval $[1/3, 1]$, while if $f(x) = 1$ then the amplitude of $|x, 0^{S-n}\rangle$ in $|\psi_x\rangle$ is in $[-1, -1/3]$.
- (b) **PP** is the class of computational decision problems that can be solved by classical randomized polynomial-time computers with success probability $> 1/2$ (however, the success probability could be exponentially close to $1/2$, i.e., **PP** is **BPP** without the 'B' for bounded-error). Show that **BQP** $\subseteq$ **PP**.

Chapter 14

QMA and the Local Hamiltonian Problem

14.1 Quantum Merlin-Arthur (QMA)

One can think of the complexity class **NP** as formalizing the standard notion of an efficiently-verifiable proof. For example, to prove that a given formula ϕ is satisfiable we give a satisfying assignment: this is easy to verify but hard to find (unless $\mathbf{P}=\mathbf{NP}$). The theory of **NP**-completeness shows that many very different computational problems (satisfiability, TSP, integer linear programming, etc.) are essentially the same computational problem, in the sense that instances from one can easily be translated to instances of another in deterministic polynomial time.

We can relax the notion of proof slightly by allowing the prover and verifier the use of randomness, and allowing them some error probability, say $1/3$. For every $x \in L$ the prover should be able to provide a polynomial-size proof or “witness” that convinces the randomized verifier (with probability $\geq 2/3$) that $x \in L$; while for $x \notin L$, no matter what purported “witness” the prover sends, the verifier should only accept with probability $\leq 1/3$. For historical reasons [34] the complexity class corresponding to such L is called *Merlin-Arthur* (**MA**), with Merlin referring to the omniscient prover and Arthur referring to the mere mortal (i.e., randomized and polynomial-time) king who is supposed to verify Merlin’s proofs.

Quantum Merlin-Arthur (**QMA**) is the proper quantum analogue of **NP**. We already mentioned it in the previous chapter, but let us formally define **QMA** here. In contrast to **NP**, which consists of languages L where every string x is either in or out of L , **QMA** is a class of *promise problems*. A promise problem L partitions the set $\{0, 1\}^*$ of all binary strings into L_1 , L_0 , and L_* . An algorithm is “promised” that it never receives inputs from L_* ; if it gets an input from L_b for $b \in \{0, 1\}$, then it has to determine b . The usual languages are promise problems where $L_* = \emptyset$.

Definition 1 A promise problem $L = (L_1, L_0, L_*)$ is in the class **QMA**, if there exists a uniform family $\{C_n\}$ of polynomial-size quantum circuits with two input registers and one output qubit, and a polynomial w (for “witness length”), such that for all $x \in \{0, 1\}^*$:

- **Completeness:** If $x \in L_1 \cap \{0, 1\}^n$, then there exists a $w(n)$ -qubit state $|\psi\rangle$ (a “proof” or “witness” state) such that the circuit C_n outputs 1 with probability $\geq 2/3$ when run on $x, |\psi\rangle$.
- **Soundness:** If $x \in L_0 \cap \{0, 1\}^n$, then for every $w(n)$ -qubit state $|\psi\rangle$, the circuit C_n outputs 1 with probability $\leq 1/3$ when run on $x, |\psi\rangle$.

If we force $|\psi\rangle$ to be classical, then we get a class called **QCM**A. If we additionally force the verifier to be classical, then we get **MA**. And if we additionally replace success probability $2/3$ by 1 , then we get **NP**.¹ Hence $\mathbf{NP} \subseteq \mathbf{MA} \subseteq \mathbf{QCM}A \subseteq \mathbf{QMA}$ follows immediately from the definitions. It is believed that $\mathbf{MA}=\mathbf{NP}$ [158], for similar reasons as why we believe $\mathbf{BPP}=\mathbf{P}$ [140]. However, **QMA** is strongly believed to be a larger class than **NP**, meaning that “quantum proofs” can prove more than classical proofs can. Moreover, as we will see later, we can identify important **QMA**-complete problems, none of which is in **NP** (unless $\mathbf{NP}=\mathbf{QMA}$).

We have fixed the error probability here to $1/3$ rather arbitrarily. We can easily reduce this to a much smaller δ like we would for a randomized algorithm (see Appendix B.2): repeat the protocol $O(\log(1/\delta))$ times and output the majority output bit of those runs. If done naively, we’d need a new witness state $|\psi\rangle$ in each run, because the measurement that produces the verifier’s output bit in each run can collapse the state. One has to be careful about soundness here, since the prover can send a large entangled state instead of a tensor product of witness states for the individual runs; however, it is not too hard to show that this cannot help the prover. This approach increases the verifier’s runtime but also the required witness-size $w(n)$ by a factor $O(\log(1/\delta))$. However, there is a beautiful and quite surprising technique (which we will not explain here) due to Marriott and Watrous [187] that achieves the same error reduction *using the same witness state*! The verifier’s runtime in that amplified protocol will still go up by a factor $O(\log(1/\delta))$, but the witness-size remains $w(n)$.

14.2 The local Hamiltonian problem

The quintessential **NP**-complete problem is satisfiability (SAT): given a formula $\phi(x_1, \dots, x_n)$ of n Boolean variables $x_1, \dots, x_n$ ², decide if there is an assignment of truth values to $x_1, \dots, x_n$ that makes the formula true. The famous Cook-Levin theorem [92, 167] says this is **NP**-complete.

A special case of this is k -SAT, where we restrict the formula ϕ to be the conjunction of clauses, each of which is the disjunction of k literals (a literal is a variable x_i or its negation). For example, the following is a 3-SAT instance with 4 clauses on $n = 5$ Boolean variables:

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_5) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_5) \wedge (x_3 \vee x_4 \vee \neg x_5). \quad (14.1)$$

It is well-known that k -SAT is still **NP**-complete if $k \geq 3$, while 2-SAT is actually in **P**.

Let us try to reformulate k -SAT in a way that looks “more quantum,” by relating it to the minimal eigenvalue of a particular Hamiltonian (recall from Section 9.1 that the Hamiltonian for a physical system is the observable corresponding to total energy) that is diagonal in the computational basis. For concreteness we will fix $k = 3$. Consider a clause $C = x_1 \vee \neg x_2 \vee x_3$. This has one non-satisfying assignment, namely $x_1 = 0, x_2 = 1, x_3 = 0$. With this clause let us associate the following diagonal

¹As mentioned in the previous chapter, requiring success probability 1 instead of $2/3$ in **QMA** itself (like in **NP**) leads to an unsatisfactory complexity class because it depends on what set of elementary gates one uses.

²To remain consistent with the literature we now use n for the number of Boolean variables, not for the actual length of the input, i.e., the number of bits needed to describe the instance ϕ . However, the latter should be at most polynomial in n .

Hamiltonian:

$$H_C = \begin{pmatrix} 0 & & & & & \\ & 0 & & & & \\ & & 1 & & & \\ & & & 0 & & \\ & & & & 0 & \\ & & & & & 0 \\ & & & & & & 0 \end{pmatrix}$$

Note that the 1 sits at the location indexed by $x_1x_2x_3 = 010$, the unique non-satisfying assignment for C . Think of H_C as giving a “penalty” of 1 to x if x doesn’t satisfy clause C . We will implicitly treat H_C as an n -qubit Hamiltonian, by tensoring it with identity for the other $n - 3$ qubits. That way, we have $\langle x|H_C|x \rangle = 0$ if clause C is satisfied by assignment x , and $\langle x|H_C|x \rangle = 1$ if not.

Now suppose we have a 3-SAT formula $\phi = C_1 \wedge \cdots \wedge C_m$ that is the conjunction of m clauses, each with 3 literals. To this we associate the following Hamiltonian:

$$H_\phi = \sum_{j=1}^m H_{C_j}.$$

Note that the eigenvalues of H_ϕ lie in the interval $[0, m]$, and that H_ϕ is a *3-local* Hamiltonian: each term involves only 3 of the qubits non-trivially (more generally, if we start with a k -SAT instance, then H_ϕ is k -local). Also note that the “energy” of assignment $x \in \{0, 1\}^n$ is

$$\langle x|H_\phi|x \rangle = \sum_{j=1}^m \langle x|H_{C_j}|x \rangle,$$

which exactly counts the number of unsatisfied clauses under assignment x . The “minimal energy” (lowest eigenvalue $\lambda_{\min}$) of H_ϕ is equal to the minimal number of unsatisfied clauses ($= m$ – the maximal number of satisfied clauses). In particular, ϕ is satisfiable iff $\lambda_{\min} = 0$.

The above Hamiltonian is diagonal, and hence rather “classical,” because it is the sum of diagonal 3-local terms. If instead we allow the terms to be arbitrary 3-local (or even k -local) Hamiltonians, we arrive at the central problem of this chapter.

Definition 2 *The k -local Hamiltonian problem is the following: given a classical description of an n -qubit Hamiltonian*

$$H = \sum_{j=1}^m H_j \tag{14.2}$$

where each H_j is k -local (i.e., it acts nontrivially on only k of the n qubits) and $0 \preceq H_j \preceq I$, and given parameters $a, b \in [0, m]$ with $b - a \geq 1/\text{poly}(n)$, promised that H ’s minimal eigenvalue $\lambda_{\min}$ is either $\leq a$ or $\geq b$, decide which is the case.

Note that this is a promise problem: some H of the form of Eq. (14.2) will have $\lambda_{\min}$ in (a, b) and hence won’t satisfy the promise. Such instances will form the set L_* of this promise problem, while the “ $\leq a$ ” instances will form L_1 and the “ $\geq b$ ” instances will form L_0 . The input H is a $2^n \times 2^n$ matrix, but because it is k -local we do not need to describe it literally. Instead the

description of H that is given as input will just consist of each of the m terms as a $2^k \times 2^k$ matrix ($m \cdot 2^{2k}$ complex numbers) and $m \lceil \log \binom{n}{k} \rceil$ bits telling us for each of the m terms on which k qubits that term acts non-trivially. Accordingly, if $m = \text{poly}(n)$, $k = O(\log n)$, and each complex entry is represented with $\text{poly}(n)$ bits, then the actual input length is $\text{poly}(n)$ bits.

The assumption that each H_j (and therefore H as well) is positive semidefinite, is not essential. If we were instead to start with the weaker condition $-I \preceq H_j \preceq I$, then by defining psd matrices $H'_j = (H_j + I)/2$ we obtain an instance where $0 \preceq H'_j \preceq I$ with a simple relation between the minimal eigenvalues $\lambda_{\min}$ of $H = \sum_j H_j$ and $\lambda'_{\min}$ of $H' = \sum_j H'_j$ (namely $\lambda'_{\min} = (\lambda_{\min} + m)/2$). This allows us to also model negative energies.

The minimal eigenvalue $\lambda_{\min}$ is known as the “ground state energy” of the Hamiltonian: it is the energy of the state(s) we get if we cool an n -qubit system governed by H to temperature 0. Distinguishing $\lambda_{\min} \leq a$ or $\lambda_{\min} \geq b$ is essentially equivalent to approximating $\lambda_{\min}$ up to an additive error $O(b - a)$ (see Exercise 3). Finding information about $\lambda_{\min}$, and more generally about the lowest eigenvalues of H and about the structure of the “ground states” (the eigenstates for eigenvalue $\lambda_{\min}$) is important for many problems in physics and chemistry, for instance in determining the properties of materials at low temperatures (including poorly-understood phenomena such as superconductivity) and the reaction speeds of chemical reactions. Much work in computational science is expended on solving such problems for particular Hamiltonians corresponding to physical systems of interest. Such Hamiltonians are typically indeed k -local for small k , at least approximately, since particles tend to significantly influence only the particles close to them.

Unfortunately the local Hamiltonian problem is **NP**-hard already for $k \geq 2$ and very large gap between a and b . This follows from our above translation from SAT: we can convert a 2-SAT instance ϕ to 2-local Hamiltonian H_ϕ where $\lambda_{\min} = m$ —the maximal number of satisfied clauses. Computing the maximal number of satisfied clauses is known as the MAX-2-SAT problem, and distinguishing between different values that are $\Omega(m)$ apart for its value is already known to be an **NP**-hard problem [136]. Since it is generally believed that **NP** $\not\subseteq$ **BQP**, it is unlikely that a quantum computer can solve local Hamiltonian efficiently in general. Even worse, as we will see next, k -local Hamiltonian turns out to be complete for the complexity class **QMA** (which is presumably larger than **NP**), already for $k \geq 2$, but with polynomially small gap between a and b .

14.3 Local Hamiltonian is QMA-complete

In this section we will show that k -local Hamiltonian is **QMA**-complete, proving a quantum analogue of the Cook-Levin theorem.

First, it is not too hard to see that the problem is in **QMA**. The witness $|\psi\rangle$ for the instances $x \in L_1$ would be a ground state, and there are several efficient ways to approximate its energy in order to verify that it is indeed $\leq a$ and not $\geq b$ (see Exercise 4).

Second, we need to show that k -local Hamiltonian is **QMA**-hard, meaning that any other problem in **QMA** can be reduced to it. So consider an arbitrary $L = (L_1, L_0, L_*) \in \mathbf{QMA}$ as in Definition 1, and fix an n -bit input $x \in L_1 \cup L_0$. We would like to convert the circuit C_n into a Hamiltonian H , such that H has a small eigenvalue iff C_n has high acceptance probability on some states $|\psi\rangle$. We will assume the error probability is $\leq 1/4T$ rather than $1/3$.

The circuit C_n acts on $n + s + w(n)$ qubits, where the first n qubits contain the fixed classical input x (which we will omit below for simplicity), the circuit uses s workspace qubits (which start out as $|0\rangle$), and the third register contains the purported $w(n)$ -qubit witness state. Since C_n

consists of some $T = \text{poly}(n)$ gates, we can write it as a product $C_n = U_T \cdots U_1$ where each U_t is an elementary gate on 1 or 2 qubits, tensored with identity on the other qubits. For a given $w(n)$ -qubit state $|\psi\rangle$, let $|\psi_0\rangle = |0^s\rangle|\psi\rangle$ and $|\psi_t\rangle = U_t|\psi_{t-1}\rangle$ for $t \in [T]$ be the initial, intermediate, and final states of the algorithm. The output qubit of the circuit is obtained by measuring the first qubit of the final state $|\psi_T\rangle$ in the computational basis.

We will now describe a Hamiltonian H that “follows” the state and gives “penalties” for every deviation from the proper sequence of states $|\psi_0\rangle, \dots, |\psi_T\rangle$, as well as penalizing a 0-output in the final measurement. In addition to the register that U acts on, we will add another register of $\lceil \log(T+1) \rceil$ qubits that acts like a “clock,” ranging from 0 to T . We will subscript the first $s + w(n)$ qubits by integers in $\{1, \dots, s + w(n)\}$, and subscript the clock register by ‘ C ’. Define

$$\begin{aligned} H_{\text{init}} &= \sum_{i=1}^s |1\rangle\langle 1|_i \otimes |0\rangle\langle 0|_C \\ H_t &= \frac{1}{2} (I \otimes (|t-1\rangle\langle t-1|_C + |t\rangle\langle t|_C) - U_t \otimes |t\rangle\langle t-1|_C - U_t^* \otimes |t-1\rangle\langle t|_C), \text{ for } t \in [T] \\ H_{\text{final}} &= |0\rangle\langle 0|_1 \otimes |T\rangle\langle T|_C \\ H &= H_{\text{init}} + \sum_{t=1}^T H_t + H_{\text{final}}. \end{aligned}$$

The number of terms in H is $m = s + T + 1$. The idea behind this Hamiltonian H is that H_{init} checks that the s workspace qubits are all 0 in the initial state, where the clock register is 0 (giving an “energy penalty” if some of those workspace qubits are 1); H_t checks that U_t is applied properly in the t -th step (the factor $1/2$ is to ensure $H_t \preceq 1$); and H_{final} checks that the output qubit in the final state is 1 (giving a penalty if it’s 0). Because the clock register uses $\lceil \log(T+1) \rceil$ bits, and each gate U_t acts on at most 2 qubits, the locality of H is $k = \lceil \log(T+1) \rceil + 2 = O(\log n)$. We will reduce this to a constant later.

14.3.1 Completeness and soundness

We now want to show that we can distinguish $x \in L_1$ and $x \in L_0$ by considering the smallest eigenvalue $\lambda_{\min}$ of the above Hamiltonian H . First, for **completeness** we want to show that if $x \in L_1$ then there is a state with small eigenvalue. Since $x \in L_1$, there is a $w(n)$ -qubit witness state $|\psi\rangle$ that leads U with initial state $|\psi_0\rangle = |0^s\rangle|\psi\rangle$ to accept (i.e., output 1) with probability $\geq 1 - 1/4T$. Consider the following state:

$$|\psi'\rangle = \frac{1}{\sqrt{T+1}} \sum_{t=0}^T |\psi_t\rangle|t\rangle.$$

This state is sometimes called the “history state” of the circuit U , and you can think of it as the quantum analogue of a satisfying assignment in classical SAT. It faithfully “follows” the intermediate states of the computation. This means $|\psi'\rangle$ gets penalty 0 from H_{init} and from each H_t . Since the probability of getting (the incorrect) measurement outcome 0 is $\leq 1/4T$, we have

$$\lambda_{\min} \leq \langle \psi' | H | \psi' \rangle = \frac{1}{T+1} \langle \psi_T | \langle T | H_{\text{final}} | \psi_T \rangle | T \rangle = \frac{1}{T+1} \langle \psi_T | (|0\rangle\langle 0|_1 \otimes I) | \psi_T \rangle \leq \frac{1}{4T(T+1)} := a.$$

Second, to prove **soundness** we'll show that if $x \in L_0$, then $\lambda_{\min}$ is at least $b := 2a$. Consider any purported witness state $|\psi'\rangle$. We can write this as

$$|\psi'\rangle = \sum_{t=0}^T \alpha_t |\phi_t\rangle |t\rangle$$

for some nonnegative reals α_t and normalized states $|\phi_t\rangle$. Note that

$$\begin{aligned} \langle \psi' | H_t | \psi' \rangle &= (\alpha_{t-1} \langle \phi_{t-1} | \langle t-1 | + \alpha_t \langle \phi_t | \langle t |) H_t (\alpha_{t-1} |\phi_{t-1}\rangle |t-1\rangle + \alpha_t |\phi_t\rangle |t\rangle) \\ &= \frac{1}{2} (\alpha_{t-1}^2 + \alpha_t^2 - \alpha_{t-1} \alpha_t \langle \phi_t | U_t | \phi_{t-1} \rangle - \alpha_{t-1} \alpha_t \langle \phi_{t-1} | U_t^* | \phi_t \rangle) \\ &= \frac{1}{2} \|\alpha_t |\phi_t\rangle - \alpha_{t-1} U_t |\phi_{t-1}\rangle\|^2. \end{aligned} \tag{14.3}$$

So intuitively, assuming $\alpha_t \approx \alpha_{t-1}$, the Hamiltonian term H_t gives a penalty proportional to how much $|\phi_t\rangle$ deviates from $U_t |\phi_{t-1}\rangle$, i.e., from a correct application of the t -th gate of C_n .

For ease of presentation we will now make the following three simplifying assumptions.³

- All α_t are equal to $1/\sqrt{T+1}$, as they would be in the history state. This assumption is reasonable because $|\phi_t\rangle$ and $U_t |\phi_{t-1}\rangle$ both have norm 1, so differences between α_t and α_{t-1} will only make the penalty of Eq. (14.3) bigger.
- $|\phi_0\rangle$ starts with s 0s, so $|\phi_0\rangle = |0^s\rangle |\psi\rangle$ for some $w(n)$ -qubit state $|\psi\rangle$. This is reasonable because if $|\phi_0\rangle$ deviates significantly from this form, then H_{init} will give a large energy penalty.
- $|\phi_T\rangle$ has acceptance probability close to 1. This is reasonable because if $|\phi_T\rangle$ has low acceptance probability, then H_{final} will give a large energy penalty.

Because of the second item and the fact that $x \in L_0$, the state $C_n |\phi_0\rangle$ must have acceptance probability close to 0. Comparing with the third item, it follows that $|\phi_T\rangle$ and $C_n |\phi_0\rangle$ must be nearly orthogonal, so their distance is close to $\sqrt{2}$, and in particular at least 1. This implies

$$\begin{aligned} 1 &\leq \| |\phi_T\rangle - C_n |\phi_0\rangle \| = \left\| \sum_{t=1}^T U_T \cdots U_{t+1} |\phi_t\rangle - U_T \cdots U_t |\phi_{t-1}\rangle \right\| \\ &\leq \sum_{t=1}^T \| U_T \cdots U_{t+1} |\phi_t\rangle - U_T \cdots U_t |\phi_{t-1}\rangle \| = \sum_{t=1}^T \| |\phi_t\rangle - U_t |\phi_{t-1}\rangle \|. \end{aligned}$$

Here the first equality uses a telescoping sum, the second inequality is the triangle inequality, and the last equality is because the operator norm is unitarily invariant ($\|Uv - Uw\| = \|v - w\|$).

Using Eq. (14.3) with $\alpha_t = \alpha_{t-1} = 1/\sqrt{T+1}$, and Cauchy-Schwarz, we now have:

$$\begin{aligned} \langle \psi' | H | \psi' \rangle &\geq \sum_{t=1}^T \langle \psi' | H_t | \psi' \rangle = \frac{1}{2(T+1)} \sum_{t=1}^T \| |\phi_t\rangle - U_t |\phi_{t-1}\rangle \|^2 \\ &\geq \frac{1}{2T(T+1)} \left(\sum_{t=1}^T \| |\phi_t\rangle - U_t |\phi_{t-1}\rangle \| \right)^2 \geq \frac{1}{2T(T+1)} = b. \end{aligned}$$

Accordingly, if $x \in L_0$ then $\lambda_{\min} = \min_{|\psi'\rangle} \langle \psi' | H | \psi' \rangle \geq b$. If $x \in L_1$ then the history state shows $\lambda_{\min} \leq a = 1/(4T(T+1))$. We also have $b - a = 1/(4T(T+1)) \geq 1/\text{poly}(n)$, as required.

³Exercise 5 shows this is without loss of generality, though the lower bound on $b - a$ becomes a worse polynomial ($1/T^6$ instead of $1/T^2$) if we drop these three assumptions and take the unary clock of the next section into account.

14.3.2 Reducing the locality

The above construction of H , with the history state as witness for $x \in L_1$, is due to Kitaev [156], who was inspired by an earlier clock construction in [112]. Our proof of soundness is a bit different from Kitaev’s. He also showed that the locality can be reduced from $O(\log n)$ to 5 by representing the clock in *unary*: $t = 0$ would now be represented by $|0^T\rangle_C$, $t = 1$ by $|10^{T-1}\rangle_C$, $t = 2$ by $|110^{T-2}\rangle_C$, etc. This now requires T qubits to represent the clock instead of $\lceil \log(T+1) \rceil$. Denoting the t -th qubit of the clock by ‘ C_t ’, for $t \in [T]$, the previous terms in H now become

$$\begin{aligned} H_{\text{init}} &= \sum_{i=1}^s |1\rangle\langle 1|_i \otimes |0\rangle\langle 0|_{C_1} \\ H_t &= \frac{1}{2} (I \otimes |100\rangle\langle 100|_{C_{t-1}, C_t, C_{t+1}} + I \otimes |110\rangle\langle 110|_{C_{t-1}, C_t, C_{t+1}} + \\ &\quad - U_t \otimes |110\rangle\langle 100|_{C_{t-1}, C_t, C_{t+1}} - U_t^* \otimes |100\rangle\langle 110|_{C_{t-1}, C_t, C_{t+1}}) \\ H_{\text{final}} &= |0\rangle\langle 0|_1 \otimes |1\rangle\langle 1|_{C_T} \end{aligned}$$

(H_1 and H_T have a slightly different form than the H_t written above, because the clock register starts resp. ends there.) We also add the following to penalize a T -bit clock register that doesn’t conform to the proper 1s-followed-by-0s format of a unary number:

$$H_{\text{clock}} = \sum_{t=1}^{T-1} |01\rangle\langle 01|_{C_t, C_{t+1}}.$$

Note that the terms in the Hamiltonian now only “touch” at most 5 qubits: in particular, each H_t touches 1 or 2 qubits for the gate U_t , and 3 qubits of the clock. This shows k -local Hamiltonian is **QMA**-complete for $k \geq 5$. Subsequently, Kempe, Kitaev, and Regev [149] showed that k -local Hamiltonian is **QMA**-complete already for $k = 2$ (in contrast to 2-SAT, which is in **P**).

All of these results assume a polynomially small gap $b - a$. Intuitively, the local Hamiltonian problem becomes easier if the gap between a and b becomes bigger, since the 1-instances and 0-instances are further apart and should be easier to distinguish. One may ask whether k -local Hamiltonian is still **QMA**-complete if $b - a \geq \Omega(m)$ instead of $\geq 1/\text{poly}(n)$. We know that it is at least **NP**-hard for all $k \geq 2$ because of the connection with MAX-2-SAT mentioned at the end of Section 14.2, but whether it is also **QMA**-hard is a longstanding open problem in the field of Hamiltonian complexity [119], known as the “quantum PCP conjecture” [6].

14.4 Other interesting problems in QMA

The main reason **NP**-completeness is a prominent notion in computer science, is that very many interesting and practically important computational problems are **NP**-complete: satisfiability, Traveling Salesman, scheduling problems, integer linear programming, protein folding, and many more. See [117] for an already very extensive list from the late-70s. Similarly (though not yet as extensively) there is a growing list of **QMA**-complete problems, often from physics or chemistry. Below we list a few without proof of their **QMA**-hardness; the fact that these problems are *in* **QMA** is usually easy to show. See [8, 61] for more.

- The local Hamiltonian problem for **Hamiltonians corresponding to actual physical systems**, such as 2-local Ising model with 1-local transverse field and a tunable 2-local transverse coupling [58], the 2D Heisenberg Hamiltonian with local magnetic fields [220], the 2D Hubbard Hamiltonian with local magnetic fields [220], and the Bose-Hubbard model [84].

- **Non-Identity check** [144]. Given a polynomial-size quantum circuit U on n qubits, determine whether U is not close to the identity up to some global phase:
 - (1) for all $\phi \in [0, 2\pi)$ we have $\|U - e^{i\phi}I\| \geq b$ or
 - (0) there is a $\phi \in [0, 2\pi)$ such that $\|U - e^{i\phi}I\| \leq a$,
 promised one of these is the case, and $b - a \geq 1/\text{poly}(n)$.
- **k -local density matrix consistency** [170, 171, 68].⁴ Given $m = \text{poly}(n)$ density matrices $\rho_1, \dots, \rho_m$ where the state of ρ_i is only on qubits $C_i \subseteq [n]$ with $|C_i| \leq k$, determine whether:
 - (1) there is an n -qubit density matrix ρ such that for all $i \in [m]$, $\text{Tr}_{[n] \setminus C_i}(\rho) = \rho_i$, or
 - (0) for all n -qubit density matrices ρ there is an $i \in [m]$ such that $\|\text{Tr}_{[n] \setminus C_i}(\rho) - \rho_i\|_{tr} \geq b$,
 promised one of these is the case, and $b \geq 1/\text{poly}(n)$.

The following problem is in **QMA**, is not known to be **QMA**-complete, but also not known to be in **NP** (in contrast, group membership is known to be in **NP**).

- **Group non-membership** [243]. Given a finite group G , subgroup $H \leq G$, and an element $g \in G$, determine whether:
 - (1) $g \notin H$, or
 - (0) $g \in H$.

Here the groups H, G could be given as multiplication tables for a set of generating elements, or by means of an oracle (black-box) for the multiplication. The witness state for case (1) is the uniform superposition over H . Exercise 9 asks you to show completeness; proving soundness is a bit harder.

14.5 Quantum interactive proofs

The prover-verifier protocols in **QMA**, just like **NP** and **MA**, only allow one message from the prover to the verifier. This is akin to submitting a proof to a journal, where the referee then verifies the correctness of the proof without further interaction with the prover. One can also allow multiple rounds of interaction, formalizing the back-and-forth situation that often occurs when a mathematician (the prover) proves a complicated theorem in front of a colleague (the verifier): the verifier can raise objections or ask questions about steps of the proof, which the prover answers (hopefully to the verifier's satisfaction) before proceeding with the next steps. A “proof” here is a very general notion: it's any polynomial interaction that convinces the verifier of true statements and cannot convince the verifier about any false statements.

The complexity class **IP** consists of those languages that can be decided by a polynomial interaction between an unbounded prover and a polynomial-time classical verifier. Again, if $x \in L_1$ then the prover should succeed in convincing the verifier to accept (with probability $\geq 2/3$), and if $x \in L_0$ then no matter what the prover does, the verifier should reject with probability $\geq 2/3$. A fundamental classical complexity theory result says that **IP=PSPACE** [181, 224, 225].

One can define *quantum IP* (**QIP**) analogously. The two main results known about **QIP** are:

1. Every **QIP** protocol can be implemented with only 3 messages, with the prover starting [245, 154]. Roughly speaking, the 3-message protocol starts from a $\text{poly}(n)$ -message protocol for an $L \in \mathbf{QIP}$, using the 3 messages to check one randomly chosen one among the $\text{poly}(n)$ messages. This results in a 3-message protocol with $1/\text{poly}(n)$ gap between completeness and

⁴A *density matrix* is a generalization of a pure state. See Chapters 15 and 18 for the notation used here.

soundness parameters for the same L . This small gap can then be amplified to a constant gap by repeating the protocol in parallel; this repetition increases the communication per message but not the number of messages, which remains 3. In contrast, it is widely believed that **IP** restricted to 3-message protocols is a much smaller class than the full class **IP**.

2. **QIP=IP** [143]. Roughly speaking, this is proved by showing that the acceptance probability of the optimal strategy of the prover in a **QIP**-protocol can be described implicitly by an exponential-size semidefinite program. Its optimal value (which is either $\geq 2/3$ or $\leq 1/3$) can then be approximated by an exponential-size but polynomial-depth circuit. Such circuits can be implemented in **PSPACE**, and we already knew that **IP=PSPACE**.

Accordingly, adding quantum to the model of interactive proofs does not change the class of languages that can be decided, but does reduce the required number of messages between prover and verifier from polynomial to constant. See [242] for much more about quantum proof systems.

Exercises

1. Give a satisfying assignment for the 3-SAT instance of Eq. (14.1).
2. Show that 1-local Hamiltonian is in **P**.
3. (H) Suppose you had an efficient quantum algorithm for the k -local Hamiltonian problem for every a, b that satisfy $b - a \geq 1/n$. Give an efficient quantum algorithm that approximates $\lambda_{\min}$ to within additive error $\pm 2/n$.
4. Show that k -local Hamiltonian is in **QMA** in two different ways, by providing details for the following two sketches:
 - (a) Choose a $j \in [m]$ uniformly at random and measure the observable H_j on the witness state. Repeat this a few times (using new witness states each time) to approximate the expected value.
 - (b) Apply phase estimation (Section 4.6) to the unitary $U = e^{iH}$ with the given witness state; U can be implemented using Hamiltonian simulation (Chapter 9).
5. This long exercise completes the proof of the soundness for the Hamiltonian of Section 14.3.2, with the unary clock and without the three simplifying assumptions of Section 14.3.1. The gap $b - a$ between completeness and soundness will now be $\Omega(1/T^6)$ instead of $\Omega(1/T^2)$.
 - (a) Assume the error probability of the **QMA**-protocol for L is $\ll 1/T^5$. Let $|\psi'''\rangle$ be an arbitrary ground state (with energy $\lambda_{\min}$) for the Hamiltonian of Section 14.3.2. Show that if $x \in L_1$, then $\lambda_{\min} \ll 1/T^6$.
 - (b) (H) For the remainder of the exercise assume $x \in L_0$. Let P_{bc} be the projector on the subspace of bad (i.e., non-unary) clock states. Show that $\|P_{bc}|\psi'''\rangle\|^2 \leq \lambda_{\min}$.
 - (c) Show that $\|H\| \leq O(T)$.
 - (d) (H) Let $|\psi''\rangle$ be the state obtained from $|\psi'''\rangle$ by removing $P_{bc}|\psi'''\rangle$ and renormalizing. Show that its energy λ'' cannot be much larger than $\lambda_{\min}$: $\lambda'' \leq O(T\lambda_{\min})$.

- (e) (H) Write $|\psi''\rangle = \sum_{t=0}^T \alpha_t |\phi_t\rangle |t\rangle$ for nonnegative reals α_t and normalized states $|\phi_t\rangle$. Show that $\sum_{t=1}^T |\alpha_t - \alpha_{t-1}|^2 \leq 2\lambda''$.
- (f) (H) Show that for all t, t' we have $|\alpha_{t'} - \alpha_t|^2 \leq 2\lambda''T$.
- (g) (H) Let $|\psi'\rangle = \frac{1}{\sqrt{T+1}} \sum_{t=0}^T |\phi_t\rangle |t\rangle$ be the state after making all amplitudes equal in $|\psi''\rangle$. Show that $\| |\psi''\rangle - |\psi'\rangle \|^2 \leq O(T^2\lambda'')$.
- (h) Modify $|\phi_0\rangle$ and $|\phi_T\rangle$ to satisfy the second and third simplifying assumptions.
- (i) Show that the energy λ' of $|\psi'\rangle$ cannot be much larger than λ'' : $\lambda' \leq O(T^3\lambda'')$.
- (j) (H) Show that $\lambda_{\min} \geq \Omega(1/T^6)$.
6. Consider a promise problem $L = (L_1, L_0, L_*) \in \mathbf{QMA}$ and a protocol as in Definition 1 with witness states of $w(n)$ qubits.
- (a) (H) Show that there is a **QMA** protocol for L with witness states of $w(n)$ qubits and error probability $\leq \frac{1}{3}2^{-w(n)}$.
- (b) Suppose we now replace the $w(n)$ qubits of the protocol of (a) with a uniformly random $w(n)$ -bit basis state. Show that if $x \in L_1$, then the acceptance probability (i.e., the probability of output 1) is $\geq \frac{2}{3}2^{-w(n)}$, while if $x \in L_0$ then it is $\leq \frac{1}{3}2^{-w(n)}$.
- (c) Use (b) to show that **QMA** with witness states restricted to $w(n) = O(\log n)$ qubits equals **BQP**.
7. (H) Let **PP** be the class of promise problems $L = (L_1, L_0, L_*)$ that can be decided by a polynomial-time classical algorithm with success probability $> 1/2$ (meaning that for inputs $x \in L_1$ the algorithm accepts with probability $> 1/2$, and for $x \in L_0$ it accepts with probability $< 1/2$). Show that **QMA** $\subseteq$ **PP**.
8. Consider the following computational decision problem. We are given a Hamiltonian H of the form of page 117, with the additional property that $w(n) = 0$. We are promised that the smallest eigenvalue $\lambda_{\min}$ of H is either $\leq 1/(4T(T+1))$ (“yes-instance”) or $\geq 1/(2T(T+1))$ (“no-instance”), and the problem is to decide which case we are in.
- (a) (H) Show that this problem is in **BQP**.
- (b) Show that this problem is **BQP**-hard, meaning that for every promise problem $L = (L_1, L_0, L_*)$ in **BQP** there exists a classical deterministic polynomial-time algorithm that maps every $x \in L_1$ to a yes-instance of the above problem and every $x \in L_0$ to a no-instance.
9. Suppose we are given (in some form) a finite group G , a subgroup $H \leq G$, and an element $g \in G$, and we can efficiently implement the unitary map V corresponding to multiplication with g (i.e., the map $V : |h\rangle \mapsto |h \circ g\rangle$). Let

$$|\psi\rangle = \frac{1}{\sqrt{|H|}} \sum_{h \in H} |h\rangle$$

be the uniform superposition over H . The prover for the non-membership problem can construct this state, though not necessarily efficiently.

- (a) Show that if $g \in H$, then $V|\psi\rangle = |\psi\rangle$.
- (b) Show that if $g \notin H$, then $V|\psi\rangle$ is orthogonal to $|\psi\rangle$.
- (c) Consider the following procedure that the verifier can use to test if $g \in H$ or not:
 - (1) prepare an auxiliary qubit in state $H|0\rangle$,
 - (2) conditioned on that qubit apply V to $|\psi\rangle$,
 - (3) apply H to the auxiliary qubit and measure it.Show that the probability of measurement outcome 0 is 1 if $g \in H$, and is $1/2$ if $g \notin H$.

Chapter 15

Quantum Encodings, with a Non-Quantum Application

15.1 Mixed states and general measurements

So far, we have restricted our states to so-called *pure states*: unit vectors of amplitudes. In the classical world we often have uncertainty about the state of a system, which can be expressed by viewing the state as a random variable that has a certain probability distribution over the set of basis states. Similarly we can define a *mixed* quantum state as a probability distribution (or “mixture”) over pure states. While pure states are written as vectors, it is most convenient to write mixed states as *density matrices*. A pure state $|\phi\rangle$ corresponds to the density matrix $|\phi\rangle\langle\phi|$, which is the outer product of the vector $|\phi\rangle$ with itself. For example, the pure state $|\phi\rangle = \alpha|0\rangle + \beta|1\rangle$ corresponds to the density matrix

$$|\phi\rangle\langle\phi| = \begin{pmatrix} \alpha \\ \beta \end{pmatrix} \cdot (\alpha^* \quad \beta^*) = \begin{pmatrix} |\alpha|^2 & \alpha\beta^* \\ \alpha^*\beta & |\beta|^2 \end{pmatrix}.$$

A mixed state that is in pure states $|\phi_1\rangle, \dots, |\phi_\ell\rangle$ with probabilities $p_1, \dots, p_\ell$, respectively, corresponds to the density matrix $\rho = \sum_{i=1}^{\ell} p_i |\phi_i\rangle\langle\phi_i|$. This ρ is sometimes called a “mixture” of the states $|\phi_1\rangle, \dots, |\phi_\ell\rangle$.¹ The set of density matrices is exactly the set of positive semidefinite (psd) matrices of trace 1. A mixed state is pure if, and only if, it has rank 1.

You can always write a mixed state ρ as a probability distribution over *orthogonal* pure states, using the diagonalization of ρ (see Appendix A.5) plus the observations that (1) the eigenvalues of a trace-1 psd matrix form a probability distribution, and (2) that the eigenvectors of a Hermitian matrix can be assumed to form an orthonormal set without loss of generality. But you can also write ρ as a convex combination of non-orthogonal states (see Exercise 1.c).

Applying a unitary U to a pure state $|\phi\rangle$ gives pure state $U|\phi\rangle$. Written in terms of rank-1 density matrices, this corresponds to the map

$$|\phi\rangle\langle\phi| \mapsto U|\phi\rangle\langle\phi|U^*.$$

¹Note that applying the probabilities p_i to the vectors $|\phi_i\rangle$ (rather than to the matrices $|\phi_i\rangle\langle\phi_i|$) does not make sense in general, because $\sum_{i=1}^{\ell} p_i |\phi_i\rangle$ need not be a unit vector. Using square roots of the probabilities also doesn't work, because $\sum_{i=1}^{\ell} \sqrt{p_i} |\phi_i\rangle$ need not be a unit vector either if the $|\phi_i\rangle$ are not pairwise orthogonal.

By linearity, this actually tells us that a unitary acts on an arbitrary mixed state by conjugation:

$$\rho \mapsto U\rho U^*.$$

What about measurements? Recall from Section 1.2.2 that an m -outcome projective measurement corresponds to m orthogonal projectors $P_1, \dots, P_m$ that satisfy $\sum_{i=1}^m P_i = I$. When applying this measurement to a mixed state ρ , the probability to see outcome i is given by $p_i = \text{Tr}(P_i \rho)$. If we get outcome i , then the state collapses to $P_i \rho P_i / p_i$ (the division by p_i renormalizes the state to have trace 1). This may look weird, but let's recover our familiar measurement in the computational basis in this framework. Suppose we measure a state $|\phi\rangle = \sum_{j=1}^d \alpha_j |j\rangle$ using d projectors $P_i = |i\rangle\langle i|$ (note that $\sum_i P_i$ is the identity on the d -dimensional space). The probability to get outcome i is given by $p_i = \text{Tr}(P_i |\phi\rangle\langle\phi|) = |\langle i|\phi\rangle|^2 = |\alpha_i|^2$. If we get outcome i then the state collapses to $P_i |\phi\rangle\langle\phi| P_i / p_i = \alpha_i |i\rangle\langle i| \alpha_i^* / p_i = |i\rangle\langle i|$. This is exactly the measurement in the computational basis as we have used it until now. Similarly, a measurement of the first register of a two-register state corresponds to projectors $P_i = |i\rangle\langle i| \otimes I$, where i goes over all basis states of the first register.

If we only care about the final probability distribution on the m outcomes, not about the resulting state, then the most general thing we can do is a POVM. This is specified by m positive semidefinite matrices $E_1, \dots, E_m$ satisfying $\sum_{i=1}^m E_i = I$. When measuring a state ρ , the probability of outcome i is given by $\text{Tr}(E_i \rho)$.

15.2 Quantum encodings and their limits

Quantum information theory studies the quantum generalizations of familiar notions from classical information theory such as Shannon entropy, mutual information, channel capacities, etc. Here we will discuss a few quantum information-theoretic results that all have the same flavor: they say that a low-dimensional quantum state (i.e., a small number of qubits) cannot contain too much *accessible* information.

Holevo's Theorem: The mother of all such results is Holevo's theorem from 1973 [138], which predates the area of quantum computing by several decades. Its proper technical statement is in terms of a quantum generalization of mutual information, but the following consequence of it (derived by Cleve et al. [90]) about two communicating parties, suffices for our purposes.

Theorem 3 (Holevo, CDNT) *Suppose Alice wants to communicate some classical string x to Bob.*

- *If Alice sends Bob m qubits, and they did not share any prior entanglement, then Bob receives at most m bits of information about x .*
- *If Alice sends Bob m qubits, and they did share some prior entangled state, then Bob receives at most $2m$ bits of information about x .*
- *If Alice sends Bob m classical bits, and they did share some prior entangled state, then Bob receives at most m bits of information about x .*

This theorem is slightly imprecisely stated here, but the intuition should be clear: if Bob makes any measurement on his state after the communication, then the mutual information between his

classical outcome and Alice's x , is bounded by m or $2m$. In particular, the first part of the theorem says that if we encode some classical random variable X in an m -qubit state², then no measurement on the quantum state can give more than m bits of information about X . If we encoded the classical information in an m -bit system instead of an m -qubit system this would be a trivial statement, but the proof of Holevo's theorem is quite non-trivial. Thus we see that an m -qubit state, despite somehow "containing" 2^m complex amplitudes, is no better than m classical bits for the purpose of storing or transmitting information. Prior entanglement can improve this by a factor of 2 because of superdense coding (see Exercise 1.12), but no more than that.

Low-dimensional encodings: Here we provide a "poor man's version" of Holevo's theorem due to Nayak [193, Theorem 2.4.2], which has a simple proof and often suffices for applications. Suppose we have a classical random variable X , uniformly distributed over $[N] = \{1, \dots, N\}$.³ Let $x \mapsto \rho_x$ be some encoding of $[N]$, where ρ_x is a mixed state in a d -dimensional space. Let $E_1, \dots, E_N$ be the POVM operators applied for decoding; these sum to the d -dimensional identity operator. Then the probability of correct decoding in case $X = x$, is

$$p_x = \text{Tr}(E_x \rho_x) \leq \text{Tr}(E_x).$$

The sum of these success probabilities is at most

$$\sum_{x=1}^N p_x \leq \sum_{x=1}^N \text{Tr}(E_x) = \text{Tr}\left(\sum_{x=1}^N E_x\right) = \text{Tr}(I) = d. \quad (15.1)$$

In other words, if we are encoding one of N classical values in a d -dimensional quantum state, then any measurement to decode the encoded classical value has average success probability at most d/N (uniformly averaged over all N values that we can encode). For example, if we encode n uniformly random bits into m qubits, we will have $N = 2^n$, $d = 2^m$, and the average success probability of decoding is at most $2^m/2^n$, which is very small unless m is nearly n .

Random access codes: The previous two results dealt with the situation where we encoded a classical random variable X in some quantum system, and would like to recover the original value X by an appropriate measurement on that quantum system. However, suppose $X = X_1 \dots X_n$ is a string of n bits, uniformly distributed and encoded by a map $x \mapsto \rho_x$, and it suffices for us if we are able to decode individual bits X_i from this with some probability $p > 1/2$. More precisely, for each $i \in [n]$ there should exist a measurement $\{M_i, I - M_i\}$ allowing us to recover x_i . M_i would correspond to output 1 and $I - M_i$ to output 0. Hence for each $x \in \{0, 1\}^n$ we should have $\text{Tr}(M_i \rho_x) \geq p$ if $x_i = 1$ and $\text{Tr}(M_i \rho_x) \leq 1 - p$ if $x_i = 0$. An encoding satisfying this is called a *quantum random access code*, since it allows us to choose which bit of X we would like to access. Note that the measurement to recover x_i can change the state ρ_x , so generally we may not be able to decode more than one bit of x (also, we cannot copy ρ_x because of the no-cloning theorem, see Exercise 1.10).

An encoding that allows us to recover (with high success probability) an n -bit string requires about n qubits by Holevo. Random access codes only allow us to recover *each* of the n bits. Can

²Via an encoding map $x \mapsto \rho_x$; we generally use upper-case letters like X to denote random variables, lower-case letters like x to denote specific values.

³NB: unlike in most of these lecture notes, N need not equal 2^n in this chapter!

they be much shorter? In small cases they can be: for instance, one can encode two classical bits into one qubit, in such a way that each of the two bits can be recovered with success probability 85% from that qubit (see Exercise 2). However, Nayak [193] proved that asymptotically quantum random access codes cannot be much shorter than classical.

Theorem 4 (Nayak) *Let $x \mapsto \rho_x$ be a quantum random access encoding of n -bit strings into m -qubit states such that, for each $i \in [n]$, we can decode X_i from $|\phi_X\rangle$ with success probability p (averaged over a uniform choice of x and the measurement randomness). Then $m \geq (1 - H(p))n$, where $H(p) = -p \log p - (1 - p) \log(1 - p)$ is the binary entropy function.*

The intuition of the proof is quite simple: since the quantum state allows us to predict the bit X_i with probability p_i , it reduces the “uncertainty” about X_i from 1 bit to $H(p_i)$ bits. Hence it contains at least $1 - H(p_i)$ bits of information about X_i . Since all n X_i ’s are independent, the state has to contain at least $\sum_{i=1}^n (1 - H(p_i))$ bits of information about X in total.

15.3 Lower bounds on locally decodable codes

Here we will give an application of quantum information theory to a *classical* problem.⁴

The development of error-correcting codes is one of the success stories of science in the second half of the 20th century. Such codes are eminently practical, and are widely used to protect information stored on discs, communication over channels, etc. From a theoretical perspective, there exist codes that are nearly optimal in a number of different respects simultaneously: they have constant rate, can protect against a constant noise-rate, and have linear-time encoding and decoding procedures. We refer to Trevisan’s survey [236] for a complexity-oriented discussion of codes and their applications.

One drawback of ordinary error-correcting codes is that we cannot efficiently decode small parts of the encoded information. If we want to learn, say, the first bit of the encoded message then we usually still need to decode the whole encoded string. This is relevant in situations where we have encoded a very large string (say, a library of books, or a large database), but are only interested in recovering small pieces of it at any given time. Dividing the data into small blocks and encoding each block separately will not work: small chunks will be efficiently decodable but not error-correcting, since a tiny fraction of well-placed noise could wipe out the encoding of one chunk completely. There exist, however, error-correcting codes that are *locally decodable*, in the sense that we can efficiently recover individual bits of the encoded string.

Definition 3 $C : \{0, 1\}^n \rightarrow \{0, 1\}^N$ is a (q, δ, ε) -locally decodable code (LDC) if there is a classical randomized decoding algorithm A such that

1. A makes at most q queries to an N -bit string y .
2. For all $x \in \{0, 1\}^n$ and $i \in [n]$, and all $y \in \{0, 1\}^N$ with Hamming distance $d(C(x), y) \leq \delta N$ we have $\Pr[A^y(i) = x_i] \geq 1/2 + \varepsilon$.

Here δ is an upper bound on the fraction of bits of the codeword that may have been corrupted (by some noise process, or by our worst enemy), and ε is a lower bound on the advantage we have

⁴There is a growing number of such applications of quantum tools to non-quantum problems. See [100] for a survey.

compared to just randomly guessing the value of the bit x_i . The notation $A^y(i)$ reflects that the decoder A has two different types of input. On the one hand there is the (possibly corrupted) codeword y , to which the decoder has oracle access and from which it can read at most q bits of its choice. On the other hand there is the index i of the bit that needs to be recovered, and which is known fully to the decoder.

The main question about LDCs is the tradeoff between the codeword length N and the number of queries q (which is a proxy for the decoding-time). This tradeoff is still not very well understood. The only case where we know the answer is the case of $q = 2$ queries.⁵ For $q = 2$ there is the Hadamard code: given $x \in \{0, 1\}^n$, define a codeword of length $N = 2^n$ by writing down the bits $x \cdot z \bmod 2$, for all $z \in \{0, 1\}^n$, with the z 's ordered in lexicographic order. For example for $n = 2$ and $x = 10$, the codeword would be

$$C(x) = (x \cdot 00, x \cdot 01, x \cdot 10, x \cdot 11) = 0011.$$

One can decode x_i with 2 queries as follows: choose $z \in \{0, 1\}^n$ uniformly at random, query the (possibly corrupted) codeword at indices z and $z \oplus e_i$ (where the latter denotes the string obtained from z by flipping its i -th bit), and output the sum of the two returned bits modulo 2. Individually, each of these two indices z and $z \oplus e_i$ is uniformly distributed. Hence for each of them, the probability that the returned bit is corrupted is at most δ . By the union bound, with probability at least $1 - 2\delta$, both queries return the uncorrupted values. Adding these two bits mod 2 gives the correct answer:

$$C(x)_z \oplus C(x)_{z \oplus e_i} = (x \cdot z) \oplus (x \cdot (z \oplus e_i)) = x \cdot e_i = x_i.$$

Thus the Hadamard code is a $(2, \delta, 1/2 - 2\delta)$ -LDC of exponential length.

The only superpolynomial *lower bound* known on the length of LDCs is for the case of 2 queries: there one needs an exponential codeword length and hence the Hadamard code is essentially optimal. This is shown via a *quantum* argument [152]—despite the fact that the result is a purely classical result, about classical codes and classical decoders. The easiest way to present this argument is to assume the following fact, which states a kind of “normal form” for the decoder.

Fact 1 (Katz & Trevisan [148] + folklore) *For every (q, δ, ε) -LDC $C : \{0, 1\}^n \rightarrow \{0, 1\}^N$, and for each $i \in [n]$, there exists a set $\mathcal{M}_i$ of $\Omega(\delta \varepsilon N / q^2)$ disjoint tuples, each of at most q indices from $[N]$, and a bit $a_{i,t}$ for each tuple $t \in \mathcal{M}_i$, such that the following holds:*

$$\Pr_{x \in \{0, 1\}^n} \left[x_i = a_{i,t} \oplus \sum_{j \in t} C(x)_j \right] \geq 1/2 + \Omega(\varepsilon/2^q), \quad (15.2)$$

where the probability is taken uniformly over x . Hence to decode x_i from $C(x)$, the decoder can just query the indices in a randomly chosen tuple t from $\mathcal{M}_i$, outputting the sum of those q bits and $a_{i,t}$.

Note that the above decoder for the Hadamard code is already of this form, with $\mathcal{M}_i$ consisting of the 2^{n-1} pairs $\{z, z \oplus e_i\}$. We omit the fairly easy proof of Fact 1, which uses purely classical ideas.

⁵For $q = 1$, LDCs don't exist once n is sufficiently large [148]. For $q = 3$, the best upper bound known on the codeword length N is roughly $2^{2^{\sqrt{\log n}}}$ [252, 104], while the best lower bound is roughly n^3 [10].

Now suppose $C : \{0, 1\}^n \rightarrow \{0, 1\}^N$ is a $(2, \delta, \varepsilon)$ -LDC. We want to show that the codelength N must be exponentially large in n . Our strategy is to show that the following N -dimensional quantum encoding is a quantum random access code for x (with some success probability $p > 1/2$):

$$x \mapsto |\phi_x\rangle = \frac{1}{\sqrt{N}} \sum_{j=1}^N (-1)^{C(x)_j} |j\rangle.$$

Theorem 4 then implies that the number of qubits of this state (which is $\lceil \log N \rceil$) is at least $(1 - H(p))n = \Omega(n)$, and we are done.

Suppose we want to recover x_i from $|\phi_x\rangle$. We'll do this by a sequence of two measurements, as follows. We turn each $\mathcal{M}_i$ from Fact 1 into a projective measurement: for each pair $(j, k) \in \mathcal{M}_i$ form the projector $P_{jk} = |j\rangle\langle j| + |k\rangle\langle k|$, and let $P_{rest} = \sum_{j \notin \cup_{t \in \mathcal{M}_i} t} |j\rangle\langle j|$ be the projector on the remaining indices. These $|\mathcal{M}_i| + 1$ projectors sum to the N -dimensional identity matrix, so they form a valid projective measurement. Applying this to $|\phi_x\rangle$ gives outcome (j, k) with probability $\|P_{jk}|\phi_x\rangle\|^2 = 2/N$ for each $(j, k) \in \mathcal{M}_i$. There are $|\mathcal{M}_i| = \Omega(\delta\varepsilon N)$ different (j, k) -pairs in $\mathcal{M}_i$, so the probability to see one of those as outcome of the measurement, is $|\mathcal{M}_i| \cdot 2/N = \Omega(\delta\varepsilon)$. With the remaining probability $r = 1 - \Omega(\delta\varepsilon)$, we'll get "rest" as outcome of the measurement. In the latter case we didn't get anything useful from the measurement, so we'll just output a fair coin flip as our guess for x_i (then the output will equal x_i with probability exactly $1/2$). In case we got one of the (j, k) as measurement outcome, the state has collapsed to the following useful superposition:

$$\frac{1}{\sqrt{2}} \left((-1)^{C(x)_j} |j\rangle + (-1)^{C(x)_k} |k\rangle \right) = \frac{(-1)^{C(x)_j}}{\sqrt{2}} \left(|j\rangle + (-1)^{C(x)_j \oplus C(x)_k} |k\rangle \right)$$

We know what j and k are, because it is the outcome of the measurement on $|\phi_x\rangle$. Now do a 2-outcome projective measurement with projectors P_0 and P_1 corresponding to the two vectors $\frac{1}{\sqrt{2}}(|j\rangle + |k\rangle)$ and $\frac{1}{\sqrt{2}}(|j\rangle - |k\rangle)$, respectively. The measurement outcome equals the value $C(x)_j \oplus C(x)_k$ with probability 1. By Eq. (15.2), if we add the bit $a_{i,(j,k)}$ to this, we get x_i with probability at least $1/2 + \Omega(\varepsilon)$. The success probability of recovering x_i , averaged over all x , is

$$p \geq \frac{1}{2}r + \left(\frac{1}{2} + \Omega(\varepsilon) \right) (1 - r) = \frac{1}{2} + \Omega(\delta\varepsilon^2).$$

Thus we have constructed a random access code that encodes n bits into $\log N$ qubits, and has success probability at least p . Applying Theorem 4 and using that

$$1 - H(1/2 + \eta) = \Theta(\eta^2) \text{ for } \eta \in [0, 1/2], \quad (15.3)$$

we obtain the following:

Theorem 5 *If $C : \{0, 1\}^n \rightarrow \{0, 1\}^N$ is a $(2, \delta, \varepsilon)$ -locally decodable code, then $N \geq 2^{\Omega(\delta^2 \varepsilon^4 n)}$.*

Exercises

1. Suppose we have a qubit whose density matrix is ρ .

- (a) Show that there exist real numbers r_0, r_1, r_2, r_3 such that $\rho = \frac{r_0}{2}I + \frac{r_1}{2}X + \frac{r_2}{2}Y + \frac{r_3}{2}Z$, where I, X, Y, Z are the Pauli matrices (see Appendix A.9).

- (b) Show that $r_0 = 1$.
- (c) Show that $\text{Tr}(\rho^2) = \frac{1}{2}(r_0^2 + r_1^2 + r_2^2 + r_3^2)$.
- (d) Show that $r_1^2 + r_2^2 + r_3^2 \leq 1$.
- (e) Show that $r_1^2 + r_2^2 + r_3^2 = 1$ iff ρ is a pure state.

Comment: One can represent a qubit by the corresponding vector $(r_1, r_2, r_3) \in \mathbb{R}^3$. By part (e) the pure states are exactly the ones that are on the boundary of the 3-dimensional sphere of radius 1. The mixed states are in the interior of the sphere, and the maximally mixed state $\rho = I/2$ is at the origin $(0, 0, 0)$. This geometric picture is called the *Bloch-sphere* representation of a qubit, and is very useful in physics. For example, single-qubit gates correspond to rotations on this sphere. Unfortunately this picture does not generalize cleanly to more than one qubit.

- 2. (a) (H) Give a quantum random access code that encodes 2 classical bits into 1 qubit, such that each of the two classical bits can be recovered from the quantum encoding with success probability $p \geq 0.85$.
- (b) Prove an upper bound of $1/2 + O(1/\sqrt{n})$ on the success probability p for a random access code that encodes n classical bits into 1 qubit.
- 3. (H) Teleportation transfers an arbitrary unknown qubit from Alice to Bob, using 1 EPR-pair and 2 classical bits of communication from Alice to Bob (see Section 1.5). Prove that these 2 bits of communication are necessary, i.e., you cannot teleport an arbitrary unknown qubit using 1 EPR-pair and only 1 classical bit of communication.
- 4. Suppose $n + 1 = 2^k$ for some integer k . For $\ell \in \{0, \dots, n\}$ define n -qubit state

$$|\psi_\ell\rangle = \frac{1}{\sqrt{\binom{n}{\ell}}} \sum_{x \in \{0,1\}^n: |x|=\ell} |x\rangle,$$

where $|x|$ denotes the Hamming weight (number of 1s) in x .

- (a) Show that $\langle \psi_\ell | \psi_{\ell'} \rangle$ equals 1 if $\ell = \ell'$, and equals 0 otherwise.
- (b) Consider a qubit $|\phi\rangle = \alpha|0\rangle + \beta|1\rangle$. Show that the n -qubit state $|\phi\rangle^{\otimes n}$ can be written as a linear combination of the states $|\psi_\ell\rangle$. Say explicitly what the coefficients of this linear combination are.
- (c) Give a unitary V , independent of α, β , that encodes $|\phi\rangle^{\otimes n}$ into a k -qubit state $|\psi\rangle$ in the sense that

$$V : |\phi\rangle^{\otimes n} \mapsto |\psi\rangle \otimes |0^{n-k}\rangle.$$

Say explicitly what your state $|\psi\rangle$ is and how it depends on α and β (you're not required to write out circuits).

- 5. Consider the Hadamard code C that encodes $n = 2$ bits $x_1 x_2$ into a codeword of $N = 4$ bits.
 - (a) Give the 4-bit codeword $C(11)$.
 - (b) What are the states $|\phi_x\rangle$ that arise as quantum random access code when we apply the LDC lower bound proof of Section 15.3 to C ? Give the 4 states, not one general formula.

- (c) What is the measurement used for recovering x_2 from $|\phi_x\rangle$ at the end of that proof? You may either describe this as a sequence of two projective measurements, or as one (combined) projective measurement.
6. (a) Let $x \in \{0,1\}^n$. Suppose we apply the $2n$ -qubit Fourier transform $F_{2^{2n}}$ on the $2n$ -bit basis state $|x\rangle|0^n\rangle$, followed by $F_{2^n}^{-1}$ on the last n qubits (and identity on the first n qubits). Show that we end up with the $2n$ -qubit state $|+\rangle^{\otimes n}|x\rangle$.
- (b) (H) Consider a circuit C that implements $F_{2^{2n}}$ in some way using arbitrary 1-qubit and 2-qubit gates (C can do anything, it need not be one of the specific QFT circuits from the lecture notes). Show that there must be $\Omega(n)$ two-qubit gates in C where the control bit lies in the first n qubits of the state and the target qubit lies in the second n qubits (or vice versa).
7. Suppose there are two classically-known mixed states ρ_0 and ρ_1 , and we are given one copy of quantum state ρ_b for a uniformly random $b \in \{0,1\}$. We want to learn b using some 2-outcome projective measurement with operators P_0 and P_1 , which we can choose ourselves depending on what ρ_0 and ρ_1 are. The success probability of such a measurement is $\frac{1}{2}(\text{Tr}(P_0\rho_0) + \text{Tr}(P_1\rho_1))$.
- (a) (H) Give a projective measurement with success probability $\geq \frac{1}{2} + \frac{1}{4}\|\rho_0 - \rho_1\|_1$, where the norm $\|A\|_1$ of a matrix A is defined as the sum of A 's singular values.
- (b) Show that every 2-outcome projective measurement P_0, P_1 has a success probability that is $\leq \frac{1}{2} + \frac{1}{4}\|\rho_0 - \rho_1\|_1$.

Chapter 16

Quantum Communication Complexity

Communication complexity was first introduced by Yao [250], and has been studied extensively in the area of theoretical computer science and has deep connections with seemingly unrelated areas, such as VLSI design, circuit lower bounds, lower bounds on branching programs, sizes of data structures, and bounds on the length of logical proof systems, to name just a few.

16.1 Classical communication complexity

First we sketch the setting for classical communication complexity. Alice and Bob want to compute some function $f : \mathcal{D} \rightarrow \{0, 1\}$, where $\mathcal{D} \subseteq X \times Y$.¹ Alice receives input $x \in X$, Bob receives input $y \in Y$, with $(x, y) \in \mathcal{D}$. A typical situation, illustrated in Fig. 16.1, is where $X = Y = \{0, 1\}^n$, so both Alice and Bob receive an n -bit input string. As the value $f(x, y)$ will generally depend on both x and y , some communication between Alice and Bob is required in order for them to be able to compute $f(x, y)$. We are interested in the *minimal* amount of communication they need.

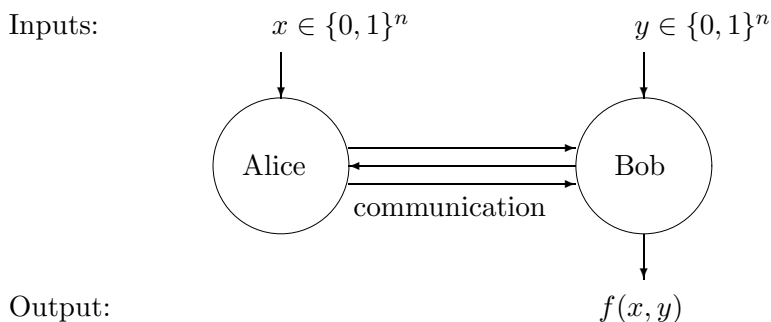


Figure 16.1: Alice and Bob solving a communication complexity problem

A communication *protocol* is a distributed algorithm where first Alice does some individual computation, and then sends a message (of one or more bits) to Bob, then Bob does some computation and sends a message to Alice, etc. Each message is called a *round*. After one or more rounds the protocol terminates and one of the parties (let's say Bob) outputs some value that should be $f(x, y)$. The *cost* of a protocol is the total number of bits communicated on the worst-case input.

¹If the domain $\mathcal{D}$ equals $X \times Y$ then f is called a *total* function, otherwise it is called a *partial* or *promise* function.

A *deterministic* protocol for f always has to output the right value $f(x, y)$ for all $(x, y) \in \mathcal{D}$. In a *bounded-error* protocol, Alice and Bob may flip coins and the protocol has to output the right value $f(x, y)$ with probability $\geq 2/3$ for all $(x, y) \in \mathcal{D}$. We could either allow Alice and Bob to toss coins individually (local randomness, or “private coin”) or jointly (shared randomness, or “public coin”). A public coin can simulate a private coin and is potentially more powerful. However, Newman’s theorem [194] says that having a public coin can save at most $O(\log n)$ bits of communication, compared to a protocol with a private coin.

To illustrate the power of randomness, let us give a simple yet efficient bounded-error protocol for the equality problem, where the goal for Alice is to determine whether her n -bit input is the same as Bob’s or not: $f(x, y) = 1$ if $x = y$, and $f(x, y) = 0$ otherwise. Alice and Bob jointly toss a random string $r \in \{0, 1\}^n$. Alice sends the bit $a = x \cdot r$ to Bob (where ‘ $\cdot$ ’ is inner product mod 2). Bob computes $b = y \cdot r$ and compares this with a . If $x = y$ then $a = b$, but if $x \neq y$ then $a \neq b$ with probability $1/2$. Repeating this a few times, Alice and Bob can decide equality with small error probability using $O(n)$ public coin flips and a constant amount of communication. This protocol uses public coins, but note that Newman’s theorem implies that there exists an $O(\log n)$ -bit protocol that uses a private coin (see Exercise 9 for an explicit protocol). Note that the correct output of the equality function depends on all n bits of x , but Bob does not need to learn all n bits of x in order to be able to decide equality with high success probability. In contrast, one can show that *deterministic* protocols for the equality problem need n bits of communication, so then Alice might as well just send x to Bob.

16.2 The quantum question

Now what happens if we give Alice and Bob a quantum computer and allow them to send each other qubits and/or to make use of EPR-pairs that they share at the start of the protocol?

Formally speaking, we can model a quantum protocol as follows. The total state consists of 3 parts: Alice’s private space, the channel, and Bob’s private space. The starting state is $|x\rangle|0\rangle|y\rangle$: Alice gets x , the channel is initialized to 0, and Bob gets y . Now Alice applies a unitary transformation to her space and the channel. This corresponds to her private computation as well as to putting a message on the channel (the length of this message is the number of channel-qubits affected by Alice’s operation). Then Bob applies a unitary transformation to his space and the channel, etc. At the end of the protocol Alice or Bob makes a measurement to determine the output of the protocol. This model was introduced by Yao [251].

In the second model, introduced by Cleve and Buhrman [89], Alice and Bob share an unlimited number of EPR-pairs at the start of the protocol, but now they communicate via a *classical* channel: the channel has to be in a classical state throughout the protocol. We only count the communication, not the number of EPR-pairs used. Protocols of this kind can simulate protocols of the first kind with only a factor 2 overhead: using teleportation, the parties can send each other a qubit using an EPR-pair and two classical bits of communication. Hence the qubit-protocols that we describe below also immediately yield protocols that work with entanglement and a classical channel. Note that an EPR-pair can simulate a public coin toss: if Alice and Bob each measure their half of the pair of qubits, they get the same random bit.

The third variant combines the strengths of the other two: here Alice and Bob start out with an unlimited number of EPR-pairs *and* they are allowed to communicate qubits. This third kind of communication complexity is in fact equivalent to the second, up to a factor of 2, again by

teleportation.

Before continuing to study this model, we first have to face an important question: *is there anything to be gained here?* At first sight, the following argument seems to rule out any significant gain. Suppose that in the classical world k bits have to be communicated in order to compute f . Since Holevo's theorem says that k qubits cannot contain more information than k classical bits, it seems that the quantum communication complexity should be roughly k qubits as well (maybe $k/2$ to account for superdense coding, but not less). Surprisingly (and fortunately for us), this argument is false, and quantum communication can sometimes be much less than classical communication complexity. The information-theoretic argument via Holevo's theorem fails, because Alice and Bob do not need to communicate the information in the k bits of the classical protocol; they are only interested in the value $f(x, y)$, which is just 1 bit. Below we will go over four of the main examples that have so far been found of differences between quantum and classical communication complexity.

16.3 Example 1: Distributed Deutsch-Jozsa

The first impressively large gaps between quantum and classical communication complexity were exhibited by Buhrman, Cleve, and Wigderson [74]. Their protocols are distributed versions of known quantum query algorithms, like the Deutsch-Jozsa and Grover algorithms. Let us start with the first one. It is actually explained most easily in a direct way, without reference to the Deutsch-Jozsa algorithm (though that is where the idea came from). The problem is a promise version of the equality problem. Suppose the n -bit inputs x and y are restricted to the following case:

Distributed Deutsch-Jozsa: either $x = y$, or x and y differ in exactly $n/2$ positions

Note that this promise only makes sense if n is an even number, otherwise $n/2$ would not be integer. In fact it will be convenient to assume n is a power of 2. Here is a simple quantum protocol to solve this promise version of equality using only $\log n$ qubits of communication:

1. Alice sends Bob the $\log n$ -qubit state $\frac{1}{\sqrt{n}} \sum_{i=1}^n (-1)^{x_i} |i\rangle$, which she can prepare unitarily from x and $\log n$ $|0\rangle$ -qubits.
2. Bob applies the unitary map $|i\rangle \mapsto (-1)^{y_i} |i\rangle$ to the state, applies a Hadamard transform to each qubit (for this it is convenient to view i as a $\log n$ -bit string), and measures the resulting $\log n$ -qubit state.
3. Bob outputs 1 if the measurement gave $|0^{\log n}\rangle$ and outputs 0 otherwise.

It is clear that this protocol only communicates $\log n$ qubits, but why does it work? Note that the state that Bob measures is

$$H^{\otimes \log n} \left(\frac{1}{\sqrt{n}} \sum_{i=1}^n (-1)^{x_i + y_i} |i\rangle \right) = \frac{1}{n} \sum_{i=1}^n (-1)^{x_i + y_i} \sum_{j \in \{0,1\}^{\log n}} (-1)^{i \cdot j} |j\rangle$$

This superposition looks rather unwieldy, but consider the amplitude of the $|0^{\log n}\rangle$ basis state. It is $\frac{1}{n} \sum_{i=1}^n (-1)^{x_i + y_i}$, which is 1 if $x = y$ and 0 otherwise because the promise now guarantees that x and y differ in exactly $n/2$ of the bits! Hence Bob will always give the correct answer.

What about efficient *classical* protocols (without entanglement) for this problem? Proving lower bounds on communication complexity often requires a very technical combinatorial analysis. Buhrman, Cleve, and Wigderson used a deep combinatorial result from [114] to prove that every classical errorless protocol for this problem needs to send at least $0.007n$ bits.

This $\log n$ -qubits-vs- $0.007n$ -bits example was the first exponentially large separation of quantum and classical communication complexity. Notice, however, that the difference disappears if we move to the *bounded-error* setting, allowing the protocol to have some small error probability. We can use the randomized protocol for equality discussed above or even simpler: Alice can just send a few (i, x_i) pairs to Bob, who then compares the x_i 's with his y_i 's. If $x = y$ he will not see a difference, but if x and y differ in $n/2$ positions, then Bob will probably detect this. Hence $O(\log n)$ classical bits of communication suffice in the bounded-error setting, in sharp contrast to the errorless setting.

16.4 Example 2: The Intersection problem

Now consider the Intersection function, which is 1 if $x_i = y_i = 1$ for at least one i . Buhrman, Cleve, and Wigderson [74] also presented an efficient quantum protocol for this, based on Grover's search algorithm (Chapter 7). We can solve Intersection if we can solve the following search problem: find some i such that $x_i = y_i = 1$, if such an i exists.² We want to find a solution to the search problem on the string $z = x \wedge y$ (which is the bit-wise AND of x and y), since $z_i = 1$ whenever both $x_i = 1$ and $y_i = 1$. The idea is now to let Alice run Grover's algorithm to search for such a solution. Clearly, she can prepare the uniform starting state herself. She can also apply the unitaries H and R herself. The only thing where she needs Bob's help, is in implementing the phase-query $O_{z,\pm}$ (which she needs to do $O(\sqrt{n})$ times, because that's how many queries Grover makes). Alice and Bob can together implement a phase-query as follows. Whenever Alice wants to apply $O_{z,\pm}$ to a state

$$|\phi\rangle = \sum_{i=1}^n \alpha_i |i\rangle,$$

she tags on her x_i 's in an extra qubit (which she can do by the unitary map $|i\rangle|0\rangle \mapsto |i\rangle|x_i\rangle$) and sends Bob the state

$$\sum_{i=1}^n \alpha_i |i\rangle |x_i\rangle.$$

Bob applies the unitary map

$$|i\rangle |x_i\rangle \mapsto (-1)^{x_i \wedge y_i} |i\rangle |x_i\rangle$$

and sends back the result. Alice sets the last qubit back to $|0\rangle$ (which she can do unitarily because she has x), and now she has the state $O_{z,\pm}|\phi\rangle$! Thus we can simulate $O_{z,\pm}$ using 2 messages of $\log(n) + 1$ qubits each. Thus Alice and Bob can run Grover's algorithm to find an intersection, using $O(\sqrt{n})$ messages of $O(\log n)$ qubits each, for total communication of $O(\sqrt{n} \log n)$ qubits. Later Aaronson and Ambainis [3] gave a more complicated protocol that uses $O(\sqrt{n})$ qubits of communication.

What about lower bounds? It is a well-known result of classical communication complexity that classical bounded-error protocols for the Intersection problem need about n bits of communication.

²This is sometimes called the *appointment-scheduling problem*: view x and y as Alice's and Bob's agendas, respectively, with a 1 at the i -th bit indicating that timeslot i is available. Then the goal is to find a timeslot where Alice and Bob are both available, so they can schedule an appointment.

Thus we have a quadratic quantum-classical separation for this problem. Could there be a quantum protocol that uses much less than $\sqrt{n}$ qubits of communication? This question was open for quite a few years after [74] appeared, until finally Razborov [202] showed that any bounded-error quantum protocol for Intersection needs to communicate about $\sqrt{n}$ qubits.

16.5 Example 3: The vector-in-subspace problem

Notice the contrast between the examples of the last two sections. For the Distributed Deutsch-Jozsa problem we get an *exponential* quantum-classical separation, but the separation only holds if we require the classical protocol to be errorless. On the other hand, the gap for the disjointness function is only *quadratic*, but it holds even if we allow classical protocols to have some error probability.

Here is a function where the quantum-classical separation has both features: the quantum protocol is exponentially better than the classical protocol, even if the latter is allowed some error:

Alice receives a unit vector $v \in \mathbb{R}^m$

Bob receives two m -dimensional projectors P_0 and P_1 such that $P_0 + P_1 = I$

Promise: either $P_0 v = v$ or $P_1 v = v$.

Question: which of the two?

As stated, this is a problem with continuous input, but it can be discretized in a natural way by approximating each real number by $O(\log m)$ bits. Alice and Bob's input is now $n = O(m^2 \log m)$ bits long. There is a simple yet efficient 1-round quantum protocol for this problem: Alice views v as a $\log m$ -qubit state and sends this to Bob; Bob measures with operators P_0 and P_1 , and outputs the measurement result (0 or 1). this takes only $\log m = O(\log n)$ qubits of communication, and Bob's output is correct with probability 1 thanks to the promise on the inputs.

The efficiency of this protocol comes from the fact that an m -dimensional unit vector can be “compressed” or “represented” as a $\log m$ -qubit state. Similar compression is not possible with classical bits, which suggests that any classical protocol will have to send the vector v more or less literally and hence will require a lot of communication. This turns out to be true, but the proof is quite hard [157]. It shows that any bounded-error protocol needs to send $\Omega(m^{1/3})$ bits.

16.6 Example 4: Quantum fingerprinting

The examples of the previous section were either exponential quantum improvements for promise problems (Deutsch-Jozsa and vector-in-subspace) or polynomial improvements for total problems (disjointness). We will now give an exponential improvement for the total problem of equality-testing, but in a restricted setting called the *simultaneous message passing* (SMP) model. Alice and Bob receive n -bit input x and y , respectively. They do not have any shared resources like shared randomness or an entangled state, but they do have local randomness. They don't communicate with each other directly, but instead send a single message to a third party, called the Referee. The Referee, upon receiving message m_A from Alice and m_B from Bob, should output the value $f(x, y)$. The goal is to compute $f(x, y)$ with a minimal amount of communication from Alice and Bob to the Referee.

We will see that for the equality problem there is an exponential savings in communication when qubits are used instead of classical bits. Classically, the problem of the bounded-error com-

munication complexity of equality in the SMP model was first raised by Yao [250], and was open for almost twenty years until Newman and Szegedy [195] exhibited a lower bound of $\Omega(\sqrt{n})$ bits. This is tight, since Ambainis [11] constructed a bounded-error protocol for this problem where the messages are $O(\sqrt{n})$ bits long (see Exercise 8). In contrast, in the quantum setting this problem can be solved with very little communication: only $O(\log n)$ qubits suffice [73].

The quantum trick is to associate each $x \in \{0,1\}^n$ with a short quantum state $|\phi_x\rangle$, called the *quantum fingerprint* of x . Just like with physical fingerprints, the idea is that a quantum fingerprint is a small object that doesn't contain very much information about the object x , but that suffices for testing if the fingerprinted object equals some other fingerprinted object. As we will see below, we can do such testing if the fingerprints are pairwise almost orthogonal. More precisely, an (n, m, ε) -quantum fingerprinting scheme maps n -bit string x to m -qubit state $|\phi_x\rangle$ with the property that for all distinct $x, y \in \{0,1\}^n$, we have $|\langle\phi_x|\phi_y\rangle| \leq \varepsilon$.

We will now show how to obtain a specific $(n, m, 0.02)$ -quantum fingerprinting scheme from an error-correcting code $C : \{0,1\}^n \rightarrow \{0,1\}^N$ where $m = \log N \approx \log n$. There exist codes where $N = O(n)$ and any two codewords $C(x)$ and $C(y)$ have Hamming distance close to $N/2$, say $d(C(x), C(y)) \in [0.49N, 0.51N]$ (we won't prove this here, but for instance a random linear code will work). Define the quantum fingerprint of x as follows:

$$|\phi_x\rangle = \frac{1}{\sqrt{N}} \sum_{j=1}^N (-1)^{C(x)_j} |j\rangle.$$

This is a unit vector in an N -dimensional space, so it corresponds to only $\lceil \log N \rceil = \log n + O(1)$ qubits. For distinct x and y , the corresponding fingerprints will have small inner product:

$$\langle\phi_x|\phi_y\rangle = \frac{1}{N} \sum_{j=1}^N (-1)^{C(x)_j + C(y)_j} = \frac{N - 2d(C(x), C(y))}{N} \in [-0.02, 0.02].$$

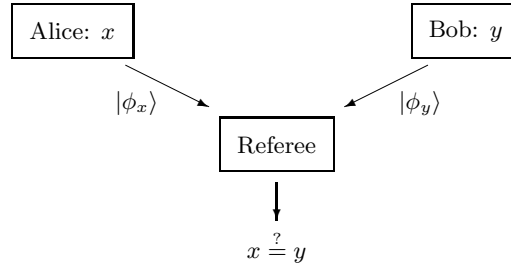


Figure 16.2: Quantum fingerprinting protocol for the equality problem

The quantum protocol is very simple (see Figure 16.2): Alice and Bob send quantum fingerprints of x and y to the Referee, respectively. The referee now has to determine whether $x = y$ (which corresponds to $\langle\phi_x|\phi_y\rangle = 1$) or $x \neq y$ (which corresponds to $\langle\phi_x|\phi_y\rangle \in [-0.02, 0.02]$). The following test (Figure 16.3), sometimes called the *SWAP-test*, accomplishes this with small error probability.

This circuit first applies a Hadamard transform to a qubit that is initially $|0\rangle$, then SWAPs the other two registers conditioned on the value of the first qubit being $|1\rangle$, then applies another Hadamard transform to the first qubit and measures it. Here SWAP is the operation that swaps the

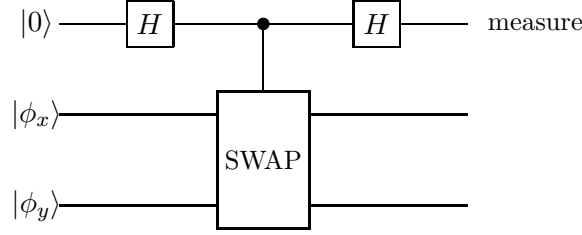


Figure 16.3: Quantum circuit to test if $|\phi_x\rangle = |\phi_y\rangle$ or $|\langle\phi_x|\phi_y\rangle|$ is small

two registers: $|\phi_x\rangle|\phi_y\rangle \mapsto |\phi_y\rangle|\phi_x\rangle$. The Referee receives $|\phi_x\rangle$ from Alice and $|\phi_y\rangle$ from Bob and applies the test to these two states. An easy calculation reveals that the outcome of the measurement is 1 with probability $(1 - |\langle\phi_x|\phi_y\rangle|^2)/2$. Hence if $|\phi_x\rangle = |\phi_y\rangle$ then we observe a 1 with probability 0, but if $|\langle\phi_x|\phi_y\rangle|$ is close to 0 then we observe a 1 with probability close to $1/2$. Repeating this procedure with several individual fingerprints can make the error probability arbitrarily close to 0.

Exercises

1. (H) Prove that classical deterministic protocols with one message (from Alice to Bob), need to send n bits to solve the equality problem.
2. (a) (H) Show that if $|\phi\rangle$ and $|\psi\rangle$ are non-orthogonal states (i.e., $\langle\phi|\psi\rangle \neq 0$), then there is no two-outcome projective measurement that perfectly distinguishes these two states, in the sense that applying the measurement on $|\phi\rangle$ always gives a different outcome from applying the same measurement to $|\psi\rangle$.
 (b) Prove that quantum protocols with one message (from Alice to Bob), need to send at least n qubits to solve the equality problem (on n -bit inputs) with success probability 1 on every input. Assume for simplicity that Bob does a projective measurement rather than a general POVM.
 (c) (H) Prove that quantum protocols with one message (from Alice to Bob), need to send at least $\log n$ qubits to solve the distributed Deutsch-Jozsa problem with success probability 1 on every input. Again assume for simplicity that Bob does a projective measurement rather than a general POVM.
3. (H) Consider *one-round* quantum communication complexity. Alice gets input $x \in \{0, 1\}^n$, Bob gets input $y \in \{0, 1\}^n$, and they want to compute some Boolean function $f(x, y)$ of their inputs. Assume that all rows of the communication matrix are different, i.e., for all x and x' there is a y such that $f(x, y) \neq f(x', y)$. They are allowed only one round of communication: Alice sends a quantum message to Bob and Bob must then be able to give the right answer with probability 1. Prove that Alice needs to send n qubits to Bob for this. You may assume that Alice's messages are pure states (this is without loss of generality).
4. Suppose Alice and Bob each have n -bit agendas, and they know that for exactly 25% of the timeslots they are both free. Give a quantum protocol that finds such a timeslot with probability 1, using only $O(\log n)$ qubits of communication.

5. (H) The *disjointness problem* of communication complexity is the following decision version of the intersection problem: Alice receives an $x \in \{0,1\}^n$, Bob receives $y \in \{0,1\}^n$, and $f(x,y) = 0$ if there is an i such that $x_i = y_i = 1$, and $f(x,y) = 1$ otherwise (i.e., f says whether x and y represent disjoint subsets of $[n]$). Suppose there exists an m -qubit one-way protocol that solves this problem, so where Alice sends Bob m qubits and then Bob outputs $f(x,y)$ with probability at least $2/3$. Prove the lower bound $m = \Omega(n)$ on the number of qubits sent.
6. (H) Consider the intersection problem: Alice has input $x \in \{0,1\}^n$, Bob has input $y \in \{0,1\}^n$, and they want to find (with success probability $\geq 2/3$) an i such that $x_i = y_i = 1$, if such an i exists. We know that using $r = O(\sqrt{n})$ messages between Alice and Bob, they can solve the intersection problem with $O(\sqrt{n} \log n)$ qubits of communication (see Section 16.4). We also know that with only $r = 1$ message (i.e., one-way communication) $\Theta(n)$ qubits of communication are necessary and sufficient (see Exercise 5). Now suppose we limit them to some $r \in \{1, \dots, \sqrt{n}\}$ messages. This r is known to Alice and Bob. Describe a communication protocol by means of which Alice and Bob can solve the intersection problem with at most r messages, and $O((n/r) \log n)$ qubits of communication in total.
7. (a) Consider the following variant of the search problem: we are given query access to a string $x \in \{0,1\}^n$, and we know a set $S \subseteq [n]$ of $k < n$ elements such that $x_i = 0$ for all $i \notin S$. Show that there is a quantum algorithm that can find a solution for this search problem (i.e., an i such that $x_i = 1$, if there is one) with success probability $\geq 2/3$, using $O(\sqrt{k})$ queries to x .
(b) Consider the following variant of the intersection problem of communication complexity: Alice holds a string $x \in \{0,1\}^n$ of Hamming weight k , and Bob holds a string $y \in \{0,1\}^n$ of Hamming weight k . Give a quantum communication protocol that finds an i such that $x_i = y_i = 1$ (if such an i exists) with success probability $\geq 2/3$, using $O(\sqrt{k} \log n)$ qubits of communication.
8. Consider an error-correcting code $C : \{0,1\}^n \rightarrow \{0,1\}^N$ where $N = O(n)$, N is a square, and any two distinct codewords are at Hamming distance $d(C(x), C(y)) \in [0.49N, 0.51N]$ (such codes exist, but you don't have to prove that).
(a) View the codeword $C(x)$ as a $\sqrt{N} \times \sqrt{N}$ matrix. Show that if you choose a row index uniformly at random and choose a column index uniformly at random, then the unique index i where these row and column intersect, is uniformly distributed over $i \in \{1, \dots, N\}$.
(b) (H) Give a classical bounded-error SMP-protocol for the equality problem where Alice and Bob each send $O(\sqrt{n})$ bits to the Referee.
9. Alice and Bob want to solve the equality problem on n -bit inputs x and y (i.e., decide whether $x = y$). They do not share randomness or entanglement but can use local (private) randomness.
(a) (H) Fix a prime number $p \in [3n, 6n]$, then the set $\mathbb{F}_p$ of integers modulo p is a finite field (i.e., it has a well-defined addition and multiplication). For $x = (x_0, \dots, x_{n-1}) \in \{0,1\}^n$, define the univariate polynomial $P_x : \mathbb{F}_p \rightarrow \mathbb{F}_p$ of degree $< n$ as $P_x(t) = \sum_{i=0}^{n-1} x_i t^i$ (note that the n bits of x are used as coefficients here, not as the argument of the polynomial).

Show that for distinct n -bit strings x and y , we have $\Pr_{t \in \mathbb{F}_p}[P_x(t) = P_y(t)] \leq 1/3$, where the probability is taken over a uniformly random $t \in \mathbb{F}_p$.

- (b) Use (a) to give a classical communication protocol where Alice sends an $O(\log n)$ -bit message to Bob, and Bob can decide whether $x = y$ with success probability $\geq 2/3$.
 - (c) Use (a) to give a quantum fingerprinting scheme $x \mapsto |\phi_x\rangle$, where quantum state $|\phi_x\rangle$ has $O(\log n)$ qubits, and $|\langle \phi_x | \phi_y \rangle| \in [0, 1/3]$ for all distinct n -bit strings x and y (prove the latter property explicitly, it's not enough to write down only the states).
10. The inner product problem in communication complexity is the function $f : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$ defined by $f(x,y) = \sum_{i=1}^n x_i y_i \bmod 2$. Suppose there exists a quantum protocol P for Alice and Bob that uses q qubits of communication (possibly using multiple messages between Alice and Bob) and computes the inner product function with success probability 1 (on every possible inputs x, y). The protocol does not assume any shared entangled state at the start.
- (a) Give a quantum protocol that uses $2q$ qubits of communication and implements the $2n$ -qubit map $|x\rangle_A |y\rangle_B \mapsto (-1)^{x \cdot y} |x\rangle_A |y\rangle_B$ (possibly with some auxiliary qubits for each of Alice and Bob; these should start and end in state $|0\rangle$).
 - (b) (H) Give a protocol where Alice transmits x to Bob using $2q$ qubits of communication.
 - (c) Derive a lower bound on q from (b) and Holevo's theorem (Theorem 3 of Chapter 15; be specific about which part of the theorem you invoke).
11. Consider the following problem in communication complexity. Alice's input has two parts: a unit vector $v \in \mathbb{R}^m$ and two orthogonal projectors P_0 and P_1 . Bob's input is an $m \times m$ unitary U . They are promised that the vector Uv either lies in the subspace corresponding to P_0 (i.e., $P_0 Uv = v$) or in the subspace corresponding to P_1 (i.e., $P_1 Uv = v$), and the problem for Alice and Bob is to find out which of these two cases holds.
- (a) Give a quantum protocol that uses two messages of $O(\log m)$ qubits (one message from Alice to Bob and one from Bob to Alice) to solve this problem with success probability 1.
 - (b) (H) Show that there exists a constant $c > 0$ such that classical protocols need to send $\Omega(m^c)$ bits of communication to solve this problem with error probability $\leq 1/3$, even when they are allowed to send many messages.
12. (H) Consider the following communication complexity problem, called the "Hidden Matching Problem." Alice's input is some $x \in \{0,1\}^n$. Bob's input is a matching M , i.e., a partition of $\{1, \dots, n\}$ into $n/2$ disjoint unordered pairs (assume n is a power of 2 for simplicity). Their goal is that Bob outputs a pair $\{i, j\} \in M$ together with the parity $x_i \oplus x_j$ of the two bits indexed by that pair. It doesn't matter which pair $\{i, j\} \in M$ Bob outputs, as long as the additional bit of output equals the parity of the two indexed bits of x . Show that they can solve this problem with success probability 1 using only a message of $\log n$ qubits from Alice to Bob (and no communication from Bob to Alice).
- Comment: One can show that classical one-way protocols need $\Omega(\sqrt{n})$ bits of communication to solve this problem with small error probability.
13. (a) Suppose you have a state $\frac{1}{\sqrt{2}}(|0\rangle|\phi\rangle + |1\rangle|\psi\rangle)$, where $|\phi\rangle$ and $|\psi\rangle$ are quantum states with real amplitudes. Suppose you apply a Hadamard gate to its first qubit and then measure that first qubit. Show that the probability of measurement outcome 0 is $\frac{1}{2}(1 + \langle \phi | \psi \rangle)$.

- (b) Suppose H is a subgroup of a finite group G , and $g \in G$ some element. Show (1) if $g \in H$ then the cosets $g \circ H$ and H are equal and (2) if $g \notin H$ then the cosets $g \circ H$ and H are disjoint.
- (c) Suppose you are given quantum state $|\psi_H\rangle = \frac{1}{\sqrt{|H|}} \sum_{h \in H} |h\rangle$ (for an unknown $H \leq G$), and an element $g \in G$. You may assume you have a unitary A available that implements the group operation, $A : |g, h\rangle \mapsto |g, g \circ h\rangle$, and you may also apply a controlled version of A . Give an algorithm that acts on $|\psi_H\rangle$ and possibly some auxiliary qubits, and that outputs 0 with probability 1 if $g \in H$, and outputs 0 with probability $\leq 1/2$ if $g \notin H$.
- (d) (H) Consider the following communication complexity problem. Alice and Bob both know a finite group G , Alice gets as input some subgroup $H \leq G$ (for instance in the form of a generating set for H) and Bob gets input $g \in G$. Give a one-way quantum protocol where Alice sends to Bob a message of $O(\log |G|)$ qubits, and then Bob decides with success probability $\geq 2/3$ whether $g \in H$.

Chapter 17

Entanglement and Non-Locality

17.1 Quantum non-locality

Entangled states are those that cannot be written as a tensor product of separate states. The most famous one is the EPR-pair:

$$\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle).$$

Suppose Alice has the first qubit of the pair, and Bob has the second. If Alice measures her qubit in the computational basis and gets outcome $b \in \{0, 1\}$, then the state collapses to $|bb\rangle$. Similarly, if Alice measures her qubit in some other basis, this will collapse the joint state (including Bob's qubit) to some state that depends on her measurement basis as well as its outcome. Somehow Alice's action seems to have an *instantaneous* effect on Bob's side—even if the two qubits are light-years apart! This was a great bother to Einstein, whose theory of relativity posits that information and causation cannot travel faster than the speed of light. Einstein called such effects of entanglement “spooky action at a distance” (in German: “spukhafte Fernwirkungen”), and viewed it as a fundamental problem for quantum mechanics [106]. In his view, quantum mechanics should be replaced by some “local realist” physical theory that would still have the same predictive power as quantum mechanics. Here “local” means that information and causation act locally, not faster than light, and “realistic” means that physical systems have definite, well-defined properties (even if those properties may be unknown to us).

Note that the above experiment where Alice measures her half of the EPR-pair doesn't actually violate locality: no information is transferred from Alice and Bob. From Bob's perspective there is no difference between the situation where Alice measured and the situation where she didn't.¹ For this experiment, a shared coin flip between Alice and Bob is a local realist physical model that has exactly the same observable consequences as measuring the qubits of the EPR-pair in the computational basis: a 50-50 distribution on outcomes $|00\rangle$ and $|11\rangle$. This shared-coin-flip model is *local* because no information is transferred between Alice and Bob, and it's *realist* because the coin flip has a definite outcome (even if that outcome is unknown to Alice and Bob before they measure).

Given this example, one might hope (and Einstein expected) that any kind of behavior that comes from entangled states can be replaced by some local realist physical model. This way, quantum mechanics could be replaced by an alternative physical theory with less counter-intuitive

¹In fact, one can show that entanglement cannot replace communication, see for example Exercise 18.7.

behavior. Surprisingly, in the 1960s, John Bell [39] devised entanglement-based experiments whose behavior *cannot be reproduced by any local realist theory*. In other words, we can let Alice and Bob do certain measurements on an entangled state, and the resulting distributions on their outputs predicted by quantum mechanics, *cannot be obtained* from any local realist theory. This phenomenon is known as “quantum non-locality.” It could of course be that the quantum mechanical predictions of the resulting correlations are just wrong. However, in the early 1980s, such experiments were actually done by Aspect and others [31], and they gave the outcomes that quantum mechanics predicted.² Note that such experiments don’t *prove* quantum mechanics, but they *disprove* any local realist physical theory.³

Such experiments, which realize correlations that are *provably impossible* to realize with local realist models, are among the deepest and most philosophical results of 20th century physics: the commonsense idea of local realism is most probably false! Since Bell’s seminal work, the concept of quantum non-locality has been extensively studied, by physicists, philosophers, and more recently by computer scientists.

In the next sections we review some interesting examples. The two-party setting of these examples is illustrated in Fig. 17.1: Alice receives input x and Bob receives input y , and they produce outputs a and b , respectively, that have to be correlated in a certain way (which depends on the game). They are not allowed to communicate. In physics language, we could assume they are “space-like separated,” which means that they are so far apart that they cannot influence each other during the course of the experiment (assuming information doesn’t travel faster than the speed of light). In the classical scenario they are allowed to share a random variable. Physicists would call this the “local hidden variable” that gives properties their definite value (that value may be unknown to the experimenter). This setting captures all local realist models. In the quantum model Alice and Bob are allowed to share entangled states, such as EPR-pairs. The goal is to show that entanglement-based strategies can do things that local realist strategies cannot.

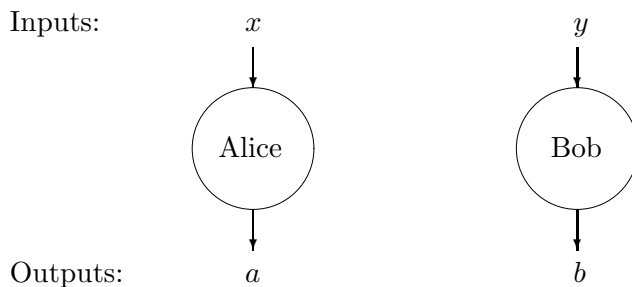


Figure 17.1: The non-locality scenario involving two parties: Alice and Bob receive inputs x and y , respectively, and are required to produce outputs a and b that satisfy certain conditions. Once the inputs are received, no communication is permitted between the parties.

²Modulo some technical “loopholes” due to imperfect photon sources, measurement devices, Alice and Bob not being sufficiently far apart etc. These are still hotly debated, but most people accept that Aspect’s and later experiments are convincing, and kill any hope of a complete local-realist explanation of nature. Recently [137] an experiment was done that simultaneously closed the two most important loopholes.

³Despite its name, non-locality doesn’t disprove locality, but rather disproves the conjunction of locality and realism—at least one of the two assumptions has to fail.

17.2 CHSH: Clauser-Horne-Shimony-Holt

In the CHSH game [87] Alice and Bob receive input bits x and y , and their goal is to output bits a and b , respectively, such that

$$a \oplus b = x \wedge y, \quad (17.1)$$

(‘ $\wedge$ ’ is logical AND; ‘ $\oplus$ ’ is parity, i.e. addition mod 2) or, failing that, to satisfy this condition with as high a probability as possible.

First consider the case of classical *deterministic* strategies, so without any randomness. For these, Alice’s output bit depends solely on her input bit x , and similarly for Bob. Let a_0 be the bit that Alice outputs if her input is $x = 0$, and a_1 the bit she outputs if $x = 1$. Let b_0, b_1 be the outputs Bob gives on inputs $y = 0$ and $y = 1$, respectively. These four bits completely characterize any deterministic strategy. Condition (17.1) becomes

$$\begin{aligned} a_0 \oplus b_0 &= 0, \\ a_0 \oplus b_1 &= 0, \\ a_1 \oplus b_0 &= 0, \\ a_1 \oplus b_1 &= 1. \end{aligned} \quad (17.2)$$

It is impossible to satisfy all four equations simultaneously, since summing them modulo 2 yields $0 = 1$. Therefore it is impossible to satisfy Condition (17.1) perfectly. Since a probabilistic strategy (where Alice and Bob share randomness) is a probability distribution over deterministic strategies, it follows that no probabilistic strategy can have success probability better than $3/4$ on every possible input (the $3/4$ can be achieved simultaneously for every input, see Exercise 4).⁴

Now consider the same problem but where Alice and Bob are supplied with a shared 2-qubit system initialized to the entangled state

$$\frac{1}{\sqrt{2}}(|00\rangle - |11\rangle).$$

Such a state can easily be obtained from an EPR-pair by local operations, for instance if Alice applies a Z -gate to her qubit. Now the parties can produce outputs that satisfy Condition (17.1) with probability $\cos(\pi/8)^2 \approx 0.85$ (higher than what is possible in the classical case), as follows.

Recall the unitary operation that rotates the qubit by angle θ : $R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$. If $x = 0$ then Alice applies $R(-\pi/16)$ to her qubit; and if $x = 1$ she applies $R(3\pi/16)$. Then Alice measures her qubit in the computational basis and outputs the resulting bit a . Bob’s procedure is the same, depending on his input bit y . It is straightforward to calculate that if Alice rotates by θ_A and Bob rotates by θ_B , the state becomes

$$\frac{1}{\sqrt{2}} (\cos(\theta_A + \theta_B)(|00\rangle - |11\rangle) + \sin(\theta_A + \theta_B)(|01\rangle + |10\rangle)).$$

After the measurements, the probability that $a \oplus b = 0$ is $\cos(\theta_A + \theta_B)^2$. Note that if $x \wedge y = 0$ then $\theta_A + \theta_B = \pm\pi/8$, while if $x \wedge y = 1$ then $\theta_A + \theta_B = 3\pi/8$. Hence Condition 17.1 is satisfied with probability $\cos(\pi/8)^2$ for all four input possibilities, showing that quantum entanglement

⁴Such statements, upper bounding the optimal success probability of classical strategies for a specific game, are known as *Bell inequalities*. This specific one is called the *CHSH inequality*.

allows Alice and Bob to win the game with a probability that's higher than what the best classical strategy can achieve. Tsirelson [86] showed that $\cos(\pi/8)^2$ is the best that quantum strategies can do for CHSH, even if they are allowed to use much more entanglement than one EPR-pair (see Exercise 6).

17.3 Magic square game

Is there a game where the quantum protocol *always* succeeds, while the best classical success probability is bounded below 1? A particularly elegant example is the following *magic square game* [25]. Consider the problem of labeling the entries of a 3×3 matrix with bits so that the parity of each row is even, whereas the parity of each column is odd. This is clearly impossible: if the parity of each row is even then the sum of the 9 bits is $0 \bmod 2$, but if the parity of each column is odd then the sum of the 9 bits is $1 \bmod 2$. The two matrices

0	0	0
0	0	0
1	1	0

0	0	0
0	0	0
1	1	1

each satisfy five out of the six constraints. For the first matrix, all rows have even parity, but only the first two columns have odd parity. For the second matrix, the first two rows have even parity, and all columns have odd parity.

Consider the game where Alice receives $x \in \{1, 2, 3\}$ as input (specifying the number of a row), and Bob receives $y \in \{1, 2, 3\}$ as input (specifying the number of a column). Their goal is to each produce 3-bit outputs, $a_1 a_2 a_3$ for Alice and $b_1 b_2 b_3$ for Bob, such that

1. They satisfy the row/column parity constraints: $a_1 \oplus a_2 \oplus a_3 = 0$ and $b_1 \oplus b_2 \oplus b_3 = 1$.
2. They are consistent where the row intersects the column: $a_y = b_x$.

As usual, Alice and Bob are forbidden from communicating once the game starts, so Alice does not know y and Bob does not know x . We shall show the best classical strategy has success probability $8/9$, while there is a quantum strategy that always succeeds.

An example of a deterministic strategy that attains success probability $8/9$ (when the input xy is uniformly distributed) is where Alice plays according to the rows of the first matrix above and Bob plays according to the columns of the second matrix above. This succeeds in all cases, except where $x = y = 3$. To see why this is optimal, note that for any other classical strategy, it is possible to represent it as two matrices as above but with different entries. Alice plays according to the rows of the first matrix and Bob plays according to the columns of the second matrix. We can assume that the rows of Alice's matrix all have even parity; if she outputs a row with odd parity then they immediately lose, regardless of Bob's output. Similarly, we can assume that all columns of Bob's matrix have odd parity.⁵ Considering such a pair of matrices, the players lose at each entry where they differ. There must be such an entry, since otherwise it would be possible to have all rows even and all columns odd with one matrix. Thus, when the input xy is chosen uniformly from $\{1, 2, 3\} \times \{1, 2, 3\}$, the success probability of any classical strategy is at most $8/9$.

⁵In fact, the game can be simplified so that Alice and Bob each output just two bits, since the parity constraint determines the third bit.

We now give the *quantum* strategy for this game. Let I, X, Y, Z be the 2×2 Pauli matrices from Appendix A.9. Each is a 1-qubit *observable* with eigenvalues in $\{+1, -1\}$.⁶ That is, each can be written as $P_+ - P_-$ where P_+ and P_- are orthogonal projectors that sum to identity, and hence define a two-outcome measurement with outcomes $+1$ and -1 . For example, $Z = |0\rangle\langle 0| - |1\rangle\langle 1|$, corresponding to a measurement in the computational basis (with $|b\rangle$ corresponding to outcome $(-1)^b$). And $X = |+\rangle\langle +| - |-\rangle\langle -|$, corresponding to a measurement in the Hadamard basis. The Pauli matrices are self-inverse, they anti-commute unless one of them is I (e.g., $XY = -YX$), and $X = iZY$, $Y = iXZ$, and $Z = iYX$. Consider the following table, where each entry is a tensor product of two Paulis:

$X \otimes X$	$Y \otimes Z$	$Z \otimes Y$
$Y \otimes Y$	$Z \otimes X$	$X \otimes Z$
$Z \otimes Z$	$X \otimes Y$	$Y \otimes X$

Because $(P_+ - P_-) \otimes (Q_+ - Q_-) = (P_+ \otimes Q_+ + P_- \otimes Q_-) - (P_+ \otimes Q_- + P_- \otimes Q_+)$, each such product is itself a $\{+1, -1\}$ -valued observable. Hence each product of Pauli matrices corresponds to a measurement on a 2-qubit space, with outcomes $+1$ and -1 .

Note that the observables along each row commute and their product is $I \otimes I$, and the observables along each column commute and their product is $-I \otimes I$. This implies that for any 2-qubit state, performing the three measurements along any row results in three $\{+1, -1\}$ -valued bits whose product is $+1$. Also, performing the three measurements along any column results in three $\{+1, -1\}$ -valued bits whose product is -1 .

We can now describe the quantum protocol. It uses two pairs of entangled qubits, each of which is in initial state

$$\frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$$

(again, such states can be obtained from EPR-pairs by local operations). Alice, on input x , applies three 2-qubit measurements corresponding to the observables in row x of the above table. For each measurement, if the result is $+1$ then she outputs 0, and if the result is -1 then she outputs 1. Similarly, Bob, on input y , applies the measurements corresponding to the observables in column y , and converts the ± 1 -outcomes into bits.

We have already established that Alice and Bob's output bits satisfy the required parity constraints. It remains to show that Alice and Bob's output bits agree at the point where the row meets the column. For that measurement, Alice and Bob are measuring with respect to the same observable in the above table. Because all the observables in each row and in each column commute, we may assume that the place where they intersect is the first observable applied. Those bits are obtained by Alice and Bob each measuring $\frac{1}{2}(|01\rangle - |10\rangle)(|01\rangle - |10\rangle)$ with respect to the observable in entry (x, y) of the table. To show that their measurements will agree for all cases of xy , we consider the individual Pauli measurements on the individual entangled pairs of the form $\frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$. Let a' and b' denote the 0/1-valued outcomes of the first measurement, and a'' and b'' denote the outcomes of the second. The measurement associated with the tensor product of two observables gives the same distribution over outcomes as measuring each individual observable and then taking the product of the two results. Hence we have $a_y = a' \oplus a''$ and $b_x = b' \oplus b''$. It is

⁶See Section 1.2.2. In particular, a ± 1 -valued observable A can be written as $A = P - Q$, where P and Q are projectors on two orthogonal subspaces such that $P + Q = I$. This corresponds to a two-outcome measurement specified by projectors P and Q with outcomes $+1$ and -1 , respectively.

straightforward to verify that if the same measurement from $\{I, X, Y, Z\}$ is applied to each qubit of $\frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$ then the outcomes will be distinct: $a' \oplus b' = 1$ and $a'' \oplus b'' = 1$. We now have $a_y = b_x$, because

$$a_y \oplus b_x = (a' \oplus a'') \oplus (b' \oplus b'') = (a' \oplus b') \oplus (a'' \oplus b'') = 1 \oplus 1 = 0. \quad (17.3)$$

17.4 A non-local version of distributed Deutsch-Jozsa

The previous two examples used small amounts of entanglement: one EPR-pair for CHSH, two EPR-pairs for magic square. In both cases we could show that classical protocols need at least *some* communication if they want to achieve the same as what entanglement-based protocols can achieve without any communication. We will now give a non-locality game that's parametrized by a number n , and where Alice and Bob's quantum strategy uses $\log n$ EPR-pairs [65]. The advantage is that we can show that classical protocols for this game need just *some* but actually *much* classical communication rather than at least some nonzero amount.

Non-local DJ problem: Alice and Bob receive n -bit inputs x and y that satisfy the DJ promise: either $x = y$, or x and y differ in exactly $n/2$ positions. The task is for Alice and Bob to provide outputs $a, b \in \{0, 1\}^{\log n}$ such that if $x = y$ then $a = b$, and if x and y differ in exactly $n/2$ positions then $a \neq b$.

They achieve this as follows

1. Alice and Bob share $\log n$ EPR-pairs, i.e., the maximally entangled state $\frac{1}{\sqrt{n}} \sum_{i=0}^{n-1} |i\rangle|i\rangle$.⁷
2. They both apply locally a conditional phase to obtain: $\frac{1}{\sqrt{n}} \sum_{i=0}^{n-1} (-1)^{x_i} |i\rangle (-1)^{y_i} |i\rangle$.
3. They both apply a Hadamard transform, obtaining

$$\begin{aligned} & \frac{1}{n\sqrt{n}} \sum_{i=0}^{n-1} (-1)^{x_i+y_i} \sum_{a \in \{0,1\}^{\log n}} (-1)^{i \cdot a} |a\rangle \sum_{b \in \{0,1\}^{\log n}} (-1)^{i \cdot b} |b\rangle \\ &= \frac{1}{n\sqrt{n}} \sum_{a,b \in \{0,1\}^{\log n}} \left(\sum_{i=0}^{n-1} (-1)^{x_i+y_i+i \cdot (a \oplus b)} \right) |a\rangle |b\rangle. \end{aligned}$$

4. They measure in the computational basis and output the results a and b , respectively.

For every a , the probability that both Alice and Bob obtain the same result a is:

$$\left| \frac{1}{n\sqrt{n}} \sum_{i=0}^{n-1} (-1)^{x_i+y_i} \right|^2,$$

⁷Note that k EPR-pairs $\left(\frac{1}{\sqrt{2}}(|0\rangle_A|0\rangle_B + |1\rangle_A|1\rangle_B) \right)^{\otimes k}$ can also be written as $\frac{1}{\sqrt{2^k}} \sum_{i \in \{0,1\}^k} |i\rangle_A |i\rangle_B$ if we reorder

the qubits, putting Alice's k qubits on the left and Bob's on the right. While these two ways of writing the state strictly speaking correspond to two different vectors of amplitudes, they still represent the same bipartite physical state, and we will typically view them as equal.

which is $1/n$ if $x = y$, and 0 otherwise. This solves the problem perfectly using prior entanglement.

What about classical protocols? Suppose there is a classical protocol that uses C bits of communication, and that wins the *non-local* Deutsch-Jozsa problem with success probability 1. If Alice and Bob ran this protocol, and then Alice communicated her output a to Bob (using an additional $\log n$ bits), then they could solve the *distributed* Deutsch-Jozsa problem since Bob could then check whether $a = b$ or $a \neq b$. But we know from Section 16.3 that solving the distributed Deutsch-Jozsa problem requires at least $0.007n$ bits of communication. Hence $C + \log n \geq 0.007n$, so $C \geq 0.007n - \log n = \Omega(n)$. Thus we have a non-locality problem that can be solved perfectly if Alice and Bob share $\log n$ EPR-pairs, while classically it needs not just *some* communication, but actually a *lot* of communication if we want to solve it perfectly.

Exercises

1. Suppose Alice and Bob share an EPR-pair $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.
 - (a) Let U be a 1-qubit unitary. Show that the following two states are the same: (1) the state obtained if Alice applies U to her qubit of the EPR-pair; (2) the state obtained if Bob applies the transpose U^T to his qubit of the EPR-pair.
 - (b) (H) What state do you get if each of Alice and Bob applies a Hadamard transform to their qubit of the EPR-pair?
2. Alice and Bob share an EPR-pair, $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Suppose they each measure their qubit with an X -observable (which corresponds to a particular projective measurement with possible outcomes $+1, -1$).
 - (a) Show that Alice's measurement outcome is uniformly distributed, so 50% probability of outcome $+1$ and 50% probability of outcome -1 .
 - (b) (H) Show that Alice's and Bob's measurement outcomes are always equal.
 - (c) Suppose we view $X \otimes X$ as one 2-qubit observable (with possible outcomes $+1, -1$) instead of two 1-qubit observables. What is the probability distribution on the two possible outcomes?
3. Alice and Bob share n EPR-pairs. Call their shared $2n$ -qubit state $|\psi\rangle_{AB}$.
 - (a) Let U be an arbitrary n -qubit unitary and $\bar{U}$ be U after conjugating its entries (without transposing). Prove that $(U \otimes \bar{U})|\psi\rangle_{AB} = |\psi\rangle_{AB}$.
 - (b) Suppose Alice receives some input x , and she does an n -qubit unitary U_x on her part of the state and then measures in the computational basis, obtaining a classical outcome $a \in \{0, 1\}^n$. What is the probability distribution over Alice's measurement outcomes, and why?
 - (c) Suppose Bob receives the same input x as Alice already received. How can he learn Alice's measurement outcome a from part (b) without communication? (you may assume Bob knows the map $x \mapsto U_x$)
4. (H) Give a classical strategy using shared randomness for the CHSH game, such that Alice and Bob win the game with probability at least $3/4$ for every possible input x, y (note the order of quantification: the same strategy has to work for every x, y).

5. “Mermin’s game” is the following. Consider three space-like separated players: Alice, Bob, and Charlie. Alice receives input bit x , Bob receives input bit y , and Charlie receives input bit z . The input satisfies the promise that $x \oplus y \oplus z = 0$. The goal of the players is to output bits a, b, c , respectively, such that $a \oplus b \oplus c = \text{OR}(x, y, z)$. In other words, the outputs should sum to 0 (mod 2) if $x = y = z = 0$, and should sum to 1 (mod 2) if $x + y + z = 2$.

- (a) Show that every classical deterministic strategy will fail on at least one of the 4 allowed inputs.
- (b) Show that every classical randomized strategy has success probability at most $3/4$ under the uniform distribution on the four allowed inputs xyz .
- (c) Suppose the players share the following entangled 3-qubit state:

$$\frac{1}{2}(|000\rangle - |011\rangle - |101\rangle - |110\rangle).$$

Suppose each player does the following: if his/her input bit is 1, apply H to his/her qubit, otherwise do nothing. Describe the resulting 3-qubit superposition.

- (d) Using (c), give a quantum strategy that wins the above game with probability 1 on every input that satisfies the promise.
6. (H) This question examines how well the best quantum protocol can do for CHSH (resulting in the so-called “Tsirelson bound”). Consider a protocol where Alice and Bob share a $2k$ -qubit state $|\psi\rangle = |\psi\rangle_{AB}$ with k qubits for Alice and k for Bob (the state can be arbitrary and need not consist of EPR-pairs). Alice has two possible ± 1 -valued observables A_0 and A_1 , and Bob has two possible ± 1 -valued observables B_0 and B_1 . Each of these observables acts on k qubits. On inputs $x \in \{0, 1\}$ and $y \in \{0, 1\}$, respectively, Alice measures her half of $|\psi\rangle$ with A_x and outputs the resulting sign $a \in \{+1, -1\}$, and Bob measures his half of $|\psi\rangle$ with B_y and outputs the resulting sign b . Note that we treat the output bits as signs instead of 0/1 now. However, the winning condition is the same: the AND of the input bits should equal the parity (XOR) of the output bits. So Alice and Bob *win* the game if $(-1)^{xy} = ab$.
- (a) Show that the expected value of the product ab on inputs x, y is $\langle \psi | A_x \otimes B_y | \psi \rangle$ (this is the same as $\text{Tr}[(A_x \otimes B_y)|\psi\rangle\langle\psi|]$).
 - (b) Define $2k$ -qubit operator $C = A_0 \otimes B_0 + A_0 \otimes B_1 + A_1 \otimes B_0 - A_1 \otimes B_1$. Show that the winning probability of the protocol (averaged over all 4 inputs pairs x, y) is $\frac{1}{2} + \frac{1}{8} \langle \psi | C | \psi \rangle$.
 - (c) Show that $C^2 = 4I + (A_0 A_1 - A_1 A_0) \otimes (B_1 B_0 - B_0 B_1)$, where I is the $2k$ -qubit identity matrix.
 - (d) Show that $\langle \psi | C | \psi \rangle \leq \sqrt{8}$.
 - (e) What can you conclude about the best-possible winning probability among all possible quantum protocols for CHSH?

Chapter 18

Quantum Cryptography

18.1 Saving cryptography from Shor

Most classical public-key cryptography in use today can be broken by a large quantum computer. In particular, the RSA system relies on the hardness of factoring integers and hence is broken by Shor’s factoring algorithm (see Exercise 5.3); and Diffie-Helman relies on the hardness of the discrete logarithm problem which was also broken by Shor (see Exercise 6.3). This could clearly become a huge problem for society if and when a large quantum computer is realized: if we cannot securely send messages, make payments or sign transactions online anymore, then much of our economy and society breaks down, or will at least need to be heavily reconfigured.

There are two ways to address this problem. On the one hand we can try to design other classical cryptographic systems, based on the assumed hardness (even for quantum computers) of computational problems other than factoring or discrete log. This part of classical cryptography is (slightly confusingly) called *post-quantum cryptography* [52]. Its most famous cryptosystem to date is “learning with errors” (LWE) [204], which relies on the assumed hardness of certain computational problems in integer lattices.

On the other hand, we can also try to design cryptographic systems that explicitly rely on quantum effects. This area is called *quantum cryptography* and is the topic of this chapter. Compared to post-quantum cryptography, this has the disadvantage that even the honest users of the scheme need to have a (simple) quantum computer at their disposal, but it has the advantage that the security against adversaries in some cases is information-theoretic, not predicated on the assumed but unproven hardness of some computational problems.

18.2 Quantum key distribution

One of the most basic tasks of cryptography is to allow Alice to send a message to Bob (whom she trusts) over a public channel, without allowing a third party Eve (for “eavesdropper”) to get any information about M from tapping the channel. Suppose Alice wants to send message $M \in \{0, 1\}^n$ to Bob. The goal here is not minimal communication, but *secrecy*. This is often done by public-key cryptography such as RSA. Such schemes, however, are only computationally secure, not information-theoretically secure: all the information about the private key can be computed from the public key, it just appears to take a lot of time to compute it—assuming of course that problems like factoring are classically hard, and that nobody builds a quantum computer...

In contrast, the following “one-time pad” scheme is information-theoretically secure. If Alice and Bob share a *secret key* $K \in \{0, 1\}^n$ then Alice can send $C = M \oplus K$ over the channel. By adding K to what he received, Bob learns M . On the other hand, if Eve didn’t know anything about K then she learns nothing about M from tapping the message $M \oplus K$ that goes over the channel. How can we make Alice and Bob share a secret key? In the classical world this is impossible, but with quantum communication it can be done!

Below we describe the famous BB84 quantum key distribution (QKD) protocol of Bennett and Brassard [51]. Consider two possible bases: basis 0 is the computational basis $\{|0\rangle, |1\rangle\}$, and basis 1 is the Hadamard basis $\{|+\rangle, |-\rangle\}$. The main property of quantum mechanics that we’ll use, is that if a bit b is encoded in an unknown basis, then Eve cannot get information about b without disturbing the state, and the latter can be detected by Alice and Bob.¹

1. Alice chooses n random bits $a_1, \dots, a_n$ and n random bases $b_1, \dots, b_n$. She sends a_i to Bob in basis b_i over the public quantum channel. For example, if $a_i = 0$ and $b_i = 1$ then the i -th qubit that she sends is in state $|+\rangle$.
2. Bob chooses random bases $b'_1, \dots, b'_n$ and measures the qubits he received in those bases, yielding bits $a'_1, \dots, a'_n$.
3. Bob sends Alice all b'_i (this also signals to Alice that Bob has measured the qubits he received), and Alice sends Bob all b_i . Note that for roughly $n/2$ of the i s, Alice and Bob used the same basis $b_i = b'_i$. For those i Bob should have $a'_i = a_i$ (if there was no noise and Eve didn’t tamper with the i -th qubit on the channel). Both Alice and Bob know for which i s this holds. Let’s call these roughly $n/2$ positions the “shared string.”
4. Alice randomly selects $n/4$ locations in the shared string, and sends Bob those locations as well as the values a_i at those locations. Bob then checks whether they have the same bits in those positions. If the fraction of errors is bigger than some number p , then they suspect some eavesdropper was tampering with the channel, and they abort.²
5. If the test is passed, then they discard the $n/4$ test-bits, and have roughly $n/4$ bits left in their shared string. This is called the “raw key.” Now they do some classical postprocessing on the raw key: “information reconciliation” to ensure they end up with exactly the same shared string, and “privacy amplification” to ensure that Eve has negligible information about that shared string.³

The communication is n qubits in step 1, $2n$ bits in step 3, $O(n)$ bits in step 4, and $O(n)$ bits in step 5. So the required amount of communication is linear in the length of the shared secret key that Alice and Bob end up with.

¹Quantum key distribution might in fact better be called “quantum eavesdropper detection.” There is another assumption underlying BB84 that should be made explicit: we assume that the classical channel used in steps 3–5 is “authenticated,” meaning that Alice and Bob know they are talking to each other, and Eve can listen but not change the bits sent over the classical channel (in contrast to the qubits sent during step 1 of the protocol, which Eve is allowed to manipulate in any way she wants). One can authenticate a classical communication channel by using some shared secret key; if this is used, then one may think of QKD as something that allows to grow an initial shared secret key, rather than as something that conjures up a shared random key out of nothing.

²The number p can for instance be set to the natural error-rate that the quantum channel would have if there were no eavesdropper.

³This can be done for instance by something called the “leftover hash lemma.”

It's quite hard to formally *prove* that this protocol yields (with high probability) a shared key about which Eve has negligible information. In fact it took more than 12 years before BB84 was finally proven secure [188, 175]. The main reason it works is that when the qubits that encode $a_1, \dots, a_n$ are going over the public channel, Eve doesn't know yet in which bases $b_1, \dots, b_n$ these are encoded (she will learn the b_i later from tapping the classical communication in step 3, but at that point this information is not of much use to her anymore). She could try to get as much information as she can about $a_1, \dots, a_n$ by some measurement, but there's an *information-vs-disturbance tradeoff*: the more information Eve learns about $a_1, \dots, a_n$ by measuring the qubits, the more she will disturb the state, and the more likely it is that Alice and Bob will detect her presence in step 4.

We won't go into the full proof details here, just illustrate the information-disturbance tradeoff for the case where Eve individually attacks the qubits encoding each bit in step 1 of the protocol.⁴ In Fig. 18.1 we give the four possible states for one BB84-qubit. If Alice wants to send $a_i = 0$, then she sends a uniform mixture of $|0\rangle$ and $|+\rangle$ across the channel; if Alice wants to send $a_i = 1$ she sends a uniform mixture of $|1\rangle$ and $|-\rangle$. Suppose Eve tries to learn a_i from the qubit on the channel. The best way for her to do this is to measure in the orthonormal basis corresponding to state $\cos(\pi/8)|0\rangle + \sin(\pi/8)|1\rangle$ and $-\sin(\pi/8)|0\rangle + \cos(\pi/8)|1\rangle$. Note that the first state is halfway between the two encodings of 0, and the second state is halfway between the two encodings of 1 (remember that $|-\rangle$ and $-|-\rangle$ are physically indistinguishable because they only differ by a global phase). This will give her the value of a_i with probability $\cos(\pi/8)^2 \approx 0.85$ (remember the 2-to-1 quantum random access code from Exercise 2 of Chapter 15). However, this measurement will change the state of the qubit by an angle of at least $\pi/8$, so if Bob now measures the qubit he receives in the same basis as Alice, then his probability of recovering the incorrect value of a_i is at least $\sin(\pi/8)^2 \approx 0.15$ (if Bob measured in a different basis than Alice, then the result will be discarded anyway). If this i is among the test-bits Alice and Bob use in step 4 of the protocol (which happens with probability $1/2$), then they will detect an error. Eve can of course try a less disturbing measurement to reduce the probability of being detected, but such a measurement will also have lower probability of telling her a_i .

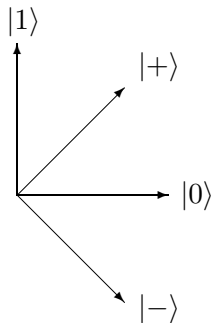


Figure 18.1: The four possible states in BB84 encoding: $|0\rangle$ and $|+\rangle$ are two different encodings of 0, and $|1\rangle$ and $|-\rangle$ are two different encodings of 1.

⁴The more complicated situation where Eve does an n -qubit measurement on all qubits of step 1 simultaneously can be reduced to the case of individual-qubit measurements by something called the *quantum De Finetti theorem*, but we won't go into the details here.

18.3 Reduced density matrices and the Schmidt decomposition

Suppose Alice and Bob share some pure state $|\phi\rangle$. If this state is entangled, it cannot be written as a tensor product $|\phi_A\rangle \otimes |\phi_B\rangle$ of separate pure states for Alice and Bob. Still, there is a way to describe Alice's local state as a mixed state, by *tracing out* Bob's part. Formally, if $C \otimes D$ is a tensor product matrix then $\text{Tr}_B(C \otimes D) = C \cdot \text{Tr}(D)$. By extending this linearly to matrices that are not of product form, the operation Tr_B is well-defined on all mixed states. Note that Tr_B removes Bob's part of the state, leaving just Alice's part of the state. If ρ_{AB} is some bipartite state (mixed or pure, entangled or not), then $\rho_A = \text{Tr}_B(\rho_{AB})$ is Alice's local density matrix. This describes all the information she has. For example, for an EPR-pair $|\phi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, the corresponding density matrix is

$$\begin{aligned}\rho_{AB} &= \frac{1}{2}(|00\rangle\langle 00| + |00\rangle\langle 11| + |11\rangle\langle 00| + |11\rangle\langle 11|) \\ &= \frac{1}{2}(|0\rangle\langle 0| \otimes |0\rangle\langle 0| + |0\rangle\langle 1| \otimes |0\rangle\langle 1| + |1\rangle\langle 0| \otimes |1\rangle\langle 0| + |1\rangle\langle 1| \otimes |1\rangle\langle 1|),\end{aligned}$$

and since $\text{Tr}(|a\rangle\langle b|) = 1$ if $a = b$ and $\text{Tr}(|a\rangle\langle b|) = 0$ if $|a\rangle$ and $|b\rangle$ are orthogonal, we have

$$\rho_A = \text{Tr}_B(\rho_{AB}) = \frac{1}{2}(|0\rangle\langle 0| + |1\rangle\langle 1|).$$

In other words, Alice's local state is the same as a random coin flip! Similarly we can compute Bob's local state by tracing out Alice's part of the space: $\rho_B = \text{Tr}_A(\rho_{AB})$. Note that the original 2-qubit density matrix ρ_{AB} is *not* equal to $\rho_A \otimes \rho_B$, because the tracing-out operation has “removed” the entanglement between the two qubits.

The Schmidt decomposition is a very useful way to write bipartite pure states, and allows us to easily calculate the local density matrices of Alice and Bob. It says the following: for every bipartite pure state $|\phi\rangle$ there is a unique integer d (called the *Schmidt rank* of $|\phi\rangle$), an orthonormal set of states $|a_1\rangle, \dots, |a_d\rangle$ for Alice's space, an orthonormal set of states $|b_1\rangle, \dots, |b_d\rangle$ for Bob's space, and positive reals $\lambda_1, \dots, \lambda_d$ whose squares sum to 1, such that

$$|\phi\rangle = \sum_{i=1}^d \lambda_i |a_i\rangle |b_i\rangle. \quad (18.1)$$

For example, an EPR-pair has Schmidt coefficients $\lambda_1 = \lambda_2 = 1/\sqrt{2}$ and hence has Schmidt rank 2. The Schmidt rank and the Schmidt coefficients of a state $|\phi\rangle$ are unique, but there is some freedom in the choice of bases if the λ_j are not all distinct. For example

$$\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = \frac{1}{\sqrt{2}}(|++\rangle + |--\rangle)$$

are two distinct Schmidt decompositions of the EPR-pair.

The existence of the Schmidt decomposition is shown as follows. Let $\rho_A = \text{Tr}_B(|\phi\rangle\langle\phi|)$ be Alice's local density matrix. This is Hermitian, so it has a spectral decomposition $\rho_A = \sum_{i=1}^d \mu_i |a_i\rangle\langle a_i|$ with orthonormal eigenvectors $|a_i\rangle$ and positive real eigenvalues μ_i . Note that d is the rank of ρ_A , and $\sum_i \mu_i = \text{Tr}(\rho_A) = 1$. Then there are c_{ij} such that

$$|\phi\rangle = \sum_{i,j=1}^d \sqrt{\mu_i} c_{ij} |a_i\rangle |j\rangle,$$

where the $|j\rangle$ are the computational basis states for Bob's space. Define $\lambda_i = \sqrt{\mu_i}$ and $|b_i\rangle = \sum_j c_{ij}|j\rangle$. This gives the decomposition of $|\phi\rangle$ of Eq. (18.1). It only remains to show that $\{|b_i\rangle\}$ is an orthonormal set, which we do as follows. The density matrix version of Eq. (18.1) is

$$|\phi\rangle\langle\phi| = \sum_{i,j=1}^d \lambda_i \lambda_j |a_i\rangle\langle a_j| \otimes |b_i\rangle\langle b_j|.$$

We know that if we trace out the B -part from $|\phi\rangle\langle\phi|$, then we should get $\rho_A = \sum_i \lambda_i^2 |a_i\rangle\langle a_i|$, but that can only happen if $\langle b_j|b_i\rangle = \text{Tr}(|b_i\rangle\langle b_j|) = 1$ for $i = j$ and $\langle b_j|b_i\rangle = 0$ for $i \neq j$. Hence the $|b_i\rangle$ form an orthonormal set. Note that from Eq. (18.1) it easily follows that Bob's local density matrix is $\rho_B = \sum_i \lambda_i^2 |b_i\rangle\langle b_i|$.

18.4 The impossibility of perfect bit commitment

Key distribution is just one of the many tasks cryptographers would like to solve. Another important primitive is *bit commitment*. In this scenario there is no eavesdropper, but Alice and Bob don't trust each other. Suppose Alice has a bit b which for the time being she doesn't want to reveal to Bob, though she would like to somehow convince Bob that she has already made up her mind about b and won't change its value later. A protocol for bit commitment comes in two stages, each of which may involve several rounds of communication:

1. In the "commit" phase Alice gives Bob a state which is supposed to commit her to the value of b (without informing Bob about the value of b).
2. In the "reveal" phase Alice sends b to Bob, and possibly some other information to allow him to check that this is indeed the same value b that Alice committed to before.

A protocol is *binding* if Alice can't change her mind, meaning she can't get Bob to "open" $1 - b$. A protocol is *concealing* if Bob cannot get any information about b before the "reveal phase."⁵

A good protocol for bit commitment would be a very useful building block for many other cryptographic applications. For instance, it would allow Alice and Bob (who still don't trust each other) to jointly flip a fair coin. Maybe they're going through a divorce, and need to decide who gets to keep their joint car. Alice can't just flip the coin by herself because Bob doesn't trust her to do this honestly, and vice versa. Instead, Alice would pick a random coin b and commit to it. Bob would then pick a random coin c and send it to Alice. Alice then reveals b , and the outcome of the coin flip is defined to be $b \oplus c$. As long as at least one of the two parties follows this protocol, the result will be a fair coin flip.

Perfect coin flipping (and hence also perfect bit commitment) are known to be impossible in the classical world. After BB84 there was some hope that perfect bit commitment (and hence also perfect coin flipping) would be possible in the quantum world, and there were some seemingly-secure proposals for quantum protocols to achieve this. Unfortunately it turns out that there is *no* quantum protocol for bit commitment that is both perfectly binding and perfectly concealing.

To show that a protocol for perfect bit commitment is impossible, consider the joint pure state $|\phi_b\rangle$ that Alice and Bob would have if Alice wants to commit to bit-value b , and they both

⁵A good metaphor to think about this: in the commit phase Alice locks b inside a safe which she sends to Bob. This commits her to the value of b , since the safe is no longer in her hands. During the reveal phase she sends Bob the key to the safe, who can then open it and learn b .

honestly followed the protocol.⁶ If the protocol is perfectly concealing, then the reduced density matrix on Bob’s side should be independent of b , i.e., $\text{Tr}_A(|\phi_0\rangle\langle\phi_0|) = \text{Tr}_A(|\phi_1\rangle\langle\phi_1|)$. The way we constructed the Schmidt decomposition in the previous section now implies that there exist Schmidt decompositions of $|\phi_0\rangle$ and $|\phi_1\rangle$ with the same λ_i ’s and the same b_i ’s: there exist orthonormal bases $\{a_i\}$ and $\{a'_i\}$ such that

$$|\phi_0\rangle = \sum_{i=1}^d \lambda_i |a_i\rangle |b_i\rangle \quad \text{and} \quad |\phi_1\rangle = \sum_{i=1}^d \lambda_i |a'_i\rangle |b_i\rangle$$

Now Alice can locally switch from $|\phi_0\rangle$ to $|\phi_1\rangle$ by just applying on her part of the state the map $|a_i\rangle \mapsto |a'_i\rangle$. Alice’s map is unitary because it takes one orthonormal basis to another orthonormal basis. But then the protocol is not binding at all: Alice can still freely change her mind about the value of b after the “commit” phase is over! Accordingly, if a quantum protocol for bit commitment is perfectly concealing, it cannot be binding at all.

18.5 More quantum cryptography

Quantum cryptography is by now a pretty large subset of the area of quantum information and computation. Here we just briefly mention a few other topics in quantum crypto (see [69]):

- There are quantum protocols for bit commitment that are partially concealing and partially binding—something which is still impossible in the classical world. A primitive called “weak coin flipping” can be implemented almost perfectly in the quantum world, and cannot be implemented at all in the classical world.
- Under assumptions on the fraction of dishonest players among a set of k parties, it is possible to implement *secure multi-party quantum computation*. This is a primitive that allows the players to compute any function of their k inputs, without revealing more information to player i than can be inferred from i ’s input plus the function value.
- One can actually do nearly perfect bit commitment, coin flipping, etc., assuming the dishonest party has *bounded quantum storage*, meaning that it can’t keep large quantum states coherent for longer times. At the present state of quantum technology this is a very reasonable assumption (though a breakthrough in physical realization of quantum computers would wipe out this approach).
- In *device-independent* cryptography, Alice and Bob want to solve certain cryptographic tasks like key distribution or randomness generation without trusting their own devices (for instance because they don’t trust the vendor of their apparatuses). Roughly speaking, the idea here is to use Bell-inequality violations to prove the presence of entanglement, and then use this entanglement for cryptographic purposes. Even if Alice or Bob’s apparatuses have been tampered with, they can still only violate things like the CHSH inequality if they actually share an entangled state.
- Experimentally it is much easier to realize quantum key distribution than general quantum computation, because you basically just need to prepare qubits (usually photons) in either the

⁶The assumption that the state is pure rather than mixed is without loss of generality.

computational or the Hadamard basis, send them across a channel (usually an optical fibre, but sometimes free space), and measure them in either the computational or the Hadamard basis. Many sophisticated experiments have already been done. Somewhat surprisingly, you can already commercially buy quantum key distribution machinery. Unfortunately the implementations are typically not perfect (for instance, we don't have perfect photon sources or perfect photon detectors), and once in a while another loophole is exposed in the implementation, which the vendor then tries to patch, etc.

Exercises

1. Here we will consider in more detail the information-disturbance tradeoff for measuring a qubit in one of the four BB84 states (each of which occurs with probability 25%).
 - (a) Suppose Eve measures the qubit in the orthonormal basis given by $\cos(\theta)|0\rangle + \sin(\theta)|1\rangle$ and $\sin(\theta)|0\rangle - \cos(\theta)|1\rangle$, for some parameter $\theta \in [0, \pi/4]$. The first basis vector corresponds to output 0, the second to output 1. For each of the four possible BB84 states, give the probabilities of outcome 0 and outcome 1 (so your answer should consist of 8 numbers, each of which is a function of θ).
 - (b) What is the average probability that Eve's measurement outcome equals the encoded bit a_i , as a function of θ ? (average taken both over the uniform distribution over the four BB84 states, and over the probabilities calculated in part (a))
 - (c) By what angle does the state change in each of the 8 cases of (a)?
2.
 - (a) What is the Schmidt rank of the state $\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$?
 - (b) Suppose Alice and Bob share k EPR-pairs. What is the Schmidt rank of their joint state?
 - (c) Prove that a pure state $|\phi\rangle$ is entangled if, and only if, its Schmidt rank is greater than 1.
3. Give the Schmidt decomposition of the state $\frac{1}{2}(|0\rangle_A|0\rangle_B + |0\rangle_A|1\rangle_B + |1\rangle_A|1\rangle_B + |1\rangle_A|2\rangle_B)$. Here Alice's space has dimension 2, and Bob's space has dimension 3. It suffices if you write down your Schmidt decomposition, being explicit about the values of the λ_i 's and what are the states $|a_i\rangle$ and $|b_i\rangle$. You can add your calculation (involving local density matrices etc.) as a justification, but you don't have to.
4. Consider a density matrix ρ on Alice's Hilbert space. A bipartite pure state $|\psi\rangle_{AB}$ is called a *purification* of ρ , if $\rho = \text{Tr}_B(|\psi\rangle\langle\psi|)$. The B -register in $|\psi\rangle_{AB}$ is called the *purifying register*.
 - (a) Show that an EPR-pair is a purification of the 1-qubit mixed state $\rho = I/2$.
 - (b) Show that if ρ is a density matrix of rank r , then there exists a purification of ρ where the purifying register has at most $\lceil \log r \rceil$ qubits.
 - (c) Show that if $|\psi\rangle_{AB}$ and $|\psi'\rangle_{AB}$ are purifications of the same ρ , then there exists a unitary U on Bob's space such that $|\psi'\rangle_{AB} = (I \otimes U)|\psi\rangle_{AB}$.
5. Suppose Alice has a 1-qubit state ρ .
 - (a) Suppose Alice chooses a uniformly random Pauli matrix (see Appendix A.9) and applies it to ρ . What is the resulting density matrix, averaged over the four cases?

- (b) Suppose Alice and Bob shared a uniformly distributed secret 2-bit string ab , which is unknown to Eve. How can Alice send ρ to Bob over a public quantum channel, without leaking any information to Eve (i.e., the quantum state sent over the channel should by itself be independent of ρ), in such a way that Bob can recover ρ ?
6. (H) Suppose we have a qubit in mixed state ρ that we want to hide from Alice and Bob individually, but in such a way that if Alice and Bob cooperate, then they can recover ρ . Describe how we can change ρ into some other 1-qubit state ρ' , what secret keys we give to Alice and Bob, why individually they can get no information about ρ from the qubit ρ' , and why jointly they can fully recover the qubit in state ρ from ρ' . The keys should be classical.
7. (H) Prove that Alice cannot give information to Bob by doing a unitary operation on her part of an entangled pure state.
8. Suppose Alice sends *two* n -bit messages M_1 and M_2 with the one-time pad scheme, reusing the *same* n -bit key K . Show that Eve can now get some information about M_1, M_2 from tapping the classical channel.
9. (a) (H) Consider a bipartite pure state shared between Alice and Bob, where Alice and Bob's local spaces have dimension d each:

$$\sum_{i,j \in \{1, \dots, d\}} \alpha_{ij} |i\rangle_A |j\rangle_B.$$

The state is given to you classically, as a list of d^2 amplitudes, each described by $O(d)$ bits. Give a classical polynomial-time algorithm to find the Schmidt coefficients and to find Alice and Bob's basis for a Schmidt decomposition.

- (b) Give a classical polynomial-time algorithm that decides whether a given bipartite pure state (given as in (a)) is entangled or not.

Comment: If the given state were mixed instead of pure, this decision problem is known to be NP-hard and hence probably not polynomial-time solvable.

Chapter 19

Quantum Machine Learning

19.1 Introduction

Machine learning tries to extract patterns and regularities from given data for the purposes of prediction and understanding. In a slogan, one could say: $\text{ML} = \text{data} + \text{optimization}$. The data is what you learn from; the optimization finds a good model or hypothesis for the given data, which hopefully has some generalization power. ML has gone through several ups and downs over the years, but currently is booming thanks to the success of so-called “deep learning,” based on neural networks.¹ ML is often subdivided into three subareas, depending on the data one has:

1. In *supervised learning* we are given labelled data, for instance pictures of animals annotated with the kind of animal that’s on the picture, and we want to learn how to predict the label.
2. In *unsupervised learning* we are just given unlabeled data, and need to find patterns in it. The canonical example is the *clustering* problem, where we are given unlabelled data items that we want to group into “similar” subsets. For example, it could be that our data consists of pictures of different kinds of animals (not labeled with the type of animal), and we somehow want to cluster the cat-pictures together, the wolf-pictures together, etc. We may or may not know in advance what the number of clusters should be.
3. In *reinforcement learning* the learner actually interacts with the environment, receiving rewards or penalties for desirable or undesirable behavior, and tries to learn from this interactive data to behave more successfully in the environment. This is roughly how a child learns.²

It is a very interesting question to see how quantum computing changes and helps machine learning. Here the learner would be a quantum computer, and the data may be classical or quantum. Quantum ML is by now a rather large area, and in this chapter we will go over a few representative results and methods for supervised and unsupervised learning, mostly with classical output. See [101] for quantum applications to reinforcement learning, and [57, 27, 222] for much more.

¹Machine learning based on neural networks has been studied for decades but quite suddenly became much more successful starting around 2012, due to the availability of more data, stronger computing hardware (incl. special-purpose GPUs for fast parallel matrix-vector calculations), and better software to do the training.

²It’s also how a computer can learn to play games. One of the big breakthroughs of machine learning was in 2016 when the AlphaGo program learned to play the game of Go so well that it beat one of the world’s best human Go players. Computers have been better than humans at chess already since the late 1990s (the IBM program “Deep Blue” beat Kasparov in 1997), but Go was viewed as a much more complicated game than chess.

19.2 Supervised learning from quantum data

19.2.1 The PAC model of learning

Let us first describe a mathematical model of what it means to learn from labeled data. This is Valiant’s PAC model [238], for “probably approximately correct” learning (see [223, 189] for more).

Assume for simplicity that the labels are just binary: 0 or 1. Our goal is to learn a Boolean function $f : \mathcal{X} \rightarrow \{0, 1\}$ from examples of the form $(x, f(x))$, where $x \in \mathcal{X}$. A typical case would be $\mathcal{X} = \{0, 1\}^n$. The last bit $f(x)$ of the example is called the *label*. Think for instance about the case where we are given 1000×1000 -pixel black-and-white pictures ($n = 1000,000$) whose labels $f(x)$ indicate whether x is the picture of a wolf or not. We would like to learn f , or some good approximation of it, to be able to recognize pictures of wolves in the future. Some x ’s are more important and more likely to appear as examples than others: many 1000×1000 -grids don’t depict anything. The assumption in PAC learning is that the examples are generated (independent and identically distributed) according to some distribution D on $\mathcal{X}$. The idea is that this D represents “the world” or “Nature,” which provides us with examples. We assume f cannot be completely arbitrary (in that case there would be an f consistent with every possible sequence of labeled examples) but comes from some known “concept class” $\mathcal{C}$ of Boolean functions. For instance, $\mathcal{C}$ could be a set of small logical formulas f on n Boolean variables, or a set of small-depth or small-size decision trees on n input bits, or neural networks with a restricted number of nodes or depth.

A learning algorithm should generate a “hypothesis” $h : \mathcal{X} \rightarrow \{0, 1\}$ that has small error compared to the unknown f that we’re trying to learn, *measured under the same distribution D that generated the data*.³ The *generalization error* of h w.r.t. the target function f is defined as

$$\text{err}_D(f, h) = \Pr_{x \sim D}[f(x) \neq h(x)].$$

This error measures how well we’ve generalized the examples, and how well we can predict the labels of future examples. We say that h is “approximately correct” if this error is small, at most some specified ε . The goal in PAC learning is to output an h that is probably approximately correct:

Definition 4 *An (ε, δ) -PAC learner for a concept class $\mathcal{C}$ w.r.t. distribution D on $\mathcal{X}$, is an algorithm that receives m labeled examples $(x_1, f(x_1)), \dots, (x_m, f(x_m))$ for a target function $f \in \mathcal{C}$, where each $x_i \sim D$, and that outputs a hypothesis h such that*

$$\Pr[\text{err}_D(f, h) \leq \varepsilon] \geq 1 - \delta.$$

The learning algorithm has to satisfy the above for every possible target function $f \in \mathcal{C}$, and the probability is over both the choice of the examples and over the internal randomness of the algorithm.

An (ε, δ) -PAC learner for a concept class $\mathcal{C}$ is an algorithm that is an (ε, δ) -PAC learner for $\mathcal{C}$ w.r.t. every possible distribution D .

Note that the first part of the definition is about learners that are only required to work correctly for one specific distribution D (for instance, the uniform distribution over $\mathcal{X}$), while the second part is “distribution-independent”: here we want a learner that works well irrespective of what

³It is important to be taught and tested according to the same distribution D . Imagine a quantum-computing course whose lectures focused on the mathematics of quantum algorithms, but with an exam that focuses on physics questions about how to implement qubits and gates—that would clearly be very unreasonable.

(unknown) distribution D generates the data. This is in keeping with the usual attitude towards algorithms in computer science: these should work well even for a worst-case input. We don't require the class $\mathcal{H}$ of possible hypotheses h to equal the class $\mathcal{C}$ of possible target functions f (if we add this requirement, then it's called *proper* PAC learning). This allows us for instance to use neural networks to learn target functions that come from some other class $\mathcal{C}$, say logical formulas.

The number of examples m that a particular learning algorithm uses is called its “sample complexity,” and the overall time or number of elementary operations it takes to output h is its “time complexity.” Clearly the latter upper bounds the former, since we need at least one operation to process one example. The sample complexity of a concept class $\mathcal{C}$ (as a function of ε, δ) is the minimal sample complexity among all PAC learners for $\mathcal{C}$. Ideally, a good learner for $\mathcal{C}$ has both small sample complexity and small time complexity (say, polynomial in n). For some concept classes $\mathcal{C}$ efficient distribution-independent PAC learners exist, for example the class of logical formulas in k -Conjunctive Normal Form (i.e., each f would be the AND of several ORs, each of at most k variables or negated variables) or the class of regular languages (with the added help of so-called “membership queries”), but there are also many $\mathcal{C}$ that are not efficiently learnable.

19.2.2 Learning from quantum examples under the uniform distribution

There are different ways to define learning from *quantum* data. One natural way, due to Bshouty and Jackson [71], is to replace each classical random example $(x, f(x))$, with $x \sim D$, by a superposition. Focusing on the typical case $\mathcal{X} = \{0, 1\}^n$, a *quantum example* would be the $(n + 1)$ -qubit state

$$\sum_{x \in \{0,1\}^n} \sqrt{D(x)} |x, f(x)\rangle.$$

Of course, the world doesn't usually present us with quantum examples, in contrast to the abundance of classical data for machine learning. So this model is only relevant in special cases, for example if we have a physical experiment producing such states.

One thing we could do with a quantum example is measure it in the computational basis, but that would just give us back a classical example $(x, f(x))$ with $x \sim D$. A more clever thing we can do is *Fourier sampling*. Suppose D is the uniform distribution. Exercise 1 shows how to convert a quantum example (with probability $1/2$) into an n -qubit state where the labels are ± 1 -phases:

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle.$$

If we apply n Hadamard gates to this state, then we get

$$\sum_{s \in \{0,1\}^n} \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{x \cdot s} (-1)^{f(x)} |s\rangle = \sum_{s \in \{0,1\}^n} \alpha_s |s\rangle.$$

If we measure this state, then we'll see outcome $s \in \{0, 1\}^n$ with probability α_s^2 . The amplitudes α_s are called the Fourier coefficients of the function $(-1)^{f(x)}$, whence the name “Fourier sampling.” In some cases Fourier sampling gives a lot of information about the f we're trying to learn.

Learning linear functions. A perfect illustration of Fourier sampling is for the following class:

$$\mathcal{C} = \{f_a \mid a \in \{0, 1\}^n, \forall x : f_a(x) = a \cdot x \bmod 2 = \sum_{i=1}^n a_i x_i \bmod 2\},$$

these are the linear functions modulo 2. It is easy to calculate that if we do Fourier sampling on a quantum example for function f_a , then $\alpha_a = 1$ and $\alpha_s = 0$ for all $s \neq a$. So one Fourier sample already tells us what a is! Hence we can learn f_a *exactly* (i.e., with $\varepsilon = 0$), with high probability, using $O(1)$ examples and $O(n)$ elementary gates. In contrast, learning linear functions from classical examples under the uniform distribution requires $\Theta(n)$ examples (see Exercise 2).

Learning DNF. A richer concept class that can be learned efficiently from uniform quantum examples is the class of s -term Disjunctive Normal Form (DNF) formulas on n Boolean variables. These are formulas of the form $f(x) = (x_1 \wedge \neg x_3) \vee (x_2 \wedge x_3 \wedge x_5)$, i.e., an OR of up to s different ANDs of variables or negations of variables. The concept class $\mathcal{C}$ of s -term DNF is not known to be efficiently PAC learnable w.r.t. the uniform distribution D classically. However, Bshouty and Jackson [71] showed that s -term DNF *can* be learned in polynomial time (in s and n) from uniform *quantum* examples. Roughly speaking, they use Fourier sampling to produce a linear function that is weakly correlated with the target DNF function f , and then use a classical “boosting” algorithm to combine multiple such weak hypotheses into one good hypothesis h . We’ll skip the details here.

19.2.3 Learning from quantum examples under all distributions

We saw a few cases where quantum examples reduce the sample and/or time complexity of learning algorithms w.r.t. a fixed data-generating distribution D , namely uniform D . But in the PAC model we ideally want a *distribution-independent* learner that works well for every possible distribution D . Can allowing quantum instead of classical examples significantly reduce the sample complexity of learning a class $\mathcal{C}$ in the distribution-independent setting? It turns out the answer is ‘no’.

Classically, the number of examples that is necessary and sufficient for (ε, δ) -PAC learning a concept class $\mathcal{C}$ is known to be [60, 132]

$$m = \Theta \left(\frac{VCdim(\mathcal{C})}{\varepsilon} + \frac{\log(1/\delta)}{\varepsilon} \right), \quad (19.1)$$

where $VCdim(\mathcal{C})$ is the so-called *VC-dimension* of $\mathcal{C}$, named after Vapnik and Chervonenkis [241] and defined as follows. We say that a set $S \subseteq \{0, 1\}^n$ is *shattered* by $\mathcal{C}$ if for each of the $2^{|S|}$ possible labelings $\ell : S \rightarrow \{0, 1\}$, there is a function $f \in \mathcal{C}$ that has the same labeling of S (i.e., $f|_S = \ell$). $VCdim(\mathcal{C})$ is the size of a largest S shattered by $\mathcal{C}$. Intuitively, larger VC-dimension corresponds to a more complex or “richer” (and hence harder to learn) concept class. We won’t prove the characterization of Eq. (19.1) here, but Exercises 4 and 5 go most of the way towards the claimed upper and lower bounds on m , respectively.

It was proven in [29] that in fact the same formula Eq. (19.1) determines the number of *quantum* examples that are necessary and sufficient for learning $\mathcal{C}$. The sufficiency is trivial: just measure the quantum examples and run the best classical PAC learner. The necessity was proved by reducing a quantum measurement problem to the problem of PAC learning $\mathcal{C}$ from quantum examples, and showing that the number of copies of the example-state required to solve that measurement problem is at least the expression of Eq. (19.1). So, up to constant factors, quantum examples are not more useful than classical examples for distribution-independent PAC learning.

19.2.4 Learning quantum states from classical data

One can generalize PAC learning from Boolean-valued to *real-valued* target functions $f : \mathcal{X} \rightarrow [0, 1]$, and then consider a hypothesis $h : \mathcal{X} \rightarrow [0, 1]$ to be approximately correct (for some small γ) if

$$\text{err}_{D,\gamma}(f, h) = \Pr_{x \sim D} [|f(x) - h(x)| > \gamma] \leq \varepsilon.$$

So now a good hypothesis h is supposed to be close to f (rather than equal) for most x .

An interesting example is the problem of learning an unknown n -qubit quantum state ρ from measurement data. Let $\mathcal{X}$ be the set of measurement elements, i.e., psd matrices M with $\|M\| \leq 1$. If we measure ρ with some POVM of which M is one element, then the probability to get the outcome corresponding to M , is $\text{Tr}(M\rho)$. Accordingly, we can define $f : \mathcal{X} \rightarrow [0, 1]$ as $f(M) = \text{Tr}(M\rho)$ and consider the class $\mathcal{C}$ of all such functions (one f for each possible ρ , so this class is uncountable). Aaronson [1] showed that this $\mathcal{C}$ is classically PAC learnable from $O(n)$ examples of the form $(x, f(x))$ (with some polynomial dependence of the sample complexity on γ, ε , and exponential time complexity). Note that we are not really learning ρ itself, but rather learn to predict the measurement probabilities. In contrast, learning a good approximation of ρ itself (with small error in trace distance) requires a number of copies of ρ that is exponential in n [126, 197]. Some positive results for learning specific classes of quantum states can be found in [180, 20, 161].

19.3 Unsupervised learning from quantum data

In this section we will look at an example of *unsupervised* learning from quantum data: dimension-reduction via Principal Component Analysis. Suppose we are given m vectors $v_1, \dots, v_m \in \mathbb{R}^d$, say unit vectors for simplicity. Let's say the dimension d of the data-vectors is very large, and we would like to reduce it to some much smaller k , say at most $k = \text{polylog}(d)$. Many machine learning tasks, for example clustering, become much easier if we can significantly reduce this dimension.

One way to achieve this dimension-reduction is to find k suitable unit vectors $c_1, \dots, c_k \in \mathbb{R}^d$ (which may or may not be in the set $\{v_i\}$ themselves), such that the projection $P_S v_i$ of the v_i 's on the k -dimensional space $S = \text{span}\{c_1, \dots, c_k\}$ typically doesn't lose much, i.e., $P_S v_i$ is close to v_i for most $i \in [m]$. Then we can replace each v_i by the k -dimensional vector $P_S v_i = \sum_{j=1}^k \alpha_j c_j$, expressed as the vector of coefficients $(\alpha_j) \in \mathbb{R}^k$ (note that $\alpha_j = \langle c_j | v_i \rangle$). How to find those k "directions"? One method that often (though not always) works well is to find the k eigenvectors corresponding to the k largest eigenvalues of the following $d \times d$ "correlation matrix":

$$A = \sum_{i=1}^m v_i v_i^T.$$

Those k eigenvectors are called the k "principal components" of A . They intuitively correspond to the k most important directions in the data, and we can choose them for dimension-reduction.

Classically, we can find those k eigenvectors by diagonalizing A , which takes times polynomial in d . In the quantum case we can do something very different, under the (very strong) assumption that we can efficiently, say in time $\text{polylog}(d)$, prepare the $\lceil \log(d) \rceil$ -qubit quantum states $|v_i\rangle$ corresponding to the vectors v_i . By choosing $i \in [m]$ uniformly at random and preparing $|v_i\rangle$, we prepare the following $\lceil \log(d) \rceil$ -qubit mixed state, which is proportional to the correlation matrix:

$$\rho = \frac{1}{m} \sum_{i=1}^m |v_i\rangle \langle v_i| = \frac{1}{m} A.$$

Let's say this has (unknown) spectral decomposition $\rho = \sum_{j=1}^d \lambda_j |c_j\rangle\langle c_j|$ with $\lambda_1 \geq \dots \geq \lambda_d \geq 0$, where the first k eigenvalues sum to something close to 1, and are not too close together, at least $1/\text{poly}(k)$ apart.⁴ We would now like to find the top- k eigenstates $|c_1\rangle, \dots, |c_k\rangle$ of this ρ .

Note that the unitary $U = e^{i\rho}$ has the same eigenstates as ρ itself, with every eigenvalue λ_j of ρ translating into eigenvalue $e^{i\lambda_j}$ of U . Lloyd et al. [174] (with more precise analysis and matching lower bound in [153]) showed that we can actually implement the power U^t up to error ε using $O(t^2/\varepsilon)$ copies of the state ρ (see Exercise 7). We now use phase estimation with the unitary U on a copy of ρ itself, with additive error $\delta = 1/\text{poly}(k)$. By Section 4.6, phase estimation with additive error δ corresponds to running controlled versions of U^t for t up to $O(1/\delta)$. Under our earlier assumptions, this only takes $\text{poly}(k) = \text{polylog}(d)$ time. Ignoring for simplicity the small errors ($\leq \delta$) that phase estimation makes in estimating the values λ_j , phase estimation transforms the copy of ρ and a few auxiliary $|0\rangle$ -qubits into the state

$$\sum_{j=1}^m \lambda_j |c_j\rangle\langle c_j| \otimes |\lambda_j\rangle\langle \lambda_j|.$$

If we measure the second register, then we obtain state $|c_j\rangle \otimes |\lambda_j\rangle$ with probability λ_j .⁵ Doing this $\text{poly}(k)$ many times, we learn the k largest values $\lambda_1, \dots, \lambda_k$, and for each of those λ_j 's we'll have a number of copies of the eigenstate $|c_j\rangle$. This is a quantum form of Principal Component Analysis.

This collection of eigenstates determines a k -dimensional subspace on which we could re-express the v_i 's (approximately), but it is not very explicit: we only have the k basis vectors of this space as quantum states! Suppose we want to express some unit vector v (which again we assume we can prepare efficiently as a state $|v\rangle$) as a linear combination of the c_j 's. One thing we can do is use a few copies of each $|c_j\rangle$ to approximate $|\langle c_j | v \rangle|^2$ for each i using the SWAP-test, which gives us at least partial information about the coefficients $\langle c_j | v \rangle$ (see Exercise 8).

"Quantum PCA" has a lot of drawbacks, but at least it shows some genuinely quantum tricks that we can use under the assumption that our input vectors can be efficiently prepared as quantum states. There have also been some quantum approaches for the prominent unsupervised learning problem of clustering, but we will not describe those here (see for instance [173, 150]).

19.4 Optimization

In the previous two sections we assumed quantum data: either the data is already given as a superposition, or we can efficiently put given classical data in superposition. However, in most real-world applications of machine learning we have classical data without the means to efficiently make this quantum. Remembering the slogan ML = data + optimization, if there's any room left for quantum improvements when data is classical, it would be in the optimization to find a well-fitting model for the data. We'll look at some examples where quantum computing might help.

⁴All these assumptions make this principal-component analysis a *heuristic* method for clustering, not something that provably always works. It is anyway not clear in this case what a correct or optimal output would be: because this is unlabeled data, we do not have a clear standard for correctness.

⁵Actually, in the second register we will see (with high probability) a δ -close approximation of λ_j rather than λ_j itself, but if we assume δ is much smaller than the spacing between the eigenvalues $\lambda_1, \dots, \lambda_k$, then such a δ -approximation is good enough to "recognize" the state in the first register as the j th principal component.

19.4.1 Variational quantum algorithms

One approach that has received a lot of attention is to optimize over *parametrized circuits*. Suppose we have a quantum circuit $U(\theta)$ with a vector θ of parameters. This could for instance be a circuit where CNOTs and single-qubit rotations are already in place, but the angles of the single-qubit gates are parameters that we can tweak. This $U(\theta)$ is then applied to a fixed starting state, say $|0\rangle$, yielding a final state $|\psi(\theta)\rangle = U(\theta)|0\rangle$. The goal is now to minimize the expected value of some observable M , i.e., to find a θ to minimize the function $f(\theta) = \langle\psi(\theta)|M|\psi(\theta)\rangle$. In the case of supervised learning applications, $U(\theta)$ could for instance represent some hypothesis (i.e., a way to predict labels of x 's), M could incorporate the given labeled examples $(x, f(x))$, and $f(\theta)$ could be the “empirical error”: the fraction of mis-predicted labels among the given examples.

Note that $f(\theta)$ can be computed approximately (for classically given θ) on a quantum computer by repeatedly preparing $|\psi(\theta)\rangle$ and measuring the observable M . If the circuits $U(\theta)$ are relatively simple (say, few qubits, few gates, low depth) and M is relatively easy to measure (say, a sum of a few n -qubit Pauli matrices with few non-identity terms) then this could already be done on a relatively small and simple quantum computer. Variational quantum algorithms (VQAs) are typically hybrid classical-quantum algorithms: the minimization over θ is usually done by a classical outer loop that iteratively improves θ . Using the ability to approximately compute f we can for instance try to do approximate gradient descent (move θ by some step-size in the direction of steepest descent of f) or some other method. This is analogous to the iterative way the weights in neural networks are optimized, and these variational quantum approaches are sometimes (with a keen sense for marketing) called “quantum neural networks” or “quantum deep learning.” For combinatorial optimization, a very structured version of the variational approach is the Quantum Approximate Optimization Algorithm (QAOA) [109]. See [78] for a general overview of VQAs.

One interesting application of this variational idea is in trying to find the smallest eigenvalue of a given Hamiltonian H . For example, H could describe the energy of a chemical system as a function of the locations of the particles (nuclei and electrons) of the system; the smallest eigenvalue of H would be the “ground-state energy” of the system, which is an important quantity in chemistry. We know from Chapter 14 that in general this problem of determining or even well-approximating this ground state energy is **QMA**-hard, even in the special case where H is a sum of 2-local terms, so in general this shouldn't be efficiently solvable on a quantum computer. However, suppose that from some general physics or chemistry intuition we have a rough idea of what the ground state of our particular Hamiltonian H should look like, something we can prepare using a simple parametrized circuit $U(\theta)$. The set of states $|\psi(\theta)\rangle = U(\theta)|0\rangle$ that we are limiting ourselves to, is called an “Ansatz” (German for “approach” or “attempt”). We can now try to optimize the parameters θ in order to minimize the expected value $f(\theta) = \langle\psi(\theta)|H|\psi(\theta)\rangle$, i.e., the energy of the state $|\psi(\theta)\rangle$. This approach is called the “variational quantum eigensolver” (VQE) [199], and is one of the best hopes for applying smallish, near-term quantum computers to problems in chemistry.

19.4.2 Some provable quantum speed-ups for optimization

The variational approach is rather heuristic: it very much depends on how good the “Ansatz” (the choice of the class of parametrized circuits $U(\theta)$) happens to be for the particular problem at hand. Here we mention some other approaches, which yield provable (albeit usually only polynomial) quantum speed-ups under some assumptions on how the input is given.

- There are many quantum speed-ups for optimization problems on graphs, typically using

Grover search (Section 7.2), Grover-based minimum-finding (Exercise 7.10), amplitude amplification (Section 7.3), or amplitude estimation (Exercise 7.8) as a subroutine. Examples are finding shortest paths [102] and approximating minimum cuts or graph sparsification [24].

- Solving linear systems and other basic linear algebra is ubiquitous in classical optimization algorithms. Since quantum states are vectors and quantum operations are matrices, one can try to improve such classical algorithms using quantum algorithms. Examples are phase estimation (Section 4.6), the block-encoding approach (Section 9.4 and [120, Section 3.2.4]), and HHL (Chapter 10). The trouble with this approach is that it often assumes the input is a quantum state (which is not always practical) and/or that it produces the output as a quantum state (which is not always useful). For example, HHL and quantum PCA have both features. See [2] for more discussion.

One interesting application of “quantum linear algebra” (with classical inputs and outputs!) is the quantum recommendation system of Kerenidis and Prakash [151], which can generate recommendations of type “you might also like” to a user of systems like Amazon or Netflix, based on the user’s and other users’ earlier behavior. Initially [151] was believed to give an exponential speed-up over classical recommendation systems, until Tang showed how to “dequantize” their quantum algorithm under similar classical access assumptions [234, 81].

- In *convex optimization* we minimize a convex function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, either over all $x \in \mathbb{R}^n$ or over all x that are constrained to lie in some convex domain $\mathcal{X} \subset \mathbb{R}^n$. This covers a big part of continuous optimization. Convexity ensures that the only local minima are also global minima, but such methods often still work to find good local minima for non-convex problems (such as training neural networks). Iterative *first-order* methods like gradient descent use the gradient of f at a given point, which in some cases can be computed more efficiently by quantum algorithms [146, 121, 95] (see Exercise 6). *Second-order* methods often solve a linear system involving the Hessian (the $n \times n$ matrix of partial second derivatives at a given point), and we can try to use quantum linear algebra. If the matrix is symmetric and diagonally dominant and the output needs to be classical, then we could use the linear solver of [24].

Quantum algorithms are known for the specific cases of linear programming (LPs) and semidefinite programming (SDPs) [64, 22, 63, 23, 21], for learning support vector machines (SVMs) [203, 221, 213, 9, 214], and for least-squares linear regression with an ℓ_1 -regularizer [80].

Exercises

1. Suppose that for some unknown Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ and amplitude-vector $(\alpha_x)_{x \in \{0, 1\}^n}$, you are given one copy of the $(n + 1)$ -qubit state

$$\sum_{x \in \{0, 1\}^n} \alpha_x |x\rangle |f(x)\rangle.$$

Show how you can convert this into state

$$\sum_{x \in \{0, 1\}^n} \alpha_x (-1)^{f(x)} |x\rangle |1\rangle$$

with success probability $1/2$, in such a way that you know when you succeeded.

2. Consider again the concept class $\mathcal{C}$ of linear functions mod 2.
 - (a) Give a classical learning algorithm to learn a linear function exactly with high success probability ($\varepsilon = 0, \delta = 1/3$) using $O(n)$ uniform random examples and $O(n^3)$ time.
 - (b) Argue that every classical PAC learner for $\mathcal{C}$ under uniform D , with $\varepsilon < 1/4$, needs $\Omega(n)$ examples.
3. In the model of *exact learning with membership queries*, the goal is to exactly learn a target function $f \in \mathcal{C}$ from queries to f (so there are no examples in this setting, or rather the learner can choose their own examples).
 Show that if $\mathcal{C}$ is the concept class of linear functions, then a target function $f \in \mathcal{C}$ can be learned with 1 quantum membership query, but requires $\Omega(n)$ classical membership queries.
4. Consider a concept class $\mathcal{C}$ of functions $f : \mathcal{X} \rightarrow \{0, 1\}$, with $|\mathcal{X}| = N$ and $VCdim(\mathcal{C}) = d$.
 - (a) Consider the following simple (and probably not very time-efficient) learning algorithm:
 Draw m examples for target function f ; output a $h \in \mathcal{C}$ consistent with these examples.
 Let $h \in \mathcal{C}$ be a function with $err_D(f, h) > \varepsilon$. Show that at the end of this algorithm, the probability that h is still consistent with the m examples is $< (1 - \varepsilon)^m$.
 - (b) Set $m = \lceil \log(3|\mathcal{C}|) / \log(1/(1 - \varepsilon)) \rceil$. Show that with probability $\geq 2/3$, the only h that are consistent with the m examples have $err_D(f, h) \leq \varepsilon$.
 - (c) Derive an upper bound $m = O(d \log(N)/\varepsilon)$ on the classical sample complexity of $(\varepsilon, 1/3)$ -PAC learning the class $\mathcal{C}$ using Sauer's lemma, which says that $|\mathcal{C}| \leq \sum_{i=0}^d \binom{N}{i}$.
5. Suppose the set $S = \{x_1, \dots, x_d\} \subseteq \mathcal{X}$ is shattered by concept class $\mathcal{C}$. Consider a distribution D that puts $1 - 4\varepsilon$ probability on x_1 and $4\varepsilon/(d - 1)$ probability on each of $x_2, \dots, x_d$.
 - (a) Let $f \in \mathcal{C}$ be the target function. Show that you need $\Omega((d - 1)/\varepsilon)$ examples $\sim D$ to see (with probability $\geq 2/3$) $(x_i, f(x_i))$ for at least 50% of the $i \in \{2, \dots, d\}$.
 - (b) Show that the sample complexity of every $(\varepsilon, 1/3)$ -PAC learner for the class $\mathcal{C}$ is at least $\Omega((d - 1)/\varepsilon)$.
6. This exercise is about efficiently finding the gradient $\nabla f(z)$ of a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ at a point $z \in \mathbb{R}^d$. The gradient is the d -dimensional real vector of the d partial derivatives $\partial f / \partial x_i$, evaluated at the point z .
 - (a) (H) Let $f(x) = a + bx$ be a linear function from $\mathbb{R}$ to $\mathbb{R}$, where the real number $b \in [0, 1)$ can be written with n bits of precision. Suppose we have a unitary O_f that maps $|x, 0\rangle \rightarrow |x, f(x)\rangle$ (assume we have enough qubits to write down x and $f(x)$). Give a quantum algorithm to compute b using one application of O_f and one application of O_f^{-1} , and some unitaries that do not depend on f .
 - (b) Let $f(x_1, \dots, x_d) = a + b_1x_1 + \dots + b_dx_d$ be a linear function from $\mathbb{R}^d$ to $\mathbb{R}$, where $a, b_1, \dots, b_d \in \mathbb{R}$. Show that the gradient $\nabla f(z)$ is equal to $(b_1, \dots, b_d)$ for every $z \in \mathbb{R}^d$.
 - (c) Assume that for the function f in (b), each coefficient b_k is $\in [0, 1)$ and can be written with n bits of precision. Suppose we have a unitary O_f that maps $|x_1, \dots, x_d, 0\rangle \rightarrow |x_1, \dots, x_d, f(x_1, \dots, x_d)\rangle$. Give a quantum algorithm that computes the gradient $\nabla f(z)$ using one application of O_f and O_f^{-1} , and some unitaries that do not depend on f .

Comment: The quantum algorithm of (c) is somewhat reminiscent of the Bernstein-Vazirani algorithm (Section 2.4.2), though that one is for functions over $\mathbb{F}_2^d$ rather than $\mathbb{R}^d$. Variants of the algorithm of (c) can also be applied to efficiently approximate the gradient of a sufficiently smooth *non-linear* function $f : \mathbb{R}^d \rightarrow \mathbb{R}$, since a smooth f can be well-approximated at a given point $z \in \mathbb{R}^d$ by a linear function whose coefficients are the entries of the gradient $\nabla f(z)$.

7. (H) Let σ and ρ be k -qubit mixed states, $\varepsilon > 0$ small, $t \geq 0$, and $U = e^{i\rho}$ be a unitary. Our goal in this exercise is to apply U^t to σ (with error $\leq \varepsilon$ in trace norm) at the expense of using some copies of the state ρ .
 - (a) Let V be the 2-qubit SWAP-gate (which maps $|a\rangle|b\rangle \rightarrow |b\rangle|a\rangle$ for all $a, b \in \{0, 1\}$). Show that $V^{-2\eta/\pi} = e^{-i\eta}e^{iV\eta}$ for all $\eta \geq 0$.
 - (b) Let W be the $2k$ -qubit unitary that swaps the first k qubits with the last k qubits, i.e., it maps $|a\rangle|b\rangle \rightarrow |b\rangle|a\rangle$ for all $a, b \in \{0, 1\}^k$. Show that for all $\eta \geq 0$, $e^{iW\eta}$ can be implemented with k 2-qubit gates.
 - (c) Show that for small $\eta \geq 0$, $U^\eta \sigma U^{-\eta} = \sigma + i\eta(\sigma\rho - \rho\sigma) + E$, where $\|E\|_1 = O(\eta^2)$.
 - (d) Let σ' be the k -qubit local state of the first register after applying the unitary $e^{iW\eta}$ to the $2k$ -qubit state $\sigma \otimes \rho$. Show that $\sigma' = \sigma + i\eta(\sigma\rho - \rho\sigma) + E'$, where $\|E'\|_1 = O(\eta^2)$.
 - (e) Show that $\|\sigma' - U^\eta \sigma U^{-\eta}\|_1 = O(\eta^2)$.
 - (f) Show that you can implement U^t on σ with error ε in trace norm, using $O(t^2/\varepsilon)$ copies of ρ and $O(kt^2/\varepsilon)$ elementary gates.
8. Suppose $|\phi\rangle$ and $|\psi\rangle$ are unknown n -qubit pure states.
 - (a) (H) Show how a quantum computer can estimate the overlap $|\langle\phi|\psi\rangle|$ (in absolute value) up to additive error $1/100$ using $O(1)$ given copies of $|\phi\rangle$ and $|\psi\rangle$, and $O(n)$ elementary gates.
 - (b) Assume the inner product $\langle\phi|\psi\rangle$ is a real number. Show that $\| |\phi\rangle - |\psi\rangle \|^2 = 2 - 2\langle\phi|\psi\rangle$.
 - (c) Assume $\langle\phi|\psi\rangle$ is real and positive. Show how a quantum computer can estimate the distance $\| |\phi\rangle - |\psi\rangle \|$ up to additive error $1/100$ using $O(1)$ copies of $|\phi\rangle$ and $|\psi\rangle$, and $O(n)$ gates.
 - (d) Can a quantum computer detect the difference between the two cases $|\psi\rangle = |\phi\rangle$ and $|\psi\rangle = -|\phi\rangle$, given arbitrarily many copies of these two states? Explain your answer.

Chapter 20

Error-Correction and Fault-Tolerance

20.1 Introduction

When Shor’s algorithm had just appeared in 1994, most people (especially experimental physicists, who were very aware of the difficulties in manipulating subatomic particles) were extremely skeptical about the prospects of actually building a quantum computer. In their view, it would be impossible to avoid errors when manipulating small quantum systems, and such errors would very quickly overwhelm the computation, rendering it no more useful than classical computation. However, in the few years that followed, the theory of quantum error-correction and fault-tolerant computation was developed. This shows, roughly speaking, that if the error-rate per operation can be brought down to something reasonably small (say 1%), and the errors between different qubits are not very correlated, then we can actually do near-perfect quantum computing for as long as we want. Below we give a succinct and somewhat sketchy introduction to this important but complex area, just explaining the main ideas. See the surveys by Gottesman [123] and Terhal [235] for more (in particular the latter for the important “surface code,” which we won’t cover here).

20.2 Classical error-correction

In the early days of classical computing, errors were all over the place: memory-errors, errors in bits sent over a channel, incorrectly applied instructions, etc.¹ Nowadays hardware is much more reliable, but we also have much better “software solutions” for errors, in particular error-correcting codes. Such codes take a string of data and encode it in a larger string (the “codeword”), adding a lot of redundancy so that a small fraction of errors on the codeword won’t be able to reduce the information about the encoded data.

The simplest example is of course the repetition code. If we want to protect a bit b , we could repeat it three times:

$$b \mapsto bbb.$$

If we want to decode the encoded bit b from the (possibly corrupted) 3-bit codeword, we just take the majority value of the 3 bits.

Consider a very simple noise model: every bit is flipped (independently of the other bits) with probability p . Then initially, before applying the code, b has probability p to be flipped. But if

¹The name “bugs” actually comes from insects getting stuck inside the computer and causing errors.

we apply the repetition code, the probability that the majority-value of the three bits is different from b , is the probability of 2 or 3 bitflips, which is $3p^2(1-p) + p^3 < 3p^2$. Hence the error-rate has been reduced from p to less than $3p^2$. If the initial error-rate p_0 was $< 1/3$, then the new error-rate $p_1 < 3p_0^2$ is less than p_0 and we have made progress: the error-rate on the encoded bit is smaller than the error-rate on the unencoded bits. If we'd like it to be even smaller, we could concatenate the code with itself, i.e., repeat each of the three bits in the code three times, so the codelength becomes 9. This would give error-rate $p_2 = 3p_1^2(1-p_1) + p_1^3 < 3p_1^2 < 27p_0^4$, giving a further improvement. As we can see, as long as the initial error-rate p was at most $1/3$, we can reduce the error-rate to whatever we want: k levels of concatenation encode one “logical bit” into 3^k “physical bits,” but the error-rate for each logical bit has been reduced to $\frac{1}{3}(3p_0)^{2^k}$.² This is a very good thing: if the initial error is below $1/3$, then k levels of concatenation increase the number of bits exponentially (in k) but reduce the error-rate *double-exponentially fast*!

Typically, already a small choice of k gets the error-rate down to negligible levels. For example, suppose we want to protect some polynomial (in some n) number of bits for some polynomial number of time-steps, and our physical error-rate is some fixed $p_0 < 1/3$. Choosing $k = 2 \log \log n$ levels of concatenation already suffices for this, because then $p_k \leq \frac{1}{3}(3p_0)^{2^k} \sim 2^{-(\log n)^2} = n^{-\log n}$ goes to 0 faster than any polynomial. In that case, by the union bound, even the probability that there exists an error anywhere among our polynomially many logical bits in polynomially many time-steps, will be negligibly small. With this choice of k , each logical bit would be encoded in $3^k = (\log n)^{2 \log(3)}$ physical bits, so we only increase the number of bits by a polylogarithmic factor.

20.3 Quantum errors

The need for error-correction is far greater for quantum computers than for classical computers, because “quantum hardware” is much more fragile than classical hardware. Unfortunately, error-correction is also substantially more difficult in the quantum world, for several reasons:

- The classical solution of just repeating a state is not available in general in the quantum world, because of the no-cloning theorem.
- The classical world has basically only bitflip-errors, while the quantum world is continuous and hence has infinitely many different possible errors.
- Measurements that test whether a state is correct can collapse the state, losing information.

Depending on the specific model of errors that one adopts, it is possible to deal with all of these issues. We will consider the following simple error model. Consider quantum circuits with S qubits, and T time-steps; in each time-step, several gates on disjoint sets of qubits may be applied in parallel. After each time-step, at each qubit, independently from the other qubits, some unitary error hits that qubit with probability p . Note that we assume the gates themselves to operate perfectly; this is just a convenient technical assumption, since a perfect gate followed by errors on its outgoing qubits is the same as an imperfect gate.

²With a bit more work we can show that this even works if the initial error-rate p is only strictly less than $1/2$ rather than $\leq 1/3$. See Exercise 9.

Let's investigate what kind of (unitary) errors we could get on one qubit. Consider the four Pauli matrices from Appendix A.9:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

These have an interpretation as possible errors: I corresponds to no-error, X is a bitflip-error, Z is a phaseflip-error, and $Y = iXZ$ is a phaseflip-error followed by a bitflip-error (and a global phase of i , which doesn't matter). These four matrices span the space of all possible 2×2 matrices, so every possible error-operation E on a qubit is some linear combination $E = \alpha_0 I + \alpha_1 X + \alpha_2 Y + \alpha_3 Z$ of the 4 Pauli matrices. More generally, every $2^k \times 2^k$ matrix can be written uniquely as a linear combinations of matrices that each are the tensor product of k Pauli matrices.

Consider for example the error which puts a small phase ϕ on $|1\rangle$:

$$E = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix} = e^{i\phi/2} \cos(\phi/2) I - ie^{i\phi/2} \sin(\phi/2) Z.$$

Note that for small ϕ most of the weight in this linear combination sits on I , which corresponds to the fact that E is close to I . The sum of squared moduli of the two coefficients is 1 in this case. That's not a coincidence: whenever we write a unitary as a linear combination of Pauli matrices, the sum of squares of the coefficients will be 1 (see Exercise 1).

The fact that all 1-qubit errors are linear combinations of I, X, Y, Z , together with the linearity of quantum mechanics, implies that if we can correct bitflip-errors (X), phaseflip-errors (Z), and their product (Y), then we can correct *all possible* unitary errors on a qubit.³ So typically, quantum error-correcting codes are designed to correct bitflip and phaseflip-errors (their product is then typically also correctable), and all other possible errors are then also handled without further work.

Our noise model does not explicitly consider errors on multiple qubits that are not a product of errors on individual qubits. However, even such a joint error on, say, k qubits simultaneously can still be written as a linear combination of products of k Pauli matrices. So also here the main observation applies: if we can just correct bitflip and phaseflip-errors on individual qubits, then we can correct all possible errors!

20.4 Quantum error-correcting codes

Quantum error-correcting codes encode a number of “logical qubits” into a larger number of “physical qubits,” in such a way that errors on some number of its qubits can be corrected. The first and simplest is Peter Shor's 9-qubit code [227], which encodes 1 logical qubit into 9 physical qubits, and can correct an error on any one of the 9 physical qubits. Here are the codewords for the two logical basis states:

$$\begin{aligned} |0\rangle \mapsto |\bar{0}\rangle &= \frac{1}{\sqrt{8}}(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle) \\ |1\rangle \mapsto |\bar{1}\rangle &= \frac{1}{\sqrt{8}}(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle) \end{aligned}$$

³We can even correct the *non-unitary* errors that arise from undesired interaction between qubits of our circuit with the environment, but we won't talk about such errors here.

These two quantum codewords $|\bar{0}\rangle$ and $|\bar{1}\rangle$ span a 2-dimensional space $\{\alpha|\bar{0}\rangle + \beta|\bar{1}\rangle\}$. This 2-dimensional subspace of the overall 2^9 -dimensional space is called the “codespace.”

Suppose an error happens on one of these 9 qubits. We would like to have a procedure that maps the resulting state back to the codespace. By linearity, it suffices if we can do this for the basis states $|\bar{0}\rangle$ and $|\bar{1}\rangle$. First consider bitflip and phaseflip-errors.

Detecting a bitflip-error. If a bitflip-error occurs on one of the first 3 qubits, we can detect its location by noting which of the 3 positions is the minority bit. We can do this for each of the three 3-qubit blocks. Hence there is a unitary that writes down in 4 auxiliary qubits (which are all initially $|0\rangle$) a number $e_b \in \{0, 1, \dots, 9\}$. Here $e_b = 0$ means that no bitflip-error was detected, and $e_b \in \{1, \dots, 9\}$ means that a bitflip-error was detected on qubit number e_b . Note that we don’t specify what should happen if more than one bitflip-error occurred.

Detecting a phaseflip-error. To detect a phaseflip-error, we can consider the relative phase for each of the three blocks $|000\rangle \pm |111\rangle$, and if they are not all the same, unitarily write down in 2 more auxiliary qubits (again, initially $|0\rangle$) a number $e_p \in \{0, 1, 2, 3\}$. Here $e_p = 0$ means that no phaseflip-error was detected, and $e_p \in \{1, 2, 3\}$ means that a phaseflip-error was detected in the e_p -th block.⁴

Together the above two procedures form one unitary U (i.e., one circuit) that acts on $9 + 4 + 2 = 15$ qubits, and that “writes down” e_b in 4 auxiliary qubits and e_p in 2 auxiliary qubits. For example, suppose we have the state $|\bar{0}\rangle$. If X_i denotes a bitflip-error on the i -th qubit ($i \in [9]$) and Z_j denotes a phaseflip-error on the j -th qubit (let $j' \in [3]$ denote the number of the block in which qubit j lies). Then after these errors our state is $X_i Z_j |\bar{0}\rangle$. After fresh auxiliary qubits $|0^4\rangle|0^2\rangle$ are added, U maps

$$X_i Z_j |\bar{0}\rangle |0^4\rangle |0^2\rangle \mapsto X_i Z_j |\bar{0}\rangle |i\rangle |j'\rangle.$$

Together, $e_b = i$ and $e_p = j'$ form the “error syndrome”; this tells us which error occurred where. The error-correction procedure can now measure this syndrome in the computational basis, and take corrective action depending on the classical outcomes e_b and e_p : apply an X to qubit e_b (or no X if $e_b = 0$), and apply a Z to one qubit in the e_p -th block (or no Z if $e_p = 0$). The case of a Y -error on the i -th qubit corresponds to the case where $i = j$ (i.e., the i -th qubit is hit by both a phaseflip and a bitflip); our procedure still works in this case. Hence we can perfectly correct one Pauli-error on any one of the 9 codeword qubits.

As we argued before, the ability to correct Pauli-errors suffices to correct all possible errors. Let’s see in more detail how this works. Consider for instance some 9-qubit unitary error E . Assume it can be decomposed as a linear combination of 9-qubit products of Paulis, each having at most one bitflip-error and one phaseflip-error:

$$E = (\alpha_0 I + \sum_{i=1}^9 \alpha_i X_i) (\beta_0 I + \sum_{j=1}^9 \beta_j Z_j).$$

⁴Note that we are not discovering exactly on which of the 9 qubits the phaseflip-error happened (in contrast to the case of bitflips), but that’s OK: we can correct the phaseflip-error by applying a Z -gate to any one of the 3 qubits in the block where the affected qubit sits.

Suppose this error occurs on $|\bar{0}\rangle$:

$$E|\bar{0}\rangle = (\alpha_0 I + \sum_{i=1}^9 \alpha_i X_i)(\beta_0 I + \sum_{j=1}^9 \beta_j Z_j)|\bar{0}\rangle = \sum_{i,j=0}^9 \alpha_i \beta_j X_i Z_j |\bar{0}\rangle,$$

where we denote $X_0 = Y_0 = I$.

If we now add auxiliary qubits $|0^4\rangle|0^2\rangle$ and apply the above unitary U , then we go into a superposition of error syndromes:

$$U(E \otimes I^{\otimes 6})|\bar{0}\rangle|0^4\rangle|0^2\rangle = \sum_{i,j=0}^9 \alpha_i \beta_j X_i Z_j |\bar{0}\rangle|i\rangle|j'\rangle.$$

Measuring the 6 auxiliary qubits will now probabilistically give us one of the syndromes $|i\rangle|j'\rangle$, with $i \in \{0, 1, \dots, 9\}$ and $j' \in \{0, 1, 2, 3\}$, and it will collapse the state to

$$X_i Z_j |\bar{0}\rangle|i\rangle|j'\rangle.$$

In a way, this measurement of the syndrome “discretizes” the continuously many possible errors to the finite set of Pauli-errors. Once the syndrome has been measured, we can apply a corrective X and/or Z to the first 9 qubits to undo the specific error corresponding to the specific syndrome we got as outcome of our measurement. It is also possible that the measurement outcome is $0^4, 0^2$; in that case the state has collapsed to $|\bar{0}\rangle|0^4\rangle|0^2\rangle$, so the syndrome measurement itself already removed the error!

So now we can correct an error on one qubit. To achieve this, however, we have substantially increased the number of locations where such an error could occur: the number of qubits has gone from 1 to 9 (even to 15 if we also count the 6 auxiliary qubits used for the syndrome measurements), and we need a number of time-steps to compute and measure the syndrome, and to correct a detected error. Hence this procedure only gains us something if the error-rate p is so small that the probability of 2 or more errors on the larger encoded system is smaller than the probability of 1 error in the unencoded qubit. We will get back to this issue below, when talking about the threshold theorem. Note also that each new application of the correction-procedure need a new, fresh 6-qubit register initialized to $|0^4\rangle|0^2\rangle$. After one run of the error-correction procedure these auxiliary qubits will contain the measured error syndrome, and we can just discard this. In a way, error correction acts like a refrigerator: a fridge pumps heat out of its system and dumps it into the environment, and error-correction pumps noise out of its system and dumps it in the environment in the form of the discarded auxiliary qubits.

The above 9-qubit code is just one example of a quantum error-correcting code. Better codes exist, and a lot of work has gone into simultaneously optimizing the different parameters: we want to encode a large number of logical qubits into a not-much-larger number of physical qubits, while being able to correct as many errors as possible. The shortest code that encodes one logical qubit and protects against one error, has five physical qubits. There are also “asymptotically good” quantum error-correcting codes; these encode k logical qubits into $O(k)$ physical qubits and can correct errors on a constant fraction of the physical qubits (rather than just an error on one of the qubits).

20.5 Fault-tolerant quantum computation

Encoding a quantum state in a quantum error-correcting code to protect it against noise is good, but not enough: we also need to be able to do *operations* on the encoded qubits (Hadamards, CNOTs, etc.). One way is to decode the logical qubits, do the operation on them, and then re-encode them. This, however, is a recipe for disaster: if an error occurs in the interval between the decoding and subsequent encoding, then we're unprotected and we cannot detect (let alone undo) errors happening during that interval. Accordingly, we need to be able to do operations on the logical qubits *while they are encoded*. Additionally, we need operations for regular stages of error-correction, i.e., measuring the syndrome and then correcting errors based on the outcomes of those measurements. These operations may also introduce errors, and the big worry is that error-correction steps may themselves introduce more errors than they correct.⁵

There is a 7-qubit code due to Steane [232] which is used often because it has some nice properties: a Hadamard on the logical qubit corresponds to $H^{\otimes 7}$ on the physical qubits, and a CNOT between two logical qubits corresponds to applying CNOTs between the 7 pairs of the two blocks of physical qubits (i.e., between the 1st qubit of one block and the 1st qubit of the other block, etc.). Such implementations are called *transversal*. Adding the T -gate ($|b\rangle \mapsto e^{ib\pi/4}|b\rangle$) to H and CNOT would yield a gate-set that suffices for universal quantum computation. Unfortunately, implementing the T -gate fault-tolerantly takes a lot more work, and we won't go into that here (see Exercise 7, though).

When designing schemes for fault-tolerant computing, it is very important to ensure that errors do not spread too quickly. Consider for instance a logical CNOT: if its control-bit is erroneous but its target bit is not, then after doing the CNOT both bits will be erroneous. The trick is to keep the errors on the physical qubits under control in such a way that regular stages of error-correction don't get overwhelmed by the errors. For example, suppose we have a code that is able to correct up to one error in each encoded block (logical qubit); then the implementation of a logical CNOT may convert two encoded blocks where only one physical qubit has an error, into two blocks each of which has a single error, but not to multiple errors within one block, because our code will be able to handle two blocks with one error each but not one block with two errors (this is why the transversal implementation of the CNOT for Steane's code is nice). In addition, we need to be able to fault-tolerantly prepare states, and measure logical qubits in the computational basis. We won't go into the many further details of fault-tolerant quantum computing here.

20.6 Concatenated codes and the threshold theorem

The idea to concatenate a code with itself, described at the end of Section 20.2 for classical codes, also applies to quantum codes as we will sketch now. Suppose we have some code that encodes one qubit into C qubits, suppose that it can correct one error on any one of its C qubits, and uses D time-steps per stage of error-correcting (each time-step may involve a number of elementary gates in parallel). Instead of only 1, we now have CD locations where an error could occur! Assuming error-rate p per-qubit-per-time-step, the probability for the code to fail on a specific logical qubit at a specific time (i.e., to have *more than 1* physical error on its CD locations) is $p' = \sum_{i=2}^{CD} \binom{CD}{i} p^i (1-p)^{CD-i}$. If p is a sufficiently small constant, then this sum is dominated by

⁵It's like being inside a leaky boat on the open seas, using a leaky bucket to scoop out water all the time to prevent the boat from filling up with water and sinking. It's doable, but not easy.

the term for $i = 2$, and we have $p' \approx (CD)^2 p^2$. Accordingly, if the initial error-rate p is below some magical constant $\approx 1/(CD)^2$, then $p' < p$ and hence each level of error-correction reduces the error-rate by a constant factor.

More generally, suppose we concatenate this code k times with itself. Then every “logical qubit” gets encoded into C^k qubits, but (by the same calculation as in Section 20.2) the error-rate for each logical qubit gets reduced to $O((CDp)^{2^k})$. Suppose we want to be able to “survive” $T = \text{poly}(n)$ time-steps without any error on the logical qubits; that is what we would need to run an efficient quantum algorithm on faulty quantum hardware. Then it suffices if we reduce the error rate to $\ll 1/T$, for which $k = O(\log \log T)$ levels of concatenation are enough. These layers of error-correction increase the number of qubits and the computation time by a factor which is exponential in k , but that is still only a polylogarithmic overhead, since $2^{O(\log \log T)} = (\log T)^{O(1)}$.⁶

The above sketch (when implemented precisely) gives us the famous “threshold theorem” [7, 159]: if the initial error-rate of the quantum hardware can be brought down below some magical constant (known as the “fault-tolerance threshold”), then we can use software-solutions like quantum error-correcting codes and fault-tolerant computing to ensure that we can quantum compute for long periods of time without serious errors. Much research has gone into finding the best value for this fault-tolerance threshold. The more efficient our basic quantum error-correcting codes are (i.e., the smaller C and D), the higher (= better) the value of the threshold is. Currently the best rigorous estimates for the threshold are around 0.1%, but there is numerical evidence that even a few percent might be tolerable. This is actually one of the most important results in the area of quantum computing, and is the main answer to the skeptics mentioned at the start of the chapter: as long as experimentalists manage to implement basic operations within a few percent of error in a scalable way, then we should be able to build large-scale quantum computers.⁷ Currently there seems to be no fundamental reason why we cannot do this; it is, however, an extremely hard engineering problem.

Exercises

1. (H) Let E be an arbitrary 1-qubit unitary. We know that it can be written as

$$E = \alpha_0 I + \alpha_1 X + \alpha_2 Y + \alpha_3 Z,$$

for some complex coefficients α_i . Show that $\sum_{i=0}^3 |\alpha_i|^2 = 1$.

2. (a) Write the 1-qubit Hadamard transform H as a linear combination of the four Pauli matrices.
(b) Suppose an H -error happens on the first qubit of $\alpha|\bar{0}\rangle + \beta|\bar{1}\rangle$ using the 9-qubit code. Give the various steps in the error-correction procedure that corrects this error.
3. Give a quantum circuit for the encoding of Shor’s 9-qubit code, i.e., a circuit that maps $|00^8\rangle \mapsto |\bar{0}\rangle$ and $|10^8\rangle \mapsto |\bar{1}\rangle$. Explain why the circuit works.
4. Shor’s 9-qubit code allows to *correct* a bit flip and/or a phase flip on one of its 9 qubits. Below we give a 4-qubit code which allows to *detect* a bitflip and/or a phaseflip. By this we

⁶Recently it was shown that one can even bring the overhead down to $O(1)$ [110].

⁷This is of course assuming our simple model of independent noise on each physical qubit is not too far off; if the noise can be correlated in devious ways it becomes much harder (though often still possible) to protect against.

mean that after the detection procedure we either have the original uncorrupted state back, or we know that an error occurred (though we do not know which one). The logical 0 and 1 are encoded as:

$$|\bar{0}\rangle = \frac{1}{2}(|00\rangle + |11\rangle) \otimes (|00\rangle + |11\rangle)$$

$$|\bar{1}\rangle = \frac{1}{2}(|00\rangle - |11\rangle) \otimes (|00\rangle - |11\rangle)$$

- (a) Give a procedure (either as a circuit or as sufficiently-detailed pseudo-code) that detects a bitflip error on one of the 4 qubits of $\alpha|\bar{0}\rangle + \beta|\bar{1}\rangle$.
 - (b) Give a procedure (either as a circuit or as sufficiently-detailed pseudo-code) that detects a phaseflip error on one of the 4 qubits of $\alpha|\bar{0}\rangle + \beta|\bar{1}\rangle$.
 - (c) Does that mean that we can now detect *any* unitary 1-qubit error on one of the 4 qubits? Explain your answer.
5. (H) Show that there cannot be a quantum code that encodes one logical qubit into $2k$ physical qubits while being able to correct errors on up to k of the physical qubits.
6. Suppose we have a qubit whose density matrix is $\rho = \alpha_0 I + \alpha_1 X + \alpha_2 Y + \alpha_3 Z$, where $\alpha_0, \alpha_1, \alpha_2, \alpha_3$ are real coefficients and I, X, Y, Z are the Pauli matrices.
- (a) Show that $\alpha_0 = 1/2$.
 - (b) Depolarizing noise (of strength $p \in [0, 1]$) acts on a qubit as follows: with probability $1 - p$ nothing happens to the qubit, and with probability p the qubit is replaced by the “completely mixed state” of a qubit, whose density matrix is $I/2$.
Show that depolarizing noise on the above qubit doesn’t change the coefficient α_0 , but shrinks each of $\alpha_1, \alpha_2, \alpha_3$ by a factor of $1 - p$.
7. Suppose we have a qubit $|\phi\rangle = \alpha|0\rangle + \beta|1\rangle$ to which we would like to apply a $T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$ gate, but for some reason we cannot. However, we have a second qubit available in state $\frac{1}{\sqrt{2}}(|0\rangle + e^{i\pi/4}|1\rangle)$, and we can apply a CNOT gate and an $S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$ gate.
- (a) What state do we get if we apply a CNOT to the first and second qubit?
 - (b) Suppose we measure the second qubit in the computational basis. What are the probabilities of outcomes 0 and 1, respectively?
 - (c) Suppose the measurement yields 0. Show how we can get $T|\phi\rangle$ in the first qubit.
 - (d) Suppose the measurement yields 1. Show how we can get $T|\phi\rangle$ in the first qubit, up to an (irrelevant) global phase.

Comment: This way of implementing the T -gate is very helpful in fault-tolerant computing, where often CNOT and S are easy to do on encoded states but T is not. What this exercise shows is that we can prepare (encodings of) the so-called “magic state” $\frac{1}{\sqrt{2}}(|0\rangle + e^{i\pi/4}|1\rangle)$ beforehand (offline, assuming we can store them until we need them), and use those to indirectly implement a T -gate using only CNOT and S -gates.

8. Consider a quantum-error correcting code that encodes k qubits (and $n - k$ $|0\rangle$ s) into an n -qubit codeword state, via the unitary encoding map

$$U : |x, 0^{n-k}\rangle \mapsto |C(x)\rangle, \text{ where } x \in \{0, 1\}^k, \text{ and } |C(x)\rangle \text{ need not be a basis state.}$$

A “weight- w Pauli error” is the tensor product of n Pauli matrices, of which at most w are not identity (e.g., something like $X \otimes I \otimes Z \otimes I \otimes I$ if $w = 2$ and $n = 5$). Suppose that there is a unitary map S on $3n$ qubits that can identify every weight- w Pauli error E on a codeword, by writing the name of E (the “error syndrome,” which we can think of as a $2n$ -bit string “ E ”, for example writing 00 for I , 10 for X , 01 for Z , 11 for Y) in a second register that’s initially 0^{2n} . In other words, for every $x \in \{0, 1\}^k$ and weight- w Pauli error E , this S maps

$$S : (E|C(x)\rangle)|0^{2n}\rangle \mapsto (E|C(x)\rangle)|E\rangle.$$

- (a) Show that if x and y are k -bit strings, and E and F are weight- w Pauli errors, then the n -qubit states $E|C(x)\rangle$ and $F|C(y)\rangle$ are orthogonal unless both $x = y$ and $E = F$.

- (b) Prove the inequality $2^k \sum_{i=0}^w \binom{n}{i} 3^i \leq 2^n$.

Comment: This inequality implies a lower bound on the required number of qubits n , in terms of the number of encoded qubits k and the weight w of errors that you can correct, but you don’t need to derive that consequence.

9. In this exercise we will see that the argument about concatenating the classical 3-bit code at the end of Section 20.2 still works if the initial error rate is $p \leq 1/2 - \varepsilon$ for some $\varepsilon > 0$.

- (a) Define the function f as $f(p) = 3p^2(1 - p) + p^3$. Show that if $p = 1/2 - \varepsilon$ for some $\varepsilon \in [0, 1/6]$, then $f(p) \leq 1/2 - (13/9)\varepsilon$.
- (b) Show that there is an $m = O(\log(1/\varepsilon))$ such that m levels of concatenation reduce the error to $p_m < 1/3$.
- (c) Show that $m + k$ levels of concatenation reduce the error to something exponentially small in 2^k .
- (d) How many bits are used to encode one logical bit in the scheme from (c)?

Appendix A

Some Useful Linear Algebra

In this appendix we quickly introduce the basic elements of linear algebra, most of which will be used somewhere or other in these notes.

A.1 Vector spaces

A *vector space* V over a field $\mathbb{F}$ is a set of objects (called vectors) satisfying that if $v, w \in V$, then $cv + dw \in V$ for all $c, d \in \mathbb{F}$. In other words, V is closed under addition and scalar multiplication. A (linear) *subspace* W is a subset $W \subseteq V$ which is itself a vector space (i.e., closed under addition and scalar multiplication). For example, $V = \mathbb{C}^d$ is the d -dimensional complex vector space, which is the set of all column vectors of d complex numbers. The set $W \subseteq V$ of vectors whose first two entries are 0 is a subspace of V . As another example, the set $V = \{0, 1\}^d$ of d -bit vectors, with entrywise addition modulo 2, is a linear space. The field here is $\mathbb{F}_2 = \{0, 1\}$. The set $W \subseteq V$ of vectors whose first two entries are equal is a subspace of V .

A set of vectors $v_1, \dots, v_m \in V$ is *linearly independent* if the only way to get $\sum_{i=1}^m a_i v_i$ equal to the zero-vector 0, is to set $a_1 = \dots = a_m = 0$. The *span* (over field $\mathbb{F}$) of a set of vectors $S = \{v_1, \dots, v_m\} \subseteq V$ is the set $\text{span}(S)$ of vectors that can be written as a linear combination $\sum_{i=1}^m a_i v_i$ (with coefficients $a_1, \dots, a_m \in \mathbb{F}$). A *basis* for V is a linearly independent set S of vectors such that $\text{span}(S) = V$. One can show that all bases of V have the same size; this size is called the *dimension* of V . If we fix an ordered basis $S = (v_1, \dots, v_m)$, then every $w \in V$ can be written uniquely as a linear combination $\sum_{i=1}^m w_i v_i$, and can also be written (in that basis) as $(w_1, \dots, w_m)$. The *support* of such a w is the set $\{i \mid w_i \neq 0\}$ of locations where w is nonzero. For example, the support of $w = (1, 5, 0, 4)$ is $\{1, 2, 4\}$.

A.2 Matrices

Matrices represent linear maps between two vector spaces with particular bases. We assume familiarity with the basic rules of matrix addition and multiplication. We use A_{ij} for the (i, j) -entry of a matrix A and A^T for its *transpose*, which has $A_{ij}^T = A_{ji}$. We use I_d to denote the $d \times d$ identity matrix, which has 1s on its diagonal and 0s elsewhere; we usually omit the subscript d when the dimension is clear from context. If A is square and there is a matrix B such that $AB = BA = I$, then we use A^{-1} to denote this B , which is called the *inverse* of A (and is unique if it exists). Note that $(AB)^{-1} = B^{-1}A^{-1}$.

In the remainder of this appendix we will mostly consider the complex field. If A is a matrix (not necessarily square), then A^* denotes its *conjugate transpose* (or *adjoint*): the matrix obtained by transposing A and taking the complex conjugates of all entries. Note that $(AB)^* = B^*A^*$. Physicists usually write $A^\dagger$ (pronounced “A-dagger”) instead of A^* , but in these notes we will stick with the A^* notation that is common in mathematics.

A.3 Inner product

For vectors v, w , we use $\langle v|w \rangle = v^*w = \sum_i v_i^* w_i$ for their *inner product*.¹ The combination of the vector space V with this inner product is called a *Hilbert space*. Two vectors v, w are *orthogonal* if $\langle v|w \rangle = 0$. A set $\{v_i\}$ of vectors is called *orthogonal* if all vectors are pairwise orthogonal: $\langle v_i|v_j \rangle = 0$ if $i \neq j$. If additionally the vectors all have norm 1, then the set is called *orthonormal*.

The inner product induces a vector *norm* $\|v\| = \sqrt{\langle v|v \rangle} = \sqrt{\sum_i |v_i|^2}$. This is the usual Euclidean norm (or “length”). The norm in turn induces a *distance* $\|v - w\|$ between vectors v and w . Note that distance and inner product are closely related:

$$\|v - w\|^2 = \langle v - w|v - w \rangle = \|v\|^2 + \|w\|^2 - \langle v|w \rangle - \langle w|v \rangle.$$

In particular, for unit vectors v and w the real part of their inner product equals $1 - \frac{1}{2}\|v - w\|^2$. Hence unit vectors that are close together have an inner product close to 1, and vice versa. The *Cauchy-Schwarz inequality* gives $|\langle v|w \rangle| \leq \|v\| \cdot \|w\|$ (see also Appendix B).

The *outer product* of v and w is the matrix vw^* .

A.4 Unitary matrices

Below we will restrict attention to square matrices, unless explicitly mentioned otherwise.

A matrix A is *unitary* if $A^{-1} = A^*$. The following conditions are equivalent:

1. A is unitary
2. A preserves inner product: $\langle Av|Aw \rangle = \langle v|w \rangle$ for all v, w
3. A preserves norm: $\|Av\| = \|v\|$ for all v
4. $\|Av\| = 1$ if $\|v\| = 1$

(1) implies (2) because if A is unitary then $A^*A = I$, and hence $\langle Av|Aw \rangle = (v^*A^*)Aw = \langle v|w \rangle$. (2) implies (1) as follows: if A is not unitary then $A^*A \neq I$, so then there is a w such that $A^*Aw \neq w$ and, hence, a v such that $\langle v|w \rangle \neq \langle v|A^*Aw \rangle = \langle Av|Aw \rangle$, contradicting (2). Clearly (2) implies (3). Moreover, it is easy to show that (3) implies (2) using the following identity:

$$\|v + w\|^2 = \|v\|^2 + \|w\|^2 + \langle v|w \rangle + \langle w|v \rangle.$$

The equivalence of (3) and (4) is obvious. Note that by (4), the eigenvalues of a unitary matrix have absolute value 1.

¹Here we follow a physics convention: mathematicians usually define $\langle v|w \rangle = vw^*$.

A.5 Diagonalization and singular values

The complex number λ is an *eigenvalue* of (square) matrix A if there is some nonzero vector v (called an *eigenvector*) such that $Av = \lambda v$.

Matrices A and B are *similar* if there is an invertible matrix S such that $A = SBS^{-1}$. Note that if $Av = \lambda v$, then $BS^{-1}v = \lambda S^{-1}v$, so similar matrices have the same eigenvalues. *Schur's lemma* states that every matrix A is similar to an upper triangular matrix: $A = UTU^{-1}$ for some unitary U and upper triangular T . Since similar matrices have the same eigenvalues and the eigenvalues of an upper triangular matrix are exactly its diagonal entries, the eigenvalues of A form the diagonal of T .

A matrix D is *diagonal* if $D_{ij} = 0$ whenever $i \neq j$. Let S be some matrix satisfying $AS = SD$ for some diagonal matrix D . Let v_i be the i -th column of S and λ_i be the i -th entry on the diagonal of D , then

$$\underbrace{\begin{pmatrix} \vdots & & \vdots \\ Av_1 & \cdots & Av_d \\ \vdots & & \vdots \end{pmatrix}}_{AS} = \underbrace{\begin{pmatrix} \vdots & & \vdots \\ \lambda_1 v_1 & \cdots & \lambda_d v_d \\ \vdots & & \vdots \end{pmatrix}}_{SD},$$

and we see that v_i is an eigenvector of A associated with eigenvalue λ_i . Conversely, if $v_1, \dots, v_d$ are eigenvectors of A with eigenvalues $\lambda_1, \dots, \lambda_d$, then we have $AS = SD$, where S has the v_i as columns and D is the diagonal matrix of λ_i . We call a square matrix A *diagonalizable* if it is similar to some diagonal matrix D : $A = SDS^{-1}$. This D then has A 's eigenvalues λ_i on its diagonal, some of which may be zero. Note that A is diagonalizable iff it has a linearly independent set of d eigenvectors. These eigenvectors will form the columns of S , giving $AS = SD$, and linear independence ensures that S has an inverse, giving $A = SDS^{-1}$. A matrix A is *unitarily* diagonalizable iff it can be diagonalized via a unitary matrix U : $A = UDU^{-1}$. If the columns of U are the vectors u_i , and the diagonal entries of D are λ_i , then we can also write $A = \sum_i \lambda_i u_i u_i^*$; this is sometimes called the *spectral decomposition* of A . By the same argument as before, A will be unitarily diagonalizable iff it has an orthonormal set of d eigenvectors.

A matrix A is *normal* if it commutes with its conjugate transpose ($A^*A = AA^*$). For example, unitary matrices are normal. If A is normal and $A = UTU^{-1}$ for some upper triangular T (which must exist because of Schur's lemma), then $T = U^{-1}AU$ and $T^* = U^{-1}A^*U$, so $TT^* = U^{-1}AA^*U = U^{-1}A^*AU = T^*T$. Hence T is normal and upper triangular, which implies (with a little work) that T is diagonal. This shows that normal matrices are unitarily diagonalizable. Conversely, if A is diagonalizable as UDU^{-1} , then $AA^* = UDD^*U^* = UD^*DU^{-1} = A^*A$, so then A is normal. Thus a matrix is normal iff it is unitarily diagonalizable. If A is not normal, it may still be diagonalizable via a non-unitary S , for example:

$$\underbrace{\begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix}}_A = \underbrace{\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}}_S \cdot \underbrace{\begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}}_D \cdot \underbrace{\begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix}}_{S^{-1}}.$$

If $A = UDU^{-1}$ then $A^* = UD^*U^{-1}$, so the eigenvalues of A^* are the complex conjugates of the eigenvalues of A .

An important class of normal (and hence unitarily diagonalizable) matrices are the *Hermitian* matrices, which are the ones satisfying $A = A^*$. Note that the last line of the previous paragraph implies that the eigenvalues of Hermitian matrices are real.

A Hermitian matrix A is called *positive semidefinite* (resp. *positive definite*), if all its eigenvalues are nonnegative (resp. positive). An equivalent definition is that A is positive semidefinite (psd) iff there exists a matrix C such that $A = C^*C$ (in other words, there exist vectors c_i such that for all i, j , we have $A_{ij} = \langle c_i | c_j \rangle$). A useful and easy to prove property is that A is psd iff $\text{Tr}(AB) \geq 0$ for all psd matrices B . By defining $A \succeq B$ iff $A - B$ is positive semidefinite, we obtain a partial ordering on the set of all Hermitian matrices. If all eigenvalues are 0 or 1, then A is called a *projection* (or *projection matrix* or *projector*). This is equivalent to requiring $A^2 = A$.

Not all matrices are diagonalizable, for instance $A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ is not. However, every matrix has a *singular value decomposition* (SVD), which is almost as useful as a diagonalization. We will derive this below for an invertible square matrix A . Since A^*A is psd, we can write $A^*A = VDV^{-1}$ for some unitary V whose columns v_i are the orthonormal eigenvectors of A^*A , and some nonnegative diagonal matrix D with the corresponding eigenvalues. The entries σ_i of the matrix $\Sigma = \sqrt{D}$ are called the *singular values* of A (some of which may be zero). Define vectors $u_i = Av_i/\sigma_i$. Note that the u_i form an orthonormal system because $u_i^*u_j = v_i^*(A^*Av_j)/\sigma_i\sigma_j = v_i^*(\sigma_j^2 v_j)/\sigma_i\sigma_j$, which is 1 if $i = j$ and 0 if $i \neq j$. Hence the matrix U that has these u_i 's as columns is unitary. We have $U = AV\Sigma^{-1}$, so we can write $A = U\Sigma V^{-1}$, which is the SVD of A . Equivalently, we can write $A = \sum_i \sigma_i u_i v_i^*$. This derivation of the SVD $A = U\Sigma V^{-1}$ can easily be extended to arbitrary $m \times n$ matrices A ; U will be an $m \times m$ unitary, Σ will be $m \times n$ (padded with 0s if the rank of A is $< m, n$), and V will be an $n \times n$ unitary.

A.6 Tensor products

If $A = (A_{ij})$ is an $m \times n$ matrix and B an $m' \times n'$ matrix, then their *tensor product* (a.k.a. *Kronecker product*) is the $mm' \times nn'$ matrix

$$A \otimes B = \begin{pmatrix} A_{11}B & \cdots & A_{1n}B \\ A_{21}B & \cdots & A_{2n}B \\ \vdots & \ddots & \vdots \\ A_{m1}B & \cdots & A_{mn}B \end{pmatrix}.$$

For example:

$$\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \otimes \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & 0 & -\frac{1}{\sqrt{2}} & 0 \\ 0 & \frac{1}{\sqrt{2}} & 0 & -\frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} & 0 \end{pmatrix}.$$

Note that the tensor product of two numbers (i.e., 1×1 matrices) is itself just a number, and the tensor product of two column vectors is itself a column vector.

The following properties of the tensor product are easily verified:

- $c(A \otimes B) = (cA) \otimes B = A \otimes (cB)$ for all scalars c
- $(A \otimes B)^* = A^* \otimes B^*$, and similarly for inverse and transpose (note that the order of the tensor factors doesn't change).
- $A \otimes (B + C) = (A \otimes B) + (A \otimes C)$

- $A \otimes (B \otimes C) = (A \otimes B) \otimes C$
- $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$

Different vector spaces can also be combined using tensor products. If V and V' are vector spaces of dimension d and d' with basis $\{v_1, \dots, v_d\}$ and $\{v'_1, \dots, v'_{d'}\}$, respectively, then their tensor product space is the $d \cdot d'$ -dimensional space $W = V \otimes V'$ spanned by $\{v_i \otimes v'_j \mid 1 \leq i \leq d, 1 \leq j \leq d'\}$. Applying a linear operation A to V and B to V' corresponds to applying the tensor product $A \otimes B$ to the tensor product space W .

A.7 Trace

The *trace* of a matrix A is the sum of its diagonal entries: $\text{Tr}(A) = \sum_i A_{ii}$. Some important and easily verified properties of $\text{Tr}(A)$ are:

- $\text{Tr}(A + B) = \text{Tr}(A) + \text{Tr}(B)$
- $\text{Tr}(AB) = \text{Tr}(BA)$, which is known as the “cyclic property” of the trace.
For example, $\text{Tr}(Avv^*) = v^*Av$.
- $\text{Tr}(A)$ is the sum of the eigenvalues of A .
(This follows from Schur and the previous item: $\text{Tr}(A) = \text{Tr}(UTU^{-1}) = \text{Tr}(U^{-1}UT) = \text{Tr}(T) = \sum_i \lambda_i$)
- $\text{Tr}(A \otimes B) = \text{Tr}(A)\text{Tr}(B)$

A.8 Rank

The *rank* of a matrix A (over a field $\mathbb{F}$) is the size of a largest linearly independent set of rows of A (linear independence taken over $\mathbb{F}$). Unless mentioned otherwise, we take $\mathbb{F}$ to be the field of complex numbers. We say that A has *full rank* if its rank equals its dimension. The following properties are all easy to show:

- $\text{rank}(A) = \text{rank}(A^*)$
- $\text{rank}(A)$ equals the number of nonzero eigenvalues of A (counting multiplicity)
- $\text{rank}(A + B) \leq \text{rank}(A) + \text{rank}(B)$
- $\text{rank}(AB) \leq \min\{\text{rank}(A), \text{rank}(B)\}$
- $\text{rank}(A \otimes B) = \text{rank}(A) \cdot \text{rank}(B)$
- A has an inverse iff A has full rank

A.9 The Pauli matrices

The four *Pauli matrices* are:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \text{and} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Note that each Pauli matrix P is both unitary and Hermitian, and hence self-inverse: $P^{-1} = P$. This implies that their eigenvalues are in $\{-1, 1\}$. Non-identity Paulis anti-commute: if $P, Q \in \{X, Y, Z\}$ are distinct then $PQ = -QP$. Note that $Y = iXZ$. Also, products of two distinct Pauli matrices have trace 0.

Define the *Hilbert-Schmidt* inner product on the space of $d \times d$ matrices as $\langle A, B \rangle = \frac{1}{d} \text{Tr}(A^* B)$. With respect to this inner product (for $d = 2$), the four Pauli matrices form an orthonormal set. This implies that every complex 2×2 matrix A can be written as a linear combination of the Pauli matrices:

$$A = \alpha_0 I + \alpha_1 X + \alpha_2 Y + \alpha_3 Z,$$

with complex coefficients α_i . If A is Hermitian, then these coefficients will be real.

We can also consider the n -qubit Paulis, which are n -fold tensor products of the above 2×2 Paulis. For example $X \otimes Z \otimes I \otimes Y \otimes Z$ is a 5-qubit Pauli. There are 4^n n -qubit Paulis, since we have 4 possibilities for each of the n tensor factors, and these 4^n matrices form an orthonormal set w.r.t. Hilbert-Schmidt inner product. Accordingly, every $2^n \times 2^n$ matrix A can be written uniquely as a linear combination of the 4^n n -qubit Paulis. Again, if A is Hermitian, then the 4^n coefficients will be real.

A.10 Dirac notation

Physicists often write their linear algebra in *Dirac notation*, and we will follow that custom for denoting quantum states. In this notation we write $|v\rangle = v$ and $\langle v| = v^*$. The first is called a *ket*, the second a *bra*. Some points about this notation:

- $\langle v|w\rangle = \langle v||w\rangle$: inner products are bra-ket (“bracket”) products.
- If matrix A is unitarily diagonalizable, then $A = \sum_i \lambda_i |v_i\rangle\langle v_i|$ for some orthonormal set of eigenvectors $\{v_i\}$.
- $|v\rangle\langle v| \otimes |w\rangle\langle w| = (|v\rangle \otimes |w\rangle)(\langle v| \otimes \langle w|)$, the latter is often abbreviated to $|v\rangle \otimes |w\rangle\langle v| \otimes \langle w|$. Abbreviating the latter further by omitting the tensor product leads to dangerous ambiguity, though sometimes it’s still clear from context.
- $(U|v\rangle)^* = \langle v|U^*$ and $(|u\rangle \otimes |v\rangle)^* = \langle u| \otimes \langle v|$ (the ordering of tensor factors doesn’t change).
- Don’t write kets inside of other kets or bras: the notation $\langle u|(\alpha|v\rangle + \beta|w\rangle)\rangle$ doesn’t really make sense.

Appendix B

Some other Useful Math and CS

Here we collect various basic but useful facts and definitions needed in parts of the lecture notes.

B.1 Some notation, equalities and inequalities

- We use $[n]$ to denote the set $\{1, \dots, n\}$, and $\delta_{a,b} \in \{0, 1\}$ to indicate whether $a = b$ or not. If P is a statement which can be true or false, then $[P] \in \{0, 1\}$ denotes its truth value. Logarithms will always be to base 2 unless stated otherwise.
- A *complex number* is of the form $c = a + bi$, where $a, b \in \mathbb{R}$, and i is the imaginary unit, which satisfies $i^2 = -1$. Such a c can also be written as $c = re^{i\phi}$ where $r = |c| = \sqrt{a^2 + b^2}$ is the *magnitude* (a.k.a. *modulus* or *norm*) of c , and $\phi \in [0, 2\pi)$ is the angle that c makes with the positive horizontal axis when we view it as a point (a, b) in the plane. Note that complex numbers of magnitude 1 lie on the unit circle in this plane. We can also write those as $e^{i\phi} = \cos(\phi) + i \sin(\phi)$. The complex conjugate c^* is $a - ib$, equivalently $c^* = re^{-i\phi}$.
- The Cauchy-Schwarz inequality: for $a = (a_1, \dots, a_n) \in \mathbb{C}^n$ and $b = (b_1, \dots, b_n) \in \mathbb{C}^n$

$$\left| \sum_{i=1}^n a_i^* b_i \right| \leq \sqrt{\sum_{i=1}^n |a_i|^2} \sqrt{\sum_{i=1}^n |b_i|^2}.$$

Equivalently, written in terms of inner products and norms of vectors: $|\langle a|b \rangle| \leq \|a\| \cdot \|b\|$.
Proof for the case with real entries: for every real λ we have $0 \leq \langle a - \lambda b | a - \lambda b \rangle = \|a\|^2 + \lambda^2 \|b\|^2 - 2\lambda \langle a|b \rangle$.
Now set $\lambda = \|a\|/\|b\|$ and rearrange (a slightly more complicated proof works if $a, b \in \mathbb{C}^n$).

- Geometric sum: $\sum_{j=0}^{m-1} z^j = \begin{cases} m & \text{if } z = 1 \\ \frac{1-z^m}{1-z} & \text{if } z \neq 1 \end{cases}$

Proof: The case $z = 1$ is obvious; for the case $z \neq 1$, observe $(1-z)(\sum_{j=0}^{m-1} z^j) = \sum_{j=0}^{m-1} z^j - \sum_{j=1}^m z^j = 1 - z^m$.
For example, if $z = e^{2\pi i r/N}$ is a root of unity, with r an integer in $\{1, \dots, N-1\}$, then $\sum_{j=0}^{N-1} z^j = \frac{1-e^{2\pi i r}}{1-e^{2\pi i r/N}} = 0$.

- The ratio in the previous line can be rewritten using the identity $|1 - e^{i\theta}| = 2|\sin(\theta/2)|$; this identity can be seen by drawing the numbers 1 and $e^{i\theta}$ as vectors from the origin in the

complex plane, and dividing their angle θ in two. Some other useful trigonometric identities: $\cos(\theta)^2 + \sin(\theta)^2 = 1$, $\sin(2\theta) = 2 \sin(\theta) \cos(\theta)$.

- $1 + x \leq e^x$ for all real numbers x (positive as well as negative).

- If $\varepsilon_j \in [0, 1]$ then $1 - \sum_{j=1}^k \varepsilon_j \leq \prod_{j=1}^k (1 - \varepsilon_j) \leq e^{-\sum_{j=1}^k \varepsilon_j}$.

Proof: The upper bound comes from the preceding item. The lower bound follows easily by induction on k , using the fact that $(1 - \varepsilon_1)(1 - \varepsilon_2) = 1 - \varepsilon_1 - \varepsilon_2 + \varepsilon_1 \varepsilon_2 \geq 1 - \varepsilon_1 - \varepsilon_2$.

B.2 Algorithms and probabilities

- When we do not care about constant factors, we'll often use big-Oh notation: $T(n) = O(f(n))$ means there exist constants $c, n_0 \geq 0$ such that for all integers $n \geq n_0$, we have $T(n) \leq cf(n)$. Similarly, big-Omega notation is used for lower bounds: $T(n) = \Omega(f(n))$ means there exist constants $c, n_0 \geq 0$ such that $T(n) \geq cf(n)$ for all $n \geq n_0$. $T(n) = \Theta(f(n))$ means that simultaneously $T(n) = O(f(n))$ and $T(n) = \Omega(f(n))$. Such notation is often used to write upper and/or lower bounds on the running time of algorithms as a function of their input length n .

- For $N = 2^n$, we can identify the integers $\{0, \dots, N-1\}$ with their n -bit binary representations as follows: the bitstring $x = x_{n-1} \dots x_1 x_0 \in \{0, 1\}^n$ corresponds to the integer $\sum_{i=0}^{n-1} x_i 2^i$. The leftmost bit x_{n-1} is called the *most significant bit* of x (since it corresponds to the largest power of two, 2^{n-1}), and the rightmost bit x_0 is its *least significant bit* (it corresponds to $2^0 = 1$, so determines whether x is an even or odd integer). For example, if $n = 3$ then the bitstring $x = x_2 x_1 x_0 = 101$ corresponds to the integer $x_2 \cdot 4 + x_1 \cdot 2 + x_0 \cdot 1 = 4 + 1 = 5$. The integer 0 corresponds to the bitstring 0^n (if we use 0 to denote a bitstring of 0s, then the value of n should be clear from context).

We can also use binary notation for non-integral numbers, with the bits to the right of the decimal dot corresponding to negative powers of two ($1/2$, $1/4$, $1/8$, etc.). For example, 0.1 denotes $1/2$ and 10.101 denotes $2 + 1/2 + 1/8 = 21/8$. Note that multiplying by two corresponds to shifting the dot to the right, and dividing corresponds to shifting to the left.

- The *union bound* says that the probability of the union (or logical “or”) of two events is at most the sum of their probabilities: $\Pr[A \vee B] \leq \Pr[A] + \Pr[B]$. This inequality should be obvious from drawing a Venn diagram. More generally, if we have T events $A_1, \dots, A_T$, then $\Pr[A_1 \vee \dots \vee A_T] \leq \sum_{i=1}^T \Pr[A_i]$.
- A (discrete) *random variable* X is an object that takes value x_i with probability p_i . Its *expected value* (or *expectation*) is $\mu = \mathbb{E}[X] = \sum_i p_i x_i$. Its *variance* is $\sigma^2 = \text{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$. Its *standard deviation* is σ .
- *Linearity of expectation* says that, for values $a_1, \dots, a_m$ and random variables $X_1, \dots, X_m$, we have $\mathbb{E}[\sum_{j=1}^m a_j X_j] = \sum_{j=1}^m a_j \mathbb{E}[X_j]$ (which is easy to verify).
- Random variable X is *independent* from random variable Y , if the value of Y does not affect the probability distribution of X , i.e., $\Pr[X = x] = \Pr[X = x \mid Y = y]$ for all possible values x, y . If X and Y are independent, then $\mathbb{E}[X \cdot Y] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$.

- Three basic upper bounds on the tails of probability distributions:

Markov: if X is a nonnegative random variable with expectation μ , then $\Pr[X \geq k\mu] \leq 1/k$.

Proof: Since X is nonnegative, $\mu \geq \Pr[X \geq k\mu] \cdot k\mu$.

Chebyshev: if X is a random variable with expectation μ and standard deviation σ , then $\Pr[|X - \mu| \geq k\sigma] \leq 1/k^2$.

Proof: Apply Markov to the random variable $|X - \mu|^2$, whose expectation is σ^2 .

Chernoff/Hoeffding: if $X = \sum_{i=1}^n X_i$ is the sum of n independent, identically distributed random variables $X_i \in \{0, 1\}$, each with expectation $\Pr[X_i = 1] = p$, then X has expectation $\mu = np$, and exponentially decreasing tail bound $\Pr[|X - \mu| \geq \alpha n] \leq 2e^{-2\alpha^2 n}$.

Proof idea: For all parameters λ , we have $\Pr[X - \mu \geq t] = \Pr[e^{\lambda X} \geq e^{\lambda(t+\mu)}]$. Upper bound the latter probability by applying Markov to the random variable $e^{\lambda X}$. This is a product of n independent random variables $e^{\lambda X_i}$, so its expectation is easy to analyze. Then choose λ to minimize the upper bound.

- A *randomized* algorithm is a classical algorithm that can flip random coins during its operation, meaning its behavior is partially determined by chance and its output is a random variable that depends on its input, rather than a deterministic function of its input. One can think of a randomized algorithm as a probability distribution over deterministic algorithms (one deterministic algorithm for each setting of the coins).
- When we say a (randomized or quantum) algorithm has error probability $\leq 1/3$, this typically means *in the worst case*: for every possible input, the algorithm produces the correct answer with probability $\geq 2/3$, where probability is taken over the random coin flips and/or quantum measurements during its operation. Such statements do not refer to “most” inputs under some distribution unless stated explicitly.
- If a (randomized or quantum) algorithm produces the correct answer in *expected* running time T (meaning *for each* input its expected running time is $\leq T$), then we can convert that into an algorithm with *worst-case* running time $3T$ and error probability $\leq 1/3$, as follows. Run the original algorithm for $3T$ steps, and just cut it off if it hasn’t terminated by itself. The probability of non-termination within $3T$ steps is at most $1/3$ by Markov’s inequality. Hence with probability $\geq 2/3$ we will have the correct answer.
- The union bound is very useful for analyzing an algorithm with T randomized subroutines, each of which has its own small failure probability $\leq 1/(3T)$. If these failure events are independent, then the probability that none of the T subroutines fails is at least $(1 - 1/(3T))^T \geq 1 - T/(3T) = 2/3$ (the inequality uses the last bullet of B.1). But what if these events are dependent on each other, which may well happen if the subroutines depend on what happens elsewhere in the algorithm? Then by the union bound the probability that there is at least one subroutine that fails is at most $T \cdot 1/(3T) \leq 1/3$. In other words, the probability that none of the T subroutines fails is still $\geq 2/3$.
- If a (classical or quantum) algorithm with 0/1-outputs has error probability $\leq 1/3$, then we can cheaply reduce this error probability to small $\varepsilon > 0$, as follows. Choose odd $n = O(\log(1/\varepsilon))$ such that $2e^{-2\alpha^2 n} \leq \varepsilon$ for $\alpha = 1/6$. Run the original algorithm n times and output the majority among the n output bits. The probability that this majority is wrong (i.e., that the number of correct output bits is more than αn below its expectation), is at most ε by the Chernoff bound. Hence we output the correct answer with probability $\geq 1 - \varepsilon$.

Appendix C

Hints for Exercises

Chapter 1

- 9. Find the maximal p such that $I - 2p|-\rangle\langle-| - 2p|1\rangle\langle1|$ is still psd.
- 10. Consider what U has to do when $|\phi\rangle = |0\rangle$, when $|\phi\rangle = |1\rangle$, and when $|\phi\rangle$ is a superposition of these two.
- 13.b. Use the facts that $\text{Tr}(D|\psi\rangle\langle\psi|) = \langle\psi|D|\psi\rangle$ and that products of 2 distinct Paulis have trace 0. This exercise is just superdense coding in disguise.

Chapter 2

- 6. Use Exercise 5.
- 7. Instead of measuring the qubit, apply a CNOT that “copies” it to a new $|0\rangle$ -qubit, which is then left alone until the end of the computation. Analyze what happens.
- 13. Use the Bernstein-Vazirani algorithm.

Chapter 3

- 6.c. Approximate the state of part (a) using the subroutine of part (b), and see what happens if you apply Hadamards to the approximate state. Use the fact that $\frac{1}{2^N} \sum_{w=0}^{N/2+2\sqrt{N}} \binom{N}{w}$ is nearly 1, because this is the probability that if you flip N fair coins, $N/2 - 2\sqrt{N}$ or more of them come up “heads.”

Chapter 4

- 3. Use $|\alpha_i^2 - \beta_i^2| = |\alpha_i - \beta_i| \cdot |\alpha_i + \beta_i|$ and the Cauchy-Schwarz inequality.
- 4.e. Use triangle inequality.
- 4.f. Drop all phase-gates with small angles $\phi < 1/n^3$ from the $O(n^2)$ -gate circuit for F_{2^n} explained in Section 4.5. Calculate how many gates are left in the circuit, and analyze the distance between the unitaries corresponding to the new circuit and the original circuit.

Chapter 5

- 1.a. You may invoke here (without proof) the Schönhage-Strassen algorithm for fast multiplication [218, 160]. This allows you to multiply two n -bit integers mod N using $O(n \log(n) \log \log(n))$

steps (where $n = \lceil \log N \rceil$).¹

3.a. The prime number theorem implies that $\Omega(N/\ln N)$ of the numbers between 1 and N are prime; also there is an efficient classical algorithm to test if a given number is prime [5]. You may use these facts, but be explicit in how many bits your primes p and q will have.

3.b. Use the result of Exercise 1 (no need to rederive that here).

3.c. The set of all possible messages forms a group of size $\phi(N)$. Euler's Theorem says that in any group G , we have $a^{|G|} = 1$ for all $a \in G$ (here '1' is the identity element in the group).

Chapter 6

4.b. For $M = \sum_{i=1}^K E_i$, show that $\|M\|^2 = \|M^2\| \leq \frac{2}{3}\|M\| + \text{a small constant}$.

5. You could use the SWAP-test from Section 16.6.

Chapter 7

4.b. Recall that if there are $i > 0$ solutions, then one variant of Grover's algorithm finds a solution using an expected number of $O(\sqrt{N/i})$ queries.

5.e. Choose γ in (d) such that applying $\lceil \tilde{k} \rceil$ rounds of amplitude amplification to $\mathcal{A}$ results in a solution for y with probability 1.

6.a. Try running the exact version of Grover (see end of Section 7.2) with different guesses for what the actual t is.

7.a. Run the basic Grover search with a cleverly chosen number of iterations.

7.b. Use binary search on top of (a).

8.d. The eigenvalues of a 2-dimensional rotation matrix over angle λ are $e^{i\lambda}$ and $e^{-i\lambda}$ (you don't need to prove this). You may also assume $a \ll 1$, so that the differences between $\sqrt{a}$, $\sin \sqrt{a}$, $\arcsin \sqrt{a}$ are negligible. You may refer to the lecture notes for phase estimation without further proof, incl. the fact that phase estimation gives a good n -bit approximation with high probability even if the phase cannot be represented exactly with n bits of precision.

8.e. Define an A that involves one query to x and where $a = t/N$. Invoke (d) with ε proportional to $1/\sqrt{N}$. You can use $|\tilde{a} - a| = |\sqrt{\tilde{a}} - \sqrt{a}| \cdot |\sqrt{\tilde{a}} + \sqrt{a}|$ in your analysis of the approximation error.

9.b. Combine amplitude amplification with the algorithm of (a), for a smart choice of k . Your answer may refer to the lecture notes for the details of amplitude amplification.

10. Start with $m = x_i$ for a random i , and repeatedly use Grover's algorithm to find an index j such that $x_j < m$ and update $m = x_j$. Continue this until you can find no element smaller than m , and analyze the number of queries of this algorithm. You are allowed to argue about this algorithm on a high level (i.e., things like "use Grover to search for a j such that..." are OK), no need to write out complete circuits. You do, however, have to take into account that the various runs of Grover each have their own error probability

11.b. What is the probability in (a) if you set s to roughly $\sqrt{N}$?

11.c. Choose a set S of size $s = O(N^{1/3})$, and classically query all its elements. First check if S contains a collision. If yes, then you're done. If not, then use Grover to find a $j \notin S$ that collides with an $i \in S$.

¹Shor used the Schönage-Strassen algorithm in his original paper. We could also invoke the more recent improvement of Harvey and van der Hoeven [135], who remove the $\log \log n$ factor.

Chapter 8

- 4.a. Choose a uniformly random vector $v \in \{0,1\}^n$, calculate ABv and Cv , and check whether these two vectors are the same.
- 4.b. Consider the case where A is the all-0 matrix.
- 4.c. Modify the algorithm for collision-finding: use a quantum walk on the Johnson graph $J(n,r)$, where each vertex corresponds to a set $R \subseteq [n]$, and that vertex is marked if there are $i, j \in R$ such that $(AB)_{i,j} \neq C_{i,j}$. Optimize over r .
- 5.b. There's no need to use the C, U, S -framework of the chapter here; the answer is much simpler. View the $3n$ -step random walk algorithm as a deterministic algorithm with an additional input $r \in \{0,1\}^n \times \{1,2,3\}^{3n}$, where the first n bits determine x , and the last $3n$ entries determine which variable of the leftmost false clauses will be flipped in the $3n$ steps of the random walk. Use Grover search on the space of all possible r , or amplitude amplification (no need to write out complete circuits here).

Chapter 9

- 3. Use induction on m , and the fact that there exists a constant c such that for A, B of small norm we have $e^{A+B} = e^A e^B + E$ for some E of norm $\|E\| \leq c\|A\| \cdot \|B\|$.
- 6. Calculate the subnormalized second-register state $(\langle 0| \otimes I)(W^{-1} \otimes I)V(W \otimes I)|0\rangle|\psi\rangle$.
- 8.d. Like in the analysis of Grover's algorithm and regular amplitude amplification (Chapter 7), the product of two reflections on $\mathcal{S}$ is a rotation of $\mathcal{S}$.
- 9. Use triangle inequality, $\|H\| \leq 1$, and the fact that $k! \geq (k/e)^k$.
- 10.b. W_2 just implements a rotation on the first qubit, by an angle that depends on A_{kj} . If you have a basis state $|0\rangle|a\rangle$ where $a \in [0,1]$ is some real number written in some fixed finite number of bits, then you can rotate the first qubit to $\sqrt{a}|0\rangle + \sqrt{1-a}|1\rangle$ by a small circuit that does some single-qubit gates on the first qubit conditioned on the bits in the $|a\rangle$ -part. That circuit is the same for all values of a , so it's independent of the particular $|0\rangle|a\rangle$ you're acting on. You may just assume you can do this circuit, without writing out its details.
- 11.d. Note that the computational basis states $|x\rangle$ are the eigenstates of P and hence also of U , so the only thing you need to do is multiply with the right phases for them.
- 11.e. By conjugating with single-qubit gates you can change non- Z Paulis to Z s, in order to reduce to the case of (d).

Chapter 10

No hints for this chapter, sorry!

Chapter 11

- 4.a. Use Exercise 2.
- 4.b. Show that the symmetrized approximate polynomial r induced by the algorithm has degree at least N .
- 6.c. Use the result of Exercise 5 for $N = 2$.
- 7.b. When defining the relation R , consider that the hardest task for this algorithm is to distinguish inputs of weight $N/2$ from inputs of weight $N/2 + 1$.
- 9.b. Consider the Boolean-valued problem of distinguishing the inputs where 0 sits at an odd

location i in the string x from those where 0 sits at an even location.

10. Show how you can use sorting to solve the Majority-problem and then use the lower bound from Exercise 7 to get an $\Omega(N)$ lower bound on sorting. (It is actually known that sorting takes $\Omega(N \log N)$ comparisons even on a quantum computer, but you don't have to show that.)

11.a. Reduce the $bs(f)$ -bit OR function (restricted to inputs of weight 0 or 1) to f and invoke the lower bound that we know for OR.

12.b. Use induction on T and triangle inequality.

12.d. Add up the inequalities of (b) and (c) over all i , and use the Cauchy-Schwarz inequality.

13.b. Compare the expected value of a monomial of degree $\leq 2T$ under distributions U and D , and then use the fact that a polynomial is a sum of monomials.

Chapter 12

2.a. Use that $I = P_0 P_0 + P_1 P_1$.

2.b. Note that $t = T$ here, so we're considering the final states of an algorithm that, for every input x , outputs the wrong value $1 - f(x)$ with probability $\leq \varepsilon$.

2.c. Use (a), the fact that $P_{f(x)} P_{1-f(y)} + P_{f(y)} P_{1-f(x)} = I$ whenever $f(x) \neq f(y)$, and the fact that $\Gamma_{xy} = 0$ whenever $f(x) = f(y)$.

2.d. Cauchy-Schwarz and the definition of operator norm imply $|\langle \phi | M | \psi \rangle| \leq \|\phi\| \cdot \|M | \psi \rangle\| \leq \|\phi\| \cdot \|M\| \cdot \|\psi\|$ for all matrices M and vectors $|\phi\rangle, |\psi\rangle$.

3.b. Note that the $|\phi_i^t\rangle$'s are pairwise orthogonal due to having a different basis state in their query register.

3.c. Show and use that $I - O_{x,\pm} O_{y,\pm} = 2 \sum_{i: x_i \neq y_i} P_i$. Also use that $(\Gamma_i)_{xy} = 0$ whenever $x_i = y_i$.

3.d. Cauchy-Schwarz and the definition of operator norm imply $|\langle \phi_i^t | (I \otimes \Gamma_i) | \phi_i^t \rangle| \leq \|I \otimes \Gamma_i\| \cdot \|\phi_i^t\|^2$.

4. Use that X and $C - \sum_{i=1}^m y_i A_i$ are both psd.

Chapter 13

1. Use binary search, running the algorithm with different choices of k to “zoom in” on the largest prime factor.

3.a. Use the last item of Appendix B.2 to make the error probability exponentially small.

3.c. Use the error analysis of Exercise 4.4.

4.a. Write $|\theta_x\rangle = \alpha|0\rangle|\phi_0\rangle + \beta|1\rangle|\phi_1\rangle$, and consider the inner product between $(Z \otimes I)|\theta_x\rangle$ and $|\theta_x\rangle$.

4.b. Use part (a). Analyze the amplitude of $|x, 0^{S-n}\rangle$ in the final state $|\psi_x\rangle$, using ideas from the proof of $\mathbf{BQP} \subseteq \mathbf{PSPACE}$ in Section 13.3. Note that in contrast to that proof, you cannot use more than polynomial time for this exercise.

Chapter 14

3. Use binary search with different values of a, b to zoom in on the right value.

5.b. Argue about the penalty given by H_{clock} , which can't be larger than $\lambda_{\min}$.

5.d. Use $\langle \psi'' | H | \psi'' \rangle = \left\| \sqrt{H} | \psi'' \rangle \right\|^2$ and triangle inequality.

5.e. Use Eq. (14.3).

5.f. Use (e) and Cauchy-Schwarz.

5.g. Sum (f) over all $(T+1)^2$ pairs t, t' to lower bound $\langle \psi' | \psi'' \rangle$.

5.j. Once all three simplifying assumptions have been satisfied, we can invoke the energy lower

bound of $\Omega(1/T^2)$ proved in Section 14.3.1.

6.a. Invoke the Marriott-Watrous result mentioned at the end of Section 14.1 (without proving it).

7. Combine ideas from Exercises 6.b and 13.4.

8.a. Note that from the description of H you can infer the circuit $C_n = U_T \cdots U_1$ for n -bit instances, and you can then run $U_t \cdots U_1$ in a controlled manner, for every $t \in \{0, \dots, T\}$ of your choice.

Chapter 15

2.a. It suffices to use pure states with real amplitudes as encoding. Try to “spread out” the 4 encodings $|\phi_{00}\rangle, |\phi_{01}\rangle, |\phi_{10}\rangle, |\phi_{11}\rangle$ in the 2-dimensional real plane as well as possible.

3. Use the fact that 1 classical bit of communication can only send 1 bit of information, no matter how much entanglement Alice and Bob share. Combine this fact with superdense coding.

6.b. Think of the first n qubits as Alice and the last n qubits as Bob; use Holevo’s theorem.

7.a. Consider the positive and negative eigenvalues in the spectral decomposition of $\rho_0 - \rho_1$, and analyze the success probability minus the error probability.

Chapter 16

1. Argue that if Alice sends the same message for distinct inputs x and x' , then Bob doesn’t know what to output if his input is $y = x$.

2.a. Argue that if P is a projector then we can’t have both $P|\phi\rangle = |\phi\rangle$ and $P|\psi\rangle = 0$.

2.c. Observe that among Alice’s possible n -bit inputs are the n codewords of the Hadamard code that encodes $\log n$ bits (see Section 15.3); each pair of distinct Hadamard codewords is at Hamming distance exactly $n/2$. Use part (a) to argue that Alice needs to send pairwise orthogonal states for those n inputs, and hence her message-space must have dimension at least n .

3. Use the fact that 2 non-orthogonal states cannot be distinguished perfectly (Exercise 2), and that a set of 2^n vectors that are pairwise orthogonal must have dimension 2^n .

5. Invoke the quantum random access lower bound, Theorem 4 of Section 15.2.

6. Partition the n positions into disjoint sets and run (in parallel) a separate r -message intersection protocol for each of these sets.

8.b. Let Alice send a random row of $C(x)$ (with the row-index) and let Bob send a random column of $C(y)$ (with the column-index).

9.a. Two distinct polynomials, each of degree $\leq d$, are equal on at most d points of the domain $\mathbb{F}_p$.

10.b. Run the protocol of part (a) on an initial state where Bob has a well-chosen superposition over many $|y\rangle$.

11.b. You can derive this from one of the communication lower bounds mentioned in this chapter, you don’t need to prove this from scratch.

12. The matching M induces a projective measurement that Bob can do on the message he receives.

13.d. Alice could send a uniform superposition over all $h \in H$.

Chapter 17

1.b. You could write this out, but you can also get the answer almost immediately from part (a) and the fact that $H^T = H^{-1}$.

2.b. It’s helpful here to write the EPR-pair in the basis $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$.

4. For every fixed input x, y , there is a classical strategy that gives a wrong output only on that

input, and that gives a correct output on all other possible inputs. Use the shared randomness to randomly choose one of those deterministic strategies.

6.b. Argue that $\frac{1}{4}\langle\psi|C|\psi\rangle = \Pr[\text{win}] - \Pr[\text{lose}]$.

6.c. Use that A_x^2 and B_y^2 are the k -qubit identity matrix.

6.d. Use Cauchy-Schwarz to show $(\langle\psi|C|\psi\rangle)^2 \leq \langle\psi|C^2|\psi\rangle$, and then upper bound the latter.

6.e. $\cos(\pi/8)^2 = \frac{1}{2} + \frac{1}{\sqrt{8}}$.

Chapter 18

6. Use the encoding of Exercise 5, so that Alice and Bob need to cooperate to learn the key used to change ρ .

7. Show that a unitary on Alice's side of the state won't change Bob's local density matrix ρ_B .

9.a. The singular value decomposition (see end of Appendix A.5) of the $d \times d$ matrix M whose entries are $M_{ij} = \alpha_{ij}$ can be computed in polynomial time, you may assume this without proof.

Chapter 19

6.a. Start with a uniform superposition over all $x \in \{0, 1\}^n$ and end with an inverse QFT. You're allowed to use a unitary like $|c\rangle \mapsto e^{2\pi ic} |c\rangle$ since it does not depend on f .

7.a. You can diagonalize V by something like a Hadamard gate on the "middle two" basis states, $|01\rangle$ and $|10\rangle$.

7.b. W consists of k 2-qubit SWAP-gates.

7.c. It's helpful to write $U^{i\rho\eta} = I + i\rho\eta + F$ for some matrix F with $\|F\|_1 = O(\eta^2)$. This follows from Taylor series and the fact that ρ has trace 1, you don't need to prove this. Here the trace norm $\|A\|_1$ of a matrix A is defined as the sum of A 's singular values.

7.d. You can first prove this for the case where $\sigma = |a\rangle\langle a|$ and $\rho = |b\rangle\langle b|$ are pure states, and then extend to general mixed states by linearity.

7.f. Apply part (d) $r = O(t^2/\varepsilon)$ times with $\eta = O(\varepsilon/t)$, choosing the constants in the $O(\cdot)$ such that $r\eta = t$ and hence $(U^\eta)^r = U^t$. Upper bound the overall error using triangle inequality.

8.a. Use the SWAP-test from Section 16.6. " $O(1)$ given copies" means you are allowed to use any number of copies of $|\phi\rangle$ and $|\psi\rangle$, as long as that number is independent of n . You may count the 3-qubit gate which is the controlled SWAP of a pair of qubits as an elementary gate here.

Chapter 20

1. Compute the trace $\text{Tr}(E^*E)$ in two ways, and use the fact that $\text{Tr}(AB) = 0$ if A and B are distinct Paulis, and $\text{Tr}(AB) = \text{Tr}(I) = 2$ if A and B are the same Pauli.

5. Given an unknown qubit $\alpha|0\rangle + \beta|1\rangle$ encoded using this code, you could split the $2k$ qubits into two sets of k qubits each, and use each to recover a copy of the unknown qubit.

Bibliography

- [1] S. Aaronson. The learnability of quantum states. *Proceedings of the Royal Society of London*, 463(2088), 2007. quant-ph/0608142.
- [2] S. Aaronson. Quantum machine learning algorithms: Read the fine print. *Nature Physics*, 11(4):291–293, April 2015.
- [3] S. Aaronson and A. Ambainis. Quantum search of spatial regions. *Theory of Computing*, 1(1):47–79, 2005. Earlier version in FOCS’03. quant-ph/0303041.
- [4] S. Aaronson and Y. Shi. Quantum lower bounds for the collision and the element distinctness problems. *Journal of the ACM*, 51(4):595–605, 2004.
- [5] M. Agrawal, N. Kayal, and N. Saxena. PRIMES is in P. *Annals of Mathematics*, 160(2):781–793, 2004.
- [6] D. Aharonov, I. Arad, and T. Vidick. Guest column: The quantum PCP conjecture. *ACM SIGACT News*, 44(2):47–79, 2013. arXiv:1309.7495.
- [7] D. Aharonov and M. Ben-Or. Fault tolerant quantum computation with constant error rate. *SIAM Journal on Computing*, 38(4):1207–1282, 2008. Earlier version in STOC’97. quant-ph/9611025.
- [8] D. Aharonov and T. Naveh. Quantum NP - a survey, 2002. quant-ph/0210077.
- [9] J. Allcock and C-Y. Hsieh. A quantum extension of SVM-perf for training nonlinear SVMs in almost linear time. *Quantum*, 4:342, 2020. arXiv:2006.10299.
- [10] O. Alrabiah, V. Guruswami, P. Kothari, and P. Manohar. A near-cubic lower bound for 3-query locally decodable codes from semirandom CSP refutation. Technical report, ECCC TR–22–101, 2022. Available at <http://www.eccc.uni-trier.de/eccc/>.
- [11] A. Ambainis. Communication complexity in a 3-computer model. *Algorithmica*, 16(3):298–301, 1996.
- [12] A. Ambainis. Quantum lower bounds by quantum arguments. *Journal of Computer and System Sciences*, 64(4):750–767, 2002. Earlier version in STOC’00. quant-ph/0002066.
- [13] A. Ambainis. Polynomial degree vs. quantum query complexity. *Journal of Computer and System Sciences*, 72(2):220–238, 2006. Earlier version in FOCS’03. quant-ph/0305028.

- [14] A. Ambainis. Quantum walk algorithm for element distinctness. *SIAM Journal on Computing*, 37(1):210–239, 2007. Earlier version in FOCS’04. quant-ph/0311001.
- [15] A. Ambainis. Quantum search with variable times. In *Proceedings of 25th Annual Symposium on Theoretical Aspects of Computer Science (STACS’08)*, pages 49–61, 2008. arXiv:1010.4458.
- [16] A. Ambainis, K. Balodis, J. Iraids, M. Kokainis, K. Prūsis, and J. Vihrovs. Quantum speedups for exponential-time dynamic programming algorithms. In *Proceedings of 30th ACM-SIAM SODA*, pages 1783–1793, 2019. arXiv:1807.05209.
- [17] A. Ambainis, A. Belovs, O. Regev, and R. de Wolf. Efficient quantum algorithms for (gapped) group testing and junta testing. In *Proceedings of 27th ACM-SIAM SODA*, pages 903–922, 2016. arXiv:1507.03126.
- [18] A. Ambainis, A. Childs, B. Reichardt, R. Špalek, and S. Zhang. Any AND-OR formula of size N can be evaluated in time $N^{1/2+o(1)}$ on a quantum computer. *SIAM Journal on Computing*, 39(6):2513–2530, 2010. Earlier version in FOCS’07.
- [19] A. Ambainis, M. Mosca, A. Tapp, and R. de Wolf. Private quantum channels. In *Proceedings of 41st IEEE FOCS*, pages 547–553, 2000. quant-ph/0003101.
- [20] A. Anshu, S. Arunachalam, T. Kuwahara, and M. Soleimanifar. Sample-efficient learning of quantum many-body systems. *Nature Physics*, 17:931–935, 2021. Earlier version in FOCS’20. arXiv:2004.07266.
- [21] J. van Apeldoorn and A. Gilyén. Quantum algorithms for zero-sum games. arXiv:1904.03180, 2019.
- [22] J. van Apeldoorn, A. Gilyén, S. Gribling, and R. de Wolf. Quantum SDP-solvers: better upper and lower bounds. *Quantum*, 4:230, 2020. Earlier version in FOCS’17. arXiv:1705.01843.
- [23] J. van Apeldoorn and A. Gilyén. Improvements in quantum SDP-solving with applications. In *Proceedings of 46th International Colloquium on Automata, Languages, and Programming*, volume 132 of *Leibniz International Proceedings in Informatics*, pages 99:1–99:15, 2019. arXiv:1804.05058.
- [24] S. Apers and R. de Wolf. Quantum speedup for graph sparsification, cut approximation and Laplacian solving. In *Proceedings of 61st IEEE Annual Symposium on Foundations of Computer Science*, pages 637–648, 2020. arXiv:1911.07306.
- [25] P. K. Aravind. A simple demonstration of Bell’s theorem involving two observers and no probabilities or inequalities. quant-ph/0206070, 2002.
- [26] S. Arunachalam, J. Briët, and C. Palazuelos. Quantum query algorithms are completely bounded forms. *SIAM Journal on Computing*, 48(3):903–925, 2019. Earlier version in ITCS’18. arXiv:1711.07285.
- [27] S. Arunachalam and R. de Wolf. Guest column: A survey of quantum learning theory. *SIGACT News*, 48(2):41–67, 2017. arXiv:1701.06806.

- [28] S. Arunachalam and R. de Wolf. Optimizing the number of gates in quantum search. *Quantum Information and Computation*, 17(4):251–261, 2017. arXiv:1512.07550.
- [29] S. Arunachalam and R. de Wolf. Optimal quantum sample complexity of learning algorithms. *Journal of Machine Learning Research*, 19, 2018. Earlier version in CCC’17. arXiv:1607.00932.
- [30] F. Arute, ..., and J. Martinis. Quantum supremacy using a programmable superconducting processor. *Nature*, 574:505–510, 2019. arXiv:1910.11333.
- [31] A. Aspect, Ph. Grangier, and G. Roger. Experimental tests of realistic local theories via Bell’s theorem. *Physical Review Letters*, 47:460, 1981.
- [32] L. Babai. Graph isomorphism in quasipolynomial time. In *Proceedings of 48th ACM STOC*, pages 684–697, 2016. arXiv:1512.03547.
- [33] L. Babai and E. M. Luks. Canonical labeling of graphs. In *Proceedings of 15th ACM STOC*, pages 171–183, 1983.
- [34] L. Babai and S. Moran. Arthur-Merlin games: a randomized proof system, and a hierarchy of complexity classes. *Journal of Computer and System Sciences*, 36(2):254–276, 1988.
- [35] Z. Bar-Yossef, T. S. Jayram, and I. Kerenidis. Exponential separation of quantum and classical one-way communication complexity. *SIAM Journal on Computing*, 38(1):366–384, 2008. Earlier version in STOC’04.
- [36] R. Beals. Quantum computation of Fourier transforms over symmetric groups. In *Proceedings of 29th ACM STOC*, pages 48–53, 1997.
- [37] R. Beals, H. Buhrman, R. Cleve, M. Mosca, and R. de Wolf. Quantum lower bounds by polynomials. *Journal of the ACM*, 48(4):778–797, 2001. Earlier version in FOCS’98. quant-ph/9802049.
- [38] S. Beauregard. Circuit for Shor’s algorithm using $2n + 3$ qubits. *Quantum Information and Computation*, 3(2):175–185, 2003. quant-ph/0205095.
- [39] J. S. Bell. On the Einstein-Podolsky-Rosen paradox. *Physics*, 1:195–200, 1964.
- [40] A. Belovs. Learning-graph-based quantum algorithm for k -distinctness. In *Proceedings of 53rd IEEE FOCS*, pages 207–216, 2012. arXiv:1205.1534.
- [41] A. Belovs. Span programs for functions with constant-sized 1-certificates. In *Proceedings of 43rd ACM STOC*, pages 77–84, 2012. arXiv:1105.4024.
- [42] A. Belovs. *Applications of the Adversary Method in Quantum Query Algorithms*. PhD thesis, University of Latvia, 2014.
- [43] A. Belovs. Quantum algorithms for learning symmetric juntas via adversary bound. *Computational Complexity*, 24(2):255–293, 2015. Earlier version in Complexity’14. arXiv:1311.6777.
- [44] A. Belovs. Variations on quantum adversary, 27 Apr 2015. arXiv:1504.06943.

- [45] A. Belovs and T. Lee. The quantum query complexity of composition with a relation. *arXiv:2004.06439*, 2020.
- [46] A. Belovs and B. Reichardt. Span programs and quantum algorithms for st-connectivity and claw detection. In *Proceedings of 20th European Symposium on Algorithms (ESA'12)*, pages 193–204, 2012. *arXiv:1203.2603*.
- [47] P. A. Benioff. Quantum mechanical Hamiltonian models of Turing machines. *Journal of Statistical Physics*, 29(3):515–546, 1982.
- [48] C. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. Wootters. Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels. *Physical Review Letters*, 70:1895–1899, 1993.
- [49] C. Bennett and S. Wiesner. Communication via one- and two-particle operators on Einstein-Podolsky-Rosen states. *Physical Review Letters*, 69:2881–2884, 1992.
- [50] C. H. Bennett, E. Bernstein, G. Brassard, and U. Vazirani. Strengths and weaknesses of quantum computing. *SIAM Journal on Computing*, 26(5):1510–1523, 1997. *quant-ph/9701001*.
- [51] C. H. Bennett and G. Brassard. Quantum cryptography: Public key distribution and coin tossing. In *Proceedings of the IEEE International Conference on Computers, Systems and Signal Processing*, pages 175–179, 1984.
- [52] D. Bernstein and T. Lange. Post-quantum cryptography. *Nature*, 549(6):188–194, 2017.
- [53] E. Bernstein and U. Vazirani. Quantum complexity theory. *SIAM Journal on Computing*, 26(5):1411–1473, 1997. Earlier version in STOC'93.
- [54] D. Berry, A. Childs, R. Cleve, R. Kothari, and R. Somma. Exponential improvement in precision for simulating sparse Hamiltonians. In *Proceedings of 46th ACM STOC*, pages 283–292, 2014. *arXiv:1312.1414*.
- [55] D. Berry, A. Childs, R. Cleve, R. Kothari, and R. Somma. Simulating Hamiltonian dynamics with a truncated Taylor series. *Physical Review Letters*, 114:090502, 2015. *arXiv:1412.4687*.
- [56] D. Berry, A. Childs, and R. Kothari. Hamiltonian simulation with nearly optimal dependence on all parameters. In *Proceedings of 56th IEEE FOCS*, pages 792–809, 2015. *arXiv:1501.01715*.
- [57] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd. Quantum machine learning. *Nature*, 549(7671), 2017. *arXiv:1611.09347*.
- [58] J. D. Biamonte and P. J. Love. Realizable Hamiltonians for universal adiabatic quantum computers. *Physical Review A*, 78(1)(012352), 2008. *arXiv:0704.1287*.
- [59] J-F. Biasse and F. Song. Efficient quantum algorithms for computing class groups and solving the principal ideal problem in arbitrary degree number fields. In *Proceedings of 27th ACM-SIAM SODA*, pages 893–902, 2016.
- [60] A. Blumer, A. Ehrenfeucht, D. Haussler, and M. K. Warmuth. Learnability and the Vapnik-Chervonenkis dimension. *Journal of the ACM*, 36(4):929–965, 1989.

- [61] A. Bookatz. QMA-complete problems. *Quantum Information and Computation*, 14(5–6):361–383, 2014. arXiv:1212.6312.
- [62] M. Boyer, G. Brassard, P. Høyer, and A. Tapp. Tight bounds on quantum searching. *Fortschritte der Physik*, 46(4–5):493–505, 1998. Earlier version in Physcomp’96. quant-ph/9605034.
- [63] F. Brandão, A. Kalev, T. Li, C. Lin, K. Svore, and X. Wu. Quantum SDP solvers: Large speed-ups, optimality, and applications to quantum learning. In *Proceedings of 46th International Colloquium on Automata, Languages, and Programming*, volume 132 of *Leibniz International Proceedings in Informatics*, pages 27:1–27:14, 2019. arXiv:1710.02581.
- [64] F. Brandão and K. Svore. Quantum speed-ups for solving semidefinite programs. In *Proceedings of 58th IEEE FOCS*, pages 415–426, 2017. arXiv:1609.05537.
- [65] G. Brassard, R. Cleve, and A. Tapp. The cost of exactly simulating quantum entanglement with classical communication. *Physical Review Letters*, 83(9):1874–1877, 1999. quant-ph/9901035.
- [66] G. Brassard, P. Høyer, M. Mosca, and A. Tapp. Quantum amplitude amplification and estimation. In *Quantum Computation and Quantum Information: A Millennium Volume*, volume 305 of *AMS Contemporary Mathematics Series*, pages 53–74. 2002. quant-ph/0005055.
- [67] G. Brassard, P. Høyer, and A. Tapp. Quantum algorithm for the collision problem. *ACM SIGACT News (Cryptography Column)*, 28:14–19, 1997. quant-ph/9705002.
- [68] A. Broadbent and A. B. Grilo. QMA-hardness of consistency of local density matrices with applications to quantum zero-knowledge. In *Proceedings of 61st IEEE FOCS*, pages 196–205, 2020. arXiv:1911.07782.
- [69] A. Broadbent and C. Schaffner. Quantum cryptography beyond quantum key distribution. *Designs, Codes and Cryptography*, 78(1):351–382, 2016. arXiv:1510.06120.
- [70] A. E. Brouwer and W. H. Haemers. *Spectra of Graphs*. Springer, 2012.
- [71] N. H. Bshouty and J. C. Jackson. Learning DNF over the uniform distribution using a quantum example oracle. *SIAM Journal on Computing*, 28(3):1136–1153, 1999. Earlier version in COLT’95.
- [72] H. Buhrman, R. Cleve, S. Massar, and R. de Wolf. Non-locality and communication complexity. *Reviews of Modern Physics*, 82:665–698, 2010. arXiv:0907.3584.
- [73] H. Buhrman, R. Cleve, J. Watrous, and R. de Wolf. Quantum fingerprinting. *Physical Review Letters*, 87(16), September 26, 2001. quant-ph/0102001.
- [74] H. Buhrman, R. Cleve, and A. Wigderson. Quantum vs. classical communication and computation. In *Proceedings of 30th ACM STOC*, pages 63–68, 1998. quant-ph/9802040.
- [75] H. Buhrman and R. Špalek. Quantum verification of matrix products. In *Proceedings of 17th ACM-SIAM SODA*, pages 880–889, 2006. quant-ph/0409035.

- [76] M. Bun and J. Thaler. Dual lower bounds for approximate degree and Markov-Bernstein inequalities. In *Proceedings of 40th ICALP*, volume 7965 of *Lecture Notes in Computer Science*, pages 303–314, 2013.
- [77] Y. Cao, J. Romero, J. Olson, M. Degroote, P. Johnson, M. Kieferová, I. Kivlichan, T. Menke, B. Peropadre, N. Sawaya, S. Sim, L. Veis, and A. Aspuru-Guzik. Quantum chemistry in the age of quantum computing. *Chemical Reviews*, 119(19):10856–11091, 2019. arXiv:1812.09976.
- [78] M. Cerezo, A. Arrasmith, R. Babbush, S. Benjamin, S. Endo, K. Fujii, J. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 1, 2021. arXiv:2012.09265.
- [79] S. Chakraborty, A. Gilyén, and S. Jeffery. The power of block-encoded matrix powers: improved regression techniques via faster Hamiltonian simulation. In *Proceedings of 46th International Colloquium on Automata, Languages, and Programming*, volume 132 of *Leibniz International Proceedings in Informatics*, pages 33:1–33:14, 2019. arXiv:1804.01973.
- [80] Y. Chen and R. de Wolf. Quantum algorithms and lower bounds for linear regression with norm constraints. arXiv:2110.13086, 2021.
- [81] N-H. Chia, A. Gilyén, T. Li, H-H. Lin, E. Tang, and C. Wang. Sampling-based sublinear low-rank matrix arithmetic framework for dequantizing quantum machine learning. In *Proceedings of 52nd ACM STOC*, pages 387–400, 2020. arXiv:1910.06151.
- [82] A. Childs. Lecture notes on quantum algorithms, 2017. Available at <https://cs.umd.edu/~amchilds/qa/>.
- [83] A. Childs, R. Kothari, and R. Somma. Quantum algorithm for systems of linear equations with exponentially improved dependence on precision. *SIAM Journal on Computing*, 46(6):1920–1950, 2017. arXiv:1511.02306.
- [84] A. M. Childs, D. Gosset, and Z. Webb. The Bose-Hubbard model is QMA-complete. *Theory of Computing*, 11(20):491–603, 2015. arXiv:1311.3297.
- [85] A. M. Childs, Y. Su, M. C. Tran, N. Wiebe, and S. Zhu. A theory of Trotter error. arXiv:1912.08854, 18 Dec 2019.
- [86] B. S. Cirel’son. Quantum generalizations of Bell’s inequality. *Letters in Mathematical Physics*, 4(2):93–100, 1980.
- [87] J. F. Clauser, M. A. Horne, A. Shimony, and R. A. Holt. Proposed experiment to test local hidden-variable theories. *Physical Review Letters*, 23(15):880–884, 1969.
- [88] R. Cleve. The query complexity of order-finding. In *Proceedings of 15th IEEE Conference on Computational Complexity*, pages 54–59, 2000. quant-ph/9911124.
- [89] R. Cleve and H. Buhrman. Substituting quantum entanglement for communication. *Physical Review A*, 56(2):1201–1204, 1997. quant-ph/9704026.

- [90] R. Cleve, W. van Dam, M. Nielsen, and A. Tapp. Quantum entanglement and the communication complexity of the inner product function. In *Proceedings of 1st NASA QCQC conference*, volume 1509 of *Lecture Notes in Computer Science*, pages 61–74. Springer, 1998. quant-ph/9708019.
- [91] R. Cleve, A. Ekert, C. Macchiavello, and M. Mosca. Quantum algorithms revisited. In *Proceedings of the Royal Society of London*, volume A454, pages 339–354, 1998. quant-ph/9708016.
- [92] S. Cook. The complexity of theorem-proving procedures. In *Proceedings of 3rd ACM STOC*, pages 151–158, 1971.
- [93] J. W. Cooley and J. W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [94] D. Coppersmith. An approximate Fourier transform useful in quantum factoring. IBM Research Report No. RC19642, quant-ph/0201067, 1994.
- [95] A. Cornelissen. Quantum gradient estimation and its application to quantum reinforcement learning. Master’s thesis, Delft University, 2018.
- [96] W. van Dam. Quantum oracle interrogation: Getting all information for almost half the price. In *Proceedings of 39th IEEE FOCS*, pages 362–367, 1998. quant-ph/9805006.
- [97] D. Deutsch. Quantum theory, the Church-Turing principle, and the universal quantum Turing machine. In *Proceedings of the Royal Society of London*, volume A400, pages 97–117, 1985.
- [98] D. Deutsch. Quantum computational networks. In *Proceedings of the Royal Society of London*, volume A425, 1989.
- [99] D. Deutsch and R. Jozsa. Rapid solution of problems by quantum computation. In *Proceedings of the Royal Society of London*, volume A439, pages 553–558, 1992.
- [100] A. Drucker and R. de Wolf. Quantum proofs for classical theorems. *Theory of Computing*, 2011. ToC Library, Graduate Surveys 2. arXiv:0910.3376.
- [101] V. Dunjko, J. Taylor, and H. Briegel. Advances in quantum reinforcement learning. *IEEE SMC*, pages 282–287, 2017. arXiv:1811.08676.
- [102] C. Dürr, M. Heiligman, P. Høyer, and M. Mhalla. Quantum query complexity of some graph problems. *SIAM Journal on Computing*, 35(6):1310–1328, 2006. Earlier version in ICALP’04. quant-ph/0401091.
- [103] C. Dürr and P. Høyer. A quantum algorithm for finding the minimum. quant-ph/9607014, 18 Jul 1996.
- [104] K. Efremenko. 3-query locally decodable codes of subexponential length. In *Proceedings of 41st ACM STOC*, pages 39–44, 2009.
- [105] H. Ehlich and K. Zeller. Schwankung von Polynomen zwischen Gitterpunkten. *Mathematische Zeitschrift*, 86:41–44, 1964.

- [106] A. Einstein, B. Podolsky, and N. Rosen. Can quantum-mechanical description of physical reality be considered complete? *Physical Review*, 47:777–780, 1935.
- [107] P. van Emde Boas. Machine models and simulations. In van Leeuwen [239], pages 1–66.
- [108] M. Ettinger, P. Høyer, and M. Knill. The quantum query complexity of the hidden subgroup problem is polynomial. *Information Processing Letters*, 91(1):43–48, 2004. quant-ph/0401083.
- [109] E. Farhi, J. Goldstone, and S. Gutmann. A quantum approximate optimization algorithm. arXiv:1411.4028, 2014.
- [110] O. Fawzi, A. Grospellier, and A. Leverrier. Constant overhead quantum fault-tolerance with quantum expander codes. In *Proceedings of 59th IEEE FOCS*, pages 743–754, 2018. arXiv:1808.03821.
- [111] R. Feynman. Simulating physics with computers. *International Journal of Theoretical Physics*, 21(6/7):467–488, 1982.
- [112] R. Feynman. Quantum mechanical computers. *Optics News*, 11:11–20, 1985.
- [113] L. Fortnow and J. Rogers. Complexity limitations on quantum computation. *Journal of Computer and System Sciences*, 59(2):240–252, 1999. Earlier version in Complexity’98. Also cs.CC/9811023.
- [114] P. Frankl and V. Rödl. Forbidden intersections. *Transactions of the American Mathematical Society*, 300(1):259–286, 1987.
- [115] R. Freivalds. Probabilistic machines can use less running time. In *Proceedings of 7th IFIP Congress*, pages 839–842, 1977.
- [116] M. Fürer. Faster integer multiplication. *SIAM Journal on Computing*, 39(3):979–1005, 2009. Earlier version in STOC’07.
- [117] M. Garey and D. Johnson. *Computers and Intractability : A Guide to the Theory of NP-completeness*. W. H. Freeman and Company, 1979.
- [118] D. Gavinsky, J. Kempe, I. Kerenidis, R. Raz, and R. de Wolf. Exponential separation for one-way quantum communication complexity, with applications to cryptography. *SIAM Journal on Computing*, 38(5):1695–1708, 2008. Earlier version in STOC’07. quant-ph/0611209.
- [119] S. Gharibian, Y. Huang, Z. Landau, and S. W. Shin. Quantum hamiltonian complexity. *Foundations and Trends in Theoretical Computer Science*, 10(3):159–282, 2015.
- [120] A. Gilyén. *Quantum Singular Value Transformation & Its Algorithmic Applications*. PhD thesis, University of Amsterdam, 2018.
- [121] A. Gilyén, S. Arunachalam, and N. Wiebe. Optimizing quantum optimization algorithms via faster quantum gradient computation. In *Proceedings of 30th ACM-SIAM SODA*, pages 1425–1444, 2019. arXiv:1711.00465.

- [122] A. Gilyén, Y. Su, G. H. Low, and N. Wiebe. Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. In *Proceedings of 51st ACM STOC*, pages 193–204, 2019. arXiv:1806.01838.
- [123] D. Gottesman. An introduction to quantum error correction and fault-tolerant quantum computation. In *Quantum Information Science and Its Contributions to Mathematics, Proceedings of Symposia in Applied Mathematics*, volume 68, pages 13–58, 2010. arXiv:0904.2557.
- [124] M. Grigni, L. Schulman, M. Vazirani, and U. Vazirani. Quantum mechanical algorithms for the nonabelian hidden subgroup problem. *Combinatorica*, 24(1):137–154, 2004. Earlier version in STOC’01.
- [125] L. K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of 28th ACM STOC*, pages 212–219, 1996. quant-ph/9605043.
- [126] J. Haah, A. W. Harrow, Z. Ji, X. Wu, and N. Yi. Sample-optimal tomography of quantum states. In *Proceedings of 48th ACM STOC*, pages 913–925, 2016. arXiv:1508.01797.
- [127] L. Hales and S. Hallgren. An improved quantum Fourier transform algorithm and applications. In *Proceedings of 41st IEEE FOCS*, pages 515–525, 2000.
- [128] S. Hallgren. Polynomial-time quantum algorithms for Pell’s equation and the principal ideal problem. *Journal of the ACM*, 54(1):653–658, 2007. Earlier version in STOC’02.
- [129] S. Hallgren, C. Moore, M. Roetteler, A. Russell, and P. Sen. Limitations of quantum coset states for graph isomorphism. *Journal of the ACM*, 57(6):34, 2010. Earlier version in STOC’06.
- [130] S. Hallgren, A. Russell, and A. Ta-Shma. The hidden subgroup problem and quantum computation using group representations. *SIAM Journal on Computing*, 32(4):916–934, 2003. Earlier version in STOC’00.
- [131] S. J. Hallgren. *Quantum Fourier Sampling, the Hidden Subgroup Problem, and Beyond*. PhD thesis, University of California, Berkeley, 2000.
- [132] S. Hanneke. The optimal sample complexity of PAC learning. *Journal of Machine Learning Research*, 17(38):1–15, 2016. arXiv:1507.00473.
- [133] G. H. Hardy and E. M. Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, New York, fifth edition, 1979.
- [134] A. Harrow, A. Hassidim, and S. Lloyd. Quantum algorithm for solving linear systems of equations. *Physical Review Letters*, 103(15):150502, 2009. arXiv:0811.3171.
- [135] D. Harvey and J. van der Hoeven. Integer multiplication in time $O(n \log n)$. *Annals of Mathematics*, 193(2):563–617, 2021. Preprint hal-02070778 2019.
- [136] J. Håstad. Some optimal inapproximability results. *Journal of the ACM*, 48(4):798–859, 2001. Earlier version in STOC’97.

- [137] B. Hensen, H. Bernien, A. E. Dréau, A. Reiserer, N. Kalb, M. S. Blok, J. Ruitenberg, R. F. L. Vermeulen, R. N. Schouten, C. Abellán, W. Amaya, V. Pruneri, M. W. Mitchell, M. Markham, D. J. Twitchen, D. Elkouss, S. Wehner, T. H. Taminiau, and R. Hanson. Loophole-free Bell inequality violation using electron spins separated by 1.3 kilometres. *Nature*, 526, 29 October 2015.
- [138] A. S. Holevo. Bounds for the quantity of information transmitted by a quantum communication channel. *Problemy Peredachi Informatsii*, 9(3):3–11, 1973. English translation in *Problems of Information Transmission*, 9:177–183, 1973.
- [139] P. Høyer, T. Lee, and R. Špalek. Negative weights make adversaries stronger. In *Proceedings of 39th ACM STOC*, pages 526–535, 2007. quant-ph/0611054.
- [140] R. Impagliazzo and A. Wigderson. $P = BPP$ if E requires exponential circuits: Derandomizing the XOR lemma. In *Proceedings of 29th ACM STOC*, pages 220–229, 1997.
- [141] G. Ivanyos, F. Magniez, and M. Santha. Efficient quantum algorithms for some instances of the non-Abelian hidden subgroup problem. *International Journal of Foundations of Computer Science*, 14(5):723–740, 2003. Earlier version in SPAA’01. quant-ph/0102014.
- [142] G. Ivanyos, L. Sanselme, and M. Santha. An efficient quantum algorithm for the hidden subgroup problem in nil-2 groups. *Algorithmica*, 62(1–2):480–498, 2012. Earlier version in LATIN’08. arXiv:0707.1260.
- [143] R. Jain, Z. Ji, S. Upadhyay, and J. Watrous. $QIP = PSPACE$. *Journal of the ACM*, 58(6):30:1–30:27, 2011. Earlier version in STOC’10. arXiv:0907.4737.
- [144] D. Janzing, P. Wocjan, and T. Beth. Non-identity check is QMA-complete. *International Journal of Quantum Information*, 3(3):463–473, 2005. quant-ph/0305050.
- [145] S. Jeffery, R. Kothari, and F. Magniez. Nested quantum walks with quantum data structures. In *Proceedings of 24th ACM-SIAM SODA*, pages 1474–1485, 2013. arXiv:1210.1199.
- [146] S. Jordan. Fast quantum algorithm for numerical gradient estimation. *Physical Review Letters*, 95:050501, 2005. quant-ph/0405146.
- [147] M. Kaplan, G. Leurent, A. Leverrier, and M. Naya-Plasencia. Breaking symmetric cryptosystems using quantum period finding. In *Proceedings of CRYPTO’16, Part II*, volume 9815 of *Lecture Notes in Computer Science*, pages 207–237, 2016. arXiv:1602.05973.
- [148] J. Katz and L. Trevisan. On the efficiency of local decoding procedures for error-correcting codes. In *Proceedings of 32nd ACM STOC*, pages 80–86, 2000.
- [149] J. Kempe, A. Yu. Kitaev, and O. Regev. The complexity of the local Hamiltonian problem. *SIAM Journal on Computing*, 35(5):1070–1097, 2006. Earlier version in FSTTCS’04. quant-ph/0406180.
- [150] I. Kerenidis, J. Landman, A. Luongo, and A. Prakash. q-means: A quantum algorithm for unsupervised machine learning. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems (NIPS’19), Paper 372*, page 4134–4144, 2019. arXiv:1812.03584.

- [151] I. Kerenidis and A. Prakash. Quantum recommendation systems. In *Proceedings of 8th Innovations in Theoretical Computer Science Conference*, volume 67 of *Leibniz International Proceedings in Informatics*, pages 49:1–49:21, 2017. arXiv:1603.08675.
- [152] I. Kerenidis and R. de Wolf. Exponential lower bound for 2-query locally decodable codes via a quantum argument. *Journal of Computer and System Sciences*, 69(3):395–420, 2004. Earlier version in STOC’03. quant-ph/0208062.
- [153] S. Kimmel, G. H. Low, C. Lin, M. Ozols, and T. Yoder. Hamiltonian simulation with optimal sample complexity. *npj Quantum Information*, 3(13), 2017. arXiv:1608.00281.
- [154] A. Kitaev and J. Watrous. Parallelization, amplification, and exponential time simulation of quantum interactive proof systems. In *Proceedings of 32nd ACM STOC*, pages 608–617, 2000.
- [155] A. Yu. Kitaev. Quantum measurements and the Abelian stabilizer problem. quant-ph/9511026, 12 Nov 1995.
- [156] A. Yu. Kitaev. Quantum NP, January 1999. Talk given at AQIP’99 conference, DePaul University, Chicago.
- [157] B. Klartag and O. Regev. Quantum one-way communication is exponentially stronger than classical communication. In *Proceedings of 43rd ACM STOC*, 2011. arXiv:1009.3640.
- [158] A. Klivans and D. van Melkebeek. Graph nonisomorphism has subexponential size proofs unless the polynomial-time hierarchy collapses. *SIAM Journal on Computing*, 31(5):1501–1526, 2002. Earlier version in STOC’99.
- [159] M. Knill, R. Laflamme, and W. Zurek. Threshold accuracy for quantum computation. quant-ph/9610011, 15 Oct 1996.
- [160] D. E. Knuth. *The Art of Computer Programming. Volume 2: Seminumerical Algorithms*. Addison-Wesley, third edition, 1997.
- [161] C-Y. Lai and H-C. Cheng. Learning quantum circuits of some T gates. *IEEE Transactions on Information Theory*, 68(6):3951–3964, 2022. arXiv:2106.12524.
- [162] F. Le Gall. Improved quantum algorithm for triangle finding via combinatorial arguments. In *Proceedings of 55th IEEE FOCS*, pages 216–225, 2014. arXiv:1407.0085.
- [163] T. Lee, F. Magniez, and M. Santha. Improved quantum query algorithms for triangle finding and associativity testing. *Algorithmica*, 77(2):459–486, 2017. arXiv:1210.1014.
- [164] T. Lee, R. Mittal, B. Reichardt, R. Špalek, and M. Szegedy. Quantum query complexity of state conversion. In *Proceedings of 52nd IEEE FOCS*, pages 344–353, 2011. arXiv:1011.3020.
- [165] A. K. Lenstra and H. W. Lenstra, Jr. *The Development of the Number Field Sieve*, volume 1554 of *Lecture Notes in Mathematics*. Springer, 1993.
- [166] H. W. Lenstra, Jr. and C. Pomerance. A rigorous time bound for factoring integers. *Journal of the American Mathematical Society*, 5:483–516, 1992.

- [167] L. Levin. Universal search problems (translated from the Russian). *Problems of Information Transmission*, 9(3):115–116, 1973.
- [168] L. Lin. Lecture notes on quantum algorithms for scientific computation, 2022. arXiv:2201.08309.
- [169] L. Lin and Y. Tong. Optimal quantum eigenstate filtering with application to solving quantum linear systems. arXiv:1910.14596, 31 Oct 2019.
- [170] Y.-K. Liu. Consistency of local density matrices is QMA-complete. In *Proceedings of 10th International Workshop on Randomization and Computation (RANDOM 2006)*, volume 4110 of *Lecture Notes in Computer Science*, pages 438–449, 2006. quant-ph/0604166.
- [171] Y.-K. Liu, M. Christandl, and F. Verstraete. Quantum computational complexity of the N-representability problem: QMA complete. *Physical Review Letters*, 98(110503), 2007. quant-ph/0609125.
- [172] S. Lloyd. Universal quantum simulators. *Science*, 273:1073–1078, 1996.
- [173] S. Lloyd, M. Mohseni, and P. Rebentrost. Quantum algorithms for supervised and unsupervised machine learning, 1 Jul 2013. arXiv:1307.0411.
- [174] S. Lloyd, M. Mohseni, and P. Rebentrost. Quantum principal component analysis. *Nature Physics*, 10:631–633, 2013. arXiv:1307.0401.
- [175] H-K. Lo and H. F. Chau. Unconditional security of quantum key distribution over arbitrarily long distances. *Science*, 283:2050–2056, 1999. quant-ph/9803006.
- [176] G. H. Low and I. L. Chuang. Hamiltonian simulation by uniform spectral amplification. arXiv:1707.05391, 17 Jul 2017.
- [177] G. H. Low and I. L. Chuang. Hamiltonian simulation by qubitization. arXiv:1610.06546, 20 Oct 2016.
- [178] G. H. Low and I. L. Chuang. Optimal Hamiltonian simulation by quantum signal processing. *Physical Review Letters*, 118(1):010501, 2017. arXiv:1606.02685.
- [179] G. H. Low, T. J. Yoder, and I. L. Chuang. Methodology of resonant equiangular composite quantum gates. *Physical Review X*, 6(4):041067, 2016. arXiv:1603.03996.
- [180] R. A. Low. Learning and testing algorithms for the Clifford group. *Physical Review A*, 80(052314), 2009. arXiv:0907.2833.
- [181] C. Lund, L. Fortnow, H. Karloff, and N. Nisan. Algebraic methods for interactive proof systems. *Journal of the ACM*, 39(4):859–868, 1992. Earlier version in FOCS’90.
- [182] F. Magniez, A. Nayak, J. Roland, and M. Santha. Search via quantum walk. *SIAM Journal on Computing*, 40(1):142–164, 2011. Earlier version in STOC’07. quant-ph/0608026.
- [183] F. Magniez, M. Santha, and M. Szegedy. Quantum algorithms for the triangle problem. In *Proceedings of 16th ACM-SIAM SODA*, pages 1109–1117, 2005. quant-ph/0310134.

- [184] U. Mahadev. Classical verification of quantum computations. In *Proceedings of 59th IEEE FOCS*, pages 259–267, 2018. arXiv:1804.01082.
- [185] Y. Manin. Vychislimoe i nevychislimoe (computable and noncomputable). *Soviet Radio*, pages 13–15, 1980. In Russian.
- [186] Y. Manin. Classical computing, quantum computing, and Shor’s factoring algorithm. quant-ph/9903008, 2 Mar 1999.
- [187] C. Marriott and J. Watrous. Quantum Arthur-Merlin games. *Computational Complexity*, 14(2):122–152, 2005. Earlier version in CCC’04. arXiv:cs/0506068.
- [188] D. Mayers. Unconditional security in quantum cryptography. quant-ph/9802025, 10 Feb 1998.
- [189] M. Mohri, A. Rostamizadeh, and A. Talwalkar. *Foundations of Machine Learning*. MIT Press, second edition, 2018.
- [190] C. Moore, D. N. Rockmore, and A. Russell. Generic quantum Fourier transforms. *ACM Transactions on Algorithms*, 2(4):707–723, 2006. quant-ph/0304064.
- [191] C. Moore, A. Russell, and L. Schulman. The symmetric group defies strong Fourier sampling. *SIAM Journal on Computing*, 37(6):1842–1864, 2008. quant-ph/0501056+66. Earlier version in FOCS’05.
- [192] M. Mosca and A. Ekert. The hidden subgroup problem and eigenvalue estimation on a quantum computer. In *Proceedings of 1st NASA QQC conference*, volume 1509 of *Lecture Notes in Computer Science*, pages 174–188. Springer, 1998. quant-ph/9903071.
- [193] A. Nayak. Optimal lower bounds for quantum automata and random access codes. In *Proceedings of 40th IEEE FOCS*, pages 369–376, 1999. quant-ph/9904093.
- [194] I. Newman. Private vs. common random bits in communication complexity. *Information Processing Letters*, 39(2):67–71, 1991.
- [195] I. Newman and M. Szegedy. Public vs. private coin flips in one round communication games. In *Proceedings of 28th ACM STOC*, pages 561–570, 1996.
- [196] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2000.
- [197] R. O’Donnell and J. Wright. Efficient quantum tomography. In *Proceedings of 48th ACM STOC*, pages 899–912, 2016. arXiv:1508.01907.
- [198] C. H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994.
- [199] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. Love, A. Aspuru-Guzik, and J. O’Brien. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 3:24, 2014.
- [200] J. Preskill. Fault-tolerant quantum computation. In H.-K. Lo, S. Popescu, and T. P. Spiller, editors, *Introduction to Quantum Computation*. World Scientific, 1998. quant-ph/9712048.

- [201] J. Preskill. Quantum computing 40 years later. In A. Hey, editor, *Feynman Lectures on Computation*. Taylor & Francis Group, second edition, 2022. arXiv:2106.10522.
- [202] A. Razborov. Quantum communication complexity of symmetric predicates. *Izvestiya of the Russian Academy of Sciences, mathematics*, 67(1):159–176, 2003. quant-ph/0204025.
- [203] P. Rebentrost, M. Mohseni, and S. Lloyd. Quantum support vector machine for big data classification. *Physical Review Letters*, 113(13):130503, 2014. arXiv:1307.0471.
- [204] O. Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):34:1–34:40, 2009. Earlier version in STOC’13.
- [205] B. Reichardt. Span programs and quantum query complexity: The general adversary bound is nearly tight for every Boolean function. In *Proceedings of 50th IEEE FOCS*, pages 544–551, 2009.
- [206] B. Reichardt. Faster quantum algorithm for evaluating game trees. In *Proceedings of 22nd ACM-SIAM SODA*, pages 546–559, 2011. arXiv:0907.1623.
- [207] B. Reichardt. Span programs are equivalent to quantum query algorithms. *SIAM Journal on Computing*, 43(3):1206–1219, 2014.
- [208] B. Reichardt and R. Špalek. Span-program-based quantum algorithm for evaluating formulas. *Theory of Computing*, 8:291–319, 2012. Earlier version in STOC’08. arXiv:0710.2630.
- [209] J. Riordan and C. E. Shannon. The number of two-terminal series-parallel networks. *Journal of Mathematics and Physics*, 21:83–93, 1942.
- [210] R. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public key cryptosystems. *Communications of the ACM*, 21:120–126, 1978.
- [211] R. L. Rivest. Cryptography. In van Leeuwen [239], pages 717–755.
- [212] T. J. Rivlin and E. W. Cheney. A comparison of uniform approximations on an interval and a finite subset thereof. *SIAM Journal on Numerical Analysis*, 3(2):311–320, 1966.
- [213] S. Saeedi and T. Arodz. Quantum sparse support vector machines, 2019. arXiv:1902.01879.
- [214] S. Saeedi, A. Panahi, and T. Arodz. Quantum semi-supervised kernel learning. *Quantum Machine Intelligence*, 3:24, 2021.
- [215] M. Saks and A. Wigderson. Probabilistic Boolean decision trees and the complexity of evaluating game trees. In *Proceedings of 27th IEEE FOCS*, pages 29–38, 1986.
- [216] M. Santha. Quantum walk based search algorithms. In *Proceedings of 5th TAMC*, pages 31–46, 2008. arXiv/0808.0059.
- [217] T. Santoli and C. Schaffner. Using Simon’s algorithm to attack symmetric-key cryptographic primitives. *Quantum Information and Computation*, 17(1& 2):65–78, 2017. arXiv:1603.07856.
- [218] A. Schönhage and V. Strassen. Schnelle Multiplikation grosser Zahlen. *Computing*, 7:281–292, 1971.

- [219] U. Schöning. A probabilistic algorithm for k -SAT and constraint satisfaction problems. In *Proceedings of 40th IEEE FOCS*, pages 410–414, 1999.
- [220] N. Schuch and F. Verstraete. Computational complexity of interacting electrons and fundamental limitations of density functional theory. *Nature Physics*, 5:732–735, 2009. arXiv:0712.0483.
- [221] M. Schuld and N. Killoran. Quantum machine learning in feature Hilbert spaces. *Physical Review Letters*, 122(13):040504, 2019. arXiv:1803.07128.
- [222] M. Schuld and F. Petruccione. *Machine Learning with Quantum Computers*. Springer, second edition, 2021.
- [223] S. Shalev-Shwartz and S. Ben-David. *Understanding machine learning: From theory to algorithms*. Cambridge University Press, 2014.
- [224] A. Shamir. $IP = PSPACE$. *Journal of the ACM*, 39(4):869–877, 1992. Earlier version in FOCS’90.
- [225] A. Shen. $IP = PSPACE$: Simplified proof. *Journal of the ACM*, 39(4):878–880, 1992.
- [226] A. Sherstov. Approximating the AND-OR tree. *Theory of Computing*, 9(20):653–663, 2013.
- [227] P. W. Shor. Scheme for reducing decoherence in quantum memory. *Physical Review A*, 52:2493, 1995.
- [228] P. W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997. Earlier version in FOCS’94. quant-ph/9508027.
- [229] V. Shoup. Lower bounds for discrete logarithms and related problems. In *Proceedings of Eurocrypt’97*, volume 1233 of *Lecture Notes in Computer Science*, pages 256–266. Springer, 1997. Revised version available at <http://www.shoup.net/papers/dlbounds1.pdf>.
- [230] D. Simon. On the power of quantum computation. *SIAM Journal on Computing*, 26(5):1474–1483, 1997. Earlier version in FOCS’94.
- [231] R. Špalek and M. Szegedy. All quantum adversary methods are equivalent. *Theory of Computing*, 2(1):1–18, 2006. Earlier version in ICALP’05, quant-ph/0409116.
- [232] A. Steane. Multiple particle interference and quantum error correction. In *Proceedings of the Royal Society of London*, volume A452, pages 2551–2577, 1996. quant-ph/9601029.
- [233] M. Szegedy. Quantum speed-up of Markov chain based algorithms. In *Proceedings of 45th IEEE FOCS*, pages 32–41, 2004. quant-ph/0401053.
- [234] E. Tang. A quantum-inspired classical algorithm for recommendation systems. In *Proceedings of 51st ACM STOC*, pages 217–228, 2019. arXiv:1807.04271.
- [235] B. M. Terhal. Quantum error correction for quantum memories. *Reviews of Modern Physics*, 87:307, 2015. arXiv:1302.3428.

- [236] L. Trevisan. Some applications of coding theory in computational complexity. *Quaderni di Matematica*, 13:347–424, 2004.
- [237] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. In *Proceedings of the London Mathematical Society*, volume 42, pages 230–265, 1936. Correction, *ibidem* (vol. 43), pages 544–546.
- [238] L. Valiant. A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142, 1984.
- [239] J. van Leeuwen, editor. *Handbook of Theoretical Computer Science. Volume A: Algorithms and Complexity*. MIT Press, Cambridge, MA, 1990.
- [240] L. Vandersypen, M. Steffen, G. Breyta, C. Yannoni, R. Cleve, and I. Chuang. Experimental realization of an order-finding algorithm with an NMR quantum computer. *Physical Review Letters*, 85(25):5452–5455, 2000. quant-ph/0007017.
- [241] V. Vapnik and A. Chervonenkis. On the uniform convergence of relative frequencies of events to their probabilities. *Theory of Probability & Its Applications*, 16(2):264–280, 1971. English translation of 1968 Russian paper in *Dokl. Akad. Nauk*. 181(4).
- [242] T. Vidick and J. Watrous. Quantum proofs. *Foundations and Trends in Theoretical Computer Science*, 11(1–2):1—215, 2015.
- [243] J. Watrous. Succinct quantum proofs for properties of finite groups. In *Proceedings of 41st IEEE FOCS*, pages 537–546, 2000. quant-ph/0011023.
- [244] J. Watrous. Quantum algorithms for solvable groups. In *Proceedings of 33rd ACM STOC*, pages 60–67, 2001.
- [245] J. Watrous. PSPACE has 2-round quantum interactive proof systems. *Theoretical Computer Science*, 292(3):575–588, 2003. Earlier version in STOC’99. arXiv:cs/9901015.
- [246] J. Watrous. Quantum computational complexity. In *Encyclopedia of Complexity and Systems Science*. Springer, 2009. arXiv:0804.3401.
- [247] J. Watrous. *The Theory of Quantum Information*. Cambridge University Press, 2018. Available at <https://cs.uwaterloo.ca/~watrous/TQI/>.
- [248] R. de Wolf. *Quantum Computing and Communication Complexity*. PhD thesis, University of Amsterdam, 2001.
- [249] W. K. Wootters and W. H. Zurek. A single quantum cannot be copied. *Nature*, 299:802–803, 1982.
- [250] A. C-C. Yao. Some complexity questions related to distributive computing. In *Proceedings of 11th ACM STOC*, pages 209–213, 1979.
- [251] A. C-C. Yao. Quantum circuit complexity. In *Proceedings of 34th IEEE FOCS*, pages 352–360, 1993.
- [252] S. Yekhanin. Towards 3-query locally decodable codes of subexponential length. *Journal of the ACM*, 55(1), 2008. Earlier version in STOC’07.